

Boot Spring Boot

스프링 부트 이용설명서

Written by honeymon

Boot Spring Boot

스프링 부트 시작하기

Honeymon(Jiheon Kim)

Version 0.9.1, 2017-11-21

Table of Contents

저자 소개	2
들어가며.....	4
책에서 다루는 내용.....	4
책을 따라하기 위해 필요한 것	5
대상독자	5
관례(Convention)	6
예제.....	7
1. '스프링 부트' 시작하기	8
1.1. 스프링 부트 소개.....	8
단독실행 가능한 스프링 애플리케이션 생성.....	8
내장 컨테이너로 톰캣, 제티 혹은 언더토우 중에서 선택가능(WAR 배포 필요없음)	8
스타터('starter') POM 을 제공하여 간결한 의존성 구성 지원	8
스프링에 대한 자동구성 제공	9
제품출시 후 운영에 필요한 상태, 건강 점검과 외부구성 등 다양한 기능 제공	9
구성을 위한 XML 코드를 필요로 하지 않으며 작성하지 않아도 됨.....	9
1.2. 시스템 요구사항	9
1.3. 스프링 부트 설치.....	11
1.4. 스프링 부트 애플리케이션 개발 시작	12
1.4.1. 프로젝트 생성	12
1.4.2. 의존성 추가	14
1.4.3. 예제: HelloController 작성	14
1.4.4. 예제: HelloController 실행하기	15
1.4.5. 실행가능한 jar 만들기.....	17
1.5. 다음 읽을꺼리	20
2. '스프링 부트' 살펴보기	21
2.1. 빌드 시스템.....	21
2.1.1. 의존성 관리	21
2.1.2. 메이븐	23
2.1.3. 부모 스프링 부트 스타터 상속	24
2.1.4. spring-boot-starter-parent 없이 사용하기.....	24
2.1.5. 스프링 부트 메이븐 플러그인 사용하기.....	26
2.1.6. 그레이들.....	26
2.1.7. 스타터(Starter).....	27
2.1.8. 래퍼(Wrapper)	27
2.2. 코드 구조.....	28
2.2.1. "기본 패키지" 이용	29
2.2.2. 애플리케이션 메인 클래스 위치.....	30
2.3. 구성(Configuration) 클래스.....	33
2.3.1. 구성클래스 불러오기.....	33
2.3.2. XML 구성 불러오기	33
2.4. 자동구성(auto-configuration)	33
2.4.1. 자동구성 대체하기.....	34
2.4.2. 특정 자동구성 비활성화하기	37
2.5. 스프링 빈(Bean)과 의존성 주입(Dependency injection)	37
2.6. @SpringBootApplication 애너테이션 이용	38

2.7. 애플리케이션 실행하기.....	39
2.7.1. IDE 에서 실행하기.....	39
2.7.2. 패키징된 애플리케이션 실행하기	40
2.7.3. 그레이들 플러그인으로 실행하기	41
2.7.4. 메이븐 플러그인으로 실행하기	41
2.7.5. 핫 스와핑	42
2.8. '스프링 부트' 개발자도구	42
2.8.1. 속성 기본정의	43
2.8.2. 자동 재시작	43
2.8.3. 전역 설정	45
2.8.4. 원격 애플리케이션.....	45
2.9. 출시를 위한 애플리케이션 패키징	46
2.10. 다음 읽을거리.....	47
3. '스프링 부트' 기능소개	48
3.1. 스프링애플리케이션(SpringApplication)	48
3.1.1. 시동 실패	49
3.1.2. 배너 재정의	49
3.1.3. SpringApplication 재정의.....	50
3.1.4. 유연한(Fluent) 빌더 API	51
3.1.5. 애플리케이션 이벤트 및 리스너.....	52
3.1.6. 웹 환경	53
3.1.7. 애플리케이션 실행인자에 접근	54
3.1.8. CommandLineRunner 혹은 ApplicationRunner 사용하기.....	54
3.1.9. 애플리케이션 종료.....	55
3.2. 외부 구성(Externalized Configuration)	55
3.2.1. 무작위(Random)값 구성	57
3.2.2. 커맨드라인 속성 접근	58
3.2.3. 애플리케이션 속성 파일.....	58
3.2.4. 프로파일 정의 속성	59
3.2.5. 속성 내 치환자(placeholder)	60
3.2.6. properties 를 대신하여 YAML 사용하기.....	60
3.2.7. 보장형(type-safe) 구성 속성	65
3.3. 프로파일	72
3.3.1. 활성화할 프로파일 추가하기	73
3.3.2. 프로파일을 프로그래밍적으로 설정	74
3.3.3. 프로파일 지정 구성 파일	74
3.4. 로깅.....	74
3.4.1. 로그 형식	74
3.4.2. 콘솔 출력	75
3.4.3. 파일 출력	75
3.4.4. 파일 출력	76
3.4.5. 로그 레벨	77
로그 구성 재정의.....	77
3.4.6. 로그백 확장	78
3.5. 웹애플리케이션 개발	80
3.5.1. '스프링 Web MVC framework'.....	80
3.5.2. 에러 처리	86

3.5.3. 스프링 HATEOAS	89
3.5.4. CORS 지원.....	89
3.5.5. 내장 서블릿 컨테이너 지원	90
3.6. 보안.....	92
3.6.1. 액추에이터 보안(Actuator Security).....	93
3.7. SQL 데이터베이스 이용	94
3.7.1. DataSource 구성.....	94
3.7.2. JdbcTemplate 사용	95
3.7.3. MyBatis 사용	96
3.7.4. JPA 그리고 '스프링 Data'	96
3.7.5. H2 사용	99
3.8. NoSQL 기술 이용.....	101
3.8.1. 레디스(Redis)	101
3.8.2. 몽고DB(MongoDB)	102
3.8.3. 엘라스틱서치(Elasticsearch)	104
3.9. 캐시.....	106
3.9.1. 지원하는 캐시 제공자(Provider)	107
3.10. REST 서비스 호출	110
3.10.1. RestTemplate 재정의	110
3.11. 스프링 세션	111
3.12. 테스트.....	112
3.12.1. 테스트 관련 의존성	112
3.12.2. 스프링 애플리케이션 테스트.....	112
3.12.3. 스프링 부트 애플리케이션 테스트.....	113
3.13. 웹소켓(WebSocket)	122
3.14. 웹 서비스(Web Service).....	122
3.15. 자동구성 직접 생성하기	122
3.15.1. 자동구성 빈 이해하기	122
3.15.2. 자동구성 대상 위치	123
3.15.3. 조건 애너테이션	123
3.15.4. 스타터 직접 생성하기	124
3.16. 다음 읽을거리.....	125
4. '스프링 부트' 출시준비: 액추에이터	126
4.1. 출시준비 기능 활성화.....	126
4.1.1. 종단점(Endpoint)	126
4.1.2. 종단점 보안	128
4.1.3. 종단점 재정의	129
4.1.4. 액추에이터 MVC 종단점 를 위한 하이퍼미디어	129
4.1.5. CORS 지원	130
4.1.6. 재정의 종단점 추가하기.....	130
4.1.7. 건강정보	130
4.1.8. HealthIndicator 보안.....	130
4.1.9. 애플리케이션 정보.....	132
4.2. HTTP 로 모니터링하고 관리하기.....	134
4.2.1. 관리 종단점 경로 재정의	134
4.2.2. 관리 서버 포트 재정의	134
4.2.3. 관리를 위한 SSL 재정의	135

4.2.4. 관리 서버 주소 재정의	135
4.2.5. HTTP 종단점 비활성화	136
4.2.6. HTTP 건강 종단점 접근 제한	136
4.3. 측량(Metrics)	137
4.4. 감시	140
4.5. 추적(Tracing)	141
4.5.1. 추적 재정의	142
4.6. 프로세스 모니터링	142
4.6.1. 확장 구성	142
4.6.2. 프로그래밍적인 처리	142
4.7. 다음 읽을꺼리	143
5. '스프링 부트' 배포	144
5.1. 클라우드 배포	144
5.2. 스프링 부트 애플리케이션 설치	144
5.2.1. 유닉스/리눅스 서비스	146
5.3. MS 윈도우즈 서비스	147
5.4. 다음 읽을꺼리	147
6. '스프링 부트' 빌드도구 플러그인	148
6.1. 스프링 부트 메이븐 플러그인	148
6.1.1. 플러그인 추가	148
6.1.2. 플러그인 추가하기	148
6.1.3. 실행가능한 jar 와 war 파일 패키징	150
6.2. 스프링 부트 그레이들 플러그인	151
6.2.1. 플러그인 추가하기	151
6.2.2. 그레이들 의존성 관리	152
6.2.3. 실행가능한 jar 와 war 파일 생성	152
6.2.4. 프로젝트를 빌드하지 않고 실행하기	153
6.2.5. 스프링 부트 플러그인 구성	153
6.3. 그 외의 빌드 시스템	155
6.3.1. 재압축한 압축파일	155
6.3.2. 라이브러리 포함하기	155
6.3.3. 메인 클래스 찾기	155
6.3.4. 재압축 구현 예제	155
6.4. 다음 읽을꺼리	156
7. '스프링 부트' 활용 안내	157
7.1. 스프링 부트 애플리케이션	157
7.1.1. 오류분석기 제작	157
7.1.2. 자동구성 문제해결	157
7.1.3. Environment 혹은 ApplicationContext 가 시작하기 전에 재정의하기	158
7.1.4. ApplicationContext 계층 조직(부모 혹은 루트 컨텍스트 추가)	159
7.1.5. 비-웹애플리케이션 생성	160
7.2. 속성(Properties) 및 구성(Configuration)	160
7.2.1. 빌드 시 자동으로 속성 확장	160
7.2.2. SpringApplication 외부 구성	161
7.2.3. 애플리케이션 속성 외부구성파일 위치변경	162
7.2.4. 커맨드라인 실행인자 축약사용	162
7.2.5. 외부속성에 대한 YAML 사용	163
7.2.6. 스프링 프로파일 활성화	163

7.2.7. 환경 의존적인 구성 변경	164
7.2.8. 외부속성에 대한 빌트인 옵션(built-in option) 발견.....	164
7.3. 내장서블릿 컨테이너	165
7.3.1. 애플리케이션에 서블릿, 필터 혹은 리스너 추가하기	165
7.3.2. HTTP 포트 변경	166
7.3.3. 할당되지 않은 HTTP 포트 무작위 사용	167
7.3.4. 실행시 HTTP 포트 할당	167
7.3.5. SSL 구성	167
7.3.6. 접근로그 구성	168
7.3.7. 톰캣 구성	168
7.3.8. 톰캣을 언더투우로 대체하기	169
7.3.9. 언더투우 구성	170
7.3.10. 언더투우 멀티 리스너 활성화	170
7.3.11. @ServerEndpoint 를 사용한 웹소켓 종단점 생성.....	171
7.3.12. HTTP 응답 압축 활성화	171
7.4. 스프링 MVC	172
7.4.1. JSON REST 서비스 작성	172
7.4.2. XML REST 서비스 작성.....	172
7.4.3. 잭슨(Jackson) 오브젝트매퍼(ObjectMapper) 재정의	173
7.4.4. @ResponseBody 렌더링 재정의	174
7.4.5. Multipart 파일 업로드 제어	174
7.4.6. 스프링 MVC DispatcherServlet 전환	175
7.4.7. 기본 MVC 구성 전환	175
7.4.8. ViewResolver 재정의	175
7.5. HTTP 클라이언트.....	176
7.5.1. RestTemplate 가 프록시를 사용하도록 구성.....	176
7.6. 로깅.....	177
7.6.1. Logback 구성.....	178
7.6.2. Log4j 구성.....	179
7.7. 데이터 접근.....	179
7.7.1. DataSource 재정의 구성.....	180
7.7.2. DataSource 복수 정의.....	182
7.7.3. 스프링 Data {repo} 이용	184
7.7.4. 스프링 구성으로부터 @Entity 정의 분리.....	184
7.7.5. JPA 속성 구성	185
7.7.6. 재정의 EntityManagerFactory 이용.....	185
7.7.7. 복수 EntityManager 이용	185
7.7.8. 전통적인 persistence.xml 이용	186
7.7.9. 스프링 Data JPA 와 몽고 {repo} 이용	187
7.7.10. REST 종단점 로 스프링 Data {repo} 노출	187
7.8. 데이터베이스 초기화	187
7.8.1. JPA 를 이용한 데이터베이스 초기화.....	187
7.8.2. Hibernate 를 이용한 데이터베이스 초기화	188
7.8.3. 데이터베이스 초기화.....	188
7.8.4. 스프링 배치 를 이용한 데이터베이스 초기화.....	188
7.8.5. 고차원 데이터베이스 마이그레이션 도구 사용	189
7.9. 배치 애플리케이션	190

7.9.1. 구동시 스프링 배치 작업 실행	190
7.10. 액추에이터	191
7.10.1. 액추에이터 종단점 의 포트 혹은 주소 변경	191
7.10.2. whitelabel 오류 페이지 재정의	191
7.11. 보안	191
7.11.1. 스프링 부트 보안 구성 전환	191
7.11.2. AuthenticationManager 변경하고 사용자 계정 추가	191
7.11.3. 프록시 서버 이용시 HTTPS 활성화	192
7.12. 핫스와핑(Hot swapping)	193
7.12.1. 정적 콘텐츠 재적재	193
7.12.2. 컨테이너 재시작없이 템플릿 재적재	193
7.12.3. 빠른 애플리케이션 재시작	193
7.12.4. 컨테이너 재시작없이 자바 클래스 재적재하기	194
7.13. 빌드	194
7.13.1. 빌드 정보 생성	194
7.13.2. 깃 정보 생성	195
7.13.3. 의존성 버전 재정의	197
7.14. 전통적 배포	197
7.14.1. 배포가능한 war 파일 생성	197
7.14.2. 오래된 서블릿 컨테이너에 배포가능한 war 파일 생성	198
7.14.3. 기존 애플리케이션을 스프링 부트 로 변환	198
7.14.4. 오래된 컨테이너(Servlet 2.5)에 war 배포	201
8. 정리	204
부록	206

저자 소개

안녕하세요, 평범한 월급쟁이개발자를 지향하는 김지현 입니다.
인터넷에서는 허니몬(honeymon) 이라는 이름으로 활동하고 있습니다.



'월급쟁이개발자'를 자처하면서 다른 분들이 쓴 책을 읽으며 오타자 찾기만하다가 다른 '개발자'가 읽을 책을 써보겠다고 나서는 무모함을 가지고 있는 평범한(?) 사람입니다(이 책에도 제가 놓친 오타자가 있겠죠. 분명히 있을 겁니다. 미리 "죄송합니다!!").

개발활동은 스쿠바다이빙, 클라이밍, 라이딩, 수영 등의 취미생활을 즐기기 위한 부업활동이라고 말하고 다니는 월급쟁이개발자지만 일하는 과정에서 문제를 해결하는 과정을 즐기고 있습니다.

제가 스프링 부트와 인연을 맺게 된 것은 잘 다니던 회사를 2014년 5월 말에 그만두고 '스쿠바 다이빙 강사'가 되겠다며 양양 동해바다에서 3개월 동안 생활하다가 복귀하면서솔루션 개발에 스프링 부트를 이용한 것이 계기가 되었습니다.

다양한 운영체제(JVM 기반)에도 실행가능한(Executable jar) 웹애플리케이션(Spring MVC)으로 개발해야한다.

위 요구사항에 제일 먼저 떠오른 것이 '스프링 부트'였습니다.

그 무렵 KSUG에서 스프링 부트와 관련된 소개하는 발표가 몇 번 있었습니다. 그래도 귀동냥한 것이 있어서 써먹을 수 있게 되었습니다. 스프링 부트로 개발을 하면서 제일 많이 보게되는 것은 '[참고문서\(refernece document\)](https://goo.gl/TRaKm3)'(<https://goo.gl/TRaKm3>)입니다(아마 이 책을 읽으시는 분들도 가장 많이 보게될 문서라고 생각합니다).

국내에서도 스프링 부트를 이용하는 사람이 많이 늘어날 것이고 나도 참고문서를 살펴보면 익힐 생각으로 '**레퍼런스 문서**'

번역을 해보기로 결심했습니다. 이 번역을 하기에 앞서 에이콘에서 'Git을 이용한 버전관리'(<https://goo.gl/X8rPFE>)라는 책을 번역했었기에 쉽게 할 수 있을 거라고 봤습니다(… 부끄럽고 죄송합니다).

'무식하면 용감하다' 라던가?

'Spring Boot Reference Guide'(<https://goo.gl/iZHGcv>)를 그대로 복사하여 번역할 계획을 세웠습니다. 이제는 요령이 생겼지만, 처음 시작할 때는 'current'와 'current-SNAPSHOT'과 특정 버전의 문서를 각각 볼 수 있다는 것과, 스프링 부트 가 빠르게 버전업된다는 것을 살피지 못했습니다. 그래도 용감하게 번역을 시작합니다.

- 스프링 부트 참고가이드: <https://github.com/ihoneymon/translate-spring-boot-reference>

저장소에 들어오면 바로 볼 수 있도록 하겠다는 생각으로 README.md 파일에 마크다운을 이용하여 무식하게 했다. 그러다가 지쳐서 탄짓 하다가 다른 분들에게 도움을 받아 80% 정도의 번역을 끝으로 방치상태로 내버려두고 있습니다(… 마무리를 지어볼까 하는 시점에는 1.3.0.RELEASE가 출시되어버렸고 초반에 제대로 구성하지 않은 문서를 가지고는 변경사항들만 적용하기도 어려워서 포기). 이 무모했던 번역문서를 만질까 말까 망설이고 있던 제게 또 다른 번역 제안이 들어왔습니다. 내용도 괜찮고 분량도 적당하다는 생각에 5개월 동안 번역하고 손질하는데 많은 시간이 들어서 'Spring MVC 4 익히기'(<https://goo.gl/4uMdgw>)라는 번역서를 내놓았습니다. 이 번역서를 쓰면서 어떻게 책을 쓰면 좋겠다는 생각을 했습니다.

그리고 현재 이 책을 쓰고 있습니다. 두둑치…!

이 책이 나올 때까지, 쓰기 싫다는 투정을 부리는 놈을 잘 방목해주신, 출판관계자분께 감사드립니다. (_ _)

들어가며

스프링 부트가 [소개\(2014.04.01\)\(https://goo.gl/MDjMZ2\)](https://goo.gl/MDjMZ2)된 지 2년이 넘는 시점에 국내에서 스프링 부트 관련 책들이 나오기 시작한다.

각각의 책은 각자의 방식으로 스프링 부트 이용방법을 소개하고 있다. 이 책을 쓰고 있는 나 역시 나만의 방식으로 소개하면서 '책을 읽는 여러분 각자의 방식으로 소화하여 자유롭게 스프링 부트 를 사용하라'고 유혹하려 한다.

스프링 부트로 개발하면서 '스프링 부트 참고 문서'를 정말 많이 봤다. 참고문서만으로 이해하기 어려운 부분에 대해서는 소스코드를 열어서 어떻게 구성되어 있고 동작할지를 살피기 시작했다. 그런 이해를 바탕으로 애플리케이션에 필요한 기능을 위한 구성을 재정의하거나 제공하는 클래스와 인터페이스를 사용하여 직접 구성하는 경험을 쌓게 됐다. 그런 경험을 전달하여 삽질(or 시행착오)을 줄기를 바란다.

이 책은 스프링 부트 참고문서를 기반으로 하여 그에 대응하는 설명과 나의 '삽질경험' 을 기술한다.

NOTE

이 책을 쓰면서 전에는 사용하지 않았던 혹은 살펴보지 않았던 기능들을 살펴볼 수 있는 좋은 기회가 됐다. "잘 알아서 책을 쓰는 게 아니라 책을 쓰면서 알아간다"라는 누군가의 이야기에 깊은 감명을 받았다.

책에서 다루는 내용

"['스프링 부트' 시작하기](#)"에서는 '스프링 부트'를 소개하면서 스프링 부트를 기반으로 하는 기본적인 개발흐름을 눈에 익힌다.

'스프링 부트'는 스프링 기반의 웹 애플리케이션을 개발하던 때와는 확연하게 다른 경험을 개발자들에게 선사한다. 이는 다른 언어가 가지고 있던 장점들을 자바 개발자들에게 제공하며 '프로토타이핑'-'개발'-'운영' 까지 이어지는 근사한 경험을 선물한다. 거기에는 **XML 기반 으로 애플리케이션에 필요한 기능에 대해서 문맥(Context)으로 정의하는 흐름이 자바코드로 구성(Configuration) 하는 개념(JavaConfig)**으로 변모하는 역동적인 변화가 발생한다.

"['스프링 부트' 살펴보기](#)"에서는 '스프링 부트' 를 본격적으로 살펴본다.

필요한 기능을 가진 라이브러리를 손쉽게 추가할 수 있는 '**의존성 관리**', 추가된 '의존성' 라이브러리들을 바로 사용할 수 있도록 스프링 부트 개발팀이 권장하는 '**자동구성(auto-config)**', 개발 생산성을 높여주고 확장성을 부여하는 '**스프링 프레임워크와의 조합**', 내장컨테이너를 이용한 빠른 실행과 재가동 시간을 줄이는 '**개발도구**' 등 스프링 부트 를 이용하는데 알아둬야할 것들을 살펴본다.

"['스프링 부트' 기능소개](#)"에서는 '스프링 부트'를 이용하게 만드는 스프링 부트 기능을 살펴본다.

스프링 부트 애플리케이션 진입점(Entry point)이 되는 **SpringApplication**을 시작으로 해서, 외부 구성 (Externalized Configuration) 기능을 이용해서 환경에 따라 애플리케이션 속성을 정의하는 편의성, 스프링에서 제공하는 프로파일 기능을 이용해서 배포운영 환경에 따라 애플리케이션 구성 분리하고, 다양한 생태계와 연동하고, 필요에 따라 추가적인 구성을 작성하는 과정 등을 살펴보게 된다. 이 장을 읽으면서 '**스프링 부트**'의 매력에 흠뻑 취하게 될 것이다.

"['스프링 부트' 출시준비: 액추에이터](#)"에서는 '빠른 개발' 이후에 고민하게 되는 '운영' 의 묘미를 살려줄 수 있는 다양한 기능을 살펴본다.

개발하고 배포하여 운영중인 스프링 부트 애플리케이션을 모니터링하고 그 정보를 수집하고 경우에 따라 쉽게 제어할 수 있는 기능들을 소개한다.

"['스프링 부트' 배포](#)"에서는 스프링 부트 로 개발된 웹 애플리케이션을 배포하는 방법을 살펴본다.

편리하고 막강한 기능을 제공하는 클라우드 서비스들이 많이 등장했다. 그 클라우드 서비스에 우리가 만든 앱을 배포하는

과정을 살펴보면서 자신에게 맞는 클라우드 서비스를 선택하고 관리해보자.

"**'스프링 부트' 빌드도구 플러그인**"에서는 '실행가능한(Executable)' 배포본(jar or war)를 만들어내는 '빌드 도구' 를 살펴보도록 한다.

'스프링 부트'는 자바 개발시 주로 사용하는 빌드도구인 메이븐과 그레이들 에서 사용할 수 있는 **빌드 도구 플러그인**을 제공하고 있다. 이 빌드 도구 플러그인을 이용하여 애플리케이션을 빌드할 때 다양한 선택사항(고려할 수 있는)들을 제공하여 개발자의 다양한 욕구를 수용하고 있다. 여기서 '**실행가능한(Executable)**' 배포본을 만들어내는 교묘한 기술 '**재포장(리패키징, Repackage)**' 에 대해 설명한다.

"**'스프링 부트' 활용 안내**"에서는 '스프링 부트' 로 개발하는 과정에서 고려해볼 다양한 확장 포인트를 살펴본다.

'스프링 부트'는 스프링 부트 개발팀에서 권장(Recommend) 하는 방식이 있다. 그렇지만 스프링 부트 를 사용하는 개발자가 확장할 수 있는 다양한 포인트를 제공한다. 이는 스프링 부트 의 근간이 되는 스프링 프레임워크가 가지고 있는 특징이기도 하다. '다 써볼 수 있을까?' 싶을만큼 너무나 다양하고 방대한 내용을 다루고 있다.

"**정리**"은 책의 내용을 정리한다.

"**'자바 개발자+스프링 프레임워크 사용자'** 라면 스프링 부트 를 능숙하게 다룰 수 있는 게 좋다" 그러니까 "스프링 부트를 살펴보자" 라는 게 결론이다.

책을 따라하기 위해 필요한 것

스프링 부트 [참고문서#시스템 요구사항](https://goo.gl/byHFzg)(<https://goo.gl/byHFzg>) 을 보면 2.0.0.RELEASE 은 **Java 8** (<https://docs.oracle.com/javase/8/>) 과 5.0.1.RELEASE 혹은 상위버전을 요구한다. Java 6도 지원을 하지만 사용할 **컨테이너의 종류**(<https://goo.gl/T54pv4>)에 따라서 제한되거나 영향을 받을 수 있다. 책에서 작성한 코드들 중에 일부는 Java 8에서 제공하는 **Optional, Stream** 등을 사용하기에 Java 8 이상을 사용하길 **권장**(recommendation)한다.

IDE 는 **STS(Spring Tool Suite, <https://spring.io/tools/sts>)** 와 **IntelliJ(<https://www.jetbrains.com/idea/>)** 등이 있는데 이 책은 **STS** 를 기준으로 한다.

NOTE

스프링 부트에 대한 **기능**(대시보드, 애플리케이션 Properties 자동완성 지원 등) 지원이 괜찮기도 하고 **이클립스** 기반이라 많은 개발자들에게 익숙하고 **무료**로 사용할 수 있다는 강점(?)이 있어 부담없이 사용할 수 있다.

대상독자

스프링 부트를 기반으로 한 개발에 관심을 가지고 이제 막 예제를 만들어보려 하거나 스프링 프레임워크에 대한 기본적인 지식이 있는 개발자를 대상으로 한다.

스프링 부트를 제대로 활용하기 위해서는 스프링 프레임워크와 그에 기반되는 지식을 가지고 있는 것이 좋다. 스프링 부트는 'spring + spring-boot + auto-config + build-tool + 3rd party' 의 조합이기 때문에 자바 프로그래밍을 처음 시작하는 이들에게는 조금은 부담스러울 수 있다. 스프링 부트를 제대로 사용하려면 스프링에 대한 공부도 병행해야 한다.

NOTE

"자바는 필요한 기능을 가진 라이브러리를 이용하려면 공부해야할 것이 많다." 라고 비판하던 어느 형님의 이야기가 떠오른다. 난 그게 당연하다고 생각했지만 누군가는 그것이 단점이라고 이야기한다. 이에 대해서도 잠시 생각을 해보자.

관례(Convention)

코드(code)는 다음과 같은 형식으로 작성된다.

```
public class Honeymon {
    private String firstName;
    private String lastName;
    private Gender gender;
    private Integer age;
    private String site;
    private Integer weight;

    public Fat eat(Food food) {
        this.weight += food.getWeight();
    }
}
```

코드 중에서 중요한 부분에 대해서는 다음과 같이 코드 오른쪽에 숫자가 매겨지고 하단에 그 숫자에 설명이 기술된다.

```
public class Honeymon {
    private String firstName;
    private String lastName;
    private Gender gender;
    private Integer age;
    private String site;
    private Integer weight;

    public Fat eat(Food food) {
        this.weight += food.getWeight(); ①
    }
}
```

① 먹은 만큼 찐다! 만고불변의 법칙!

문장 중간에 노출되는 클래스명과 같은 코드성 표현에 대해서는 **Honeymon**으로 표현한다.

중요한 단어나 문장에 대해서는 **굵은 글씨** 처리한다. **스프링 부트**는 스프링 프레임워크를 기반으로 하는 개발플랫폼이다.

중요해서 기억해두었으면 하는 부분에 대해서는 다음과 같이 **노트**로 표기한다.

NOTE | 노트

알아두면 유용한 것에 대해서는 다음과 같이 **유용함**으로 표기한다.

TIP | 유용함!

인용한 것에 대해서는 **인용**으로 표기한다.

인용

용어나 개념을 소개하는 경우에는 한국어로 표현하는 것을 선호하는 편이다. 첫 표기시에는 **자동구성(Configuration)**과 **설정(Set-up)**과 같이 한국어와 영어를 병행표기하고 이후 표기에서는 한국어만 표기한다. 적절한 한국어 표현을 찾기 어려운 경우에는 **액추에이터(Actuator)** 처럼 외래어 표기법을 따른다.

이 표기에 대해서는 의견이 분분하다. 어떤 이들은 영어를 그대로 표기하는 게 좋다하는 사람도 있지만, 나는 억지스러울지도 모르겠지만 한국어로 표기하고자 하는 노력을 기울이고 있다. 그렇다고 해서 나만 이해할 수 있는 엉터리 표기는 피하려고 노력한다. 이에 대해서는 각자의 방식으로 접근하고 가장 효율적인 방법을 선택하는 것이 적합하다고 본다.

예제

이 책에서 사용된 예제는 <https://github.com/ihoneymon/boot-spring-boot> 에서 살펴볼 수 있다. 메이븐이나 그레이들없이 실행할 수 있도록 래퍼(wrapper)가 구성되어 있지만 이 래퍼를 실행하기 위해서는 JDK(Java Development Kit)이 설치되어있어야 한다. 프로젝트는 Java 8 을 기준으로 작성됐다. 필요하다면 {repo}를 복제(clone) 해서 Java 7 에서도 실행가능하게 구성하고 공유해주면 다른 개발자들에게 도움이 될 것이다.

Chapter 1. '스프링 부트' 시작하기

'스프링 부트를 이용한 기본적인 개발흐름'을 살펴본다. 스프링 부트는 개발자들에게 스프링 기반의 웹 애플리케이션을 개발하던 때와는 확연하게 다른 개발경험을 선사해준다. 다른 언어가 제공했던 '빠른 프로토타이핑' 및 '개발/운영' 까지 이어지도록 해주는 근사한 경험을 자바 개발자들에게도 제공한다. 그 안에는 개발자가 애플리케이션에 필요한 구조를 XML로 문맥(**Context**)을 작성하던 방식에서 애플리케이션에 필요한 컴포넌트와 설정을 조합하는 Java로 구성(**Configuration**) 하는 방식으로 역동적인 변화가 있었다.

스프링 부트가 제공하는 신세계를 경험하러 가보자.

1.1. 스프링 부트 소개

스프링 부트는 '개발 플랫폼'이다. 스프링 프레임워크를 기반으로 자바 개발환경에서 유행가는 기능들을 쉽게 추가하고 관례적인 자동구성을 제공하고 '프로토타이핑'-'개발'-'운영/모니터링' 까지 이어지는 다양한 기능을 지원한다.

스프링 부트 프로젝트 페이지(<http://projects.spring.io/spring-boot>)에 작성되어 있는 **스프링 부트 기능 정의**를 살펴보면 스프링 부트 의 **특징**을 이해할 수 있다.

- 단독실행 가능한 스프링 애플리케이션 생성
- 내장 컨테이너로 톰캣, 제티 혹은 언더토우 중에서 선택가능(WAR 배포 필요없음)
- 스타터('starter') POM 을 제공하여 간결한 의존성 구성 지원
- 스프링에 대한 자동구성 제공
- 제품출시 후 운영에 필요한 상태, 건강 점검과 외부구성 등 다양한 기능 제공
- 구성을 위한 XML 코드를 필요로 하지 않으며 작성하지 않아도 됨

위 여섯가지 기능정의에 대해 간단하게 설명하자면 다음과 같다:

단독실행 가능한 스프링 애플리케이션 생성

순수형태의 자바 애플리케이션을 작성하고 WAS 에 배포하는 일련작업없이도 내장 컨테이너를 이용하여 애플리케이션을 실행할 수 있다. 이 과정은 스프링 애플리케이션을 빠르고 쉽게 생성 및 배포할 수 있게 해준다. 별도로 WAS를 설치하고 설정하는 번거로움을 줄여주고, 최소단위로 애플리케이션을 설치하고 확장하는 것이 가능해진다.

내장 컨테이너로 톰캣, 제티 혹은 언더토우 중에서 선택가능(WAR 배포 필요없음)

스프링 부트 의 **spring-boot-starter-web** 는 기본적으로는 톰캣을 포함하지만, 개발자가 원한다면 언제든지 **톰캣**을 제거하고 **제티** 혹은 **언더토우**로 대체할 수 있다. 대체하는 방법은 '**톰캣을 언더토우로 대체하기**'에서 자세히 다룰 예정이다.

스타터('starter') POM 을 제공하여 간결한 의존성 구성 지원

메이븐을 이용해서 관련되거나 필요한 기타 의존성을 함께 선언할 수 있다. 스프링 부트 프로젝트에 있는 **pom.xml** 혹은 **build.gradle** 을 열었을 때 볼 수 있는 의존성(**dependency**) 부분에 **spring-boot-starter-*** 들은 각각의

컴포넌트가 실행되는데 필요한 의존성들을 내포하고 있다. 이에 대해서는 '[스타터\(Starter\)](#)'에서 다룰 예정이다.

스프링에 대한 자동구성 제공

`@EnableAutoConfiguration` 애너테이션을 이용해서 각 컴포넌트별 기본적인 자동구성을 지원한다. 이에 대해서는 '[자동구성\(auto-configuration\)](#)'에서 자세히 살펴보도록 하자.

제품출시 후 운영에 필요한 상태, 건강 점검과 외부구성 등 다양한 기능 제공

개발을 마친 애플리케이션을 배포한 후에는 애플리케이션에서 발생하는 오류 뿐만 아니라 애플리케이션이 동작하면서 사용하는 자원의 상태 및 살아있는지 여부를 확인할 수 있는 방법이 필요하다. 이런 기능을 별도로 개발하지 않아도 될만큼 다양한 기능을 제공해준다. 이를 활용하는 방법에 대해서는 '[스프링 부트](#)' [출시준비: 액추에이터](#)에서 상세히 다룬다.

구성을 위한 XML 코드를 필요로 하지 않으며 작성하지 않아도 됨

스프링 부트에서 가장 큰 변화 중 하나는 더이상 **XML 구성**을 필요로 하지 않는다는 것이다. 자바구성(**JavaConfig**)을 통해서 애플리케이션에 필요한 컴포넌트 스캔(**ComponentScan**)과 스프링 빈(**Bean**) 등록 등을 할 수 있게 됐다. 이마저도 관례적인 자동구성(**auto-config**)을 제공하여 애플리케이션 구성에 대한 부담감을 줄여준다. 스프링 부트를 기반으로 개발하는 개발자는 환경구성에 대한 부담없이 비즈니스 로직구현에만 집중할 수 있도록 해준다. 물론 `@Configuration`가 선언된 클래스에서 `@ImportResource` 애너테이션을 이용해서 패키지에 포함된 **XML** 파일을 읽어올 수 있는 기능을 제공한다.

NOTE

스프링 부트를 사용하기로 결정했으면, XML설정은 떨쳐내고 자바구성을 사용해보자. 익숙해지면 쉽다.

스프링 부트의 여섯가지 기능들이 어울어져 자바개발자에게는 정말 좋은 도구가 됐다. 이 도구를 잘 익혀두면 자신이 만들고자 하는 애플리케이션에서 **비즈니스 로직(해결하고자 하는 문제, 달성하고자 하는 목표)**에만 집중하면 **스프링 부트**가 나머지 부분(필요한 기능을 가진 라이브러리를 추가하고 기능을 구성하고 구현하고 배포)을 처리해준다.

1.2. 시스템 요구사항

스프링 부트2.0.0.RELEASE는 기본적으로 **Java 8**과 **5.0.1.RELEASE** 혹은 **상위버전을 요구한다**. 스프링 부트 버전에 따라 적재된 스프링 프레임워크 버전이 다르니 사용된 버전을 확인하기 바란다. 확인하는 방법은 '[시스템 요구사항](#)' [항목](#) (<https://goo.gl/5Gs4k2>)을 살펴보면 된다. 스프링 부트 버전에 따라 이용하는 스프링 프레임워크 버전이 달라지므로 이에 고려하여 **애플리케이션의 기능을 설계하고 구현하기를 권한다**. 필요하다면 Java 버전과 스프링버전을 변경할 수는 있지만 그에 따라 별도로 구성을 작성해야한다. 빌드도구 그레이들 4.x+과 메이븐 3.2+ 이상을 사용해야 제대로 이용가능하다.

스프링 부트에서 사용하는 모듈버전은 **spring-boot-dependencies**에 있는 **pom.xml**을 살펴보면 된다. **2.0.0.RELEASE**에 들어간 모듈 및 라이브러리의 버전을 알고 싶다면 살펴보려는 버전을 탐색하여 **Project Object Model(POM)** 을 읽어보면 된다.

The Central Repository

SEARCH | ADVANCED SEARCH | BROWSE | QUICK STATS

[About Central](#) [Advanced Search](#) [API Guide](#) [Help](#)

SEARCH



[All Day DevOps - Register Now!](#)

[org.springframework.boot : spring-boot-dependencies : 1.5.8.RELEASE](#)

Click on a link above to browse the repository.

TIP

Project Information

GroupId: org.springframework.boot
ArtifactId: spring-boot-dependencies
Version: 1.5.8.RELEASE

Dependency Information

Apache Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-dependencies</artifactId>
  <version>1.5.8.RELEASE</version>
</dependency>
```

Apache Buildr

'org.springframework.boot:spring-boot-dependencies:ja

Apache Ivy

```
<dependency org="org.springframework.boot" name="
```

Project Object Model (POM)

```
</developer>
</developers>
<properties>
  <!-- Dependency versions -->
  <activemq.version>5.14.5</activemq.version>
  <antlr2.version>2.7.7</antlr2.version>
  <appengine-sdk.version>1.9.58</appengine-sdk.version>
  <artemis.version>1.5.5</artemis.version>
  <aspectj.version>1.8.11</aspectj.version>
  <assertj.version>2.6.0</assertj.version>
  <atomikos.version>3.9.3</atomikos.version>
  <bitronix.version>2.1.4</bitronix.version>
  <caffeine.version>2.3.5</caffeine.version>
  <cassandra-driver.version>3.1.4</cassandra-driver.version>
  <classmate.version>1.3.4</classmate.version>
  <commons-beanutils.version>1.9.3</commons-beanutils.version>
  <commons-collections.version>3.2.2</commons-collections.version>
  <commons-codec.version>1.10</commons-codec.version>
  <commons-dbc.version>1.4</commons-dbc.version>
  <commons-dbc2.version>2.1.1</commons-dbc2.version>
  <commons-digester.version>2.1</commons-digester.version>
  <commons-pool.version>1.6</commons-pool.version>
  <commons-pool2.version>2.4.2</commons-pool2.version>
  <couchbase-client.version>2.3.7</couchbase-client.version>
  <couchbase-cache-client.version>2.1.0</couchbase-cache-client.version>
  <crashub.version>1.3.2</crashub.version>
```

가능하면 스프링 부트 개발팀에서는 권장하는 **Java 8 이상**(<https://goo.gl/YCPntK>)을 사용하기 바란다. 스프링 프레임워크는 4.X 부터 "Java 8"을 완벽하게 지원했다. 스트림(Stream) API와 람다식의 혜택을 누릴 수 있도록 "Java 8" 사용을 권한다. "스프링 프레임워크 5"부터는 "Java 9"를 지원한다(<https://goo.gl/iv4BY5>).

NOTE

- Java 8 새로운 점
- 씹고 뜯고 맛보고 즐기는 '스트림API'(<https://goo.gl/3b2M5D>)
- 나만 모르고 있던 - Java 9 (Java9 빠르게 훑어 보기)(<https://goo.gl/uZMUem>)
- Java 9 - 권남님 위키(<http://kwonnam.pe.kr/wiki/java/9>)

애플리케이션에 필요한 기능을 제공하는 라이브러리 의존성을 관리하려면 빌드도구를 사용하는 것이 여러모로 편리하다. 빌드도구는 라이브러리 의존성 관리 외에도 애플리케이션 패키징, 빌드전후처리 등의 작업을 지원한다. 스프링 부트는 자바 개발환경의 대표적인 빌드도구인 그레이들과 메이븐에서 이용할 수 있는 플러그인을 제공한다.

NOTE

이 책에서는 **그레이들을 기준으로** 설명한다. 그레이들 래퍼(wrapper)를 이용하게 되면 다른 팀원들은 로컬개발환경에 그레이들을 설치하지 않아도 프로젝트 빌드가 가능해진다. 이에 대해서는 "**빌드 시스템**"에서 자세히 설명할 예정이다.

TIP

메이븐도 래퍼가 존재한다. 사용하는 방식은 그레이들과 유사하다. 이에 대한 정보는 [Maven Wrapper \(https://github.com/takari/maven-wrapper\)](https://github.com/takari/maven-wrapper) 와 [Maven Wrapper 소개와 사용 - 윤병화, pop-it \(https://goo.gl/F27iwq\)](https://goo.gl/F27iwq)에서 볼 수 있다.

스프링 부트에서 Java 사용버전을 고려하는데 가장 큰 영향을 주는 것은 **사용하는 컨테이너가 무엇이나 하는 것이다**. 컨테이너 스펙에 따라서 요구하는 Java 버전이 다르기 때문에 컨테이너를 우선적으로 고려한다. 컨테이너의 버전에 따라 지원하는 서블릿 스펙이 달라진다. 그러면 컨테이너에서 사용할 수 있는 기능이 제한될 수 있다. 서블릿 3.0의 경우에는 Java 6 까지 지원하며, 서블릿 3.1은 Java 7 부터 지원한다. 보다 자세한 내용은 다음과 같다.

1. 내장 서블릿 컨테이너별 지원사양

Name	Servlet Version	Java Version
Tomcat 8.x+	3.1	Java 7+
Tomcat 7	3.0	Java 6+
Jetty 9.3+	3.1	Java 8+
Jetty 9.2	3.1	Java 7+
Jetty 8	3.0	Java 6+
Undertow 1.3	3.1	Java 7+

NOTE

이 외에도 스프링 부트는 서블릿 3.0+ 이상을 지원하는 컨테이너라면 어디에든 배포가능하다.

- 참고: [Servlet container\(https://goo.gl/T54pv4\)](https://goo.gl/T54pv4)

NOTE

웹소켓 등 컨테이너에 따라 의존성을 가지고 있는 라이브러리 등이 영향을 받기 때문에 컨테이너를 선정할 때는 이런 영향도를 고려해야 한다. 간단한 예로 웹소켓([JSR-356](#))은 컨테이너에 따라 구현체가 각기 다르기에 컨테이너 변경시 많은 영향을 받을 수 있다.

- [Tomcat WebSocket\(org.apache.tomcat.websocket\)](#)
- [Jetty WebSocket\(org.eclipse.jetty.websocket\)](#)

1.3. 스프링 부트 설치

스프링 부트를 이용하기 위해서는 JDK(Java Development Kit)가 설치되어 있어야 한다. 라이브러리 의존성 관리를 위해서는 빌드도구가 필요한데, 빌드도구 래퍼를 이용하면 프로젝트 구성자 이외에 그레이들이나 메이븐을 로컬개발환경에 설치할 필요가 없다. 'SPRING INITIALIZR(<http://start.spring.io/>)'로 프로젝트를 생성하면서 빌드도구를 선택하면 그에 맞춰 래퍼를 포함하고 있는 스프링 부트 프로젝트가 생성된다. 래퍼에 대한 설명은 "[래퍼\(Wrapper\)](#)"를 살펴보기 바란다.

스프링 부트가 나온 이후 대부분의 IDE에서는 다양한 스프링 부트 편의기능을 제공한다. STS와 인텔리제이 경우 스프링 부트 대시보드를 제공하여 시작, 디버깅과 원격지 배포실행과 애플리케이션 공통속성 자동완성 등의 기능을 제공하고 있다.

NOTE

많은 책에서 스프링 부트 CommandLine Client(CLI)와 관련된 설치, 실행방법과 프로젝트 생성 방식을 다루지만 이 책에서는 다루지 않는다. **프로젝트 실행하는데 스프링 부트 CommandLine Client를 써본 적이 없다**는 개인적인 경험(—人-;; 나만 그런가?)을 기준으로 한다. 그래도 익혀볼 의사가 있다면 [스프링 부트 참고문서#CLI \(https://goo.gl/HQENXT\)](https://goo.gl/HQENXT)를 살펴보기 바란다.

NOTE

글로 설명하기에는 쓸데없이 화면캡처만 늘어날만한 IDE에서 스프링 부트 프로젝트 생성 등과 관련된 설명은 영상으로 설명하겠다.

- 'Boot Spring Boot' 관련 영상(<https://goo.gl/XAc9P8>)

1.4. 스프링 부트 애플리케이션 개발 시작

스프링 부트 애플리케이션을 개발하기 위해서는 플러그인과 의존성을 포함한 프로젝트를 생성해야 한다.

1.4.1. 프로젝트 생성

STS(Spring Tool Suite, <https://spring.io/tools/sts>) 와 IntelliJ(<https://www.jetbrains.com/idea/>) 에서 프로젝트를 생성한다. 이후에는 STS 를 기준으로 진행한다(무료이고 이클립스 기반이고 스프링 부트 대시보드 등의 지원기능이 꽤 쓸만하다).

IDE에서 스프링 부트 프로젝트를 생성하는 방식은 'Spring Initializer' 사이트에서 생성한 **zip** 파일을 내려받아 이를 지정된 위치에 압축해제한 후에 불러내기(import) 한다.

NOTE

'Spring Initializer' 에서 프로젝트 초기파일을 내려받는 방식이기 때문에 인터넷 연결이 되지 않는 경우에는 스프링 부트 프로젝트를 생성하기 어렵다. 이를 해결하기 위한 방법으로 인트라넷에 'Spring initializer' 프로젝트를 가동해야 한다. 여기에 오픈소스의 위대함이 깃드는데, [Spring Initializr \(https://github.com/spring-io/initializr/\)](https://github.com/spring-io/initializr/) 프로젝트에서 소스를 내려받아 인트라넷에 구성할 수 있다.

STS(or Eclipse): 스프링 부트 프로젝트 생성

- 영상: [STS에서 스프링부트 프로젝트 생성하기\(https://youtu.be/ad03h6usqYc\)](https://youtu.be/ad03h6usqYc)
 1. 'File' - 'New' - 'Spring Starter Project' 선택
image::[Spring Starter Project 생성하기]
 2. 프로젝트 정보를 다음과 같이 입력하고 '[Next >]' 버튼 클릭 image::[프로젝트 정보 입력]
- 'Type' 선택
 - 'Maven'
 - 'Gradle(Buildship)'
 - 'Gradle(STS)' ← 선택
- 'Packaging' 선택
 - 'Jar': WAS 에 배포하지 않고 커맨드로 실행시 선택
 - 'War': WAS 에 배포하는 경우 선택

- 'Java' 선택
 - '1.8' ← 권장
 - '1.7'
 - '1.6'
- 'Language' 선택
 - 'Java'
 - 'Groovy'
 - 'Kotlin'
- group: `io.honeymon.boot`
- artifact: `boot-spring-boot`
- package: `io.honeymon.boot.springboot`
 1. 스프링 부트 버전 및 의존성을 선택하고 '[Finish]' 를 누른다. image::[선택항목]
- 선택항목: `web`, `jpa`, `h2`, `lombok`, `thymeleaf`
 - `web`
 - `jpa`
 - `h2`
 - `lombok` ← `@Getter`, `@Setter` 애너테이션을 통해 Getter/Setter 메서드를 손쉽게 생성가능
- '[Make Default]' 버튼을 누르면 'Frequently Used' 항목에 추가된다.
- '[Next >]'를 누르는 경우 다음과 같은 정보를 확인할 수 있다. image::images/01-04-spring-initializer.png
[Spring Boot initializer Site]

```
http://start.spring.io/starter.zip?name=boot-
demo&groupId=io.honeymon&artifactId=bootdemo&version=0.0.1-
SNAPSHOT&description=Demo+project+for+Spring+Boot&packageName=io.honeymon.
bootdemo&type=gradle-
project&packaging=jar&javaVersion=1.8&language=java&bootVersion=2.0.0.R
ELEASE&dependencies=lombok&dependencies=data-
jpa&dependencies=h2&dependencies=web
```

해당 창에 있는 'Full URL' 항목을 브라우저 주소창에 복사해서 붙여넣으면 zip 파일이 'artifact' 에 입력값을 파일명으로 가지는 `bootdemo.zip` 와 같은 파일이 다운로드 된다. 이 파일을 특정위치로 옮겨 압축해제하고 IDE에서 불러오는 것도 가능하다.

IntelliJ: 스프링 부트 프로젝트 생성

- 영상: [인텔리제이에서 스프링부트 프로젝트 생성하기\(https://goo.gl/Ng5gMF\)](https://goo.gl/Ng5gMF)

1. 'File' → 'New' 선택
2. 'Spring Initializer' 선택
 - a. SDK 선택
 - b. [Next] 선택
3. 'New Project' 창에서 프로젝트 기본정보 등록 → [Next]
4. 사용될 '의존성'(Dependencies) 선택 → [Next]
5. 프로젝트 설치 위치 지정 → [Finish]

1.4.2. 의존성 추가

프로젝트에서 사용하는 의존성 라이브러리가 선언된 부분이다.

build.gradle

```
dependencies {  
    compile('org.springframework.boot:spring-boot-starter-data-jpa')  
    compile('org.springframework.boot:spring-boot-starter-web')  
  
    runtime('com.h2database:h2')  
  
    compileOnly('org.projectlombok:lombok')  
  
    testCompile('org.springframework.boot:spring-boot-starter-test')  
}
```

그레이들에서는 **build.gradle** 파일 내에 **dependencies** 영역 안에 의존성을 선언한다.

NOTE

"Dependency Management Basics"(<https://goo.gl/iLaf7v>) 를 살펴보면 **compile**, **testCompile** 등의 정보를 확인할 수 있다.

1.4.3. 예제: **HelloController** 작성

"코드 구조" 에서 설명하겠지만, 애플리케이션 메인 클래스를 비롯한 새롭게 생성하는 클래스들은 기본 **default** 패키지에 두지 않기를 권한다. 지금 작성하는 코드는 어디까지나 간단하게 코드작성법을 설명하기 위한 예제일 뿐이다.

앞에서 보여준 과정을 따라 했다면 **BootSpringBootApplication** 혹은 변경한 프로젝트 명을 접두사로 가지는 **~Application** 클래스가 자동생성되어 있다.

BootSpringBootApplication.java

```
package io.honeymon.boot.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BootSpringBootApplication {

    public static void main(String[] args) {
        SpringApplication.run(BootSpringBootApplication.class, args);
    }
}
```

NOTE

`BootSpringBootApplication` 클래스는 수정할 일이 별로 없다. 여기에 스프링 부트가 부린 마술이 담겨있다. 이 마술에 대해서는 "[@SpringBootApplication 애너테이션 이용](#)"에서 설명한다.

이 클래스 위치(`io.honeymon.boot.springboot`)에서 `api` 패키지(`io.honeymon.boot.springboot.api`)를 생성 한다. 그 후 `HelloController` 이란 이름으로 클래스를 생성하자. 이 클래스는 이후에 여러분이 자주 작성하게될 API 클래스의 기본구조를 가진다.

```
package io.honeymon.boot.springboot.api;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController ①
public class HelloController {

    @GetMapping("/greeting") ②
    public String greeting() {
        return "Hello, Boot Spring Boot!!";
    }
}
```

- ① `HelloController` 클래스에서 제일 먼저 선언된 `@RestController` 은 스프링 4.0 때 추가된 메타 애너테이션이다. `@Controller` 와 `@ResponseBody` 애너테이션 을 포함하고 있다.
- ② 스프링 4.3 에 추가된 메타 애너테이션 으로 `@RequestMapping(method = RequestMethod.GET)` 을 짧게 쓸 수 있다. '그렇다면? 혹시...' 하고 생각하는데로 {enum} `RequestMethod` 에 선언되어 있는 `GET`, `HEAD`, `POST`, `PUT`, `PATCH`, `DELETE`, `OPTIONS`, `TRACE` 에 대응하는 ~Mapping 들이 존재한다.

1.4.4. 예제: HelloController 실행하기

이제 예제를 실행해보자. `BootSpringBootApplication` 를 우클릭하고 팝업메뉴에서 'Run as' → 'Spring Boot App' 을 선택한다.

'Spring Boot Dashboard' 의 항목이 변경된다. 콘솔(Console) 창에 스프링부트가 실행되는 과정에서 로그가 출력된다.

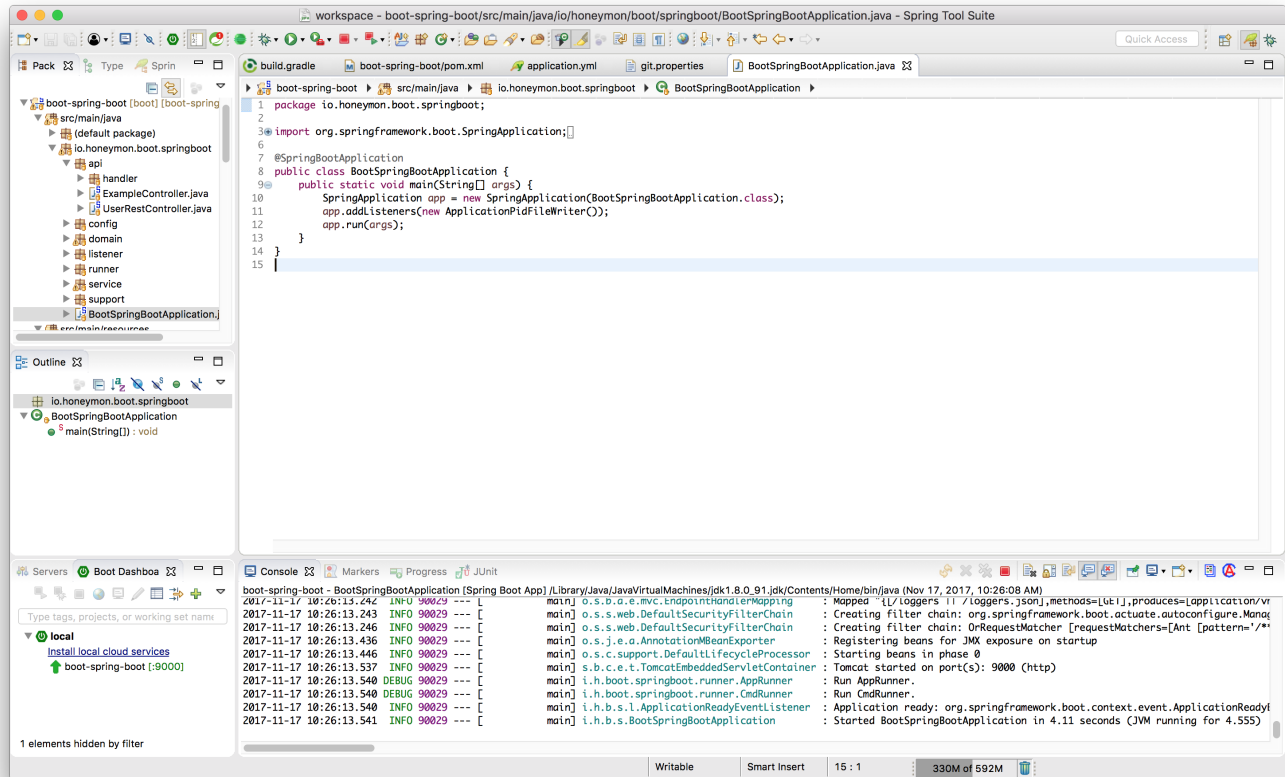
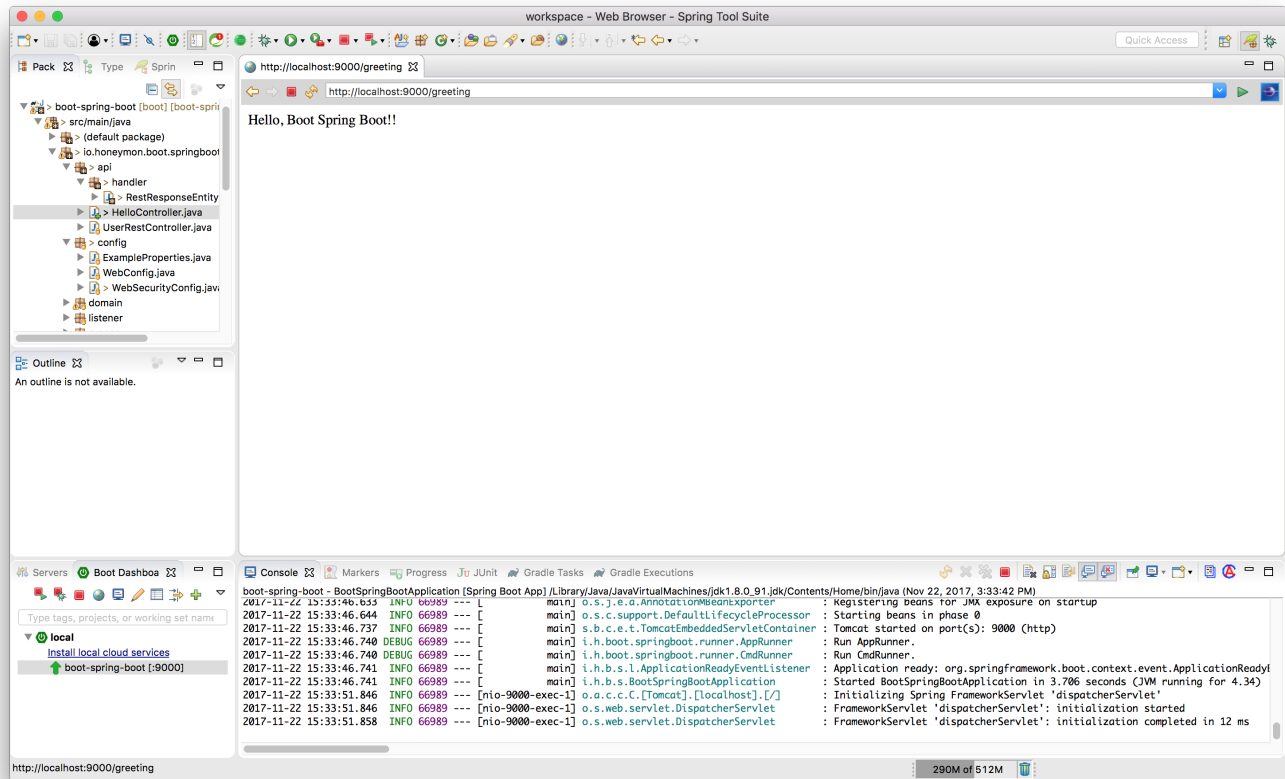


Figure 1. BootSpringBootApplication 실행모습

브라우저에서 <http://localhost:8080/greeting> 으로 접근해보면 "Hello, Boot Spring Boot!!" 가 출력되는 것을 볼 수 있다.



애플리케이션을 중지시키려면 콘솔이나 IDE의 정지버튼을 누르면 된다.

1.4.5. 실행가능한 jar 만들기

예제의 마지막 단계로 내장 컨테이너(Embedded Container)를 포함한, 실제로 운영에서도 사용가능한, 실행가능한 jar를 생성해보자. 실행가능한 jar('Fat Jars'라고도 불림)는 우리가 작성한 코드를 컴파일한 클래스와 이를 실행하는데 필요한 모든 의존성 jar가 함께 압축되어 있다(여기에는 톰캣과 같은 WAS도 내장 컨테이너로 포함되는데 여기에는 스프링 부트 빌드도구 플러그인의 **bootRepackage**가 관여한다).

실행가능한 jar를 생성하기 위해서는 **build.gradle**에 **spring-boot-gradle-plugin**가 추가되어야 한다. **build.gradle** 첫번째 줄에 다음 코드를 추가하자.

```

buildscript {
    ext {
        springBootVersion = '2.0.0.RELEASE'
    }
    repositories {
        mavenCentral()
    }
    dependencies {
        classpath("org.springframework.boot:spring-boot-gradle-plugin:
${springBootVersion}")
    }
}

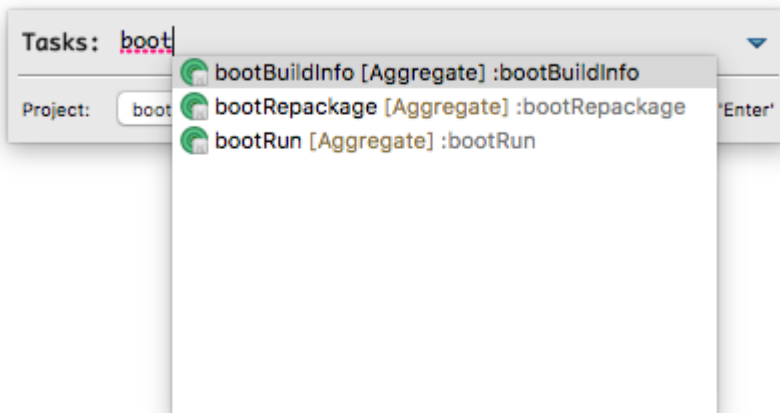
```

① **buildscript** 선언을 통해서 그레이들 빌드시 **spring-boot-gradle-plugin** 을 이용할 수 있게 된다.

NOTE

스프링 부트 플러그인을 설치하면 **bootRepackage**, **bootRun** 태스크(task)를 실행할 수 있게 된다. 이 태스크들에 대해서는 "**스프링 부트 그레이들 플러그인**" 에서 설명한다. 그레이들의 **buildscript** 에 대한 설명은 [Gradle#Using your plugin in another project\(https://goo.gl/vdE77Q\)](https://goo.gl/vdE77Q) 을 읽어보길 바란다.

프로젝트를 선택하고 우클릭하여 'Run AS' → 'Gradle' → 'Task Quick Launcher' 를 클릭한다. 팝업된 실행창에서 **bootRepackage** 를 입력하자. 자동완성 기능을 제공하기 때문에 **boot** 까지 입력하면 **bootRun** 과 **bootRepackage** 가 나열될 것이다.



```

[sts] -----
[sts] Starting Gradle build for the following tasks:
[sts]     bootRepackage
[sts] -----
:compileJava
:processResources UP-TO-DATE
:classes
:findMainClass
:jar
:bootRepackage

BUILD SUCCESSFUL

Total time: 1.31 secs
[sts] -----
[sts] Build finished successfully!
[sts] Time taken: 0 min, 1 sec
[sts] -----

```

위와 같은 내용이 콘솔에 출력될 것이다. `boot-spring-boot` 프로젝트에서 `build/libs` 폴더를 보면 `boot-spring-boot-1.0.0.RELEASE.jar` 와 `boot-spring-boot-1.0.0.RELEASE.jar.original` 이 생성되어 있다. `boot-spring-boot-1.0.0.RELEASE.jar.original` 는 우리가 작성한 코드가 컴파일되어 있다. 이를 스프링 부트 플러그인에서 실행에 필요한 컨테이너와 의존성을 추가하여 재압축(repackage) 한 것이 `boot-spring-boot-1.0.0.RELEASE.jar` 으로 대략 10MB 정도의 용량이 늘어나게 된다. 이에 대해서는 "[스프링 부트 빌드 도구 플러그인](#)" 에서 자세히 설명한다.

`jar tvf` 명령으로 `boot-spring-boot-1.0.0.RELEASE.jar`에 포함되어 있는 것을 확인할 수 있다.

```

$ jar tvf build/libs/boot-spring-boot-1.0.0.RELEASE.jar
  0 Wed Nov 22 15:50:40 KST 2017 META-INF/
 258 Wed Nov 22 15:50:40 KST 2017 META-INF/MANIFEST.MF
  0 Wed Nov 22 15:50:40 KST 2017 BOOT-INF/
  0 Wed Nov 22 15:50:40 KST 2017 BOOT-INF/classes/
 306 Wed Nov 22 15:50:38 KST 2017 BOOT-INF/classes/AvoidDefaultPackagePosition.class
  0 Wed Nov 22 15:50:38 KST 2017 BOOT-INF/classes/io/
  0 Wed Nov 22 15:50:38 KST 2017 BOOT-INF/classes/io/honeymon/
  0 Wed Nov 22 15:50:38 KST 2017 BOOT-INF/classes/io/honeymon/boot/
  0 Wed Nov 22 15:50:38 KST 2017 BOOT-INF/classes/io/honeymon/boot/springboot/
  0 Wed Nov 22 15:50:38 KST 2017 BOOT-INF/classes/io/honeymon/boot/springboot/api/
  0 Wed Nov 22 15:50:38 KST 2017 BOOT-INF/classes/io/honeymon/boot/springboot/api/handler/
  // 생략

```

`bootRepackage` 을 통해서 생성된 `boot-spring-boot-1.0.0.RELEASE.jar` 파일은 `java -jar` 명령으로 커맨드라인 에서 실행할 수 있다.

```
$ java -jar build/libs/boot-spring-boot-1.0.0.RELEASE.jar
... 스프링부트 로고
:: Spring Boot ::          (v2.0.0.RELEASE)

... i.h.s.book.BootSpringBootApplication      : Starting
BootSpringBootApplication on jake-mac.local with PID 42838
(/Users/honeymon/workspace/boot-spring-boot/build/libs/boot-spring-boot-1.0.0.RELEASE.jar started by honeymon in /Users/honeymon/workspace/boot-spring-boot)
... i.h.s.book.BootSpringBootApplication      : No active profile set,
falling back to default profiles: default
... 로그 생략 ..
... [main] s.b.c.e.t.TomcatEmbeddedServletContainer : Tomcat
started on port(s): 8080 (http)
... [main] i.h.s.book.BootSpringBootApplication      : Started
BootSpringBootApplication in 2.668 seconds (JVM running for 3.09)
```

실행중인 애플리케이션은 `Ctrl + C` 를 누르면 종료된다.

1.5. 다음 읽을꺼리

스프링 부트 기본 구성과 동작원리를 설명한다.

Chapter 2. '스프링 부트' 살펴보기

기능 구현에 필요한 라이브러리와 모듈을 손쉽게 추가할 수 있는 '의존성 관리', 추가된 '의존성' 라이브러리들을 바로 사용할 수 있도록 스프링 부트 개발팀이 권장하는 '자동구성(auto-config)', 개발 생산성을 높여주고 확장성을 부여하는 '스프링 프레임워크', "내장 컨테이너"를 이용한 빠른 실행과 재가동 시간을 줄이는 '개발자도구' 등 스프링 부트를 이용하는데 알아둬야할 것들을 살펴본다.

2.1. 빌드 시스템

스프링 부트 개발팀에서는 "의존성 관리" 에서 제시하는 것을 강력하게 권장하고 있으며, 의존성과 관련된 부분들은 "메이븐 중앙" 리파지토리에 배포하고 있다. 대표적인 빌드도구 그레이들과 메이븐에 대해서 설명하고 이후에는 그레이들을 중심으로 설명을 진행한다.

NOTE

스프링 부트 개발팀에서는 그레이들과 메이븐 중에서 사용하길 권한다. 다른 빌드도구를 사용하는 경우 제대로 지원하지 못한다고 밝히고 있다(맷고 끊고가 분명해서 마음에 든다).

2.1.1. 의존성 관리

스프링 부트는 출시할 때마다 지원하는 "의존성 라이브러리 버전"을 정리한 목록([스프링 부트 부록#Dependency versions\(https://goo.gl/H1VShv\)](https://goo.gl/H1VShv))을 제공한다. 목록에 명시된 의존성 라이브러리 버전을 개발자가 관리할 필요가 없다. 스프링 부트 버전을 업그레이드하면 **spring-boot-dependencies** 프로젝트 **pom.xml** 에 명시되어 있는 버전에 따라 함께 업그레이드 된다.

TIP

사용자가 필요하다면 스프링 부트에서 권장하는 버전을 자신이 원하는 버전으로 재정의할 수 있다(이런 경우 발생하는 충돌이나 호환성의 문제는 사용자가 처리해야한다). 사용하고 있는 스프링 부트 가 사용하는 의존성 모듈의 버전은 **spring-boot-dependencies** 프로젝트에 있는 **pom.xml** 을 살펴보거나 사용하는 버전의 스프링 부트 참고문서를 살펴보면 된다.

선별된 목록에는 스프링 부트와 함께 사용할 수있는 모든 스프링 모듈과 서드파티(3rd party) 라이브러리 목록이 포함되어 있다. **spring-boot-parent** 프로젝트의 부모 프로젝트로 **spring-boot-dependencies** 가 정의되어 있다.

spring-boot-parent/pom.xml 중 일부

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-dependencies</artifactId>
    <version>2.0.0.RELEASE</version>
    <relativePath>../spring-boot-dependencies</relativePath> ①
  </parent>
  <!-- 중략 -->
```

① 스프링 부트 출시버전 및 의존성 라이브러리 버전이 정의되어 있는 프로젝트

spring-boot-dependencies 프로젝트 pom.xml 을 살펴보면 <properties>..</properties> 에는 스프링 부트에서 사용하는 의존성 라이브러리 버전만 정의되어 있는 것을 확인할 수 있다.

\$./gradlew dependencies 명령을 통해서 그레이들 태스크 compileJava 와 test 시에 구성되는 의존성 관계를 확인할 수 있다.

그레이들 :: Java 플러그인 의존성 구성(<https://goo.gl/PIDuVX>)

```
$ ./gradlew dependencies | more
:dependencies

-----
Root project
-----

archives - Configuration for archive artifacts.
No dependencies

compile - Dependencies for source set 'main'.
\--- org.springframework.boot:spring-boot-starter-web: -> 2.0.0.RELEASE
     +--- org.springframework.boot:spring-boot-starter:2.0.0.RELEASE
          |    +--- org.springframework.boot:spring-boot:2.0.0.RELEASE
          // 중략

compileClasspath - Compile classpath for source set 'main'.
\--- org.springframework.boot:spring-boot-starter-web: -> 2.0.0.RELEASE
     +--- org.springframework.boot:spring-boot-starter:2.0.0.RELEASE
          |    +--- org.springframework.boot:spring-boot:2.0.0.RELEASE
          // 중략

compileOnly - Compile dependencies for source set 'main'.
\--- org.springframework.boot:spring-boot-starter-web: -> 2.0.0.RELEASE
```

```

+--- org.springframework.boot:spring-boot-starter:2.0.0.RELEASE
|    +--- org.springframework.boot:spring-boot:2.0.0.RELEASE
// 종락

testCompileClasspath - Compile classpath for source set 'test'.
+--- org.springframework.boot:spring-boot-starter-web: -> 2.0.0.RELEASE
|    +--- org.springframework.boot:spring-boot-starter:2.0.0.RELEASE
|    |    +--- org.springframework.boot:spring-boot:2.0.0.RELEASE
// 종락

testCompileOnly - Compile dependencies for source set 'test'.
+--- org.springframework.boot:spring-boot-starter-web: -> 2.0.0.RELEASE
|    +--- org.springframework.boot:spring-boot-starter:2.0.0.RELEASE
|    |    +--- org.springframework.boot:spring-boot:2.0.0.RELEASE
// 종락

testRuntime - Runtime dependencies for source set 'test'.
+--- org.springframework.boot:spring-boot-starter-web: -> 2.0.0.RELEASE
|    +--- org.springframework.boot:spring-boot-starter:2.0.0.RELEASE
|    |    +--- org.springframework.boot:spring-boot:2.0.0.RELEASE
// 종락

(*) - dependencies omitted (listed previously)

BUILD SUCCESSFUL

Total time: 4.04 secs

```

TIP 커맨드라인에서 사용하는 **파이프라인(|)**은 앞에 있는 명령어의 결과를 뒤로 전달하는 기능을 한다. **./gradlew dependencies | more** 명령어는 **./gradlew dependencies** 으로 빠르게 출력되는 목록을 **more** 명령을 내려서 한화면씩 멈추도록 한다.

- 영상: [프로젝트 의존성 계층 확인하기](#)

IMPORTANT 출시된 스프링 부트는 출시버전마다 스프링 프레임워크와 연계되어 있으므로 스프링 프레임워크 버전을 **직접 지정하지 않는 것이 좋다.**

2.1.2. 메이븐

메이븐 사용자는 **spring-boot-starter-parent** 프로젝트를 상속받아 기본값을 얻어올 수 있다. 상위 프로젝트는 다음과 같은 기능을 제공한다:

- 기본 컴파일러를 Java 1.6 사용
- UTF-8 소스 인코딩
- "[의존성 관리](#)" 에서, **spring-boot-dependencies** 를 상속한 경우 선별된 의존성에 대해서는 **<version>** 태그 생략가능

- 손쉬운 리소스 필터링(<https://goo.gl/L3peFs>)
- 손쉬운 플러그인 구성(exec plugin(<https://goo.gl/kIO2TY>), surefire(<https://goo.gl/Scd9qE>), Git commit ID (<https://goo.gl/oZXhu8>), shade(<https://goo.gl/P9ZgqN>))
- 프로파일 선언 파일(예: `application-foo.properties` 과 `application-foo.yml`) 을 포함한 `application.properties` 과 `application.yml` 에 대한 리소스 필터링 제공

마지막으로, 메이븐 에서 사용하는 위치변환자(placeholder) `@..@` 를 스프링 스타일 `${...}` 로 변경할 수 있다(메이븐 속성 `resource.delimiter` 를 오버라이드 할 수 있다)

2.1.3. 부모 스프링 부트 스타터 상속

다음에서 볼 수 있는 것처럼 `parent` 를 설정하는 것으로 간단하게 프로젝트에서 `spring-boot-starter-parent` 를 상속할 수 있다.

```
<!-- Inherit defaults from Spring Boot -->
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.0.0.RELEASE</version>
</parent>
```

TIP

스프링 부트와 관련된 버전에 대해서는 여기에서 사용자가 원하는 대로 선언하면 된다. 이후에 `<dependencies>` 영역에서 스타터 를 추가할 때는 버전을 생략해도 `spring-boot-starter-parent` 버전에 따라 적용된다.

프로젝트에서 의존성에 대해서 개별적으로 버전을 정의할 수도 있다. 예를 들어 스프링 Data release train 을 추가하고 싶다면 다음과 같이 `pom.xml` 에 추가하면 된다.

```
<properties>
  <spring-data-releasetrain.version>Fowler-SR2</spring-data-releasetrain.version>
</properties>
```

NOTE

지원하는 속성들에 대해서 궁금하다면 `spring-boot-dependencies`(<https://goo.gl/tUruhG>) `pom` 을 살펴보자.

2.1.4. spring-boot-starter-parent 없이 사용하기

모든 이가 `spring-boot-starter-parent` POM 사용을 원하는 것은 아니다. 기업 표준 parent POM 을 사용하거나 모든 메이븐 구성을 명확하게 선언하기를 원하는 경우가 있을 것이다.

`spring-boot-starter-parent` 사용은 하고 싶지 않지만 의존성 관리(플러그인 관리는 안됨)의 장점은 유지하고 싶다면 `scope=import`를 이용하면 된다.


```

<dependencyManagement>
  <dependencies>
    <dependency>
      <!-- 스프링 부트로부터 의존성관리 불러오기 -->
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-dependencies</artifactId>
      <version>2.0.0.RELEASE</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

```

이 설정은 위에 설명대로 속성(property)을 이용한 개별적인 의존성을 재정의 할 수 없다. 부모 스프링 부트 스타터 상속에서 프로퍼티만 정의했던 것과 동일한 결과를 얻기 위해서는 **dependencyManagement** 엔트리 안에서 **spring-boot-dependencies** 엔트리 **이전에** 필요한 것을 추가해야 한다. 예를 들자면, **pom.xml** 에 스프링 Data release train 을 추가하는 방법은 다음과 같다:

```

<dependencyManagement>
  <dependencies>
    <!-- 스프링 부트에서 제공하는 스프링 Data release train 재정의 -->
    <dependency>
      <groupId>org.springframework.data</groupId>
      <artifactId>spring-data-releasetrain</artifactId>
      <version>Fowler-SR2</version>
      <scope>import</scope>
      <type>pom</type>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-dependencies</artifactId>
      <version>2.0.0.RELEASE</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

```

앞의 예제에서 **BOM(Bills of Materials)** 타입을 정의했지만 저런 방식으로 어떤 의존성 유형이든 정의할 수 있다.

NOTE

BOM 은 메이븐에서 정의한 라이브러리와 관련된 의존성을 한데 묶어서 버전을 관리하는 기능을 제공한다. 스프링 부트는 **spring-boot-dependencies** 에 사전에 정의해둔 BOM을 정의하고 있다.

`spring-boot-starter-parent` 는 매우 보수적인 자바 호환성을 선택했다(Java 6). 권장사항에 따라서 최신 자바 버전을 사용하고 싶다면 다음처럼 `java.version` 속성을 추가하면 된다.

```
<properties>
  <java.version>1.8</java.version>
</properties>
```

2.1.5. 스프링 부트 메이븐 플러그인 사용하기

"스프링 부트 메이븐 플러그인" 을 사용하면 프로젝트를 실행가능한 jar로 압축할 수 있다. 이를 사용하고 싶다면 `<plugins>` 영역에 다음과 같이 추가하면 된다:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

NOTE

스프링 부트 starter parent pom 을 사용하고 있다면 플러그인을 추가하기만 하면 된다. parent 에 정의된 설정을 변경하지 않았다면 별다른 구성을 하지 않아도 된다.

2.1.6. 그레이들

그레이들 사용자는 `dependencies` 영역에 바로 'starter'를 불러올 수 있다. 메이븐과는 다르게, 몇몇 구성을 공유하기 위해 불러올 수 있는 "super parent" 가 없다.

```
repositories {
  jcenter()
}

dependencies {
  compile("org.springframework.boot:spring-boot-starter-
web:2.0.0.RELEASE")
}
```

"스프링 부트 그레이들 플러그인" 는 실행가능한 jar를 생성하고 소스를 바로 실행할 수 있는 태스크를 지원한다.

"그레이들 의존성 관리"가 제공하는 기능으로 스프링 부트가 관리되는 의존성 라이브러리들의 버전을 생략할 수 있다.

```

plugins {
    id 'java'
    id 'org.springframework.boot' version '2.0.0.RELEASE'
}

repositories {
    jcenter()
}

dependencies {
    compile("org.springframework.boot:spring-boot-starter-web")
    testCompile("org.springframework.boot:spring-boot-starter-test")
}

```

2.1.7. 스타터(Starter)

스타터는 애플리케이션이 이용할 수 있는 **의존성을 기술하고 있는 집합체**다. 필요한 기능을 가진 의존성 라이브러리를 모두 기술할 필요없이 필요한 의존성 라이브러리를 한번에 담을 수 있다. 예를 들자면, 스프링과 데이터베이스 접속을 위한 JPA를 사용하고 싶다면, 프로젝트 빌드스크립트(**build.gradle** or **pom.xml**) **spring-boot-starter-data-jpa**에 대한 의존성을 추가하고 프로젝트를 갱신하면 이와 관련된 별다른 구성을 하지 않아도 바로 사용할 수 있다. 스타터는 일관성있게 지원되는 의존성 라이브러리 세트를 제공하여 기능을 구현하는데 필요한 의존성 라이브러리 목록을 포함하고 있다.

스타터 이름 설명

공식 스타터는 동일한 작명방식(네이밍 패턴, naming pattern)을 따른다; **spring-boot-starter-**, 는 특정한 기능 유형을 정의한다. 이 작명구조는 필요한 starter 를 쉽게 찾을 수 있도록 해준다. 많은 IDE 의 메이븐 통합기능을 통해 이름으로 의존성을 검색가능하다. 예를 들어, 이클립스 혹은 STS 에 플러그인을 설치하면, POM 에디터에서 "spring-boot-starter"를 입력한 후 자동완성기능을 이용하면 목록에서 원하는 항목을 검색할 수 있다.

- 영상: [pom.xml 에서 starter 자동완성 찾기](#)

NOTE

추가적으로 커뮤니티에서 제공하는 starters 를 보고 싶다면 깃헙의 **spring-boot-starters** (<https://goo.gl/tNem58>) 프로젝트를 살펴보면 된다.

2.1.8. 래퍼(Wrapper)

그레이들 혹은 메이븐을 개발자 로컬환경 CI 서버에 설치하지 않고 프로젝트에 함께 포함시켜 배포할 수 있다.

TIP

스프링 부트 프로젝트를 메이븐으로 구성하게 되는 경우에는 메이븐 래퍼 를 사용할 수 있는 스크립트 (**mvnw** or **mvnw.cmd**)가 포함된다. **\$./mvnw** 를 실행하면 래퍼 가 실행된다. 처음 실행할 경우에는 로컬 저장소(**~/.m2/io/takari**) 에 래퍼 와 관련된 것을 다운로드 한 후 실행가능한 상태가 된다.

프로젝트를 처음 구성하는 개발자는 그레이들 혹은 메이븐이 설치되어 있어야 한다. 설치 방법은 아래 링크를 확인하기 바란다.

NOTE

- 그레이들 설치: <https://gradle.org/install/>
- 메이븐 설치: <https://maven.apache.org/install.html>

윈도우즈의 경우에는 **gradlew.bat** 를 실행하면 되고 유닉스 기반 시스템(MacOS, Linux) 에서는 **gradlew** 를 실행하면 된다. \$ **./gradlew** 를 처음 실행할 경우 실행위치에서 **.gradle/wrapper**에 **gradle-wrapper.jar** 이 없는 경우에는 원격저장소에 있는 최신의 배포본을 받아온다.

```
$ gradlew wrapper --gradle-version 4.3 ①
Downloading https://services.gradle.org/distributions/gradle-4.3-bin.zip
.....
Unzipping /Users/honeymon/.gradle/wrapper/dists/gradle-4.3-
bin/daoimhu7k5rlo48ntmxw2ok3e/gradle-4.3-bin.zip to
/Users/honeymon/.gradle/wrapper/dists/gradle-4.3-
bin/daoimhu7k5rlo48ntmxw2ok3e
Set executable permissions for:
/Users/honeymon/.gradle/wrapper/dists/gradle-4.3-
bin/daoimhu7k5rlo48ntmxw2ok3e/gradle-4.3/bin/gradle
Starting a Gradle Daemon (subsequent builds will be faster)
Cleaned up directory '/Users/honeymon/workspace/boot-spring-
boot/build/classes/main'
Cleaned up directory '/Users/honeymon/workspace/boot-spring-
boot/build/resources/main'
:wrapper

BUILD SUCCESSFUL

Total time: 7.554 secs
```

① 4.3은 그레이들에서 제공하는 버전에 맞춰 변경한다.

그레이들 래퍼 에 대해 보다 자세히 알고 싶다면 아래 문서를 살펴보기 바란다.

NOTE

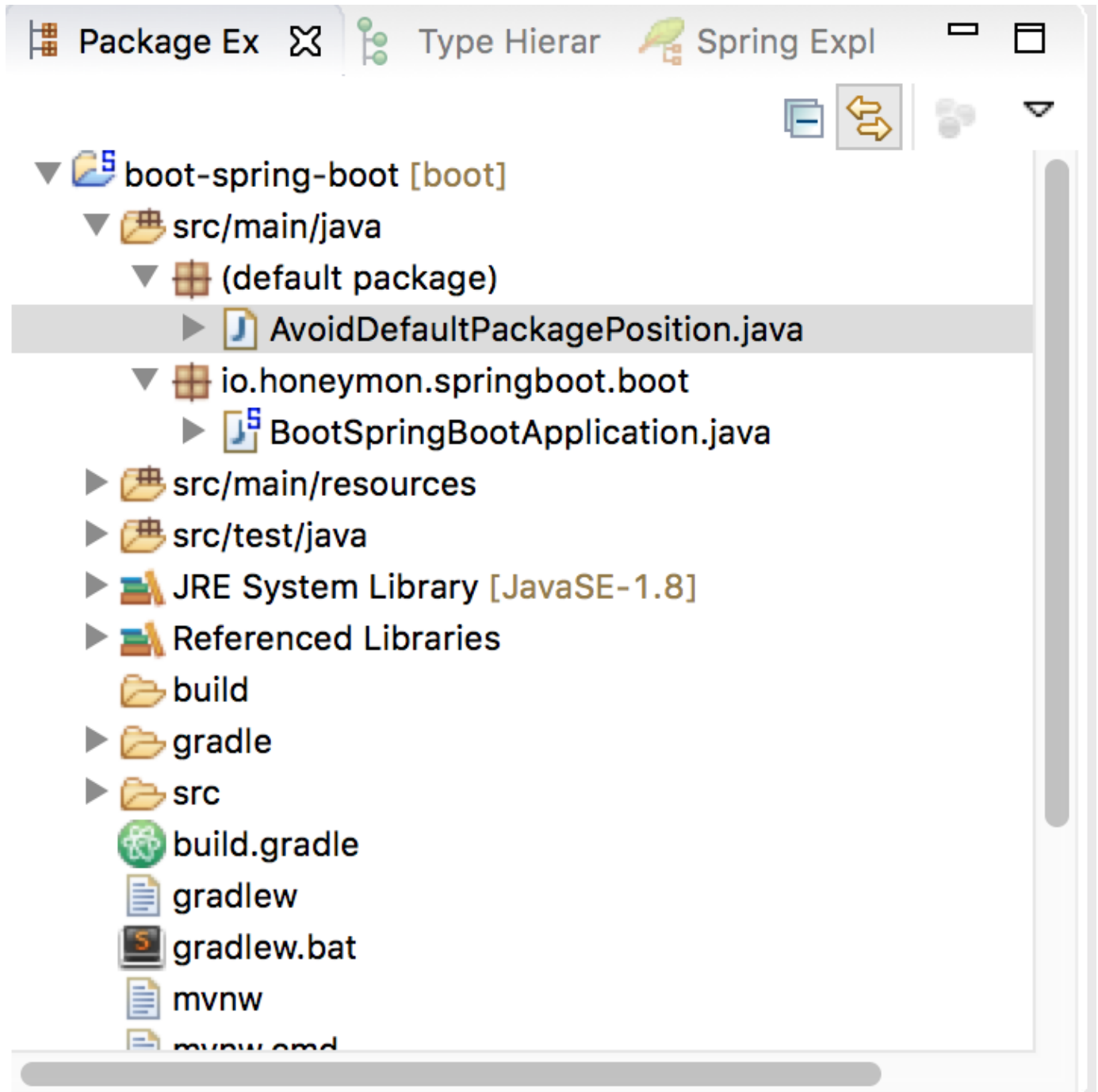
- 그레이들 가이드#래퍼(<https://goo.gl/zdGK3l>)
- Gradle wrapper - 권남(<https://goo.gl/7NpPp5>)

2.2. 코드 구조

스프링 부트는 특별히 어떤 계층구조로 코드를 작성하라고 강요하지는 않는다. 도움이 될만한 몇 가지 가이드가 있다. 이외에는 팀과 조직의 관례(컨벤션, Convention, 규칙)를 따르기를 바란다.

2.2.1. "기본 패키지" 이용

기본패키지("Default package")로 간주되는 되는 곳에 클래스를 작성하는 것은 좋지 않다. 기본 패키지 (`AvoidDefaultPackagePosition` 위치, <https://goo.gl/uCm2zQ>) 사용은 피하길 바란다.



프로젝트에서 `@ComponentScan`, `@EntityScan` 혹은 `@SpringBootApplication` 애너테이션이 선언된 클래스가 "기본 패키지(default package)"에 위치할 경우 애플리케이션이 사용하는 모든 jar 에서 모든 클래스를 탐색하는 상황이 발생할 수 있다.

패키지명은 자바에서 권장하는 패키지명 작성관례를 따르거나 도메인명을 뒤집어서 사용하는 것을 권장한다(예: `io.honeymon.boot.springboot`).

TIP

고전적으로 사용된 방식은 도메인명을 뒤집는 것이지만 패키지명이 길어지는 것이 싫다면, **개발팀 내의 합의**가 있다면, 축약해서 사용해도 된다. 특별히 형식이 정해져 있는 것은 아니니 자신만의 방식을 사용하여 작성해보자(예: `io.honeymon.boot.springboot` → `io.honey.bsb`).

2.2.2. 애플리케이션 메인 클래스 위치

메인 애플리케이션 클래스는 **최상위 패키지(예: `io.honeymon.boot.springboot`)에 위치하는 것을 권장**한다. `@EnableAutoConfiguration` 처럼 애플리케이션 구성에 필요한 것을 탐색하는 것이 명백한 행위에 대한 선언은 메인 애플리케이션 클래스에 위치하는 경우가 많다(실제로 `@SpringBootApplication` 에 기본선언되어 있음). 예를 들어, JPA 를 사용한다면 `@EnableAutoConfiguration`이 선언되어 있다면 `@Entity`가 선언된 클래스를 탐색한다. 최상위 패키지에 있는 클래스에 `@ComponentScan` 를 선언하면 `basePackage` 속성을 정의할 필요가 없다. 최상위 패키지에 있는 메인 클래스에 `@SpringBootApplication` 을 대신 사용할 수도 있다.

여기 기본적인 스프링 부트 프로젝트 구조가 있다:

최상위 패키지에 메인클래스가 있고 `@SpringBootApplication` 가 있는 경우

```
io
+- honeymon
  +- boot.springboot
    +- BootSpringBootApplication.java
    |
    +- domain
    |   +- Customer.java
    |
    +- repository
    |   +- CustomerRepository.java
    |
    +- service
    |   +- CustomerService.java
    |
    +- web
    |   +- CustomerController.java
```

`BootSpringBootApplication` 는 다음처럼 간단해진다.

```

@SpringBootApplication ①
public class BootSpringBootApplication {

    public static void main(String[] args) {
        SpringApplication.run(BootSpringBootApplication.class, args);
    }
}

```

① 스프링 부트 1.2.0.RELEASE 버전부터 적용된 `@SpringBootApplication` 덕분이다

그런데 `BootSpringBootApplication`의 위치가 `app` 디렉토리로 변경된다면 많은 번거로움이 생긴다.

메인클래스 `app`으로 이동한 경우

```

io
+- honeymon
  +- boot.springboot
    +- app
      +- BootSpringBootApplication.java
    +- domain
      +- Customer.java
    +- repository
      +- CustomerRepository.java
    +- service
      +- CustomerService.java
    +- web
      +- CustomerController.java

```

`BootSpringBootApplication`에 `scanBasePackages` 속성에 탐색해야 할 패키지를 지정해야 하는 번거로움이 생긴다.

의외로 '@SpringBootApplication' 이 선언된 메인 클래스 위치를 임의로 변경하고 애플리케이션이 실행되지 않는다(혹은 에러가 발생한다)'는 질문을 올리는 이들이 많다(제법 많다!!). @SpringBootApplication 이 선언된 메인 클래스 위치가 변경되면 해야할 작업들이 많아져서 번거롭다.

```
@SpringBootApplication(scanBasePackages = {
    "io.honeymon.boot.springboot.domain",
    "io.honeymon.boot.springboot.service",
    "io.honeymon.boot.springboot.web" })
@EnableJpaRepositories("io.honeymon.boot.springboot.domain")
@EntityScan("io.honeymon.boot.springboot.domain")
public class BootSpringBootApplication {
    public static void main(String[] args) {
        SpringApplication.run(BootSpringBootApplication.class,
            args);
    }
}
```

위처럼 선언하지 않으면 Controller 등의 빈(Bean)을 탐색하지 못해 예외가 발생한다. 다음처럼 말이다:

NOTE

```
Caused by:
org.springframework.beans.factory.NoSuchBeanDefinitionException
: No qualifying bean of type
[io.honeymon.boot.springboot.domain.CustomerRepository] found
for dependency
[io.honeymon.boot.springboot.domain.CustomerRepository]:
expected at least 1 bean which qualifies as autowire candidate
for this dependency. Dependency annotations:
{@org.springframework.beans.factory.annotation.Autowired(required=true)}
    at
org.springframework.beans.factory.support.DefaultListableBeanFactory.raiseNoMatchingBeanFound(DefaultListableBeanFactory.java:
1406)
    // 생략
```

스프링 애플리케이션 구성에 어느 정도 익숙해지면 개인적인 역량에 따라 메인 클래스 위치를 변경할 수 있겠지만, 최상위 패키지에 메인 클래스를 위치시키는 게 여러모로 편리하다는 것을 알아두길 바란다. 프로젝트 패키지 구성은 사소한듯 하면서도 신경 쓸 부분들이 많은데, 이와 관련해서는 [패키지 구조는 어떻게 가져가는 것이 좋을까?\(https://slipp.net/questions/36\)](https://slipp.net/questions/36), [Get Your Java Package Structures Right\(https://goo.gl/p6j1kc\)](https://goo.gl/p6j1kc) 와 [Package by feature, not layer\(https://goo.gl/Qz2kqg\)](https://goo.gl/Qz2kqg) 등을 읽어본 후에 팀원들과 이야기를 나누며 적절한 구조를 잡길 바란다.

2.3. 구성(Configuration) 클래스

스프링 부트는 자바기반 구성(JavaConfig, <https://goo.gl/YhA1qc>)을 선호한다. 물론 XML 소스를 이용하는 것도 가능하지만 기본적으로는 `@Configuration`와 자바코드로 선언하는 것을 권장한다. `@Configuration` 선언된 클래스에 `main` 메서드를 정의해두는 것은 보완책을 마련해두는 좋은 방법이다.

TIP

인터넷에서 찾아볼 수 있는 예제 중에는 XML로 구성하는 경우가 많다. 대부분의 경우 XML구성을 동등한 자바기반으로 구성할 수 있다. `~AutoConfiguration` 애너테이션들을 찾아보는 것부터 시작해보자.

2.3.1. 구성클래스 불러오기

굳이 모든 것을 `@Configuration`가 선언된 하나의 클래스에 몰아 넣을 필요는 없다. `@Import` 애너테이션을 이용하여 추가적인 구성 클래스를 불러와 사용할 수 있다. 이에 대한 대안으로 `@ComponentScan`을 사용하면 `@Configuration`를 포함한 모든 모든 스프링 컴포넌트를 사용할 수 있다.

`@Import`를 이용해서 `@Configuration` 클래스 불러오기

```
@Configuration
@Import(EnableWebMvcConfiguration.class)
```

TIP

`@SpringBootApplication`를 열고 `@EnableAutoConfiguration`에 포함되어 있는 `@AutoConfigurationPackage`를 살펴보면 자동구성과 관련된 클래스를 불러오는 과정을 살펴볼 수 있다.

2.3.2. XML 구성 불러오기

`@Configuration` 클래스로 시작하는 것을 권하지만 그래도! 굳이! XML 기반으로 구성을 하겠다면 할 수도 있다. `@ImportResource` 애너테이션을 사용해서 XML 구성 파일들을 불러오면 된다.

NOTE

사용방법에 대해서는 [Spring @ImportResource Annotation Example\(https://goo.gl/dvLFv4\)](https://goo.gl/dvLFv4)를 살펴보자.

2.4. 자동구성(auto-configuration)

스프링 부트 "자동구성"은 추가된 의존성을 기반으로 애플리케이션을 자동구성하는 기능이다. 예를 들어 `h2database` 라이브러리가 클래스패스 상에 존재한다면 데이터베이스에 대한 어떤 구성을 하지 않아도 스프링 부트가 알아서 인-메모리 데이터베이스로 자동구성한다.

NOTE

`EmbeddedDatabaseCondition`의 조건이 발동되면서 `DataSourceAutoConfiguration.EmbeddedDatabaseConfiguration` 구성이 활성화되어 인-메모리로 구성한다.

```
@Configuration
public class BootSpringBootApplication {
    // 중략
}
```

자동구성 기능을 활성화 하려면 `@Configuration` 이 선언된 클래스에 `@EnableAutoConfiguration`을 추가하면 된다.

```
@EnableAutoConfiguration
@Configuration
public class BootSpringBootApplication {
    // 중략
}
```

스프링 부트에서는 반복적으로 사용되는 애너테이션을 쓰는 번거로움을 줄일 수 있도록

`@EnableAutoConfiguration`와 `@SpringBootConfiguration(@Configuration 을 포함하고 있음)`을 합친 `@SpringBootApplication`를 제공한다. 스프링 부트 개발팀에서 제공하는 자동구성이 어떻게 구성되어 있는지를 살펴보면 `~AutoConfiguration` 클래스(예: `WebMvcAutoConfiguration`)를 찾아보기 바란다.

WebMvcAutoConfiguration

```
@Configuration
@ConditionalOnWebApplication
@ConditionalOnClass({ Servlet.class, DispatcherServlet.class,
    WebMvcConfigurerAdapter.class })
@ConditionalOnMissingBean(WebMvcConfigurationSupport.class)
@AutoConfigureOrder(Ordered.HIGHEST_PRECEDENCE + 10)
@AutoConfigureAfter({ DispatcherServletAutoConfiguration.class,
    ValidationAutoConfiguration.class })
public class WebMvcAutoConfiguration
    // 생략
```

2.4.1. 자동구성 대체하기

자동구성은 비침투적이기에, 자동구성 특정 부분에 대해서는 재정의한 부분으로 시작하는 것이 가능하다. 예를 들자면, `DataSource` 빈을 추가한다면, 기본 내장 데이터베이스 지원은 비활성화된다.

현재 제공되고 있는 자동구성이 무엇인지 찾아보려고 한다면, 애플리케이션이 시작될 때 `--debug`를 활성화하면 된다. 디버그 로그가 활성화되어 자동구성 보고서가 콘솔에 출력된다.

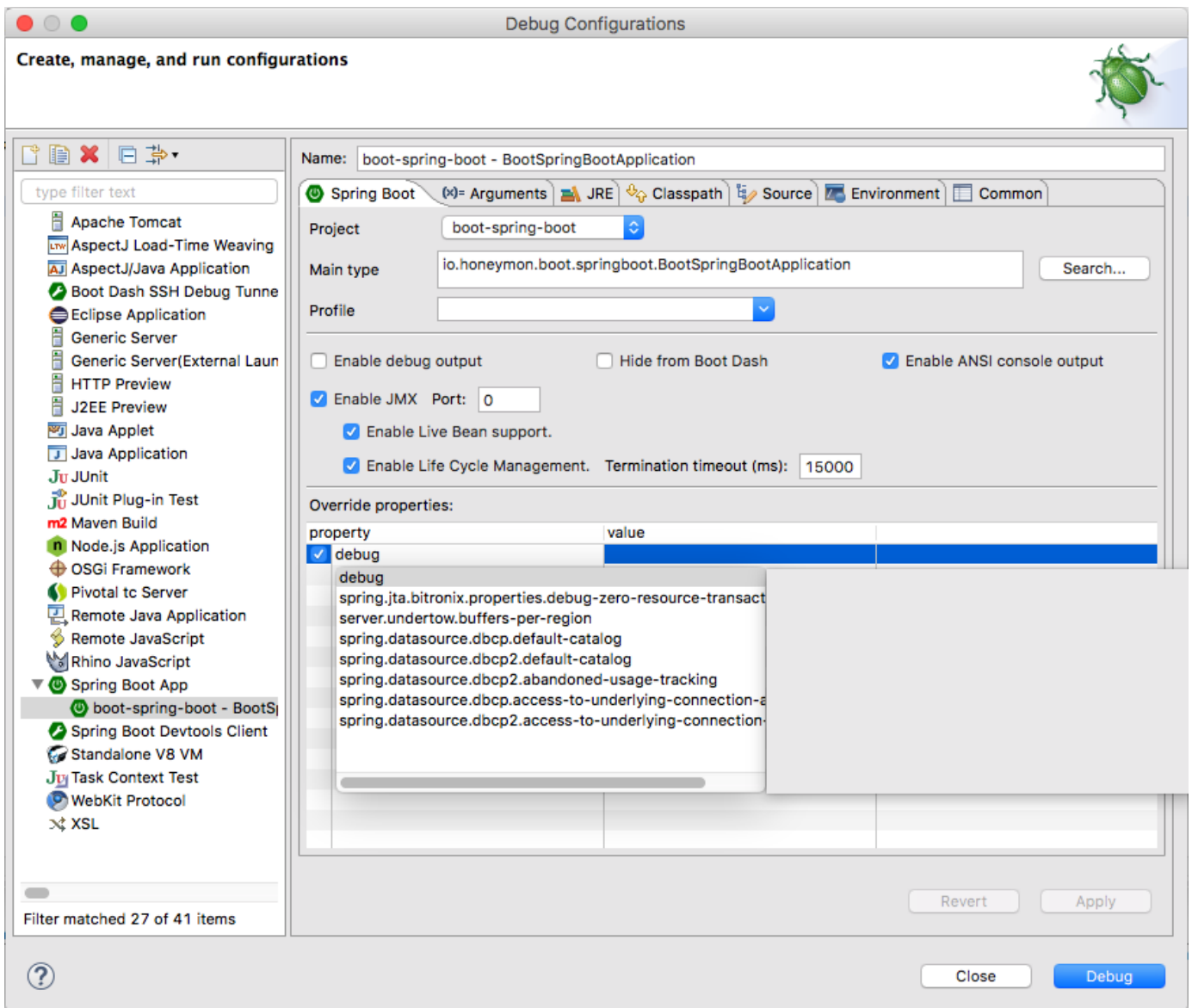


Figure 2. --debug 스위치 설정

이렇게 애플리케이션을 실행시키면 다음과 같은 로그 출력결과를 확인할 수 있다:

```
=====
auto-config REPORT
=====
```

Positive matches:

// 종략

WebMvcAutoConfiguration matched:

- @ConditionalOnClass found required classes

'javax.servlet.Servlet',

'org.springframework.web.servlet.DispatcherServlet',

'org.springframework.web.servlet.config.annotation.WebMvcConfigurerAdapter'

```

'; @ConditionalOnMissingClass did not find unwanted class
(OnClassCondition)
    - @ConditionalOnWebApplication (required) found
StandardServletEnvironment (OnWebApplicationCondition)
    - @ConditionalOnMissingBean (types:
org.springframework.web.servlet.config.annotation.WebMvcConfigurationSuppo
rt; SearchStrategy: all) did not find any beans (OnBeanCondition)

WebMvcAutoConfiguration#hiddenHttpMethodFilter matched:
    - @ConditionalOnMissingBean (types:
org.springframework.web.filter.HiddenHttpMethodFilter; SearchStrategy:
all) did not find any beans (OnBeanCondition)

WebMvcAutoConfiguration#httpPutFormContentFilter matched:
    - @ConditionalOnProperty (spring.mvc.formcontent.putfilter.enabled)
matched (OnPropertyCondition)
    - @ConditionalOnMissingBean (types:
org.springframework.web.filter.HttpPutFormContentFilter; SearchStrategy:
all) did not find any beans (OnBeanCondition)

// 종락

Negative matches:
-----
// 종락

DataSourceAutoConfiguration:
Did not match:
    - @ConditionalOnClass did not find required class
'org.springframework.jdbc.datasource.embedded.EmbeddedDatabaseType'
(OnClassCondition)

DataSourceTransactionManagerAutoConfiguration:
Did not match:
    - @ConditionalOnClass did not find required classes
'org.springframework.jdbc.core.JdbcTemplate',
'org.springframework.transaction.PlatformTransactionManager'
(OnClassCondition)

// 종락

DispatcherServletAutoConfiguration.DispatcherServletConfiguration#multipar
tResolver:
Did not match:
    - @ConditionalOnBean (types:
org.springframework.web.multipart.MultipartResolver; SearchStrategy: all)
did not find any beans (OnBeanCondition)

// 종락

```

Exclusions:

None

Unconditional classes:

org.springframework.boot.autoconfigure.context.ConfigurationPropertiesAutoConfiguration

org.springframework.boot.autoconfigure.web.WebClientAutoConfiguration

org.springframework.boot.autoconfigure.context.PropertyPlaceholderAutoConfiguration

org.springframework.boot.autoconfigure.info.ProjectInfoAutoConfiguration

2.4.2. 특정 자동구성 비활성화하기

찾은 특정 자동구성 클래스를 사용하고 싶지 않다면, `@EnableAutoConfiguration`의 `exclude` 속성을 이용해서 **비활성화** 할 수 있다. 다음 예제에서는 `DataSourceAutoConfiguration` 을 비활성화하고 있다.

```
import org.springframework.boot.autoconfigure.*;
import org.springframework.boot.autoconfigure.jdbc.*;
import org.springframework.context.annotation.*;

@Configuration
@EnableAutoConfiguration(exclude={DataSourceAutoConfiguration.class})
public class MyConfiguration {
}
```

클래스패스 상에 클래스가 없다면 `excludeName` 속성을 이용해서 등록된 빈의 이름 전체를 정의하는 방법을 사용할 수 있다. 마지막으로, `application.yml` 에서 `spring.autoconfigure.exclude` 속성을 통해서 자동구성 클래스 목록을 제외하는 것도 가능하다.

TIP | 애너테이션 레벨과 속성(property)에서 제외선언이 가능하다.

2.5. 스프링 빈(Beans)과 의존성 주입(Dependency injection)

스프링 프레임워크 표준기술을 사용하여 빈과 의존성 주입을 자유롭게 사용할 수 있다. 간단히 말해, `@ComponentScan` 을 사용해서 빈을 찾아내고, 생성자에서 `@Autowired`를 선언하여 생성자 의존성 주입방식을 사용할 수 있다.

만약 위에서 제안했던 것처럼 최상위 패키지에 애플리케이션 클래스를 위치시켰다면 `@ComponentScan`을 추가할 때 다른 속성을 정의하지 않아도 된다. 모든 애플리케이션 컴포넌트(`@Component`, `@Service`, `@Repository`, `@Controller` 등등)가 스프링 빈으로 자동등록된다.

여기 `RiskAssessor` 빈을 필요로 하여 생성자 주입방식을 사용한 `@Service` 빈 예제가 있다:

```
@Service
public class DatabaseAccountService implements AccountService {

    private final RiskAssessor riskAssessor;

    @Autowired
    public DatabaseAccountService(RiskAssessor riskAssessor) {
        this.riskAssessor = riskAssessor;
    }
    // ...
}
```

그리고 빈에 생성자가 하나만 있다면 `@Autowired`를 생략가능하다:

```
@Service
public class DatabaseAccountService implements AccountService {

    private final RiskAssessor riskAssessor;

    public DatabaseAccountService(RiskAssessor riskAssessor) {
        this.riskAssessor = riskAssessor;
    }
    // ...
}
```

TIP

생성자 주입방식을 사용할 때에는 `riskAssessor` 필드를 `final` 로 표기해두면 빈이 생성된 이후에 변경되지 않는다는 것을 **명시적으로** 알려주는 역할을 한다.

2.6. @SpringBootApplication 애너테이션 이용

많은 스프링 부트 개발자들이 메인 클래스에 `@Configuration`, `@EnableAutoConfiguration` 와 `@ComponentScan`를 선언했었다. 이 애너테이션들은 자주 함께 사용한다. 스프링 부트 개발팀에서 간단하게 쓸 수 있는 `@SpringBootApplication` 를 제공한다(위에서 살펴본 "[코드 구조](#)"에서 `BootSpringBootApplication`에 선언되어 있다).

`@SpringBootApplication` 애너테이션 을 사용하면 `@Configuration`, `@EnableAutoConfiguration` 와 `@ComponentScan`을 사용한 것과 동일하게 동작한다.

```
@SpringBootApplication
public class BootSpringBootApplication {
    public static void main(String[] args) {
        SpringApplication.run(BootSpringBootApplication.class, args);
    }
}
```

NOTE

`@SpringBootApplication`은 `@EnableAutoConfiguration`와 `@ComponentScan`의 속성을 재정의하는 앨리어스를 제공한다. 이 애너테이션은 스프링 부트 1.2.0.RELEASE (<https://goo.gl/UveDGP>)에서 추가됐다. 1.4 버전(<https://goo.gl/Bzv2E5>)에서는 `@SpringBootConfiguration`이 `@Configuration`을 대체했다.

`@SpringBootConfiguration`는 선언된 클래스가 `@SpringBootConfiguration`를 제공함을 나타낸다. 테스트에서 `@Configuration` 대안으로 사용하여 자동구성을 찾을 수 있다. 스프링 부트 애플리케이션에서는 하나(`@SpringBootApplication`에 포함되어 있음)만 사용해야 한다.

2.7. 애플리케이션 실행하기

애플리케이션을 JAR로 패키징하면서 내장된 HTTP 서버를 포함해서 얻을 수 있는 가장 큰 혜택 중 하나는 **어디서나 애플리케이션을 실행할 수 있다**는 것이다. 스프링 부트 애플리케이션을 디버깅하는 것도 쉽다. 특별히 IDE 플러그인이나 확장기능을 필요로 하지 않는다.

기본 스프링 애플리케이션이었다면 IDE에서는 톰캣 등의 WAS를 내려받고 내려받은 서버를 설치하고 서버에서 실행하기 위한 설정작업을 해야했지만 스프링 부트는 이런 설정없이 실행할 수 있는 강점을 가지고 있다.

2.7.1. IDE에서 실행하기

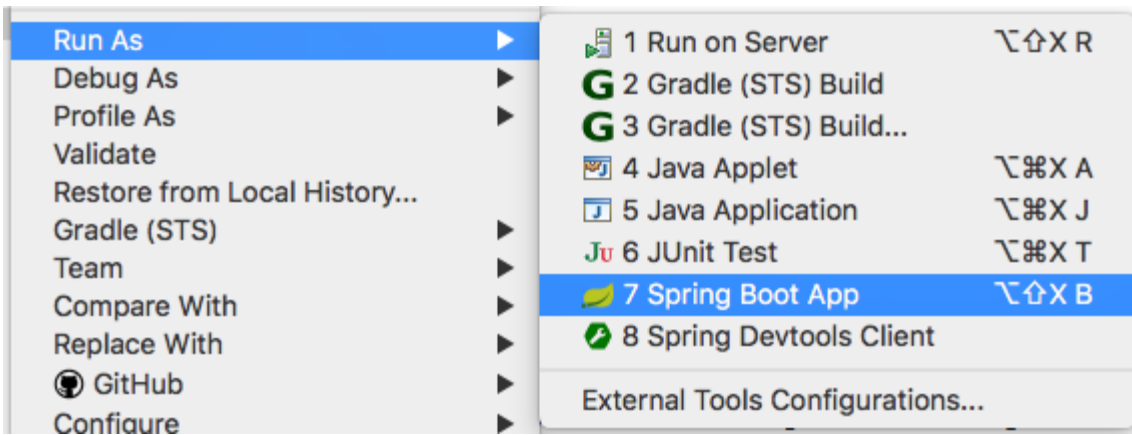
스프링 부트 애플리케이션을 IDE에서 간단한 자바 애플리케이션처럼 실행할 수 있다. 대부분의 IDE에는 그레이들과 메이븐 빌드시스템을 지원하고 있기 때문에, 프로젝트를 구성하고 있는 빌드 설정에 맞춰 애플리케이션을 불러오면 된다.

TIP

웹 애플리케이션을 실행했을 때 "Port already in use" 에러가 발생하는 상황이 생길 수 있다. STS 사용자라면 **Run** 보다는 **Relaunch**를 사용하여 이미 실행되고 있는 인스턴스를 종료시키고 시작할 수 있다.

STS 실행방법

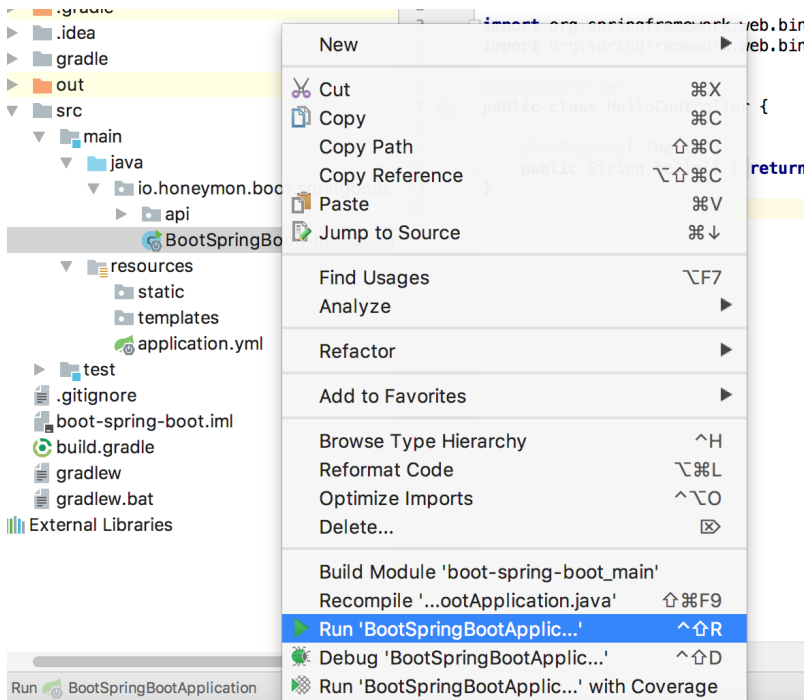
`@SpringBootApplication`가 선언되어 있는 메인클래스를 호출한 상태에서 단축키를 누르면 애플리케이션이 실행된다. 혹은 프로젝트를 선택한 상태에서 우클릭하여 'Run As → Spring Boot App'을 선택해도 된다.



- 영상: <https://youtu.be/auqisIJJR3k>

인텔리제이 실행방법

@SpringBootApplication 가 선언되어 있는 메인클래스를 호출한 상태에서 단축키를 누르면 애플리케이션 이 실행된다. 혹은 프로젝트를 선택한 상태에서 우클릭하여 'Run As → Spring Boot App' 을 선택해도 된다.



- 영상: <https://youtu.be/BOUIQm-75cl>

2.7.2. 패키징된 애플리케이션 실행하기

스프링 부트 메이븐 혹은 그레이들 플러그인을 사용하고 있다면 실행가능한 jar를 생성하여 `java -jar` 명령으로 애플리케이션을 사용할 수 있다. 예를 들어:

```
$ java -jar build/lib/boot-spring-boot-1.0.0.RELEASE.jar
```


- 영상: <https://youtu.be/zr5oMR34VFU>

실행중인 패키징된 애플리케이션에 대해서 원격디버깅 지원이 가능하다. 다음처럼 하면 패키징된 애플리케이션 디버거에 접근하는 것을 허용할 수 있다.

```
$ java -Xdebug  
-Xrunjdwp:server=y,transport=dt_socket,address=8000,suspend=n \  
-jar build/lib/boot-spring-boot-1.0.0.RELEASE.jar
```

2.7.3. 그레이들 플러그인으로 실행하기

스프링 부트 그레이들 플러그인은 애플리케이션을 실행할 수 있는 **bootRun** 태스크를 포함하고 있다. **bootRun** 태스크는 **spring-boot-gradle-plugin** 을 통해서 추가된다.

```
$ ./gradlew bootRun
```

- 영상: <https://youtu.be/hT8DrtPlWgA>

메이븐과 마찬가지로 운영체제 환경변수를 유용하게 사용할 수 있다.

```
$ export JAVA_OPTS=-Xmx1024m -XX:MaxPermSize=128M
```

TIP

선언한 운영체제 환경변수는 다음 방법으로 해제가능하다.

```
$ export JAVA_OPTS
```

2.7.4. 메이븐 플러그인으로 실행하기

스프링 부트 메이븐에는 **run** 골이 추가되어 있어 애플리케이션을 빠르게 컴파일하고 실행할 수 있다. 애플리케이션 실행은 IDE에서 실행하는 것과 유사하다.

```
$ ./mvnw spring-boot:run
```

- 영상: <https://youtu.be/L5-9TnBtYRU>

또한 운영체제 환경변수를 사용할 수 있다:

```
$ export MAVEN_OPTS=-Xmx1024m -XX:MaxPermSize=128M
```

2.7.5. 핫 스와핑

스프링 부트 애플리케이션은 정말 순수한 자바 애플리케이션이다. JVM 핫 스와핑은 즉시 사용가능하지만 바이트코드를 대체하는 정도로 사용이 제한되기 때문에, 이를 보다 완벽하게 지원할 수 있는 방법으로 [JRebel](#) 혹은 [Spring Loaded](#) 프로젝트를 사용할 수 있다.

"[스프링 부트' 개발자도구](#)"과 "[핫스와핑\(Hot swapping\)](#)"에서 상세한 내용을 살펴볼 수 있다.

NOTE

가벼운 애플리케이션을 개발하는 것이라면 "[스프링 부트' 개발자도구](#)" 만으로도 충분히 만족스런 효과를 얻을 수 있다. JRebel을 사용하다가 개발자도구 생각이 나서 사용을 해봤는데 큰 불편함이 없었다.

2.8. '스프링 부트' 개발자도구

스프링 부트는 애플리케이션 개발경험을 향상시키는데 도움이 되는 도구들을 포함하고 있다. [spring-boot-devtools](#) 모듈은 어느 프로젝트에서든 개발할 때 필요한 기능들을 제공하고 있다. 개발자도구를 포함시키는 방법은 모듈 의존성을 빌드 스크립트에 추가하면 된다:

그레이들 [build.gradle](#)

```
dependencies {  
    compile("org.springframework.boot:spring-boot-devtools")  
}
```

메이븐 [pom.xml](#)

```
<dependencies>  
    <dependency>  
        <groupId>org.springframework.boot</groupId>  
        <artifactId>spring-boot-devtools</artifactId>  
        <optional>true</optional>  
    </dependency>  
</dependencies>
```

NOTE

개발자도구는 **패키징된 애플리케이션이 실행될 때 자동으로 비활성화**된다. 애플리케이션 실행시 [java -jar](#) 혹은 별도의 클래스로더를 통해서 시작됐다면 "운영 애플리케이션" 이라고 판단한다. 의존성을 옵션([optional](#))으로 표시하는 것은 프로젝트가 다른 모듈에 개발자도구를 적용하는 것을 방지하기 위해서다. 그레이들에서는 [optional](#) 의존성 제한 기능을 제공하지 않는데 혹시나 그런 기능이 필요하다면 [propdeps-plugin\(https://goo.gl/4ODlcs\)](#) 을 살펴보기 바란다.

TIP

기본적으로 리패키징된 압축파일에는 개발자도구가 포함되지 않는다. "[원격 개발자도구 기능을 포함](#)"하고 싶다면, [excludeDevtools](#) 빌드속성을 비활성화해야한다. 이 속성에 대해서는 메이븐과 그레이들 플러그인에서 지원하고 있다.

2.8.1. 속성 기본정의

스프링 부트가 지원하는 여러 라이브러리는 캐시를 사용하면 더욱 효과적이다. 예를 들어, "템플릿 엔진"이 컴파일된 템플릿을 캐시처리하면 템플릿 파일을 반복적으로 파싱하는 것을 피할 수 있다. 또한, 스프링 MVC는 정적자원을 제공할 때 HTTP 캐싱 헤더를 추가할 수 있다.

캐싱은 운영할 때는 매우 유용하지만, 개발할 때는 애플리케이션의 변경사항을 바로 볼 수 없게 만들어 생산성을 저하시킬 수도 있다. 그런 이유로, **spring-boot-devtools**는 기본적으로 캐시 처리를 비활성화한다.

캐시에 대한 활성화 여부는 **application.yml** 파일에서 언제든지 구성이 가능하다. 예를 들어 타임리프의 캐싱 처리와 관련해서 **spring.thymeleaf.cache** 속성을 제공한다. **spring-boot-devtools**은 이러한 속성들을 수동으로 설정하지 않아도 되도록 개발시 구성을 적절한 타이밍에 적용한다.

TIP

이러한 이유로, 스프링 부트에서 사용가능한 라이브러리의 속성들은 대부분 **비활성화 상태**를 기본으로 하고 있다(개발시 필요한 부분들에 대해서는 활성화 되어있다). 운영에 들어갈 때에는 사용하는 라이브러리의 속성을 점검하고 필요한 기능에 대해서는 활성화할 수 있도록 설정해야 한다. 이와 관련해서는 "**외부 구성(Externalized Configuration)**" 부분에서 설명하겠다.

사용하는 기능에 대한 구성과 관련된 속성은 프로파일에 따라 명시적으로 선언해주는 것이 좋다.

NOTE

개발자도구 에서 비활성화 처리하는 목록을 보고 싶다면 **DevToolsPropertyDefaultsPostProcessor** 을 살펴보면 된다.

2.8.2. 자동 재시작

애플리케이션에서 **spring-boot-devtools**을 사용하면 클래스패스 내에 파일이 변경되면 자동으로 재시작된다. 이 기능은 IDE에서 작업 시 코드가 변경된 결과를 빠르게 확인할 수 있어 매우 유용하다. 기본적으로는 클래스패스를 기준으로 하위에 있는 파일들의 변경사항을 살피게 된다. 정적 자원과 뷰 템플릿 과 같은 자원이 변경되었을 때(클래스패스 하위에 있다면)는 **재시작할 필요가 없다**.

재시작 트리거

개발자도구는 클래스패스를 지켜보다가 클래스패스가 갱신되면 재시작 트리거가 동작한다. 클래스패스를 이용하는 방법은 사용하는 IDE에 따라 달라진다. Eclipse 의 경우에는 파일을 변경하고 저장하면 클래스패스파일이 변경되면서 재시작 트리거가 동작한다. 인텔리제이에서는 프로젝트를 빌드(**Build** → **Make Project**)하면 동일한 효과를 볼 수 있다.

NOTE

그래서 인텔리제이는 빌드를 저장하는 매크로를 이용해야 한다. 코바님의 블로그에 다음 내용을 살펴보자.

- [IntelliJ 에서 Spring Devtools 활용하기 팁\(http://sbcoba.tistory.com/36\)](http://sbcoba.tistory.com/36)

TIP

개발자도구는 재시작 동안에는 애플리케이션 컨텍스트 셧다운 혹은 잠궜다. 셧다운 혹은 비활성화 하면 이 기능이 동작하지 않는다(**SpringApplication.setRegisterShutdownHook(false)**).

TIP

개발자도구는 클래스패스가 변경되었을 때 `spring-boot`, `spring-boot-devtools`, `spring-boot-autoconfigure`, `spring-boot-actuator`과 `spring-boot-starter` 같은 이름을 가진 프로젝트를 제외하고 재시작되는 진입점을 결정한다.

재시작 vs 재적재

스프링 부트 에서 제공하는 재시작 기술은 두 가지 클래스로더를 사용한다. 클래스가 변경되지 않았을 때(예: 서드파티 jar 에 있는 클래스)는 '**기본 클래스로더**'에 적재되어 있다. 개발하는 과정에서 변경된 클래스는 '**재시작 클래스로더**'에 적재되어 있다. **애플리케이션이 재시작될 때 기존 '재시작 클래스로더'를 던져버리고 새로운 것을 만든다.** 이 접근법을 통해서 '기본 클래스로더'가 유지되고 있고 필요한 것들을 가지고 있기 때문에 애플리케이션 재시작은 꺾다 켜는 것에 비해 매우 빠르게 처리된다.

만약에 애플리케이션 재시작 속도가 만족스럽지 못하거나 클래스적재와 관련된 문제가 생긴다면 ZeroTurnaround의 [JRebel](https://zeroturnaround.com/software/jrebel/)(<https://zeroturnaround.com/software/jrebel/>) 사용을 고려해보길 바란다. 클래스가 적재될 때 클래스를 재작성하여 적재하기 쉽도록 만드는 방식으로 동작한다. [스프링 Loaded](https://github.com/spring-projects/spring-loaded) (<https://github.com/spring-projects/spring-loaded>)는 다른 선택사항들을 제공하지만 많은 프레임워크를 지원하지 않으며 상업적인 지원도 없다.

P.S. 돈이 넉넉하다면 **JRebel**을 써보는 것도 나쁘지 않다. 30일 체험판 사용해보고 마음에 들면 구!매!해보자.

재시작 대상 제외

확실히 리소스는 변경사항이 생겨도 재시작할 필요가 없다. 예를 들어, 타임리프 템플릿은 그 위치에서 바로 수정할 수 있다. 기본적으로 `/META-INF/maven`, `/META-INF/resources`, `/resources`, `/static`, `/public` 혹은 `/templates`의 자원이 변경되어도 트리거는 동작하지 않는다. `spring.devtools.restart.exclude` 속성을 사용해서 제외 대상을 재정의할 수 있다. 예를 들어 `/static` 과 `/public` 만 제외하고 싶을 때는 다음과 같이 설정할 수 있다:

```
spring.devtools.restart.exclude: static/**,public/**
```

TIP

기본 제외목록을 유지하면서 추가적으로 제외하고 싶다면 `spring.devtools.restart.additional-exclude`을 대신 사용하면 된다.

재시작 감시경로 추가

클래스패스 이외의 경로에서 변경사항이 생겼을 때 애플리케이션을 재시작하고 싶은 경우가 있다. 그 때는 `spring.devtools.restart.additional-paths` 속성을 정의하면 변경사항이 발생하는 지점을 감시할 수 있다.

재시작 비활성화

재시작 기능을 사용하고 싶지 않다면 `spring.devtools.restart.enabled` 속성을 정의한다. 대부분의 경우에는 `application.properties` 혹은 `application.yml` 에 설정할 것이다(재시작 클래스로더가 초기화될 때 파일 변경사항을 감시하지 않는다).

예를 들어 특정 라이브러리에는 적용되지 않는 경우가 있는데 이런 경우에 **완벽하게** 재시작 기능을 비활성화하고 싶다면

`SpringApplication.run(...)` 이 호출되기 전에 `System` 속성을 설정해야 한다. 예를 들어:

```
public static void main(String[] args) {
    System.setProperty("spring.devtools.restart.enabled", "false");
    SpringApplication.run(`BootSpringBootApplication`.class, args);
}
```

트리거파일 사용

IDE를 이용해서 작업한다면 변경된 파일은 지속적으로 컴파일되는데, 특정 시점에만 재시작 트리거가 동작하길 원할 것이다. 이를 위해서 특정 파일이 변경되었을 때 재시작을 확인하는 "트리거 파일" 사용할 수 있다. 파일이 변경되면 트리거 검사가 시작되고 개발자도구가 뭔가를 해야한다는 것을 감지한 경우에만 재시작된다. 트리거 파일은 수동으로 갱신하거나 IDE 플러그인을 이용할 수 있다.

`spring.devtools.restart.trigger-file` 속성을 정의하여 트리거 파일을 정의할 수 있다.

2.8.3. 전역 설정

홈디렉토리(`$HOME`)에 `.spring-boot-devtools.properties` 라는 이름의 파일을 추가하면 개발자도구 에 대한 전역설정을 할 수 있다(파일이름이 '.'으로 시작해야 한다는 것을 기억하기 바란다.). 이 파일에 속성들을 추가하면 개발자도구를 사용하는 모든 스프링 부트 애플리케이션이 이 속성을 적용한다. 예를 들어, 다음과 같이 항상 트리거 파일을 사용하여 재시작하도록 설정할 수 있다:

`~/.spring-boot-devtools.properties`

```
spring.devtools.reload.trigger-file=.reloadtrigger
```

NOTE

파일명을 "."으로 시작하는 이유는, MacOS 와 Linux 와 같은 Unix 계열의 운영체제에서는 숨김 (Hidden)파일로 인식하기 때문이다. 파일 조회나 탐색기 등에서 노출되지 않는다는 특성을 가지게 된다.

2.8.4. 원격 애플리케이션

스프링 부트 개발자도구 는 로컬 개발에만 국한되지 않는다. 원격지에서 애플리케이션 이 실행되고 있을 때에도 여러 가지 기능을 사용할 수 있다. 리팩키지할 때 개발자도구를 포함하도록 처리하면 원격지원 기능을 추가할 수 있다.

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <excludeDevtools>false</excludeDevtools>
      </configuration>
    </plugin>
  </plugins>
</build>
```

```
bootRepackage {
    excludeDevtools = false
}
```

개발자도구를 추가하는 경우에는 `spring.devtools.remote.secret` 속성을 설정해야 한다. 예를 들어:

```
spring.devtools.remote.secret: mysecret
```

WARNING

원격 애플리케이션 에서 `spring-boot-devtools`를 사용하면 보안위험이 생겨난다. 가급적이면 운영 배포시에는 기능을 **활성화하지 말라**.

원격 개발자도구 지원은 두 가지 방법을 제공한다. 하나는 서버쪽 종단점에 접근하는 방법이 있고, 다른 하나는 IDE 클라이언트 애플리케이션을 실행하는 것이다. 서버 컴포넌트는 `spring.devtools.remote.secret` 속성을 설정하면 자동으로 설정된다. 클라이언트 컴포넌트는 수동으로 실행해야 한다.

```
spring.devtools.remote.secret=mysecret
```

2.9. 출시를 위한 애플리케이션 패키징

{executable-jar}는 운영배포에 사용할 수 있다. 컨테이너를 포함하고 있기 때문에 클라우드 기반 배포에 최적화되어 있다.

스프링 부트는 "운영준비"를 위한 기능으로 헬스체크, 감시 및 측량 종단점에 대한 REST 혹은 JMX 방식으로 접근할 수 있는 기능을 `spring-boot-actuator`를 통해서 제공하니 운영을 고려할 때 모듈추가를 고려해보자. 보다 자세한 내용은 '[스프링 부트](#)' 출시준비: 액츄에이터 을 살펴보기 바란다.

2.10. 다음 읽을꺼리

스프링 부트를 어떻게 사용하는지 살펴보았다. "'스프링 부트' 기능소개"을 보다 깊이 살펴볼 예정인데 "'스프링 부트' 출시준비: 액츄에이터"에 관심이 있다면 넘어가면 된다.

Chapter 3. '스프링 부트' 기능소개

이번 장에서는 스프링 부트를 자세히 살펴본다. 이 과정에서 개발자가 사용하거나 재정의하고 싶은 기능을 파악할 수 있다. 아직 준비가 되지 않았다고 생각한다면 '스프링 부트' 시작하기와 '스프링 부트' 살펴보기에서 배경지식을 갖춘 후에 읽기 시작하는 것이 좋다.

스프링 부트의 진입점(Entry point) `SpringApplication`을 시작으로, 패키징된 애플리케이션을 배포환경에 따라 별도로 빌드할 필요없이 배포환경에 따라 각각의 속성을 부여하는 방법, 프로파일을 통해서 '로컬','테스트','개발','운영' 등의 실행환경을 구분하고, 다양한 서드파티 생태계와 연동하고, 필요에 따라 구성(Configuration)을 작성하는 과정 등을 살펴보게 된다. 이 장을 읽으면서 스프링 부트의 매력에 흠뻑 취하게 될 것이다. 팔라가 될지도 모른다.

3.1. 스프링애플리케이션(SpringApplication)

`SpringApplication` 클래스는 스프링 애플리케이션 실행시 `main()` 메서드를 통해 여러가지 편의를 제공한다. 여러 가지 상황 속에서 기본적으로 사용되는 `SpringApplication.run` 메서드는 다음과 같은 형태를 띈다:

```
public static void main(String[] args) {
    SpringApplication.run(BootSpringBootApplication.class, args);
}
```

애플리케이션이 실행되면 다음과 유사한 코드를 볼 수 있다.

```
.
/\ \ / ---' _ _ _ _ _ ( _ ) _ _ _ _ _ \ \ \ \ \
( ( ) \ _ _ | ' _ | ' _ | ' _ \ / _ ' | \ \ \ \
\ \ / _ _ _ ) | | _ ) | | | | | | | ( _ | | ) ) )
' | _ _ _ | . _ _ | | _ _ | | _ _ , | / / / /
=====|_|=====| _ _ / = / _ / _ /
:: Spring Boot :: v2.0.0.RELEASE

2017-05-31 06:48:03.250 INFO 3925 --- [ main]
i.h.s.boot.BootSpringBootApplication : Starting
BootSpringBootApplication on honeymon-ThinkPad with PID 3925

// 애플리케이션 시동과 관련된 로그가 출력된다.

2017-05-31 06:48:11.694 INFO 3925 --- [ main]
i.h.s.boot.BootSpringBootApplication : Started
BootSpringBootApplication in 10.572 seconds (JVM running for 12.216)
```

애플리케이션이 시작되면서 보여주는 INFO 로깅 메시지로 보여주는 것은 애플리케이션 시작과 관련된 상세한 정보를 포함하고 있다.

3.1.1. 시동 실패

애플리케이션이 시동 중 실패하면, 등록되어있는 **FailureAnalyzers** 를 통해서 발생한 문제를 출력하고 이 문제를 해결할 수 있는 기회를 제공한다. 예를 들어, **8080**포트를 이미 사용하고 있는 상황에서 동일한 포트를 사용하는 웹 애플리케이션을 시작한다면 다음과 같은 메시지를 보게된다.

```
*****
APPLICATION FAILED TO START
*****
```

Description:

Embedded servlet container failed to start. Port 8080 was already in use.

Action:

Identify and stop the process that's listening on port 8080 or configure this application to listen on another port.

TIP

스프링 부트는 다수의 **FailureAnalyzer** 구현체를 제공하고 있으며 필요에 따라 쉽게 추가할 수 있다.
[Create your own FailureAnalyzer](#)

실패를 알려줄 실패분석기가 없다하더라도, 자동구성 보고서를 통해 어디가 잘못되었는지 살펴볼 수 있으면 유용하다. 이를 위해

org.springframework.boot.autoconfigure.logging.AutoConfigurationReportLoggingInitializer 와 관련된 **debug** 속성을 활성화하면 된다.

예를 들어, **java -jar** 명령으로 애플리케이션 실행하면 **debug** 속성을 활성화할 수 있다.

```
$ java -jar myproject-0.0.1-SNAPSHOT.jar --debug
```

혹은 **application.yml** 속성에 **debug** 속성을 정의해도 된다.

```
debug: true
```

3.1.2. 배너 재정의

실제 서비스 운영과는 크게 관련이 없지만(!), 개발자들이 크게 환호할 만한 기능이 하나 있다. 바로 **배너 재정의** 기능이다. 애플리케이션이 시동될 때 출력되는 배너에 왜 그렇게 열광하는 지 이해가 잘 되지 않는다.

NOTE

그렇게 말하면서, 설현 사진을 올려서 배너에 출력되는 것을 보며 자랑스럽게 캡처해서 올려 본 적이 있다...

banner.txt 파일을 클래스패스 혹은 특정 위치에 두고 **banner.location**로 정의하면 된다. 기본 인코딩(**UTF-8**)을

사용하지 않고 별도의 인코딩을 사용할 경우에는 `banner.charset`을 설정한다. 텍스트 파일 외에도 `banner.gif` `banner.jpg` 혹은 `banner.png` 이미지를 클래스패스 혹은 `banner.location` 속성으로 정의한 곳에 위치시키면 이미지 파일을 아스키(ASCII) 형식으로 텍스트 배너처럼 출력한다.

`banner.txt` 파일 내에 다음과 같은 위치변환자를 사용수 있다:

Table 1. 배너 변수

변수	설명
<code>\${application.version}</code>	MANIFEST.MF 에 정의한 애플리케이션 버전. 예를 들어 <code>Implementation-Version: 1.0</code> 이라고 정의하면 <code>1.0</code> 이 출력된다.
<code>\${application.formatted-version}</code>	MANIFEST.MF 에 정의한 애플리케이션 버전을 정형화(괄호로 감싸고 <code>v</code> 접두사 추가)된 형태, 예를 들면 <code>(v1.0)</code> 으로 출력된다.
<code>\${spring-boot.version}</code>	스프링 부트 버전을 이용한다. 예를 들어 <code>2.0.0.RELEASE</code> 가 출력된다.
<code>\${spring-boot.formatted-version}</code>	스프링 부트 버전을 정형화(괄호로 감싸고 <code>v</code> 접두사 추가)된 형태, 예를 들어 <code>(v2.0.0.RELEASE)</code> 으로 출력된다.
<code>\${Ansi.NAME}</code> (혹은 <code>\${AnsiColor.NAME}</code> , <code>\${AnsiBackground.NAME}</code> , <code>\${AnsiStyle.NAME}</code>)	NAME 은 ANSI 이스케이프 코드의 이름이다.
<code>\${application.title}</code>	MANIFEST.MF 에 정의한 애플리케이션 이름. 예를 들자면, <code>Implementation-Title: BootSpringBoot</code> 라고 정의하면 <code>BootSpringBoot</code> 가 출력된다.

TIP

`SpringApplication.setBanner(...)` 메서드를 이용해서 배너 생성을 프로그래밍할 수 있다. `org.springframework.boot.Banner` 인터페이스의 `printBanner` 메서드를 구현하면 된다.

`spring.main.banner-mode` 속성을 사용하여 배너 출력방식을 설정할 수 있다.

```
spring.main.banner-mode: console # System.out 으로 출력
spring.main.banner-mode: log    # Logger 로 출력
spring.main.banner-mode: off    # 배너 출력 끄기
```

배너 출력은 `springBootBanner` 라는 이름의 빈으로 등록할 수 있다.

3.1.3. SpringApplication 재정의

`SpringApplication` 은 기본적인 설정이 마음에 들지 않을 때는 로컬 인스턴스를 만들면서 사용자정의할 수 있다. 예를 들어, 배너를 끄려면 다음과 같이 작성하면 된다:

```
public static void main(String[] args) {
    SpringApplication app = new SpringApplication
    (BootSpringBootApplication.class);
    app.setBannerMode(Banner.Mode.OFF);
    app.run(args);
}
```

또한, 실행가능한 jar 형태로 실행할 때 유용한 것 중에 하나가 애플리케이션 PID 파일을 생성하는 것인데 다음과 같이 작성하면 된다:

```
public static void main(String[] args) {
    SpringApplication app = new SpringApplication
    (BootSpringBootApplication.class);
    app.addListeners(new ApplicationPidFileWriter());
    app.run(args);
}
```

NOTE

SpringApplication가 전달하는 생성인자들은 스프링 빈을 위한 구성요소다. 대부분 **@Configuration**에서 참조하는 편이지만, XML 구성을 참조하거나 패키지를 스캔할 수도 있다.

또한 **SpringApplication**을 구성할 때 **application.yml(혹은 properties)** 파일을 사용할 수 있다. **외부 구성(Externalized Configuration)**에서 보다 자세한 내용을 확인할 수 있다.

3.1.4. 유연한(Fluent) 빌더 API

ApplicationContext 계층(hierarchy, 부모/자식 관계의 다중 컨텍스트)을 구성해야하거나, 유연한(**Fluent**)형태의 빌더 API를 선호한다면 **SpringApplicationBuilder**를 사용할 수 있다.

SpringApplicationBuilder는 **parent**와 **child** 메서드를 포함하고 있어 계층을 구성하거나 여러 메서드를 연이어 호출수 있다. 예를 들어:

```
new SpringApplicationBuilder()
    .sources(Parent.class)           ①
    .child(Application.class)        ②
    .bannerMode(Banner.Mode.OFF)    ③
    .run(args);                      ④
```

- ① 애플리케이션 에 추가하려는 구성 클래스와 컴포넌트
- ② 자식 컨텍스트지정
- ③ 배너모드 비활성화
- ④ 실행인자 전달

NOTE

`ApplicationContext` 계층을 생성할 때 몇가지 제약이 있다. 예를 들자면, 웹컴포넌트는 하위 컨텍스트 내에 포함되어야 하고 부모와 자식 컨텍스트에는 동일한 `Environment`가 사용된다. 보다 자세한 내용은 `SpringApplicationBuilder Javadoc`에서 살펴볼 수 있다. 꼭 살펴보길 바란다.

3.1.5. 애플리케이션 이벤트 및 리스너

`SpringApplication`은 `ContextRefreshedEvent`와 일반적인 스프링 프레임워크 이벤트 외에도 추가적인 이벤트를 전송한다.

NOTE

몇 가지 이벤트는 `ApplicationContext` 전에 발생하기 때문에 `@Bean`처럼 리스너로 등록할 수 없다. `SpringApplication.addListeners(...)` 혹은 `SpringApplicationBuilder.listeners(...)` 메서드를 이용해서 등록할 수 있다. 만약에 리스너가 자동으로 등록되길 바란다면 프로젝트 내에 `META-INF/spring.factories` 파일을 추가하고 `org.springframework.context.ApplicationListener`를 키로 해서 리스너들을 참조하도록 하면 된다.

```
#Listener
org.springframework.context.ApplicationListener=\
io.honeymon.springboot.boot.listner.MyEventListener
```

애플리케이션이 실행될때 애플리케이션 이벤트는 다음 순서로 발생한다: `ApplicationStartingEvent`는 구동후 리스너와 초기화가 등록되기 전에 발생한다. `ApplicationEnvironmentPreparedEvent`는 `Environment`가 사용할 컨텍스트를 알고 있지만 컨텍스트가 생성되기 전에 전송한다. `ApplicationPreparedEvent`는 정의한 빈이 적재된 후 새로고침이 진행되기 전에 전송한다. `ApplicationReadyEvent`는 새로고침 후 관련된 콜백이 처리된 후 애플리케이션이 서비스 요청을 처리할 준비가 되었을 때 전송한다. `ApplicationFailedEvent`는 시동 중에 예외가 발생했을 때 전송한다.

TIP

굳이 애플리케이션 이벤트를 사용하지 않아도 되지만, 이벤트가 있었다는 것을 알면 유용하다. 스프링 부트는 내부적으로 이벤트를 통해 다양한 작업을 처리한다.

`ApplicationListener`는 다음과 같이 작성되어 있는 인터페이스다. 이벤트가 발생하면 `onApplicationEvent` 메서드로 `ApplicationEvent`가 전달된다. `ApplicationEvent`는 `EventObject`를 확장한 추상클래스다.

```
public interface ApplicationListener<E extends ApplicationEvent> extends
EventListener {
    /**
     * Handle an application event.
     * @param event the event to respond to
     */
    void onApplicationEvent(E event);
}
```

이벤트에 대한 리스너는 다음처럼 작성할 수 있다:

```

@Slf4j
public class ApplicationStartingEventListener implements
ApplicationListener<ApplicationStartingEvent> {
    @Override
    public void onApplicationEvent(ApplicationStartingEvent event) {
        log.info("Application Start: {}", event);
    }
}

@Slf4j
public class ApplicationReadyEventListener implements ApplicationListener
<ApplicationReadyEvent> {

    @Override
    public void onApplicationEvent(ApplicationReadyEvent event) {
        log.info("Application ready: {}", event);
    }
}

```

이 리스너들을 자동으로 등록하려면 다음처럼 `META-INF/spring.factories`를 작성하면 된다.

```

#Listener
org.springframework.context.ApplicationListener=\
io.honeymon.springboot.boot.listener.ApplicationStartingEventListener,\
io.honeymon.springboot.boot.listener.ApplicationReadyEventListener

```

TIP 여러 개를 등록할 때는 쉼표(,)로 구분하며 줄을 변경시에는 \를 사용한다.

3.1.6. 웹 환경

`SpringApplication`은 사용자에게 유용한 `ApplicationContext` 를 만드려한다. 기본적으로, 웹애플리케이션이나 일반애플리케이션을 개발하는지에 따라 `AnnotationConfigApplicationContext` 혹은 `AnnotationConfigEmbeddedWebApplicationContext`을 사용한다.

'웹 환경'을 구분짓는 방법은 정말 간단하다(몇 가지 클래스를 통해 식별). 기본 방법을 수동으로 설정하고자 한다면 `setWebEnvironment(boolean webEnvironment)`를 사용하면 된다.

`setApplicationContextClass(...)`를 호출하면 `ApplicationContext` 유형을 완벽하게 제어하는 것이 가능하다.

TIP `SpringApplication`를 JUnit 테스트할 때는 `setWebEnvironment(false)`가 유용하다.

3.1.7. 애플리케이션 실행인자에 접근

애플리케이션 구동시 전달받은 실행인자에 접근하고 싶을 때는 `SpringApplication.run(...)`
`org.springframework.boot.ApplicationArguments` 빈을 이용하면 된다. `ApplicationArguments`
인터페이스는 원본 `String[]` 인자를 `option`과 `non-option` 인자로 분류하여 접근할 수 있는 기능을 제공한다.

```
@Slf4j
@Component
public class AppArgsAccessor {

    @Autowired
    public AppArgsAccessor(ApplicationArguments args) {
        boolean debug = args.containsOption("debug");
        log.debug("debug={}", debug);
        List<String> files = args.getNonOptionArgs();
        log.debug("files={}", files);
        // "--debug logfile.txt"라는 실행인자를 주어 애플리케이션을 실행하면
        debug=true, files=["logfile.txt"]를 확인해볼 수 있다.
    }
}
```

위와 같이 작성한 후 `java -jar boot-spring-boot-1.0.0.RELEASE.jar --debug logfile.txt` 으로 실행하면 출력되는 로그 중 다음과 같이 출력되는 것을 발견할 수 있다.

```
2017-06-03 12:31:59.261 DEBUG 24838 --- [          main]
i.h.s.boot.support.AppArgsAccessor      : debug=true
2017-06-03 12:31:59.264 DEBUG 24838 --- [          main]
i.h.s.boot.support.AppArgsAccessor      : files=[logfile.txt]
```

TIP

스프링 부트는 스프링에 `Environment`에 `CommandLinePropertySource`를 등록한다. 이를 통해서 애플리케이션 실행인자를 `@Value` 애너테이션을 통해 사용할 수 있다.

3.1.8. `CommandLineRunner` 혹은 `ApplicationRunner` 사용하기

스프링 부트 애플리케이션 이 실행될 때 `SpringApplication` 와 함께 실행(`run`)되도록 빈을 정의할 수 있는 인터페이스가 `CommandLineRunner` 과 `ApplicationRunner` 이다. 둘의 차이점이라면 `CommandLineRunner`는 애플리케이션 실행인자를 문자열로 받는데 반해 `ApplicationRunner`는 `ApplicationArguments` 배열로 받는다는 것이다.

이 인터페이스의 용도는 애플리케이션 이 실행될 때 반드시 한번 실행되어야 하는 기능을 구현하는데 사용할 수 있다. 하나의 컴포넌트로 인식되기 때문에 애플리케이션 컨텍스트를 공유한다.

```
@Slf4j
@Component
public class CmdRunner implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
        log.debug("Run CmdRunner.");
    }
}
```

```
@Slf4j
@Component
public class AppRunner implements ApplicationRunner {
    @Override
    public void run(ApplicationArguments args) throws Exception {
        log.debug("Run AppRunner.");
    }
}
```

`org.springframework.core.Ordered` 인터페이스를 구현하거나 `org.springframework.core.annotation.Order` 애너테이션을 `CommandLineRunner` 혹은 `ApplicationRunner` 등의 여러 빈에 정의하여 특정순서로 호출되도록 정의할 수 있다.

3.1.9. 애플리케이션 종료

`SpringApplication`는 종료호출을 통해서 `ApplicationContext`를 보호하면서 깔끔하게 종료할 수 있다. 표준 스프링 라이프사이클 콜백(`DisposableBean`를 구현했거나 `@PreDestroy` 애너테이션을 사용한)이 사용된다.

추가적으로 애플리케이션이 종료되었을 때 지정된 종료 코드를 반환하도록 `org.springframework.boot.ExitCodeGenerator` 인터페이스를 구현할 수도 있다.

3.2. 외부 구성(Externalized Configuration)

스프링 부트는 동일한 애플리케이션을 각기 다른 환경에서 동작할 수 있도록 애플리케이션 구성을 외부에서 조작할 수 있다. YAML 파일, properties 파일, 환경변수 그리고 커맨드라인 실행인자를 이용해 외부 구성을 사용할 수 있다. 이 속성값은 `@Value` 애너테이션을 통해서 스프링 빈에 주입하거나 스프링 `Environment`를 이용해서 접근하거나 `@ConfigurationProperties`를 통해 **구조화된 객체**내에 연결할 수도 있다.

스프링 부트는 매우 특별한 `PropertySource` 순서에 따라 값을 섬세하게 덮어쓰기할 수 있도록 설계되어있다. 속성은 다음의 순서로 적용된다:

1. 사용자 홈 디렉토리에 있는 **개발자도구 전역 설정 속성**(개발자도구 가 활성화상태일 때 `~/.spring-boot-devtools.properties`이 있다면 적용됨)

2. 테스트에서 `@TestPropertySource` 애너테이션을 선언한 경우
3. 테스트에서 `@SpringBootTest#properties` 애너테이션을 선언한 경우
4. 커맨드라인 실행인자
5. `SPRING_APPLICATION_JSON`로 정의된 속성(JSON 형식을 띄고 환경변수나 시스템속성으로 정의)
6. `ServletConfig` 초기화 파라미터
7. `ServletContext` 초기화 파라미터
8. `java:comp/env`으로 설정한 JNDI 속성
9. 자바 시스템속성(`System.getProperties()`)
10. 운영체제 환경변수(클라우드 플랫폼에서 사용하는 경우 애플리케이션 속성 확장에 사용한다)
11. `random.*` 속성을 가진 `RandomValuePropertySource`
12. 패키징된 jar 외부에서 선언한 `프로파일 정의 속성`(`application-프로파일.properties` 과 YAML 변형)
13. jar 파일 내부에 있는 `프로파일 정의 속성`(`application-프로파일.properties` 과 YAML 변형)
14. 패키징된 jar 외부에서 선언한 애플리케이션 속성들(`application-프로파일.properties` 과 YAML 변형)
15. jar 파일 내부에 있는 애플리케이션 속성들(`application-프로파일.properties` 과 YAML 변형)
16. `@Configuration` 클래스에 있는 `@PropertySource` 애너테이션
17. 기본 속성들(`SpringApplication.setDefaultProperties`을 이용해서 정의한 경우)

name 속성주입 예

```
import org.springframework.stereotype.*
import org.springframework.beans.factory.annotation.*

@Component
public class MyBean {
    @Value("${name}")
    private String name; ①
    // ...
}
```

① `name` 속성의 값을 주입해본다고 가정하자

애플리케이션 클래스패스 상(jar 파일 안에 있는)에서 `application.yml`에 `name`에 대한 기본값을 선언했다고 가정하자.

```
name: Boot Spring Boot
```

`application.yml`에 선언한 `name`에 대한 속성을 새로운 환경에서 실행할 때, 일회성 테스트, 커맨드라인 실행시점

(예: `java -jar boot-spring-boot.jar --name="Boot Spring Boot"`) 등 다양한 상황에서 변경할 수 있다.

커맨드라인 혹은 환경변수에서 JSON 형식으로 속성을 정의하는 `SPRING_APPLICATION_JSON`를 사용할 수 있다. 유닉스 계열 셸에서는

```
$ SPRING_APPLICATION_JSON='{ "foo":{ "bar": "spam" } }' java -jar boot-spring-boot.jar
```

이런 예의 경우 스프링 `Environment` 에는 `foo.bar=spam`으로 등록된다. 또한 시스템 변수 `spring.application.json` 에서도 JSON을 지원한다:

TIP

```
$ java -Dspring.application.json='{ "foo": "bar" }' -jar boot-spring-boot.jar
```

혹은 커맨드라인 실행인자를 지정해도 된다.

```
java -jar boot-spring-boot.jar --spring.application.json='{ "foo": "bar" }'
```

혹은 `java:comp/env/spring.application.json`을 이용해서 JNDI 변수를 선언할 수 있다.

3.2.1. 무작위(Random)값 구성

`RandomValuePropertySource`는 무작위값을 주입하는데 유용하다(테스트 혹은 비밀을 위한 무작위값). 32비트 정수(int), 64비트 정수(long), UUID(Universally Unique Identifier, <http://www.ietf.org/rfc/rfc4122.txt>) 혹은 문자열을 제공한다.

```
my:
  secret: ${random.value}           ①
  number: ${random.int}
    less.than.ten: ${random.int(10)} ②
    in.range: ${random.int[1024,65536]}
  bignumber: ${random.long}
  uuid: ${random.uuid}
```

① `my.secret` 속성키

② `my.number.less.than.ten` 속성

`random.int*` 문법에서 `OPEN value (, max) CLOSE` 로 사용가능하며 `OPEN`, `CLOSE` 는 어떤 문자나 가능하고 `value`, `max`는 정수여야 한다. 만약 `max` 값을 등록하면 `value`는 최소값이 되고 `max`가 최대값이 된다.

3.2.2. 커맨드라인 속성 접근

`SpringApplication`은 기본적으로 어떤 커맨드라인 실행인자(`--server.port=9000`처럼 `--`로 시작하는)를 스프링 `Environment`에 속성으로 추가한다. 앞에서 언급했듯이 커맨드라인 속성은 언제나 다른 유형의 속성정의보다 우선한다.

만약 커맨드라인 실행인자가 `Environment`에 추가되는 것을 비활성화하려면

`SpringApplication.setAddCommandLineProperties(false)`를 사용하라.

3.2.3. 애플리케이션 속성 파일

`SpringApplication`은 다음 위치에 있는 `application.yml(or properties)` 파일을 읽어서 `Environment`에 속성을 재한다:

1. `/config`와 하위 디렉토리
2. 현재 디렉토리
3. 클래스패스 `/config` 패키지
4. 클래스패스

목록은 우선순위에 따라 정렬되어 있다(목록의 상위 위치에 정의된 속성은 하위 위치에 정의된 속성보다 우선한다).

NOTE

기본적으로 `$PROJECT_HOME/src/main/resources/application.yml(== CLASSPATH/application.yml)`에 위치하도록 하는 편이다.

스프링 부트 참조문서에서는 `.properties`파일을 기준으로 하고 있지만, 나는 YAML 파일(`.yaml`)을 이용해서 속성정의를 선호하는 편이다. `.properties`파일로 작성할 경우 프로파일에 따라서서 구성파일이 여러 개로 나뉘어야 하는데 반해, `.yaml`은 하나의 문서파일 내에서 `---` 구분자를 이용해서 각각의 프로파일을 구분지을 수 있어 한번에 문서를 파악할 수 있으며 프로파일에 따라 덮어쓰는 속성들을 파악하기가 용이하다.

혹시나 `application.yml`이라는 구성파일명이 마음에 들지 않는다면 `spring.config.name`을 이용해서 다른 이름으로 대체할 수 있다. 또한 `spring.config.location` 속성을 이용해서 특정 위치를 참조하도록 할 수 있다 (파일이나 디렉토리가 여러개인 경우에는 쉼표(,)로 구분짓는다).

```
$ java -jar boot-spring-boot.jar --spring.config.name=myproject
```

혹은

```
java -jar boot-spring-boot.jar
--spring.config.location=classpath:/default.properties,classpath:/override
.properties
```

WARNING

`spring.config.name`과 `spring.config.location`은 애플리케이션 시동시에 어떤 파일을 적재해야하는지 결정하기 위해 매우 일찍 사용되므로 환경속성(운영체제 환경변수, 시스템 속성 또는 커맨드라인 실행인자)으로 정의해야 한다. `application.yml(or .properties)`에서 정의하는 것은 반영되지 않는다.

`spring.config.location`에 정의된 디렉토리는 `/`으로 끝나야 한다(그래야 적재되기 전에 `spring.config.name`에서 읽어온, 프로파일로 정의된 파일을 포함하여, 파일들이 추가된다). `spring.config.location`으로 정의한 특정 파일명까지 상세히 정의한 경우에는 프로파일 변환과 속성 덮어쓰기 기능들을 사용할 수 없다.

NOTE

대부분의 운영체제에서는 마침표(`.`)로 구분된 키를 사용할 수 없기 때문에, 시스템 속성이 아닌 환경변수를 사용할 때는 마침표 대신 밑줄(`_`)을 사용한다(예: `system.config.name` 대신 `SPRING_CONFIG_NAME`).

```
$ java -jar boot-spring-boot.jar --SPRING_CONFIG_NAME=myproject
```

TIP

클라우드 플랫폼을 사용하는 경우에는 커맨드라인 실행인자를 전달하는 대신 사전에 운영체제 환경변수를 정의하는 상황이 발생한다. 운영체제 환경변수를 정의해야하는 경우에는 밑줄(`_`)로 환경변수를 정의하는 것을 기억해두기 바란다.

3.2.4. 프로파일 정의 속성

반면 `application.properties`를 사용하는 경우에는 프로파일 선언을 위해 `application-local.properties`, `application-test.properties`, `application-dev.properties`, `application-prd.properties`처럼 `application-{프로파일}.properties` 형식의 `.properties`파일을 만들어서 각 프로파일에서 사용될 속성을 선언해야 한다(특정 프로파일이 활성화 되지않거나 프로파일에 구분없이 사용할 속성에 대해서는 기본[`default`]을 사용한 `application-default.properties`에 정의해두면 된다).

지정 프로파일 속성파일은 `applicatoin.properties`가 있는 위치와 동일해야 하며, 지정 프로파일 속성파일은 패키징된 jar 파일의 내외부 관계없이 다른 비지정 프로파일 속성파일을 대체한다(`--spring.profiles.active=프로파일` 선언으로 지정된 프로파일이 우선한다).

만약 여러개 프로파일을 지정하는 경우 마지막 프로파일이 적용된다. 예를 들자면, `spring.profiles.active`으로 지정한 프로파일은 `SpringApplication` API를 통해 구성된 프로파일 다음에 추가되므로 우선순위가 높아진다.

NOTE

`spring.config.location` 위치에 여러가지 속성파일이 있다고 해서 프로파일 변환이 일어나지는 않는다. `spring.config.location`에 디렉토리를 지정해야 프로파일 변환을 사용할 수 있다. 이에 대해서는 "애플리케이션 속성 파일"에서 언급했다.

NOTE

개인적으로 애플리케이션 외부구성용 속성파일로 `.properties`보다는 `.yml`을 선호한다. `.properties`의 경우는 프로파일에 따라서 파일이 구분해서 작성해야 하는데 `.yml`의 경우는 하나의 파일 안에서 프로파일별로 속성을 정의하고 상속의 개념을 활용할 수도 있기 때문이다. 이에 대해서는 "properties를 대신하여 YAML 사용하기"에서 설명한다.

3.2.5. 속성 내 치환자(placeholder)

`application.yml`에 정의된 값은 `Environment`에 존재할 경우 필터링 되기 때문에 앞서 정의한 값을 다음처럼 사용할 수 있다:

```
app:
  name: Boot Spring boot
  descr: ${app.name} is a Spring Boot Application ①
```

① 순차적으로 값을 등록하기 때문에 순서가 뒤바뀌면 오류가 발생한다.

TIP

이 방법을 스프링 부트 속성에 존재하는 축약변환에도 사용가능하다. 보다 자세한 내용은 [커맨드라인 실행인자 축약사용](#)에서 볼 수 있다.

3.2.6. `properties` 를 대신하여 `YAML` 사용하기

`YAML`은 `JSON`의 확장판이라고 할 수 있다. `YAML`은 계층적으로 구성된 데이터를 사람이 쉽게 읽을 수 있는 형식으로 표현한다. `SpringApplication` 클래스는 클래스패스 상에 `SnakeYAML`을 가지고 있다면 자동으로 `properties`를 대신하여 `YAML`을 지원할 수 있다.

NOTE

스타터(`Starters`)를 사용한다면 `SnakeYAML`은 `spring-boot-starter`를 통해서 자동으로 제공된다.

YAML 적재

스프링 프레임워크는 `YAML` 문서 적재에 사용하기 위한 두가지 편의 클래스를 제공한다.

`YamlPropertiesFactoryBean`는 `Properties`에 `YAML`을 적재하고 `YamlMapFactoryBean`는 `Map`에 `YAML`을 적재한다.

예를 들어, 다음과 같은 `YAML` 문서가 있다면:

```
environments:
  dev:
    url: http://dev.honeymon.io
    name: Development Setup
  prod:
    url: http://honeymon.io
    name: Production App
```

`properties`로 변환되면 다음과 같을 것이다:

```
environments.dev.url=http://dev.honeymon.io
environments.dev.name=Development Setup
environments.prod.url=http://honeymon.io
environments.prod.name=Production App
```

YAML 리스트는 `[index]` 형식으로 표현된다, 예로:

```
my:
  servers:
    - dev.honeymon.io
    - honeymon.io
```

properties 로 변환되면 다음과 같다:

```
my.servers[0]=dev.honeymon.io
my.servers[1]=honeymon.io
```

스프링 `DataBinder` 유틸리티(`@ConfigurationProperties`를 사용)를 사용한 것처럼 속성을 연결할 수 있다. 대상 빈에 `java.util.List`(혹은 `Set`) 그리고 {setter}나 변경가능한 값을 제공하는 것이다. 다음 예에 앞서 작성한 properties 값이 연결될 것이다.

```
@ConfigurationProperties(prefix="my")
public class Config {
    private List<String> servers = new ArrayList<String>(); ①
    public List<String> getServers() {
        return this.servers;
    }
}
```

① `my.servers[]` 값이 연결된다.

NOTE

구성을 리스트로 관리할 때는 우리 예상대로 동작하지 않을 수 있기에 추가적인 관리가 필요하다. 앞선 예에서 `my.servers` 가 여러 곳에서 재정의 될 때, 반드시 `List` 에만 연결되리라 보장할 수는 없다. 더 높은 우선순위를 가진 `PropertySource`가 리스트를 오버라이드할 수 있게 하려면, 다음처럼 하나의 속성으로 정의해야 한다:

```
my:
  servers: dev.bar.com,foo.bar.com
```

스프링 Environment 에서 properties처럼 YAML사용하기

YamlPropertySourceLoader 클래스는 스프링 Environment의 PropertySource 처럼 YAML을 사용할 수 있다. 이는 위치변환자 문법과 @Value 애너테이션을 사용하여 YAML 속성에 접근하는 것을 허용한다.

NOTE 나는 개인적으로 .properties 파일보다는 .yaml 을 이용한 설정을 선호한다. [_\)](#)

YAML 문서로 다중 프로파일 다루기

내가 YAML을 선호하는 주된 이유 중 하나다. 하나의 YAML 문서에서 spring.profiles 키를 이용해서 다중 프로파일을 다룰 수 있다.

```
# default ①
logging:
  level:
    io.honeymon.springboot.boot: DEBUG
server:
  port: 9000
---
spring:
  profiles: local ②
  logging:
    level:
      io.honeymon.springboot.boot: TRACE
---
spring:
  profiles: test ③
server:
  port: ${random.int[8000,9000]}
---
spring:
  profiles: dev ④
---
spring:
  profiles: prod ⑤
logging:
  level:
    io.honeymon.springboot.boot: INFO
server:
  port: 5000 # AWS Elastic Beanstalk 에서는 5000번을 사용해야한다. 단
SERVER_PORT 로 정의해야 한다.
---
```

- ① 모든 프로파일에 공통적용되는 속성 정의, 프로파일을 별도로 지정하지 않은 경우(default) 적용됨
- ② local: server.port는 9000 유지, io.honeymon.springboot.boot 로깅레벨은 TRACE 로 조정
- ③ test: server.port는 8000~9000 사이에서 무작위 부여, io.honeymon.springboot.boot 로깅레벨은

DEBUG 유지

- ④ `dev: server.port`는 9000 유지, `io.honeymon.springboot.boot` 로깅레벨은 DEBUG 유지
- ⑤ `prd: server.port`는 5000 변경, `io.honeymon.springboot.boot` 로깅레벨은 INFO 유지(운영레벨)

반면 `application.properties` 를 사용하는 경우에는 프로파일 선언을 위해 `application-local.properties`, `application-test.properties`, `application-dev.properties`, `application-prd.properties` 를 만들어서 각각의 설정을 추가해야한다.

기본(default) 프로파일은 애플리케이션 컨텍스트가 구동될 때 특정 프로파일을 지정하지 않은 경우에도 적용된다. YAML에서 다음과 같이 `security.user.password` 가 default 프로파일에만 정의되어 있을 때:

```
server:
  port: 8000
---
spring:
  profiles: default
security:
  user:
    password: weak
```

이 예제에서 보면, 비밀번호는 다른 프로파일에는 추가하지 않았지만 어느 프로파일에서고 적용되며, 필요에 따라 다른 프로파일에서 필요에 따라 재설정이 가능하다:

```
server:
  port: 8000
---
spring:
  profiles: default
security:
  user:
    password: weak
---
spring:
  profiles: prod
security:
  user:
    password: strong
```

스프링 프로파일은 `spring.profiles` 요소를 '!'와 함께 조합하여 다양한 방법으로 사용할 수 있도록 설계되어 있다. 단일 문서파일에서 지정 프로파일과 '!'를 사용한 프로파일이 사용된다면, 일치하는 지정 프로파일이 있다면 '!'를 사용한 프로파일은 적용되지 않는다.

YAML 단점

YAML 파일은 `@PropertySource` 애너테이션에 적재되지 않는다. 그래서 `@PropertySource`에 값을 적재하기 위해서는 `properties`파일을 사용해야한다.

NOTE

`@PropertySource`을 사용하는 방법은 다음과 같다.

```
@Configuration
@PropertySource("classpath:/com/myco/a.properties")
public class ConfigA { }
```

YAML 목록 병합(Merging)

앞에서 살펴본 `YAML 적재` 것처럼, YAML 콘텐츠는 최종적으로 `properties` 로 변환된다. 이 과정은 프로파일 에 의해서 결정된다.

예로, `MyPojo` 객체에는 `name`과 `description` 필드가 있으며 기본값은 `null`이다. `ExampleProperties`로부터 `MyPojo` 리스트에 값을 연결해보자.

```
@ConfigurationProperties("example")
public class ExampleProperties {
    private final List<MyPojo> list = new ArrayList<>();
    public List<MyPojo> getList() {
        return this.list;
    }
}
```

다음처럼 구성되어 있다고 생각하자:

```
example:
  list:
    - name: my name
      description: my description
---
spring:
  profiles: dev
foo:
  list:
    - name: my another name
```

만약 `dev` 프로파일이 활성화되지 않는다면 `ExampleProperties#list`에는 앞서 정의한 `MyPojo` 한개가 포함되어 있다. 만약 `dev` 프로파일 이어도 한개(`name`에는 `my another name`이 있고 `description`는 `null`)일 것이다. 이 구성으로는 리스트에 두번째 `MyPojo` 인스턴스가 추가되지는 않는다.

컬렉션은 지정된 다중 프로파일 중에서 가장 우선순위가 높은 것이 사용된다:

```
example:
  list:
    - name: my name
      description: my description
    - name: another name
      description: another description
---
spring:
  profiles: dev
example:
  list:
    - name: my another name
```

이 예제의 경우, `dev` 프로파일이 활성화 되면 `ExampleProperties#list`에는 `MyPojo` 객체가 한개만 존재한다. 프로파일마다 동일한 키값이 있는 경우에 활성화된 프로파일에 속성값이 최종적으로 덮어쓰기된다.

3.2.7. 보장형(type-safe) 구성 속성

```

package io.honeymon.boot.springboot.config;

import java.net.InetAddress;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

import org.springframework.boot.context.properties.ConfigurationProperties;

import lombok.Data;

@ConfigurationProperties("example")
public class ExampleProperties {
    private boolean enabled;                                ①
    private InetAddress remoteAddress;                      ②
    private final Security security = new Security();
    public boolean isEnabled() { ... }
    public void setEnabled(boolean enabled) { ... }
    public InetAddress getRemoteAddress() { ... }
    public void setRemoteAddress(InetAddress remoteAddress) { ... }
    public Security getSecurity() { ... }

    public static class Security {
        private String username;                            ③
        private String password;                            ④
        private List<String> roles = new ArrayList<>(Collections.singleton(
"USER")); ⑤
        public String getUsername() { ... }
        public void setUsername(String username) { ... }
        public String getPassword() { ... }
        public void setPassword(String password) { ... }
        public List<String> getRoles() { ... }
        public void setRoles(List<String> roles) { ... }
    }
}

```

① `example.enabled`는 기본값 `false`

② `example.remote-address`는 `String`에서 강제변환된다.

③ `example.security.username`에서 중간에 있는 `security`는 속성의 이름으로부터 결정된다. 특정 리턴타입을 지정하지 않으면 `SecurityProperties` 일수도 있다.

④ `foo.security.password`

⑤ `foo.security.roles` 는 `String` 으로부터 배열처리된다. 별도로 선언하지 않은 경우에는 `USER`값만 가지고 있다.

{getter}와 {setter}를 항상 작성하는 것은, 스프링 MVC 와 마찬가지로 표준 Java Beans 속성 기술에 의해 바인딩 되기때문에 필수 항목이다. 다음 경우에는 {setter}를 생략할 수 있다:

- Map은 초기화시 바인더에 의해 변경할 수 있기 때문에 {getter}는 필요하지만 반드시 {setter}가 필요하지는 않다.
- 컬렉션과 배열은 인덱스(YAML을 사용하는 경우)를 사용하거나 쉼표(,)로 구분된 값을 사용(properties를 사용하는 경우)하여 접근할 수 있다. 이러한 경우에는 {setter}가 필요하다. 이런 유형에 대해서는 항상 {setter}를 추가하는 것을 권장한다. 만약 컬렉션을 초기화한다면, 변경가능(Not immutable)하도록 만들어야 한다.
- POJO 내부 속성이 초기화(위의 예제에서 **Security** 필드처럼) 됐다면 {setter}는 필요없다. 바인더가 기본 생성자를 사용해서 인스턴스를 즉성에서 만들게 하려면 {setter}가 필요하다.

몇몇 사람들(나를 포함한)은 'Project Lombok'을 사용하여 {getter}와 {setter}를 자동생성한다. 롬복(lombok)이 컨테이너에서 객체 인스턴스화 하는 것과 같은 특정 유형의 생성자를 생성하지 않도록 하는 것이 좋다. 기본적으로 클래스 작성시 사용하는 **getter(=@Getter)/setter(=@Setter), toString(=@ToString), equals, hashCode(=@EqualsAndHashCode)** 등의 메서드를 자동생성하는 목적으로 사용하면 편리하다.

위의 예제에 롬복을 적용한 예제

NOTE

```
@Data
@ConfigurationProperties("example") ①
public class ExampleProperties {
    private boolean enabled;
    private InetAddress remoteAddress;
    private final Security security = new Security();

    @Data
    public static class Security {
        private String username;
        private String password;
        private List<String> roles = new ArrayList<>
(Collection.singleton("USER"));
    }
}
```

- ① **@ConfigurationProperties** 애너테이션을 사용하면 IDE에서 **When using @ConfigurationProperties it is recommended to add 'spring-boot-configuration-processor' to your classpath to generate configuration metadata** 라는 메시지를 보여준다. **spring-boot-configuration-processor**를 추가해주자.

TIP

@ConfigurationProperties 유효성검사에서 차이점을 확인할 수 있다.

@EnableConfigurationProperties 애너테이션을 이용하면 필요한 Properties 클래스 목록을 등록할 수 있다.

```
@Configuration
@EnableConfigurationProperties(ExampleProperties.class)
public class MyConfiguration {
}
```

NOTE

`@ConfigurationProperties` 빈을 위의 방법으로 등록했을 때, 빈은 관례적 작명(<prefix>-<fqcn>)을 따른다. <prefix>는 `@ConfigurationProperties`에서 사용한 값 `example`이 적용되고 <fqcn>는 빈의 Fully Qualified Name 이다. 만약에 애너테이션에 별도의 접두사를 제공하지 않은 경우에는 빈 이름으로 `fqcn`만 사용된다. 위에 예제의 경우에는 빈의 이름은 `example-io.honeymon.boot.springboot.config.ExampleProperties`가 된다.

위의 구성으로 `ExampleProperties`에 대한 빈을 작성하더라도 `@ConfigurationProperties`는 환경을 구성하는데만 사용하고 컨텍스트에 있는 다른 빈에는 부분적으로 주입하지 않는 것이 좋다. 즉, `@EnableConfigurationProperties` 애너테이션은 `Environment`로부터 `@ConfigurationProperties` 선언된 빈을 구성하여 프로젝트에서 사용할 수 있도록 만든다. 위의 `MyConfiguration`을 살펴보지 않아도 `ExampleProperties`가 빈으로 등록됐다는 것을 다음처럼 표시할 수도 있다:

```
@Component
@ConfigurationProperties(prefix="example")
public class ExampleProperties {
    // ... see above
}
```

이런 식으로 구성하면 `SpringApplication` 외부 YAML 구성에서 특히 잘 동작한다:

`application.yml`

```
example:
  remote-address: 192.168.1.1
  security:
    username: admin
    roles:
      - USER
      - ADMIN
# additional configuration as required
```

`@ConfigurationProperties` 빈은 다른 빈과 마찬가지로 빈주입 후 사용할 수 있다:

```

@Slf4j
@Service
public class BootService {

    private ExampleProperties properties;

    @Autowired
    public BootService(ExampleProperties properties) {    ❶
        this.properties = properties;
    }

    @PostConstruct
    public void init() {
        log.debug("Injected properties: {}", this.properties);
    }
}

```

❶ 생성자 주입방식으로 **ExampleProperties**빈을 주입했다.

NOTE

로그를 살펴보면 다음과 같은 결과를 확인해볼 수 있다.

```

i.h.boot.springboot.service.BootService : Injected properties:
ExampleProperties(enabled=false, remoteAddress=/192.168.1.1,
security=ExampleProperties.Security(username=admin,
password=null, roles=[USER, ADMIN]))

```

TIP

@ConfigurationProperties를 사용하면 메타데이터 파일을 만들고 이를 이용해서 IDE 자동완성키 기능을 사용할 수 있다. 자세한 내용은 [메타데이터 구성](#)을 읽어보기 바란다.

서드파티 구성

@ConfigurationProperties는 클래스 레벨에서 선언해도 잘 동작하지만, **public @Bean** 메서드들에도 사용할 수 있다. 이 방법은 서드파티 컴포넌트에 속성을 연결하고 외부에서 제어할 때 특히 유용하다.

```

@Bean
@ConfigurationProperties(prefix = "bar")
public BarComponent barComponent() {
    //...
}

```

위에서 살펴봤던 **ExampleProperties**와 유사한 형식으로 **BarComponent** 빈에 **bar**를 접두사로 가진 값들이 연결된다.

느슨한 연결

스프링 부트는 `Environment` 속성에서 `@ConfigurationProperties` 빈으로 연결할 때 몇가지 느슨한 규칙을 사용하기 때문에 `Environment` 속성이름과 빈 속성이름을 정확하게 맞추지 않아도 된다. 일반적인 예로 대시보드(-)로 구분된 경우(예: `context-path`는 `contextPath`에 연결됨)와 대소문자(예: `PORT`는 `port`에 연결됨) 환경 속성에 유용하다.

예로, 다음과 같은 `@ConfigurationProperties` 클래스가 있다:

```
@Getter @Setter
@ConfigurationProperties(prefix="person")
public class OwnerProperties {
    private String firstName;
}
```

다음의 속성 모두 사용가능하다:

Property	Name
<code>person.firstName</code>	표준 카멜(Camel)식 문법.
<code>person.first-name</code>	<code>.properties</code> 과 <code>.yaml</code> 파일에서 사용하길 권장하는 대시(-) 표기법
<code>person.first_name</code>	<code>.properties</code> 과 <code>.yaml</code> 파일에서 대안으로 형식하는 밑줄(_) 표기법
<code>PERSON_FIRST_NAME</code>	대문자 형식. 시스템 환경변수에서 사용할 때 권장.

Properties 관례

스프링은 `@ConfigurationProperties` 빈에 외부 애플리케이션 속성을 연결할 때 적절한 유형으로 강제변환을 시도한다. 만약 재정의의 유형 변환이 필요하다면 `ConversionService` 빈(ID `conversionService`) 혹은 재정의속성편집기(`CustomEditorConfigurer` 빈 같은) 혹은 재정의 `Converters` (`@ConfigurationPropertiesBinding`이 선언된 빈)을 사용하라.

NOTE

`@ConfigurationProperties` 빈은 애플리케이션 생애주기에서 초기에 요청되기 때문에, `ConversionService`를 사용할 때 의존성을 제한해야 한다. 일반적으로 필요한 족속성은 생성과정에서 완전히 초기화되지 않을 수 있다. 재정의한 `ConversionService`의 이름을 변경하고 싶다면

구성 키 강제 변환에 필요하지 않은 경우 재정의한 `ConversionService`의 이름을 바꾸고 `@ConfigurationPropertiesBinding`으로 정규화 된 재정의 `Converters`에만 의존 할 수 있습니다.

@ConfigurationProperties 유효성검사

스프링 부트는 스프링의 `@Validated` 애너테이션 을 `@ConfigurationProperties` 클래스에 선언하여 유효성을 검사할 수 있다. JSR-303 `javax.validation`에 있는 애너테이션을 구성 클래스에 사용할 수 있다. 클래스패스 상에 JSR-303 구현체가 있다면 클래스 필드에 강제 애너테이션을 추가하는 것은 간단하다.

```

@Validated ①
@ConfigurationProperties("example")
public class ExampleProperties {
    // 중략
    @NotNull
    private InetAddress remoteAddress;
    // 중략
}

```

① 스프링에 포함되어 있음: `org.springframework.validation.annotation.Validated`

내부 속성의 값에 대해서 유효성검사를 하려할 때는 `@Valid`를 필드에 선언하면 된다. 예를 들어, `ExampleProperties`를 예제로 들어보면:

```

@Component
@Data
@Validated
@ConfigurationProperties("example")
public class ExampleProperties {
    private boolean enabled;

    @NotNull
    private InetAddress remoteAddress;

    @Valid
    private final Security security = new Security();

    @Data
    public static class Security {

        @NotEmpty
        private String username;

        private String password;

        private List<String> roles = new ArrayList<>(Collections.
singleton("USER"));
    }
}

```

만약에 유효성 검사결과에 적합하지 않은 경우 실행 중 다음과 같은 로그를 볼 수 있다:

```
*****
APPLICATION FAILED TO START
*****
```

Description:

```
Binding to target ExampleProperties(enabled=false,
remoteAddress=/192.168.1.1,
security=ExampleProperties.Security(username=admin, password=null,
roles=[USER, ADMIN])) failed:
```

```
Property: example.security.password ①
Value: null
Reason: may not be empty
```

① Security.password에 @NotEmpty를 선언한 경우

또한, `configurationPropertiesValidator`라는 빈 정의를 생성하여 재정의된 스프링 `Validator`를 추가 할 수 있다. `@Bean` 메서드는 **정적(static)**으로 선언되어야 한다. 구성속성유효성심사기는 {application}의 생애주기 초기에 생성되어야 하며 `@Bean` 메서드를 정적으로 선언하면 `@Configuration` 클래스를 인스턴스화하지 않고도 빈을 만들 수 있다. 이 방법은 초기 인스턴스 생성으로 발생할 수 있는 문제를 피할 수 있다. [속성검증예제](#)가 있으니 어떻게 설정하는지 살펴볼 수 있다.

TIP

`spring-boot-actuator` 모듈은 종단점(endpoint)과 관련한 모든 `@ConfigurationProperties`를 보여주는 기능을 포함하고 있다. `/configprops`를 웹브라우저로 살펴보거나 유사한 기능을 제공하는 `JMX` 종단점을 사용하면 된다. [종단점\(Endpoint\)](#)에서 보다 자세한 내용을 살펴볼 수 있다.

@ConfigurationProperties 대 @Value

`@Value`는 컨테이너 핵심기능이며 보장형(type-safe) 구성 속성을 제공하지는 못한다. 다음 테이블에서 `ConfigurationProperties`와 `@Value`의 차이점을 비교했다:

Feature	@ConfigurationProperties	@Value
느슨한 연결	지원	미지원
메타데이터 지원	지원	미지원
SpEL 지원	미지원	지원

컴포넌트를 작성시 구성키를 정의할 때는 `@ConfigurationProperties`를 정의한 **POJO**를 사용하는 것을 권장한다. `@Value`는 느슨한 연결을 지원하지 않는다는 것을 상기해두고 환경변수를 사용하는 값을 제공하려할 때 고려할만한 대상은 아니라는 것도 알아두기 바란다.

3.3. 프로파일

스프링 프로파일은 애플리케이션 구성은 각 환경에 따라서 구성을 분리하여 사용할 수 있도록 해준다. 어느 `@Component`

(`@Controller`, `@Service`와 `@Repository` 포함) 혹은 `@Configuration`이든지 `@Profile`을 사용하여 적재되는 상황을 제한할 수 있다:

```
@Configuration
@Profile("prod")
public class ProdConfiguration {
    // 중략
}
```

스프링에서는 일반적으로 `spring.profiles.active`이라는 `Environment`속성을 통해서 활성화할 프로파일을 다중으로 지정할 수 있다. `application.yml`내에 다음과 같은 형태로 속성을 지정할 수 있다:

```
spring.profiles.active: local,h2
```

혹은 커맨드라인 실행인자에서 사용할 수도 있다 `--spring.profiles.active=prod,prod-maria`

3.3.1. 활성화할 프로파일 추가하기

`spring.profiles.active` 속성은 최상위의 `PropertySource`부터 정해진 규칙에 따라 우선순위를 따른다. 이 말인 즉슨 `application.yml`에서 지정한 프로파일을 커맨드라인에서 대체할 수 있다는 이야기다.

경우에 따라서는 프로파일을 대체하는 대신 활성화할 프로파일을 추가하는 방법도 유용하다.

`spring.profiles.include` 속성을 사용하여 지정 프로파일이 아니라 하더라도 활성화할 수 있다.

`SpringApplication` 진입점에서도 (`spring.profiles.active` 속성으로 활성화된 것에 추가적으로) 프로파일을 추가설정할 수 있는 Java API를 제공한다(`setAdditionalProfiles()` 메서드를 살펴볼 수 있다).

예를 들어, `--spring.profiles.active=prod`으로 활성화 되었을 때 `prod-db`와 `prod-cache` 속성도 활성화시킨다면:

```
#####
## Default properties
#####
---
spring:
  profiles: prod
  include: prod-db, prod-cache
```

NOTE

YAML 문서에서는 `spring.profiles` 속성을 선언한 영역을 부분적으로 문서화하여 구성하는데 사용할 수 있다는 것을 기억해두기 바란다. 매우 유용하게 사용할 수 있다. 자세한 내용은 [환경 의존적인 구성 변경](#)을 살펴보기 바란다.

3.3.2. 프로파일을 프로그래밍적으로 설정

애플리케이션이 실행되기 전에 `SpringApplication.setAdditionalProfiles(...)`를 호출하여 프로파일을 프로그래밍적으로 지정할 수 있다. 또한 스프링 `ConfigurationEnvironment` 인터페이스를 사용하여 활성화 프로파일을 지정하는 것도 가능하다.

3.3.3. 프로파일 지정 구성 파일

프로파일 지정은 `application.yml`(혹은 `application.properties`)과 `@ConfigurationProperties`이 참조하는 파일 등에서 치환가능하다. 보다 자세한 내용은 [프로파일 정의 속성](#)을 살펴보기 바란다.

3.4. 로깅

스프링 부트는 모든 내부 로깅에 `Commons Logging`을 사용하지만, 기본 구성은 그대로 유지한다. `Java Util Logging`, `Log4J2` 그리고 `Logback`을 위한 기본 구성을 제공한다. 로거는 기본적으로 콘솔로 출력하도록 사전에 구성되어 있으며 선택적으로 파일출력도 사용할 수 있다.

스타터를 사용한다면 기본적으로 로깅에는 로그백(Logback)을 이용한다. Java Util Logging, Commons Logging, Log4J 또는 SLF4J를 사용하는 종속라이브러리가 모두 올바르게 동작하도록 Logback 라우팅을 포함한다.

TIP

위에서 나열한 것들은 자바에서 사용가능한 로깅 프레임워크들이다. 많은 숫자에 당황할 필요는 없다. 로깅 의존성을 변경할 필요없이 스프링 부트 기본 로깅구성으로도 충분히 잘 동작한다.

3.4.1. 로그 형식

스프링 부트에서 출력되는 기본적인 형식은 다음과 같다:

```
2017-06-11 09:43:41.201 INFO 22691 --- [main]
o.apache.catalina.core.StandardService : Starting service Tomcat
2017-06-11 09:43:41.204 INFO 22691 --- [main]
org.apache.catalina.core.StandardEngine : Starting Servlet Engine: Apache
Tomcat/8.5.14
2017-06-11 09:43:41.460 INFO 22691 --- [ost-startStop-1]
o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded
WebApplicationContext
2017-06-11 09:43:41.461 INFO 22691 --- [ost-startStop-1]
o.s.web.context.ContextLoader : Root WebApplicationContext:
initialization completed in 3104 ms
2017-06-11 09:43:41.779 INFO 22691 --- [ost-startStop-1]
o.s.b.w.servlet.ServletRegistrationBean : Mapping servlet:
'dispatcherServlet' to [/]
```

출력된 형식을 하나씩 살펴보면:

- 일시: 밀리세컨드단위 정밀도로 쉽게 정렬가능
- 로그레벨: `ERROR`, `WARN`, `INFO`, `DEBUG` 혹은 `TRACE`

- 프로세스ID
- --- 구분자로 이후에는 로그가 실제로 시작
- 스레드명: 대괄호로 감싸져 있음(콘솔 출력을 위해 일부 자름)
- 로거명: 로그를 출력하는 소스 클래스이름 사용(줄여쓰기됨)
- 로그 메시지

NOTE | 로그백에서는 **FATAL** 레벨이 없다(**ERROR**로 대응)

3.4.2. 콘솔 출력

기본 로그 구성은 로거에서 출력하는 로그 메시지를 콘솔에 출력한다. 기본적으로 **ERROR**, **WARN**과 **INFO** 레벨의 메시지가 출력된다. 애플리케이션 구동시 **--debug** 플래그를 이용해서 "디버그" 모드로 출력할 수도 있다.

```
$ java -jar boot-spring-boot.jar --debug
```

NOTE | **application.yml**에서 **debug: true**로 지정할 수도 있다.

디버그 모드가 활성화되었을때, 주요 로거(내장 컨테이너, 하이버네이트와 스프링 부트)를 선택도록 구성되어 추가적인 정보를 출력한다. 디버그 모드가 활성화된다고 해서 애플리케이션의 모든 **DEBUG** 레벨 메시지가 출력되지는 않는다.

선택적으로, 애플리케이션이 시작될 때 **--trace** 플래그(**trace:true**를 **application.yml**에 지정) "trace" 모드로 시동하는 것도 가능하다. 주요 로거(내장 컨테이너, 하이버네이트 스키마 생성과 스프링 포트폴리오)가 선택되어 트레이스 로깅이 활성화된다.

3.4.3. 파일 출력

터미널에서 안시(ANSI)을 지원하는 경우, 컬러로 출력되면 가독성에 더욱 도움이 될 것이다.

spring.output.ansi.enabled 속성을 지정하여 지원하는 값(**ALWAYS**, **DETECT**, **NEVER**)으로 설정을 변경할 수 있다.

%cr 관례를 이용해서 색상을 변경할 수 있다. 가장 간단한 형태로 컨버터는 로그 수준에 따라서 출력색상을 지정한다, 예를 들자면:

```
%clr(%5p)
```

로그 레벨과 대응하는 색상은 다음과 같다:

레벨	색상
FATAL	빨강
ERROR	빨강
WARN	노랑

레벨	색상
INFO	녹색
DEBUG	녹색
TRACE	녹색

대안으로 관례로 제공하는 선택사항 `%r` 사용하여 색상이나 형식을 변경지정할 수도 있다. 예를 들어, 텍스트를 노랑색으로 변경해보자면:

```
%clr(%d{yyyy-MM-dd HH:mm:ss.SSS}){yellow}
```

지원하는 색상과 스타일은 다음과 같다:

- blue
- cyan
- faint
- green
- magenta
- red
- yello

3.4.4. 파일 출력

스프링 부트는 기본적으로 콘솔에만 로그를 출력하고 로그파일을 작성하지는 않는다. 만약 콘솔 출력에 추가적으로 로그파일을 작성하길 원한다면 `logging.file` 혹은 `logging.path` 속성을 설정해야 한다(`application.yml` 내에 작성).

`logging.*` 속성은 다음과 같다:

<code>logging.file</code>	<code>logging.path</code>	예	설명
(미지정)	(미지정)		콘솔에만 출력됨
파일 지정	(미지정)	<code>app.log</code>	지정된 로그파일을 작성한다. 이름으로 현재 디렉토리에 대한 상대경로나 위치를 지정
(미지정)	지정 디렉토리	<code>/var/log</code>	지정된 디렉토리에 <code>spring.log</code> 로 작성한다. 이름으로 현재 디렉토리나 위치를 지정할 수 있다.

로그 파일은 10MB 단위로 절삭되며 `ERROR`, `WARN` 그리고 `INFO` 레벨 메시지가 기본적으로 기록된다.

NOTE

로깅 시스템은 애플리케이션 생애주기 중 초기에 구성되기 때문에 `@PropertySource` 애너테이션 등이 적재될 때 로깅과 관련된 속성을 찾지 못할 수도 있다.

TIP

로깅 속성은 실제 로깅 기반에 의존적이다. 그 때문에 스프링 부트에서는 로깅 시스템에서 사용하는 구성키 (`logback.configurationFile` 같은)를 관리하지 못한다.

3.4.5. 로그 레벨

지원되는 모든 로깅 시스템에 대해서는 스프링 `Environment`의 `logging.level.*={LEVEL}` (LEVEL: TRACE, DEBUG, INFO, WARN, ERROR, OFF 중 하나) 이용해서 로거 레벨을 설정할 수 있다. `root` 로거는 `logging.level.root`를 사용하여 구성할 수 있다.

```
logging:
  level:
    root: WARN
    org.springframework.web: DEBUG
    org.hibernate: ERROR
```

NOTE

스프링 부트는 타임리프 `INFO` 메시지를 `DEBUG` 레벨로 기록한다. 이는 로그 출력을 표준화 하고 잡음을 감소시키는 효과를 가져오는데, `LevelRemappingAppender`를 살펴보면 구성에서 어떻게 사용하는지 이해할 수 있다.

로그 구성 재정의

<http://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/#boot-features-custom-log-configuration> 다양한 로깅 시스템이 라이브러리를 적절한 클래스패스에 포함하고 있고 클래스패스 최상위에 적합한 구성 파일을 제공하거나, 스프링 `Environment` 속성 `logging.config`로 위치를 지정하면 로그 설정을 재정의 할 수 있다.

스프링 부트에서 `org.springframework.boot.logging.LoggingSystem` 시스템 속성을 이용해서 특정 로깅 시스템을 사용하도록 강제할 수 있다. 대응하는 값으로 `LoggingSystem` 구현체의 절대경로를 포함한 클래스명을 사용한다. 혹은 `none` 값을 사용하여 스프링 부트 로깅 구성을 완전히 비활성화할 수도 있다.

NOTE

로깅은 `ApplicationContext`가 생성되기 전에 초기화 되기 때문에 스프링 `@Configuration` 파일에서 `@PropertySource`으로 로깅 시스템을 제어하는 것은 불가능하다. 시스템 속성 및 스프링 부트 외부 구성 파일에서는 잘 동작한다.

로깅시스템에 따라, 다음의 파일들이 적재된다:

로깅 시스템	재정의 파일명
Logback	<code>logback-spring.xml</code> , <code>logback-spring.groovy</code> , <code>logback.xml</code> 혹은 <code>logback.groovy</code>
Log4j2	<code>log4j2-spring.xml</code> 혹은 <code>log4j2.xml</code>
JDK(Java Util Logging)	<code>logging.properties</code>

NOTE

로깅 구성을 위해 **-spring** 접미사를 사용하길 권장한다(예를 들어 **logback.xml** 보다는 **logback-spring.xml**). 로깅 재정의 파일이 지정된 위치에 있다면, {spring}은 로그 초기화를 완벽하게 제어할 수 있다.

WARNING

실행가능한 jar에서 실행될 때 Java Util Logging의 클래스로딩 이슈(<https://goo.gl/MPELec>, <https://goo.gl/9rfccZ>)로 인해 문제가 발생할 수 있다. 그러니 이 문제를 가급적이면 피하기를 권한다(Logback, Log4j2, Slf4j 등의 라이브러리를 이용).

스프링 **Environment** 속성과 시스템 속성 중 일부를 이용하여 재정의가 가능하다:

스프링 Environment	시스템 속성	설명
logging.exception-conversion-word	LOG_EXCEPTION_CONVERSION_WORD	로깅 예외가 발생했을 때 사용할 단어 정의
logging.file	LOG_FILE	지정된 파일을 기본구성에 사용
logging.path	LOG_PATH	지정된 파일을 기본구성에 사용
logging.pattern.console	LOGGING_LOG_PATTERN	콘솔(stdout)에서 사용하는 로그 패턴(로그백 기본설정만 지원)
logging.pattern.file	FILE_LOG_PATTERN	파일에서 사용할 로그 패턴(LOG_FILE 활성화 시)(로그백 기본설정만 지원).
logging.pattern.level	LOG_LEVEL_PATTERN	로그 레벨(기본 %5p) 표현방식(로그백 기본설정만 지원).
PID	PID	현재 프로세스 ID(운영체제 환경변수로 정의하지 않은 경우에 발견할 수 있음)

지원하는 모든 로깅 시스템에서는 구성 파일을 파싱할 때 시스템 속성을 사용할 수 있다. **spring-boot.jar** 에 기본 구성을 살펴보자.

TIP

로깅 속성에 대해서 위치변환자를 사용하길 원한다면, 로깅시스템의 문법 대신 "**속성 내 치환자 (placeholder)**" 처럼 스프링 부트의 **\${property key}**를 사용할 수 있다. 로그백을 사용한다면 **:-**이 아닌 **:**을 속성명 사이에 구분자로 사용할 수 있다(로그백에서는 **:-** 연산자로 키값을 정의한다, <https://goo.gl/lqyhtU>).

3.4.6. 로그백 확장

스프링 부트는 로그백에 대한 많은 확장포인트를 가지고 있어서 보다 나은 구성을 할 수 있다. **logback-spring.xml** 구성파일을 확장할 수 있다.

NOTE

너무 빠르게 적재되는 표준 **logback.xml** 구성 파일에서는 확장을 사용할 수 없다. **logback-spring.xml** 혹은 **logging.config** 속성으로 정의된 경우 사용가능하다.

확장은 로그백 구성탐색(<https://goo.gl/FBVlxu>)을 사용할 수는 없다. 만약에 시도를 한다면 구성 파일을 변경하면 다음과 같은 결과를 보게될 것이다.

WARNING

```
ERROR in ch.qos.logback.core.joran.spi.Interpreter@4:71 -
no applicable action for [springProperty], current
ElementPath is [[configuration][springProperty]]
ERROR in ch.qos.logback.core.joran.spi.Interpreter@4:71 -
no applicable action for [springProfile], current
ElementPath is [[configuration][springProfile]]
```

프로파일 지정 구성

`<springProfile>` 태그는 스프링 프로파일에 기반하여 구성된 특정영역을 포함하거나 제외하는 것을 허용한다. 프로파일 영역에 대해서는 `<configuration>` 요소와 함께 어디서나 사용가능하다. `name` 속성을 사용하여 구성 프로파일 정의 항목에 접근가능하다. 쉼표(,)를 사용하면 다중 프로파일을 지정가능하다.

```
<springProfile name="stag">
  <!-- configuration to be enabled when the "staging" profile is active -->
</springProfile>

<springProfile name="dev, stag">
  <!-- configuration to be enabled when the "dev" or "staging" profiles
are active -->
</springProfile>

<springProfile name="!prod">
  <!-- configuration to be enabled when the "production" profile is not
active -->
</springProfile>
```

환경 속성

로그백을 사용할 때 스프링 `Environment`을 통해서 `<springProperty>` 태그를 사용할 수 있다. 이를 이용하면 로그백 구성에서 `application.yml`의 값에 접근할 수 있어 매우 유용하다. 태그는 로그백 표준 `<property>` 태그와 유사하게 동작하지만 (`Environment`로부터) 속성에 `source`를 지정하면 값이 바로 정의된다. `local` 스코프가 아닌 다른 곳에 저장해야하는 경우 `scope` 속성을 사용할 수 있다. `Environment` 에서 속성에 값을 설정하지 못하는 때를 대비해서 `defaultValue` 속성을 사용할 수 있다.

```
<springProperty scope="context" name="fluentHost" source=
"myapp.fluentd.host"
    defaultValue="localhost" />
<appender name="FLUENT" class=
"ch.qos.logback.more.appenders.DataFluentAppender">
    <remoteHost>${fluentHost}</remoteHost>
    ...
</appender>
```

TIP

Environment 속성접근에는 `RelaxedPropertyResolver`을 사용한다. `source`를 지정할 때 대시(-)를 사용(my-property-name)하면 다양한 변형에 대해 시도해볼 수 있다(myPropertyName, MY_PROPERTY_NAME 등).

3.5. 웹애플리케이션 개발

스프링 부트는 웹애플리케이션 개발에 최적화되어 있다. 내장컨테이너(톰캣, 제티 혹은 언더토우)를 사용하여 손쉽게 HTTP 서버를 제작할 수 있다. 대부분의 웹애플리케이션은 `spring-boot-starter-web` 모듈을 사용하여 빠른 실행이 가능하다.

3.5.1. '스프링 Web MVC framework'

스프링 Web MVC 프레임워크(간단히 줄여서 'Spring MVC')는 'Model View Controller' 웹 프레임워크다. 스프링 MVC는 외부 HTTP 요청에 대해서 `@Controller` 혹은 `@RestController`를 만들어 대응한다. 컨트롤러의 메서드에는 `@RequestMapping`애너테이션을 사용하여 HTTP 요청에 대응한다.

여기 JSON 데이터를 제공하는 전형적인 `@RestController`가 있다:


```

@RestController
@RequestMapping(value="/users")
public class UserRestController {

    @RequestMapping(value="/{user}", method=RequestMethod.GET)
    public User getUser(@PathVariable Long user) {
        // ...
    }

    @RequestMapping(value="/{user}/customers", method=RequestMethod.GET)
    List<Customer> getUserCustomers(@PathVariable Long user) {
        // ...
    }

    @RequestMapping(value="/{user}", method=RequestMethod.DELETE)
    public User deleteUser(@PathVariable Long user) {
        // ...
    }

}

```

스프링 MVC는 주요 스프링 프레임워크 중 일부이며 자세한 내용은 [참조 문서](#)를 살펴보기 바란다. 스프링 MVC에 대해 다루는 <https://spring.io/guides>에서도 여러가지를 안내받을 수 있다.

스프링 MVC 자동구성

스프링 부트는 스프링 MVC에 대한 자동구성을 제공하며 대부분의 애플리케이션에서 큰 무리없이 잘 동작한다.

자동구성은 스프링 기본구성에 다음 기능을 추가한다:

- `ContentNegotiatingViewResolver`와 `BeanNameViewResolver` 빈 포함
- 정적자원 지원 및 `WebJars`지원기능 포함
- `Converter`, `GenericConverter`, `Formatter` 빈 자동등록
- `HttpMessageConverter` 지원
- `MessageCodeResolver` 자동등록
- 정적 `index.html` 지원
- `Favicon` 재정의 지원
- `ConfigurableWebBindingInitializer` 빈 자동지원

스프링 부트 MVC 기능을 유지하면서 `MVC 구성`(인터셉터, 포맷터, 뷰 컨트롤러 등)을 추가하고 싶다면 `@EnableWebMvc`없이 `WebMvcConfigurerAdapter` 타입의 클래스에 `@Configuration` 추가하면 된다. `RequestMappingHandlerMapping`, `RequestMappingHandlerAdapter` 혹은 `ExceptionHandlerExceptionResolver`에 대한 인스턴스를 재정의하고 싶다면

`WebMvcRegistrationsAdapter`를 이용할 수 있다.

만약 스프링 MVC를 완벽히 제어하고 싶다면 `@Configuration` 선언에 추가적으로 `@EnableWebMvc`를 선언하면 된다.

HttpMessageConverters

스프링 MVC는 HTTP 요청과 응답에 `HttpMessageConverter` 인터페이스를 사용한다. 기본적인 처리방식은 이미 정해져 있는데, 예를 들어 객체를 자동으로 JSON(Jackson 라이브러리 사용) 혹은 XML(JAXB를 사용하거나 가능하다면 Jackson XML 확장을 사용)로 변환가능하다.

컨버터를 추가하거나 재정의하려는 경우 스프링 부트의 `HttpMessageConverters`를 사용할 수 있다:

```
import org.springframework.boot.autoconfigure.web.HttpMessageConverters;
import org.springframework.context.annotation.*;
import org.springframework.http.converter.*;

@Configuration
public class MyConfiguration {
    @Bean
    public HttpMessageConverters customConverters() {
        HttpMessageConverter<?> additional = ...
        HttpMessageConverter<?> another = ...
        return new HttpMessageConverters(additional, another);
    }
}
```

어떤 `HttpMessageConverter` 빈이든지 목록에 추가하면 컨버터 목록에 추가되어 컨텍스트에 적용된다. 기본 컨버터도 동일한 방식으로 덮어쓸 수 있다.

JSON 직렬화와 역직렬화 사용자정의

JSON 데이터에 대해서 직렬화(serialize)와 역직렬화(deserialize)에 Jackson을 사용한다면 `JsonSerializer`와 `JsonDeserializer` 클래스를 작성할 수 있다. 재정의한 직렬화기(Serializers)는 Jackson 모듈로 등록가능하지만, 스프링 부트는 대안으로 `@JsonComponent` 애너테이션을 통해서 스프링 빈으로 바로 등록가능하다.

`JsonSerializer` 혹은 `JsonDeserializer` 구현체에 `@JsonComponent`를 사용할 수 있다. 혹은 내부 클래스에 포함하는 방식을 사용할 수도 있다. 예를 들자면:

```
import java.io.*;
import com.fasterxml.jackson.core.*;
import com.fasterxml.jackson.databind.*;
import org.springframework.boot.jackson.*;

@JsonComponent
public class Example {
    public static class Serializer extends JsonSerializer<SomeObject> {
        // ...
    }
    public static class Deserializer extends JsonDeserializer<SomeObject>
    {
        // ...
    }
}
```

`ApplicationContext`에 있는 모든 `@JsonComponent` 빈은 Jackson과 함께 자동으로 등록된다.

`@JsonComponent`는 `@Component`처럼 메타애너테이션 이기에 컴포넌트 스캔규칙에 적용된다.

스프링 부트는 `JsonObjectSerializer`과 `JsonObjectDeserializer` 기본 클래스를 제공하여 객체 표준화와 관련하여 표준 Jackson 버전에 대한 유용한 대안을 제공한다. 자세한 내용에 대해서는 Javadoc 문서를 살펴보기 바란다.

MessageCodeResolver

스프링 MVC는 발생한 에러에 대응하여 `MessageCodeResolver`을 바탕으로 에러 메시지를 렌더링하여 에러코드를 생성하는 전략을 가지고 있다. 스프링 부트는 `spring.mvc.message-codes-resolver.format` 속성에 `PREFIX_ERROR_CODE` 혹은 `POSTFIX_ERROR_CODE`를 설정할 수 있다(사용가능한 enum은 `DefaultMessageCodesResolver.Format`에 정의되어 있다).

정적 자원

스프링 부트는 기본적으로 클래스패스에 있는 `/static`(혹은 `/public` 혹은 `/resources` 혹은 `/META-INF/resources`) 디렉토리 혹은 `ServletContext` 최상위에 있는 정적 자원을 제공한다. 스프링 MVC의 `ResourceHttpRequestHandler`를 사용하며 `WebMvcConfigurerAdapter`의 `addResourceHandlers`를 재작성하여 행동을 변경할 수 있다.

단독실행 웹애플리케이션에서는 스프링이 자원에 대한 제어를 결정하지 않으면 컨테이너의 기본 서블릿이 활성화되고 `ServletContext`의 최상위 콘텐츠가 제공되고 요청에 응답한다. 스프링은 언제나 `DispatcherServlet`을 통해서 요청을 제어하기 때문에 이런 상황은 대부분 발생하지 않는다(기본 MVC 구성을 변경하지 않는 한).

기본적으로는 자원에 대해서는 `/`으로 대응하지만 `spring.mvc.static-path-pattern` 속성으로 조정 가능하다. 예를 들자면, 다음처럼 `/resources/`으로 모든 자원 위치를 재조정할 수 있다:

```
spring.mvc.static-path-pattern: /resources/**
```

`spring.resources.static-locations` 속성(앞에서 나열했던 기본 디렉터리 위치를 대체)을 사용하면 정적자원

위치에 대해서 재정의할 수 있다. 만약 기본 환영페이지(보통 `index.html`) 위치를 재정의한 위치가 애플리케이션의 시작페이지가 될 것이다.

Webjars 콘텐츠의 경우에는 앞에서 살펴본 '표준' 정적 자원 위치와는 다르다. 어떤 자원이든지 Webjar 형식으로 패키징된 jar 파일로부터 `/webjars/**` 에 파일이 위치하게 된다.

TIP

jar로 패키징한 애플리케이션에서 `src/main/webapp` 디렉토리를 사용하지 않는다. 이 디렉토리는 공통표준이지만 오직 war 패키징에서만 동작하며 jar파일을 생성할 때는 조용히 무시처리된다(WEB-INF도 마찬가지).

스프링 부트는 스프링 MVC에서 제공하는 자원 제어 기능보다 효과적인 방식(캐시 버스팅 정적 자원 혹은 Webjars를 위한 버전 무시전략)을 제공한다.

Webjars를 위한 버전무시 전략은, `webjars-locator` 의존성을 간단히 추가하면 된다. Webjars로 jQuery를 사용한다고 가정했을 때 `/webjars/jquery/x.y.z/dist/jquery.min.js`를 `/webjars/jquery/dist/jquery.min.js`로 처리한다(x.y.z는 Webjars 버전이다).

캐시 버스팅 사용의 경우, 다음 구성을 따르면 모든 정적 자원에 대해서 캐시 버스팅 적용이 구성되어 모든 자원 URL에 대해서 해시가 추가(예: `<link href="/css/spring-2a2d595e6ed9a0b24f027f2b63b134d6.css"/>`)된다.

NOTE

정적자원에 대한 정보를 해시코드화하여 이 정보가 변경되지 않는 경우에는 캐시를 유지하도록 하여 자원에 대한 요청을 하지 않도록 하여 화면구성을 빠르게할 수 있다.

```
spring.resources.chain.strategy.content.enabled: true
spring.resources.chain.strategy.content.paths: /**
```

NOTE

자동설정된 타임리프와 프리마커 경우에는 `ResourceUrlEncodingFilter`가 연결된 자원에 대한 재작성을 수행한다. JSP 를 사용하는 경우에는 수동으로 설정해야 한다. 다른 템플릿 엔진에서는 자동지원을 하지 않지만 `ResourceUrlProvider` 을 사용하거나 템플릿 매크로/헬퍼(helper)를 재정의하여 사용할 수 있다.

만약 동적으로 리소스 접근시 자바스크립트 모듈 로더처럼 이름이 변경되면 안되는 경우가 있다. 이런 때는 파일 이름을 변경하지 않고 URL에 정적 자원 버전을 추가하는 'fixed' 전략을 추가하면 된다.

```
# properties
spring.resources.chain.strategy.content.enabled=true
spring.resources.chain.strategy.content.paths=/**
spring.resources.chain.strategy.fixed.enabled=true
spring.resources.chain.strategy.fixed.paths=/js/lib/
spring.resources.chain.strategy.fixed.version=v12

# yaml
spring.resources.chain.strategy.content:
  enabled: true
  paths: /**
spring.resources.chain.strategy.fixed:
  enabled: true
  paths: /js/lib/
  version: v12
```

이렇게 구성하면 `/js/lib/` 하위에 위치한 자바스크립트 모듈에 대해서 고정된 버전관리 전략을 사용하여 `/v12/js/lib/mymodule.js` 으로 처리하며, 다른 자원들에 대해서는 `<link href="/css/spring-2a2d595e6ed9a0b24f027f2b63b134d6.css" />` 처럼 처리할 것이다.

`ResourceProperties`을 살펴보면 보다 자세한 선택사항을 살펴볼 수 있다.

TIP | 이 기능에 대한 보다 자세한 설명은 [블로그 포스트](#)와 스프링 프레임워크 [참조문서](#)에 기술되어 있다.

파비콘Favicon 사용자 정의

스프링 부트는 정의되어 있는 정적 콘텐츠 위치와 클래스패스 최상위에서 `favicon.ico`를 찾는다(순서에 따라). 만약 파일이 존재한다면 애플리케이션 파비콘으로 적용한다.

ConfigurableWebBindingInitializer

스프링 MVC는 `WebBindingInitializer`를 사용하여 특정 요청에 대응하는 `WebDataBinder`를 초기화할 수 있다. 만약 `ConfigurableWebBindingInitializer` 빈을 작성했다면, 스프링 부트는 자동으로 스프링 MVC를 구성하는데 사용할 것이다.

템플릿 엔진

훌륭한 REST 웹 서비스에서도, 스프링 MVC를 이용해서 동적인 HTML 콘텐츠를 제공할 수 있다. 스프링 MVC는 타임리프, 프리마커와 JSP를 포함한 다양한 템플릿 기술을 지원한다. 그외에도 스프링 MVC에서 지원하는 다양한 템플릿 엔진을 적용할 수 있다.

스프링 부트에서 자동구성을 지원하는 템플릿 엔진은 다음과 같다:

- [프라마커\(FreeMarker\)](#)
- [그루비\(Groovy\)](#)
- [타임리프\(Thymeleaf\)](#)

- 머스태쉬(Mustache)

TIP

JSP는 내장 서블릿 컨테이너를 사용할 때 알려져 있는 [여러가지 제약사항](#)들이 있어서 가급적이면 사용하지 않길 바란다.

기본구성과 함께 템플릿 엔진을 사용할 때, `src/main/resources/templates` 에서 템플릿 파일을 찾는다.

NOTE

인텔리제이 는 애플리케이션 실행 방법에 따라 클래스패스를 다르게 정렬한다. IDE에서 메인 메서드를 이용해서 애플리케이션을 실행할 때와 메이븐 과 그레이들 을 사용하여 실행할 때와 패키징된 jar를 이용해서 실행할 때 각기 다른 방식을 사용한다. 이런 경우 스프링 부트가 클래스패스에서 템플릿을 찾을 때 실패하게 된다. 이런 문제를 피할 수 있는 대안으로 클래스패스 상에서 모든 템플릿 파일을 탐색하도록 템플릿 접미사를 `classpath*:/templates/`로 구성할 수 있다(자동구성에서는 `classpath:/templates/`으로 정의되어 있다. `classpath*`(<https://goo.gl/8hQFei>)를 사용하는 경우에는 프로젝트로 가뇌려된 모든 jar를 탐색한다).

3.5.2. 에러 처리

스프링 부트는 모든 에러에 대해서 `/error` 대응을 제공하며, 이를 이용해서 서블릿 컨테이너에서 '전역' 에러페이지를 등록할 수 있다. 클라이언트를 위해서 에러에 대한 상세한 내용(HTTP 상태와 예외 메시지)을 JSON 응답으로 제공할 수도 있다. 브라우저 클라이언트는 동일한 데이터가 HTML로 렌더링 되어 'whitelabel' 에러 뷰로 처리된 것을 보게 될 것이다 (`error` 에 대응한 `View`를 추가하면 재정의 된다). `ExceptionHandler`를 구현하고 빈으로 등록거나 `ErrorAttributes` 유형의 빈을 추가하면 콘텐츠가 변경되긴 하지만 기본 동작을 완벽하게 대체할 수 있다.

TIP

`ExceptionHandler`는 재정의 `ExceptionHandler`를 위한 기본 클래스로 사용할 수 있다. 이것은 새로운 콘텐츠 유형(기본적으로 모든 것에 대해서 `text/html`으로 대응)에 대한 제어를 하고 싶을 때 유용하다. `ExceptionHandler`를 확장하고 public 메서드에 `@RequestMapping` 에 `produces` 속성과 함께 추가하고 새로운 유형의 빈으로 등록할 수 있다.

`@ExceptionHandler`를 정의하면서 특정 컨트롤러 혹은 예외 유형에 대해서 JSON 응답양식을 재정의하는 것도 가능하다.

```

@ControllerAdvice(basePackageClasses = HelloController.class) ①
public class RestResponseEntityExceptionHandler extends
ResponseEntityExceptionHandler {

    @ExceptionHandler(YourException.class) ②
    @ResponseBody
    ResponseEntity<?> handleControllerException(HttpServletRequest request,
    Throwable ex) {
        HttpStatus status = getStatus(request);
        return new ResponseEntity<>(new CustomErrorType(status.value(),
    ex.getMessage()), status); ③
    }

    private HttpStatus getStatus(HttpServletRequest request) {
        Integer statusCode = (Integer) request.getAttribute(
    "javax.servlet.error.status_code");
        if (statusCode == null) {
            return HttpStatus.INTERNAL_SERVER_ERROR;
        }
        return HttpStatus.valueOf(statusCode);
    }
}

```

① **HelloController** 와 동일한 패키지에 있는 컨트롤러들 적용

② **YourException**이 발생하는 경우

③ **ErrorAttributes**로 표현하는 대신 **CustomErrorType** POJO를 JSON으로 표현하여 제공한다.

에러 페이지 재정의

상태 코드에 따라 재정의한 HTML 에러 페이지를 보여주고 싶다면, **/error**폴더에 파일을 추가하면 된다. 에러 페이지는 정적 HTML 파일(정적 자원 폴더 아래에 추가하거나)이거나 템플릿 엔진에 의해 구성될 수 있다. 파일의 이름은 상태 코드와 정확하게 일치하거나 일련의 번호가 일치해야 한다.

예를 들어, 다음과 같은 폴더 구조에서 **404**에 대응하는 파일을 둘 수 있다:

```

src/
+- main/
  +- java/
    |   + <source code>
  +- resources/
    +- public/
      +- error/
        |   +- 404.html
      +- <other public assets>

```

혹은 5xx 번호에 대응한 타임리프 템플릿 파일이 있을 수 있다:

```
src/
+- main/
  +- java/
  |   + <source code>
  +- resources/
    +- templates/
      +- error/
      |   +- 5xx.html
      +- <other templates>
```

보다 복잡한 대응전략을 가지고 싶다면 `ErrorViewResolver` 인터페이스를 구현체를 빈으로 추가하면 된다.

```
public class MyErrorViewResolver implements ErrorViewResolver {

    @Override
    public ModelAndView resolveErrorView(HttpServletRequest request,
        HttpStatus status, Map<String, Object> model) {
        // Use the request or status to optionally return a ModelAndView
        return ...
    }

}
```

그에 더해 기존 스프링 MVC 기능에서 제공하던 `@ExceptionHandler`와 `@ControllerAdvice`를 사용할 수 있다. `ErrorController`는 제어하지 않는 예외들을 처리할 것이다.

스프링 MVC 영역 외부에서 에러 페이지 매핑

스프링 MVC를 사용하지 않는 애플리케이션이라면 `ErrorPageRegistrar`를 이용해서 `ErrorPages`를 바로 등록할 수 있다. 이 추상화는 내장 컨테이너에 있어도 잘 동작하며 스프링 MVC `DispatcherServlet`가 없어도 잘 동작한다.


```

@Bean
public ErrorPageRegistrar errorPageRegistrar(){
    return new MyErrorPageRegistrar();
}

// ...

private static class MyErrorPageRegistrar implements ErrorPageRegistrar {

    @Override
    public void registerErrorPages(ErrorPageRegistry registry) {
        registry.addErrorPages(new ErrorPage(HttpStatus.BAD_REQUEST, "
/400"));
    }

}

```

3.5.3. 스프링 HATEOAS

하이퍼미디어를 사용하는 RESTful API를 개발하고 있다면, 스프링 부트는 대부분의 애플리케이션에서 동작하는 스프링 HATEOAS를 위한 자동구성을 제공한다. 자동구성을 대체하기 위해서는 `@EnableHypermediaSupport`를 사용해야 하며 `LinkDiscoverers`(클라이언트측 지원용)를 포함하고 하이퍼미디어 기반 애플리케이션을 등록해야 하고 `ObjectMapper`가 구성되어 있어야 응답에 대해서 마샬링할 수 있다. `ObjectMapper`는 `spring.jackson.*` 속성을 기반으로 재정의하거나 `Jackson2ObjectMapperBuilder` 빈을 이용할 수 있다.

`@EnableHypermediaSupport`를 사용하여 스프링 HATEOAS 구성을 제어할 수 있다. 그렇게 되면 앞에서 설명한 `ObjectMapper` 재정의는 비활성화 될 것이다.

3.5.4. CORS 지원

`Cross-origin_resource_sharing`(CORS)는 W3C 명세서에 있는 것을 대부분의 브라우저에서 구현하고 있다. IFRAME 혹은 JSONP 같은 강력한 연계를 줄이고 인증을 하거나 보안성을 약화시키는 것을 대신하여 크로스 도메인(다른 도메인) 사이에 요청에 대한 유연한 방법을 기술하고 있다.

스프링 MVC 4.2 버전까지는 CORS 지원을 하지 않았다. 스프링 부트 애플리케이션에서는 CORS를 구성하려는 메서드에서 `CrossOrigin` 애너테이션을 사용하면 별다른 구성을 필요로 하지 않는다. 전역 CORS 구성은 `WebMvcConfigurer` 빈에서 `addCorsMappings(CorsRegistry)` 메서드를 이용해서 재정의하여 등록가능하다:

```

@Configuration
public class MyConfiguration {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurerAdapter() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/api/**");
            }
        };
    }
}

```

3.5.5. 내장 서블릿 컨테이너 지원

스프링 부트는 톰캣, 제티와 언더투 서버를 내장 컨테이너로 사용할 수 있다. 개발자 대부분이 스타터를 통해 완벽하게 구성된 인스턴스를 간단하게 사용할 수 있다. 내장 서버는 기본적으로 8080 포트를 기준으로 한다.

WARNING

CentOS에서 톰캣을 사용하기로 결정했다면 임시 디렉터리에 컴파일된 JSP 및 파일 업로드를 저장하도록 해야 한다. 이 디렉터리는 애플리케이션이 실행되는 동안 실패하게 되면 `tmpwatch`에 의해 정리가 삭제될 가능성이 있다. 이를 피하기 위해서 `tomcat.*` 설정을 통해서 `tmpwatch` 구성을 재정의하여 삭제하지 않도록 하거나 `server.tomcat.basedir` 설정으로 내장톰캣이 다른 위치를 사용하도록 구성하면 된다.

서블릿, 필터 와 리스너

내장 서블릿 컨테이너를 사용할 때 서블릿, 필터와 서블릿 사양(예, `HttpSessionListener`)에 있는 모든 리스너를 스프링 빈 혹은 서블릿 컴포넌트 탐색으로 등록할 수 있다.

서블릿, 필터와 리스너를 스프링 빈으로 등록하기

어떤 `Servlet`, `Filter` 혹은 서블릿 `*Listener` 인스턴스든지 스프링 빈으로 내장 컨테이너에 등록할 수 있다. 이러한 특징은 구성을 하는 동안 `application.yml`의 값을 이용하도록 하여 특별한 혜택을 얻을 수 있다.

기본적으로 컨텍스트는 단독 서블릿이 `/`에 대해서만 대응하도록 되어 있다. 다양한 서블릿 빈즈의 경우 빈의 이름이 경로에 대한 접두사로 사용된다. 필터는 `/*`에 대응한다.

관례를 기반으로 한 대응만으로는 유연한 대응이 어렵지만 `ServletRegistrationBean`, `FilterRegistrationBean`과 `ServletListenerRegistrationBean` 클래스를 이용하면 완벽하게 대응 가능하다.

서블릿 컨텍스트 초기화

내장 서블릿 컨테이너에서 서블릿 3.0+ 이상의 `javax.servlet.ServletContainerInitializer` 인터페이스나 스프링의 `org.springframework.web.WebApplicationInitializer` 인터페이스를 바로 실행할 수는 없다. 이는 war 파일을 실행하도록 설계되어 있는 서드파티 라이브러리의 전략을 스프링 부트에서 우회하여

위험을 줄이기 위한 고의적인 설계 전략이다.

만약 스프링 부트 애플리케이션에서 서블릿 컨텍스트 초기화를 수행해야 할 상황이라면, `org.springframework.boot.context.embedded.ServletContextInitializer` 구현체를 빈으로 등록하면 된다. `onStartup` 메서드에서 제공하는 `ServletContext`에 접근하고 필요하다면 `WebApplicationInitializer` 을 사용할 수 있다.

서블릿, 필터와 리스너 탐색

내장 컨테이너를 사용할 때 `@WebServlet`, `@WebFilter`와 `@WebListener`를 선언한 클래스는 `@ServletComponentScan`을 통해서 자동으로 등록된다.

TIP

독립형 컨테이너로 구동시에는 컨테이너의 빌트인 발견 기작을 사용하기 때문에 `@ServletComponentScan`는 효과가 없다.

EmbeddedWebApplicationContext

스프링 부트는 내장 서블릿 컨테이너 지원을 위하여 새로운 타입의 `ApplicationContext`을 사용한다. `EmbeddedWebApplicationContext`는 구동시 `EmbeddedServletContainerFactory`에 탐색되는 특별한 `WebApplicationContext` 유형이다. 사용하는 컨테이너에 따라서 `TomcatEmbeddedServletContainerFactory`, `JettyEmbeddedServletContainerFactory` 혹은 `UndertowEmbeddedServletContainerFactory` 중에서 하나가 자동구성된다.

NOTE

위에서 거론한 구현체 클래스를 사용할 필요는 없다. 애플리케이션 대부분 자동구성되고 이 과정에서 `ApplicationContext`와 `EmbeddedServletContainerFactory`가 만들어지는데 이것들만으로도 충분히 사용가능할 것이다.

내장 서블릿 컨테이너 재정의

서블릿 컨테이너 설정 시 스프링 `Environment` 속성을 사용할 수 있다. 보통 설정은 `application.yml`에 정의한다.

공통적인 서버설정에 포함된 것들은:

- 네트워크 설정: HTTP 요청 포트(`server.port`), 인터페이스 주소(`server.address`), 기타등등.
- 세션 설정: 세션 영속성(`server.session.persistence`), 세션 시간초과(`server.session.timeout`), 세션데이터 위치(`server.session.store-dir`) 그리고 세션쿠키 구성(`server.session.cookie.*`).
- 에러 관리: 에러 페이지 위치(`server.error.path`), 기타등등.
- [SSL 구성](#)
- [HTTP 응답 압축 활성화](#)

스프링 부트는 가능한 공통 설정을 적용하려고 시도하지만 항상 가능한 것은 아니다. 이런 경우에는 서버명세로 제공되는 네임스페이스를 제공한다(예: `server.tomcat`과 `server.undertow`). 그 예로 [접근기록](#)은 내장 서블릿 컨테이너에 명세된 기능을 구성할 수 있다.

TIP

전체 목록은 `ServerProperties`에서 볼 수 있다.

JSP 제한

스프링 부트 애플리케이션이 실행될 때 내장 서블릿 컨테이너(실행가능한 jar인 경우도 포함해서)를 사용하는 경우 JSP 지원에 몇가지 제약이 있다.

- **톰캣**: WAR 패키징 혹은 실행가능한 WAR인 경우에는 동작한다. 실행가능한 jar 인 경우에는 톰캣 에 하드코딩 되어 있는 파일 패턴 때문에 동작하지 않는다.
- **제티**: WAR 패키징할 경우 동작, 실행가능한 WAR인 경우에도 동작하며 어떤 표준 컨테이너에도 배포가능하다.
- **{underdow}**: JSP 지원하지 않음

NOTE

톰캣의 경우 JSP 명세를 구현하기 위해 **Jasper 2 JSP Engine**을 사용한다. 이때 JSP 페이지를 컴파일하여 서블릿 코드로 변환할 때 **WEB-INF/ 디렉터리를 주요 경로로 하여 하드코딩 설정되어 있다. 실행가능한 jar가 되는 경우에는 WEB-INF 디렉터리를 무시하기 때문에 WAR와는 달리 JAR로 배포할 때는 JSP를 지원하지 못한다. 그렇기 때문에 실행가능한 jar로 배포하려는 경우 JSP 외의 템플릿 엔진을 고려**해야한다.**

3.6. 보안

스프링 시큐리티가 클래스패스 상에 존재하고 웹애플리케이션이라면 모든 HTTP 종단점(EndPoints)에는 '기본(basic)' 인증방식이 보안기본이 된다. **@EnableGlobalMethodSecurity**를 추가로 사용하면 웹 애플리케이션의 메서드 수준까지 보안을 확대할 수 있다. 추가적인 정보는 [스프링 시큐리티 참조문서](#)를 살펴보기 바란다.

기본 **AuthenticationManager**에는 유저가 1명 등록되어 있다('유저'의 username과 무작위 비밀번호는 애플리케이션이 시동될 때 **INFO** 레벨로 출력된다)

Using default security password: 78fa095d-3f4c-48b1-ad50-e24c31d5cf35 ①

① 애플리케이션이 실행될 때 한번 출력된다.

NOTE

로깅 구성을 미세하게 조정하는 편이라면 **org.springframework.boot.autoconfigure.security** 카테고리에 대해서 **INFO** 레벨로 확보하지 않으면 기본 패스워드가 출력되지 않을 것이다.

security.user.password 속성을 정의하여 비밀번호를 변경할 수 있다. 이 속성 외에도 **SecurityProperties** ('security' 접두사를 가지고 있는 속성들)에 외부확장할 수 있는 다양한 속성들이 있다.

기본 보안 구성은 **SecurityAutoConfiguration**을 구현한 것이며 관련된 클래스를 불러오기 하고 있다(웹 보안을 위한 **SpringBootWebSecurityConfiguration**과 웹 애플리케이션이 아니어도 사용가능한 인증 구성을 위한 **AuthenticationManagerConfiguration**).

```

@Configuration
@ConditionalOnClass({ AuthenticationManager.class,
    GlobalAuthenticationConfigurerAdapter.class })
@EnableConfigurationProperties
@Import({ SpringBootWebSecurityConfiguration.class,
    AuthenticationManagerConfiguration.class,
    BootGlobalAuthenticationConfiguration.class,
    SecurityDataConfiguration.class })
public class SecurityAutoConfiguration {

```

기본 웹 애플리케이션 보안구성은 `@EnableWebSecurity` 으로 비활성화된다(액추에이터의 보안이나 인증 매니저 구성이 비활성화되는 것은 아니다). 외부 속성을 재정의하여 사용하거나 `WebSecurityConfigurerAdapter` 타입의 빈과 외부속성을 재정의하여 사용할 수 있다(폼(Form) 방식 로그인 추가). `AuthenticationManager` 타입의 빈을 추가하여 인증 매니저 구성을 비활성화할 수 있다. 혹은 `@Configuration` 클래스에서 `AuthenticationManagerBuilder` 를 주입받아 전역 `AuthenticationManager`를 구성할 수도 있다. [스프링 부트 예제](#)에 있는 다양한 보안 애플리케이션을 참고하여 시작할 수 있을 것이다.

웹 애플리케이션에 적용되는 기본 기능은 다음과 같다:

- 하나의 유저와 함께 인-메모리에 저장되는 `AuthenticationManager`(유저에 관한 속성은 `SecurityProperties.User`를 보라)
- 정적 자원 위치에 대한 무시(비보안) 처리(`/css/`, `/js/`, `/images/`, `/webjars/` 그리고 `**/favicon.ico`)
- 모든 종단점에 대한 HTTP 기본 보안
- 스프링 `ApplicationEventPublisher`를 이용한 보안 이벤트 게시(인증 성공과 실패 그리고 접근 거부).

위에서 거론된 것들은 외부 속성을 사용하여 활성화 및 비활성화 혹은 변경이 가능하다(`security.*`). 접근규칙에 대해서는 다른 자동구성 기능을 변경하지 않아도 `WebSecurityConfigurerAdapter` 타입에 `@Bean`의 빈에 `@Order(SecurityProperties.ACCESS_OVERRIDE_ORDER)`를 함께 사용하면 필요에 따라 구성이 가능하다.

NOTE `WebSecurityConfigurerAdapter`는 기본적으로 어느 경로에나 적용된다. 만약 스프링 부트의 자동구성되어 있는 접근 규칙을 모두 덮어쓰기를 원하지 않는다면 어댑터를 이용해서 원하는 경로에 대해서만 명확하게 구성가능하다.

3.6.1. 액추에이터 보안(Actuator Security)

액추에이터를 사용한다면 다음과 같은 상황이 벌어진다:

- 애플리케이션 종단점에 대해서 보안관리를 하지 않는다해도 관리(management) 종단점은 보안관리 대상이 된다.
- 보안 이벤트는 `AuditEvent` 인스턴스로 변형되어 `AuditEventRepository`로 발행된다.
- 기본 유저는 `USER` 역할과 유사한 `ACTUATOR` 역할을 가진다.

액추에이터 보안기능은 외부 속성(`management.security.*`)에 의해 변경가능하다. 애플리케이션 접근규칙에 `WebSecurityConfigurerAdapter` 유형의 `@Bean` 을 덮어쓴후 액추에이터 접근규칙을 덮어쓰기 하고 싶지 않다면 `@Order(SecurityProperties.ACCESS_OVERRIDE_ORDER)` 을 사용하고 액추에이터 접근 규칙을 덮어쓰고 싶다면 `@Order(ManagementServerProperties.ACCESS_OVERRIDE_ORDER)` 을 사용하면 된다.

3.7. SQL 데이터베이스 이용

스프링 프레임워크는 SQL 데이터베이스를 위한 다양한 지원을 제공한다. `JdbcTemplate`를 이용해서 직접 JDBC에 접근하는 것부터 하이버네이트 같은 '객체관계매핑(ORM)' 기술에 이르기까지. 스프링 Data는 인터페이스로부터 `Repository` 구현체를 생성하고 메서드명으로 쿼리를 생성하는 등의 추가적인 기능을 제공한다.

3.7.1. DataSource 구성

자바 `javax.sql.DataSource` 인터페이스는 데이터베이스 연결을 위한 표준 메서드를 제공한다. 전통적으로 DataSource는 URL과 인증을 통해 데이터베이스연결을 진행한다.

TIP 자세한 내용은 '[DataSource 재정의 구성](#)'을 살펴보기 바란다.

내장 데이터베이스 지원

애플리케이션 개발시 인-메모리 내장 데이터베이스를 사용하면 여러모로 편리하다. 인-메모리 데이터베이스는 저장된 내용에 대해서 영속화를 지원하지 않는다. 애플리케이션이 구동되어 준비되는 과정에서 필요한 데이터를 준비하는 과정이 필요할 수도 있다.

TIP 이에 대해서는 '[데이터베이스 초기화](#)'를 살펴보기 바란다.

스프링 부트는 내장 `H2`, `HSQL`과 `Derby` 데이터베이스에 대한 자동구성을 지원한다. 연결URL 정보를 제공할 필요도 없고 필요하다면 내장 데이터베이스에 대한 의존성을 빌드에 추가할 수 있다.

예를 들어, `build.gradle` 에 의존성을 추가한다면:

```
dependencies {
    compile('org.springframework.boot:spring-boot-starter-data-jpa') ①
    runtime('com.h2database:h2') ②
}
```

① `spring-boot-starter-data-jpa`를 추가하면 `spring-boot-starter-aop`, `spring-boot-starter-jdbc`에 대한 의존성이 추가되며 하이버네이트 구현체 대한 의존성이 추가된다.

② parent POM에 버전이 정의되어 있기 때문에 버전을 별도로 작성하지 않아도 된다. 개발과 운영 배포시에는 `mariaDB`를 사용할 예정이기 때문에 로컬에서 실행시에만 사용한다.

TIP

내장 데이터베이스에 URL 연결 구성시 데이터베이스의 자동종료 기능을 비활성화 시켜야만 한다. H2를 사용하는 경우에는 URL 마지막에 `DB_CLOSE_ON_EXIT=FALSE` 항목을 추가해야 하고, HSQLDB를 사용시에는 `shutdown=true`는 사용하면 안된다. 데이터베이스 자동종료 기능을 비활성화시켜야 더이상 데이터베이스를 연결할 필요가 없을 때 스프링 부트에서 데이터베이스 연결종료를 제어할 수 있다.

운영 데이터베이스 연결하기

운영 데이터베이스 연결은 풀링 `DataSource`를 이용하여 자동구성할 수 있다. 다음 항목들을 살펴보고 선택하기 바란다.

- 스프링 부트에서는 톰캣 풀링 `DataSource`를 기본사용한다.
- HikariCP를 사용하는 경우에도 풀링 설정이 가능하다.
- 톰캣 풀링 `DataSource` 혹은 HikariCP를 사용하지 않는 경우에는 Commons DBCP를 사용할 수 있지만 Deprecated 될 예정이라 사용하지 않는 편이 낫다.
- Commons DBCP2는 사용 가능하다. 만약 `spring-boot-starter-jdbc` 혹은 `spring-boot-starter-data-jpa` 스타터를 사용하면 자동으로 `tomcat-jdbc` 의존성이 추가된다.

NOTE

`spring.datasource.type` 속성으로 사용하려는 커넥션 풀을 정의하면 얼마든지 변경이 가능하다. 일반적으로 톰캣 컨테이너를 사용하면 `tomcat-jdbc`가 제공된다.

TIP

추가적으로 커넥션 풀은 언제나 수동구성이 가능하다. `DataSource` 빈을 정의하면 자동구성은 진행되지 않는다.

`DataSource` 구성은 `spring.datasource.*` 속성을 통해서 외부구성으로 제어가 가능하다. `application.yml`에서는 다음과 같이 정의가 가능하다:

```
spring.datasource:
  url: jdbc:mariadb://localhost:3306/test
  username: honeymon
  password: honeymonpass
  driver-class-name: org.mariadb.jdbc.Driver
```

NOTE

`spring.datasource.url` 속성을 정의하면 URL을 사용하고 그렇지 않은 경우 스프링 부트는 내장 데이터베이스를 자동구성한다.

TIP

스프링 부트는 URL로부터 대부분의 데이터베이스를 구분하기 때문에 별도로 `driver-class-name`을 지정할 필요는 없다.

보다 자세한 지원 선택사항에 대해서는 `DataSourceProperties`을 살펴보기 바란다. 실제 구현에 상관없이 동작하는 표준 선택사항들이다. 또한 각 구현체에 따라 명시적으로 접미사로 구분하여 정의하는 것도 가능하다

(`spring.datasource.tomcat.`, `spring.datasource.hikari.`, `spring.datasource.dbcp2.*`).

보다 자세한 사항은 커넥션 풀 구현체에 관한 문서를 참조하기 바란다.

3.7.2. JdbcTemplate 사용

스프링의 `JdbcTemplate`와 `NamedParameterJdbcTemplate` 클래스는 자동구성되어 있기 때문에 `@Autowired`로 작성하는 컴포넌트 안에 바로 주입하여 사용할 수 있다.

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;

@Component
public class MyJdbcTemplateBean {
    private final JdbcTemplate jdbcTemplate;

    @Autowired
    public MyJdbcTemplateBean(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    // 구현사항
}
```

3.7.3. MyBatis 사용

3.7.4. JPA 그리고 '스프링 Data'

Java Persistence API(줄여서 JPA)는 객체를 데이터베이스에 관계지어 매핑하는 표준기술이다. **spring-boot-starter-data-jpa**는 빠르게 시작할 수 있는 방법을 제공한다. 다음의 주요 의존성이 따라온다:

- 하이버네이트(Hibernate): 가장 인기있는 JPA 구현체 중 하나
- 스프링 Data JPA: JPA 기반 {repo} 구현체를 쉽게 만들 수 있음
- 스프링 ORM: - 스프링 프레임워크의 주요 ORM 기능 지원

TIP 여기서 스프링 Data JPA에 대해서 깊이 다루지는 않겠다. [스프링 Data JPA](#)와 [하이버네이트](#) 참조문서를 살펴보기 바란다. 혹은 에이콘에서 나온 '[자바 ORM 표준 JPA 프로그래밍](#), 김영한 저'를 살펴보기 바란다.

NOTE 스프링 부트는 기본적으로 하이버네이트 5.x 버전을 사용하지만 다른 버전을 사용하는 것도 가능하다.

엔티티 클래스

일반적으로 JPA 엔티티 클래스는 **persistence.xml** 파일에 정의한다. 그러나 스프링 부트는 '엔티티 탐색'을 사용하기 때문에 이 파일이 필요하지 않다. 기본적으로 메인 구성 클래스(**@EnableAutoConfiguration** 혹은 **@SpringBootApplication** 이 선언되어 있는)를 기준으로 하위에 모든 패키지를 탐색한다.

@Entity, **Embeddable** 혹은 **@MappedSourceclass** 가 선언되어 있는 클래스가 대상이 된다. 일반적인 엔티티 클래스는 다음과 같다:


```

import java.io.Serializable;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;

import org.springframework.util.Assert;

import lombok.AccessLevel;
import lombok.EqualsAndHashCode;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.ToString;

@Entity
@Getter
@NoArgsConstructor(access=AccessLevel.PROTECTED) ①
@ToString
@EqualsAndHashCode(of = { "id" })
public class User implements Serializable {
    @Id
    @GeneratedValue
    private Long id;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String password;

    @Column(length = 50)
    private String name;

    // 필드 추가 설정

    public User(String email, String password) {
        Assert.hasText(email, "User required email");
        Assert.hasText(password, "User required password");

        this.email = email;
        this.password = password;
    }

    // 기타 코드
}

```

① JPA 명세서에서는 기본생성자를 요구한다. **protected** 접근자료를 통해 다른 패키지에서는 바로 사용할 수 없도록

제한하자.

TIP | `@EntityScan` 애너테이션을 이용해서 스캐닝 위치를 변경할 수 있다.

스프링 Data JPA {repo}

스프링 Data {repo}는 데이터 접근을 정의하는 인터페이스다. JPA 쿼리는 메서드명을 통해 자동생성된다. 예를 들어 `UserRepository` 인터페이스에 정의되어 있는 `findByEmail(String email)` 메서드는 주어진 이메일에 일치하는 모든 사용자를 찾을 것이다.

복잡한 쿼리의 경우에는 스프링 Data의 `Query` 애너테이션을 사용할 수 있다.

스프링 Data 저장소는 `Repository` 혹은 `CrudRepository` 인터페이스를 확장하여 사용한다. 만약 자동구성을 사용하고 있다면, 저장소는 메인 구성 클래스 하위 패키지를 탐색할 것이다.

여기 일반적인 스프링 Data 저장소가 있다:

```
package io.honeymon.boot.springboot.domain;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {

    List<User> findByEmail(String email);

}
```

TIP | 우리는 스프링 Data JPA의 극히 일부만 살펴봤다. 자세한 사항은 [참조 문서](#)를 보기 바란다.

JPA 데이터베이스 생성 및 삭제

기본적으로 JPA 데이터베이스는 내장 데이터베이스(H2, HSQL 혹은 Derby)를 사용할 때만 자동생성된다.

`spring.jpa.*` 속성을 통해 JPA 설정을 구성할 수 있다. 예를 들어 애플리케이션 구동시 테이블을 생성하고 종료시 테이블을 삭제하고 싶은 경우에는 `application.yml`에 다음과 같이 설정하면 된다:

```
spring.jpa.hibernate.ddl-auto: create-drop
```

NOTE

이 설정은 테스트 환경에서만 정의하기 바란다. 프로파일을 분명하게 구분하여 정의하기 바란다. 이 기능에 대해서 제대로 살펴보지 않고 사용하면 운영 데이터베이스의 데이터가 날아가는 광경을 목격할 수도 있다. 데이터베이스 스키마에 대한 관리의 별도의 DB 마이그레이션 도구(FlywayDB, Liquibase 등)를 사용하는 경우가 많으며 이를 위해 `spring.jpa.hibernate.ddl-auto: validated`로 선언하여 사용한다.

뷰에서 엔티티매니저 열기

웹 애플리케이션을 실행하면 스프링 부트는 'Open EntityManager in View' 패턴을 위해 `OpenEntityManagerInViewInterceptor`를 기본등록한다. 웹 뷰에서 지연읽기를 지원한다. 이런 행동을 원하지 않는 경우에는 `application.yml(or properties)`파일에서 `spring-jpa.open-in-view`를 `false`로 정의하면 된다.

NOTE

하이버네이트 내부 속성을 정의해야할 경우에는 `spring.jpa.properties.*`를 사용하기 바란다. 접두사는 하이버네이트 엔티티 매니저에 전달되기 전에 제거된다. 예를 들자면:

```
spring.jpa.properties.hibernate.globally_quoted_identifiers:
true
```

가 하이버네이트 엔티티 매니저에 전달될 때는 `hibernate.globally_quoted_identifiers`가 된다.

기본적으로 DDL 실행 및 검증은 `ApplicationContext`가 시작되고 나서 진행된다. 이에 대해서는 `spring.jpa.generate-ddl`으로 설정가능하지만 하이버네이트가 자동고성되는 경우에는 `ddl-auto`가 더 상위에 있기에 비활성화 된다.

3.7.5. H2 사용

지금은 로컬에 mariaDB를 설치하여 사용하고 있지만, 이전에는 [H2 데이터베이스](http://h2database.com/html/main.html) (<http://h2database.com/html/main.html>)를 개발에 사용해왔다. 파일로 저장하도록 하여 데이터 영속화도 처리하면서 개발에 적용했다. 서비스 운영에 적용하기는 곤란한 부분들이 있지만 개발시에는 간단하게 파일DB를 생성할 수 있다. 그래서 `application.yml`에 프로파일을 구분지어 로컬, 테스트 시에는 h2database를 사용하고, 개발, 운영에서는 mariadDB를 사용하도록 구성해왔다.

프로파일 구성

스프링 프로파일 기능을 이용해서 각 환경별 데이터베이스 구성을 살펴보자.

```
# default: 공통Common 적용
# 공통 속성 생략
---
spring:
  profiles: local
  datasource:
    url: jdbc:h2:~/ .boot-spring-boot;DB_CLOSE_ON_EXIT=FALSE
    username: sa
    password:
  jpa:
    hibernate:
      ddl-auto: update
---
# test profile 사용시 적용
```

```

spring:
  profiles: test
  datasource:
    url: jdbc:h2:mem:test;DB_CLOSE_ON_EXIT=FALSE
    username: sa
    password:
  jpa:
    hibernate:
      ddl-auto: create-drop
server:
  port: ${random.int[8000,9000]}

---
# 개발서버 적용
spring:
  profiles: dev
  datasource:
    url: jdbc:mariadb://<<url>>:3306/dev
    username: honeymon
    password: devhoneymon
  jpa:
    hibernate:
      ddl-auto: validate
server:
  port: 9000

---
# 운영서버 적용
spring:
  profiles: prd
  datasource:
    url: jdbc:mariadb://<<url>>:3306/prd
    username: honeymon
    password: prdhoneymon
  jpa:
    hibernate:
      ddl-auto: validate
server:
  port: 5000 # AWS Elastic Beanstalk 에서는 5000번을 사용해야한다. 단
SERVER_PORT 로 정의해야 한다.

```

NOTE

위의 구성으로 **local**, **test** 에서는 H2를 사용하고, **dev**, **prd**에서는 mariaDB를 사용하도록 구성하였다. 애플리케이션 구동시 **spring.profiles.active** 속성을 반드시 부여해야 한다.

H2 웹콘솔 사용

H2에 대해서 스프링 부티 자동구성하도록 하면 H2 웹콘솔을 사용할 수 있다. 웹콘솔은 브라우저 기반의 콘솔이라고 보면 된다. 웹콘솔이 자동구성되는 조건은 다음과 같다.

- 웹 애플리케이션을 개발
- `com.h2database:h2`가 클래스패스에 존재
- 스프링 부트 개발자도구 사용

TIP

스프링 부트 개발자도구 사용하지 않는 상황에서 H2 웹콘솔을 사용하고 싶을 때가 있다. 그 때에는 `spring.h2.console.enabled` 속성을 `true`로 지정하면 된다. H2 웹콘솔은 개발동안에만 사용하도록 주의하면서 운영모드에서는 비활성화 처리하도록 하자.

H2 웹콘솔 경로 변경

웹콘솔의 기본경로는 `/h2-console`이다. `spring.h2.console.path` 속성으로 웹콘솔의 경로를 재정의할 수 있다.

3.8. NoSQL 기술 이용

스프링 Data는 [MongoDB](https://goo.gl/RJJJc3)(https://goo.gl/RJJJc3), [Neo4J](https://goo.gl/Hdc8AV)(https://goo.gl/Hdc8AV), [Elasticsearch](https://goo.gl/MKAR13)(https://goo.gl/MKAR13), [Solr](https://goo.gl/mNRmpb)(https://goo.gl/mNRmpb), [Redis](https://goo.gl/QYRr7q)(https://goo.gl/QYRr7q), [Gemfire](https://goo.gl/u3xwfs)(https://goo.gl/u3xwfs), [Cassandra](https://goo.gl/BFEbxF)(https://goo.gl/BFEbxF), [Couchbase](https://goo.gl/rGX4y6)(https://goo.gl/rGX4y6)와 [LDAP](https://goo.gl/vZqCmB)(https://goo.gl/vZqCmB)을 포함한 다양한 NoSQL 기술에 대한 접근을 도와주는 프로젝트들을 제공한다. 스프링 부트는 Redis, MongoDB, Neo4j, Elasticsearch, Solr Cassandra, Couchbase와 LDAP를 위한 자동구성을 제공한다. 다른 프로젝트를 사용하고 있다면 필요한 구성을 별도로 할 수 있다. 이와 관련된 내용에 대해서는 <http://projects.spring.io/spring-data>를 참조하기 바란다.

3.8.1. 레디스(Redis)

[레디스](http://redis.io/)(http://redis.io/)는 캐시, 메시지 브로커 그리고 풍부한 기능을 가진 키-값 저장소다. 스프링 부트는 [Jedis](https://goo.gl/qBrTLa)(https://goo.gl/qBrTLa)클라이언트 라이브러리를 위한 기초적인 자동구성과 [Redis](https://goo.gl/QYRr7q)(https://goo.gl/QYRr7q)에서 제공하는 것에 대한 추상체를 제공한다.

레디스 연결하기

여느 스프링 빈과 같이 자동구성된 `RedisConnectionFactory`, `StringRedisTemplate` 혹은 순수한 `RedisTemplate` 인스턴스를 주입받을 수 있다. 기본 인스턴스는 `localhost:6379`를 사용하여 레디스 서버에 연결된다.

```
@Component
public class RedisBean {
    private StringRedisTemplate template;
    @Autowired
    public RedisBean(StringRedisTemplate template) {
        this.template = template;
    }
    // ...
}
```

자동구성된 유형에 `@Bean`을 추가하면 기본값으로 바뀐다(`RedisTemplate`를 제외하고 다른 유형의 빈 이름이

`redisTemplate`가 된다). 만약 `common-pool2`가 클래스패스에 있다면 기본적으로 풀링된 커넥션 팩토리를 얻을 수 있다.

3.8.2. 몽고DB(MongoDB)

몽고DB(<https://www.mongodb.com/>)는 전통적인 테이블 기반 관계형 데이터를 대신하여 JSON과 유사한 스키마를 사용하는 오픈소스 NoSQL 문서 데이터베이스다. 스프링 부트는 `spring-boot-starter-data-mongodb` 스타터를 통해서 몽고DB에서 동작하는 다양한 편이를 제공한다.

몽고DB 데이터베이스 연결

몽고DB 데이터베이스에 접근하기 위한 자동구성된

`org.springframework.data.mongodb.MongoDbFactory`을 주입할 수 있다. 인스턴스는 기본적으로 `mongodb://localhost/test` URL을 사용하여 몽고DB에 연결한다:

```
import org.springframework.data.mongodb.MongoDbFactory;
import com.mongodb.DB;

@Component
public class MongoDbBean {
    private final MongoDbFactory mongo;

    @Autowired
    public MongoDbBean(MongoDbFactory mongo) {
        this.mongo = mongo;
    }
    // ...
    public void getDb() {
        DB db = mongo.getDb();
        // ...
    }
}
```

`spring.data.mongodb.uri` 속성을 이용해서 URL을 변경할 수 있으며 레플리카(replica) 구성에 대해서도 설정할 수 있다:

```
spring.data.mongodb.uri:
mongodb://user:secret@mongo1.example.com:12345,mongo2.example.com:23456/test
```

몽고 2.x에서는 `host/port`를 사용할 수 도 있다. 예를 들어, `application.yml`에 다음과 같이 정의가능하다:

```
spring.data.mongodb.host: mongoserver
spring.data.mongodb.port: 27017
```

NOTE

몽고 3.0 자바 드라이버를 사용한다면 `spring.data.mongodb.host`와 `spring.data.mongodb.port`를 지원하지 않는다. 이런 경우에는 모든 구성에 대해서 `spring.data.mongodb.uri`를 사용할 수 있다.

TIP

만약 기본포트 `27017`를 사용한다면 위의 예제에서 `spring.data.mongodb.port`줄은 제거해도 된다. `MongoProperties`에서 기본포트로 `27017`를 사용한다.

TIP

스프링 Data 몽고를 사용하지 않는다면 `MongoDbFactory`를 사용하는 대신에 `com.mongodb.Mongo`를 주입할 수 있다.

MongoTemplate

스프링 Data 몽고는 스프링의 `JdbcTemplate`와 매우 유사한 형태의 `MongoTemplate`를 제공한다. `JdbcTemplate`처럼 스프링 부트에서 자동구성한 빈을 간단히 주입할 수 있다:

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.stereotype.Component;

@Component
public class MongoBean {
    private final MongoTemplate mongoTemplate;
    @Autowired
    public MongoBean(MongoTemplate mongoTemplate) {
        this.mongoTemplate = mongoTemplate;
    }
    // ...
}
```

보다 자세한 내용은 `MongoOperations` 문서를 살펴보기 바란다.

스프링 Data 몽고DB {repo}

스프링 Data는 몽고DB를 지원하는 {repo}를 포함하고 있다. 앞서 살펴봤던 JPA {repo}처럼 기본적인 규칙은 메서드명을 기반으로 해서 쿼리를 자동생성한다.

사실 스프링 Data JPA와 스프링 Data MongoDB는 같은 공통 인프라스트럭처를 공유하고 있다. 앞서 살펴봤던 JPA 예제처럼, `City`라는 몽고 데이터 클래스를 가지고 동작하는 방식은 비슷하다:

```
import org.springframework.data.domain.*;
import org.springframework.data.repository.*;

public interface CityRepository extends Repository<City, Long> {
    Page<City> findAll(Pageable pageable);
    City findByNameAndCountryAllIgnoringCase(String name, String country);
}
```

풍부한 객체 매핑 기술을 포함한 스프링 Data MongoDB에 대해 보다 자세히 알고 싶다면 [MongoDB 참조문서 \(https://goo.gl/RJJJc3\)](https://goo.gl/RJJJc3)를 살펴보기 바란다.

내장 몽고

스프링 부트는 [내장 몽고\(https://goo.gl/V4UuhW\)](https://goo.gl/V4UuhW)에 대한 자동구성을 제공한다. 사용하기 위해서는 `de.flapdoodle.embed:de.flapdoodle.embed.mongo` 의존성을 스프링 부트 애플리케이션에 추가해야 한다. `MongoAutoConfiguration`에 의해 생성된 `MongoClient`는 무작위로 할당받은 포트로 자동구성된다.

몽고에서 사용하는 포트는 `spring.data.mongodb.port` 속성으로 정의할 수 있다. 포트값에 0을 설정하면 할당가능한 포트 중 하나가 무작위로 사용된다.

NOTE

포트를 재정의하지 않았다면 내장 몽고는 기본적으로 무작위 포트(27017를 사용하기 보다는)를 사용한다.

만약 클래스패스 상에 SLF4J 가 있다면, 몽고는 자동으로

`org.springframework.boot.autoconfigure.mongo.embedded.EmbeddedMongo` 라는 로거명으로 출력한다.

`IMongodConfig`와 `IRuntimeConfig`빈을 정의해서 몽고 인스턴스의 구성과 로깅 설정을 제어할 수 있다.

3.8.3. 엘라스틱서치(Elasticsearch)

[엘라스틱서치\(http://www.elasticsearch.org/\)](http://www.elasticsearch.org/)는 오픈소스, 분산형, 실시간 검색 및 분석엔진이다. 스프링 부트는 엘라스틱 서치에 대한 기본적인 자동구성과 [Elasticsearch\(https://goo.gl/MKAR13\)](https://goo.gl/MKAR13)에서 제공하는 추상체를 제공한다. `spring-boot-starter-data-elasticsearch` 스타터를 추가하면 간단하다. 스프링 부트는 또한 [Jest\(https://goo.gl/kvC4Jb\)](https://goo.gl/kvC4Jb)도 지원한다.

Jest 를 통해 엘라스틱서치 연결

클래스패스 상에 `Jest`가 있다면 기본적으로 `http://localhost:9200/`를 겨냥한 자동구성된 `JestClient`를 주입받을 수 있다. 클라이언트가 어떻게 조정되어 있는지 살펴보자:

```
spring.elasticsearch.jest.uris=http://search.example.com:9200
spring.elasticsearch.jest.read-timeout=10000
spring.elasticsearch.jest.username=user
spring.elasticsearch.jest.password=secret
```


보다 향상된 재정의의 할 수 있도록 임의의 `HttpClientConfigBuilderCustomizer` 구현체를 등록할 수 있다. 다음 예제의 경우 HTTP 설정을 추가적으로 조정하고 있다:

```
static class HttpSettingsCustomizer implements
HttpClientConfigBuilderCustomizer {
    @Override
    public void customize(HttpClientConfig.Builder builder) {
        builder.maxTotalConnection(100).defaultMaxTotalConnectionPerRoute
(5);
    }
}
```

스프링 Data를 사용하여 엘라스틱서치 연결하기

여느 스프링 빈과 마찬가지로 자동구성된 `ElasticsearchTemplate` 와 엘라스틱 `Client` 인스턴스를 주입받을 수 있다. 기본적으로 인스턴스는 로컬 인-메모리 서버(엘라스틱서치에서 말하는 `Node`)를 내장하고 있으며 서버의 홈디렉토리는 현재 작업 디렉토리를 사용한다. 다음 설정에서 엘라스틱 서치가 어디에 저장되는지를 정의할 수 있다:

```
spring.data.elasticsearch.properties.path.home: /foo/bar
```

이에 대한 대안으로 쉼표(,)로 구분되는 `host:port` 목록을 정의할 수 있는 `spring.data.elasticsearch.cluster-nodes` 속성을 사용하여 원격 서버(예: `TransportClient`)로 전환할 수도 있다.

```
spring.data.elasticsearch.cluster-nodes: localhost:9300
```

```
@Component
public class ElasticsearchBean {
    private ElasticsearchTemplate template;
    @Autowired
    public ElasticsearchBean(ElasticsearchTemplate template) {
        this.template = template;
    }
    // ...
}
```

`ElasticsearchTemplate` 유형의 빈을 추가하면 기본 빈을 대체하게 된다.

스프링 Data 엘라스틱서치 {repo}

스프링 Data {repo}는 엘라스틱서치도 지원한다. 앞서 살펴봤던 JPA {repo} 처럼 기본 원리는 메서드 명을 기반으로 해서 쿼리가 자동생성된다.

사실 스프링 Data JPA와 스프링 Data Elasticsearch는 동일한 인프라스트럭처를 공유하고 있기 때문에 JPA 예제에서

봤던 것과 마찬가지로 엘라스틱서치 `@Document` 애너테이션이 정의되어 있는 클래스가 JPA의 `@Entity`와 동일한 방식으로 동작한다.

TIP

스프링 Data Elasticsearch에 대해 보다 자세한 내용을 알고 싶다면 [엘라스틱서치 참조문서 \(https://goo.gl/MKAR13\)](https://goo.gl/MKAR13)를 살펴보기 바란다.

3.9. 캐시

스프링 프레임워크는 애플리케이션에 캐싱을 추가할 수 있는 기능을 제공하고 있다. 코어 적용된 추상화는 메서드에 캐싱을 적용하여 캐시에서 사용가능한 정보를 기반으로 실행횟수를 줄인다. 캐싱 로직은 호출자에 대한 간섭없이 투명하게 적용된다. 스프링 부트는 `@EnableCaching` 애너테이션을 통해 캐싱 지원이 활성화되면 캐시 인프라를 자동구성한다.

NOTE

관련된 보다 자세한 내용은 스프링 프레임워크의 [관련 내용\(https://goo.gl/jPCTQh\)](https://goo.gl/jPCTQh)을 살펴보기 바란다.

서비스도 동작에 캐싱을 추가하는 것은 애너테이션만 추가하면 손쉽게 처리된다:

```
import org.springframework.cache.annotation.Cacheable
import org.springframework.stereotype.Component;

@Component
public class MathService {
    @Cacheable("piDecimals")
    public int computePiDecimal(int i) {
        // ...
    }
}
```

이 예제에서는 어쩌면 호사스러운 방식으로 캐싱을 사용하고 있다고 볼 수도 있다. `computePiDecimal`가 호출되면 `piDecimals` 캐시에서 `i` 인자에 일치하는 값을 찾아보게 된다. 만약 일치하는 것을 찾으면 캐시에 콘텐츠가 바로 반환되고 메서드가 호출되지 않는다. 반면, 메서드가 호출되면 계산값이 반환되기 전에 캐시가 갱신된다.

NOTE

표준 JSR-107(JCache) 애너테이션을 사용할 수도 있다. 그러나 스프링 부트팀에서는 스프링 프레임워크의 `@Cacheable` 등과는 혼용해서 사용하지 않기를 권장한다.

특정 캐시 라이브러리를 추가하지 않았다면 스프링 부트는 메모리에 `ConcurrentHashMap`을 이용해서 구현한 간단한 프로바이더(Simple Provider)를 자동구성한다. 이 프로바이더는 캐시가 필요하면 즉시 생성한다. 프로바이더는 운영시에 사용하는 것은 권장하지 않지만 기능을 이해하는 것을 돕고 시작하기에는 정말 좋다. 캐시 프로바이더를 사용하는 것에 어느정도 익숙해졌다면 애플리케이션에서 사용할 캐시를 어떻게 구성할지에 대해서는 문서를 살펴보고 좀 더 살펴볼길 권한다. 사실 모든 프로바이더는 애플리케이션에서 사용할 캐시에 대해 정확하게 구성해야 한다.

`spring.cache.cache-names` 속성을 이용해서 기본 캐시에 대한 재정의할 수 있는 몇가지 방법을 제공한다.

TIP

Simple Provider는 스프링 프레임워크에서 제공하는 애너테이션 `갱신(update, @CachePut)` 혹은 `환원(evict, @CacheEvict)`을 사용가능하다.

NOTE

인터페이스 기반이 아닌 빈으로 캐시 인프라스트럭처를 사용하려면 `@EnableCaching` 애너테이션의 `proxyTargetClass` 속성을 활성화하면 된다.

3.9.1. 지원하는 캐시 제공자(Provider)

캐시 추상화는 실제 저장소를 제공하지 않으며 `org.springframework.cache.Cache`와 `org.springframework.cache.CacheManager` 인터페이스를 통해 추상화를 구현한 것에 의존한다.

만약 `CacheManager` 타입의 빈이나 `cacheResolver`라는 이름 `CacheResolver`를 정의(`CachingConfigurer`를 살펴보라)하지 않았다면, 스프링부트는 다음순서로 프로바이더를 찾아내려고 시도한다.

- `Generic`(<https://goo.gl/zuGf1T>)
- `JCache` (JSR-107)(<https://goo.gl/4GnnSe>)(EhCache 3, Hazelcast, Infinispan, 기타등등)
- `EhCache 2.x`(<https://goo.gl/ayjHaX>)
- `Hazelcast` (<https://goo.gl/LBqC4N>)
- `Infinispan` (<https://goo.gl/ZNqKzV>)
- `Couchbase` (<https://goo.gl/7sd6tb>)
- `Redis`(<https://goo.gl/BFVbKL>)
- `Caffein` (<https://goo.gl/c5Vdaj>)
- `Guava`(deprecated) (<https://goo.gl/KH5M4a>)
- `Simple`(<https://goo.gl/Fez4S>)

TIP

`spring.cache.type` 속성을 통해서 캐시 프로바이더를 강제로 지정할수도 있다. 이 속성은 캐싱을 완전히 비활성화(<https://goo.gl/S8bBBN>)하는 것이 명확한 환경에서 사용한다(예: 테스트).

TIP

`spring-boot-starter-cache` 스타터를 사용하면 기초적인 캐시 의존성을 빠르게 추가할 수 있다. 스타터는 `spring-context-support`에 대한 의존성을 가지는데, 수동으로 의존성을 관리하는데 `JCache`, `EhCache 2.x` 혹은 `Guava`를 지원해야한다면 `spring-context-support`를 반드시 포함시켜야 한다.

스프링 부트에 의해 자동구성된 `CacheManager`라면 `CacheManagerCustomizer` 인터페이스를 구현한 빈을 정의하는 것으로 완벽히 초기화 되기전에 구성을 조정할 수 있다. 다음 null 값을 기본 맵Map으로 전달하는 경우를 설정한다.

```

@Bean
public CacheManagerCustomizer<ConcurrentMapCacheManager>
cacheManagerCustomizer() {
    return new CacheManagerCustomizer<ConcurrentMapCacheManager>() {
        @Override
        public void customize(ConcurrentMapCacheManager cacheManager) {
            cacheManager.setAllowNullValues(false);
        }
    };
}

```

NOTE

위 예제에서는 자동구성된 `ConcurrentMapCacheManager`을 기대했다. 만약 이런 경우가 아니라면(자동구성된 다른 캐시 프로바이더를 구성한 것이 제공된다면), 커스터마이저는 전혀 호출되지 않는다. `@Order` 혹은 `Ordered`을 사용하면 많은 커스터마이저의 순서를 지정하여 사용할 수 있다.

Generic

제네릭 캐시는 컨텍스트에 `org.springframework.cache.Cache` 빈 하나만 정의되어 있을 때 사용된다. 생성되는 모든 유형의 빈은 `CacheManager`으로 감싸진다.

JCache (JSR-107)

JCache는 클래스패스상에 `javax.cache.spi.CachingProvider`의 존재(예: JSR-107을 구현한 캐싱 라이브러리)가 있다면 발동되며 `spring-boot-starter-cache` 스타터에 의해 `JCacheCacheManager`를 제공한다. 다양한 구현 라이브러리가 있으며 스프링 부트는 EhCache 3, Hazelcast와 Infinispan에 대한 의존성을 관리한다. 다른 구현 라이브러리를 추가해도 잘 동작한다.

다수의 프로바이더가 존재하는 경우에 프로바이더를 명시적으로 지정해야할 경우가 있다. JSR-107 표준이 구성파일의 위치를 정의하는 표준화된 방법을 제공하지 않더라도, 스프링 부트는 구현 세부사항을 수용하기 위해 최선을 다한다.

```

# 다수의 프로바이더가 존재할 때 프로바이더를 명시적으로 지정해야할 경우
spring.cache.jcache:
    provider: com.acme.MyCachingProvider
    config: classpath:acme.xml

```

NOTE

캐시 라이브러리가 기본 구현과 JSR-107 지원을 모두 제공할 수 있지만 스프링 부트는 JSR-107 지원을 선호하므로 다른 JSR-107 구현으로 전환해도 동일하게 기능을 사용할 수 있다.

TIP

스프링 부트는 `Hazelcast`를 위한 일반적인 지원을 제공(<https://goo.gl/1nA9p5>)한다. `HazelcastInstance`만 존재하는 상황에서, `spring.cache.jcache.config`이 지정되지 않았다면 자동으로 `CacheManager`으로 재사용된다.

`javax.cache.cacheManager`를 재정의하는 몇가지 방법이 있다:

- `spring.cache.cache-names` 속성을 통해 시동시 캐시를 생성할 수 있다.
`javax.cache.configuration.Configuration` 빈을 재정의하면 재정의된 캐시를 사용한다.
- `org.springframework.boot.autoconfigure.cache.JCacheManagerCustomizer` 빈은 완벽한 재정의의 위해 `CacheManager`를 참조한다.

TIP

표준 `javax.cache.CacheManager` 빈이 정의되어 있다면, 자동적으로 `org.springframework.cache.CacheManager` 구현체에 감싸여 있을 것이다. 이런 경우에는 재정의할 수 없다.

EhCache 2.x

EhCache 2.x는 클래스패스 최상위에서 `ehcache.xml` 이란 파일을 찾는다. EhCache 2.x를 사용한다면 `spring-boot-starter-cache` 스타터에서 `EhCacheCacheManager`를 제공하며 캐시 매니저가 시작할 때도 저 파일을 이용한다. 그에 대한 대안으로 다음과 같이 사용할 수 있다:

```
spring.cache.ehcache.config: classpath:config/another-config.xml
```

Redis

Redis가 구성 및 사용가능하다면, `RedisCacheManager`가 자동구성된다. `spring.cache.cache-names` 속성을 사용하여 시동시에 캐시를 생성하는 것이 가능하다.

NOTE

두 개의 캐시가 동일한 키를 사용하는 경우 Redis는 기본적으로 겹치는 키를 가지고 잘못된 값을 반환할 수 있도록 키에 접두사가 추가된다. `RedisCacheManager`를 생성하는 경우 이 설정을 활성화해두는 것이 좋다.

Simple

프로바이더를 찾을 수 없을 경우에는 캐시를 저장하는 용도로 `ConcurrentHashMap`를 사용하는 간단한 구현체가 구성된다. 애플리케이션에 캐싱 라이브러리가 없는 경우에 적용된다. 캐시는 기본적으로 직접 작성하지만 `cache-names` 속성을 사용하여 가능한 캐시를 제한할 수 있다. 예를 들어, `foo` 와 `bar` 캐시만 사용하려는 경우는 다음과 같다:

```
spring.cache.cache-names: foo,bar
```

이렇게하고 애플리케이션에서 나열하지 않은 캐시를 사용하면 캐시가 필요할 때 실행시 실패하겠지만 사용하기로 정의한 캐시와 일치하지 않는 경우에는 시작도 되지 않는다.

None

구성클래스에 `@EnableCaching`이 사용됐다면 적절한 캐시가 구성될거라 기대할 수 있다. 특정 환경에서 캐시를 비활성화할 필요가 있다면 캐시 유형을 `none`으로 강제지정하여 구현체를 사용하지 않도록 할 수 있다:

```
spring.cache.type: none
```

3.10. REST 서비스 호출

애플리케이션에서 원격에 있는 REST 서비스를 호출해야할 때 스프링 프레임워크의 `RestTemplate`를 사용할 수 있다. `RestTemplate` 인스턴스는 사용되기 전에 재정의되는 것이 일반적이기 때문에 스프링 부트에서는 `RestTemplate` 빈을 자동구성하여 제공하지 않는다. 그러나 `RestTemplate`가 필요할 때 인스턴스를 생성할 수 있도록 자동구성된 `RestTemplateBuilder`를 제공한다. 자동구성된 `RestTemplateBuilder`는 `HttpMessageConverters`를 지원하는 `RestTemplate` 인스턴스를 만들어낸다.

일반적인 예제는 다음과 같다:

```
@Service
public class RestClientExampleBean {

    private final RestTemplate restTemplate;

    @Autowired
    public RestClientExampleBean(RestTemplateBuilder builder) {
        this.restTemplate = builder.build();
    }

    public String findOne() {
        return this.restTemplate.getForObject(
            "https://jsonplaceholder.typicode.com/posts/1", String.class);
    }
}
```

TIP

`RestTemplateBuilder`는 `RestTemplate`를 빠르게 구성하는데 사용할 수 있는 유용한 여러 메서드를 제공한다. 예를 들어, `builder.basicAuthorization("user", "password").build()`으로 기본인증을 추가할 수 있다.

3.10.1. RestTemplate 재정의

사용목적에 따라서 `RestTemplate`를 재정의하는 방법은 세 가지로 구분할 수 있다.

가능한 필요한 순간에 재정의하는 방법으로 자동구성된 `RestTemplateBuilder`를 주입받아서 각 메서드가 호출되는 시점에 생성하는 방법이 있다. 각 방법은 빌더를 이용해 재정의한 새로운 인스턴스가 반환된다.

애플리케이션 전체에 적용하려면 `RestTemplateCustomizer` 빈으로 재정의하여 추가하면 된다. 자동구성된 `RestTemplateBuilder`를 사용하는 빈에 재정의한 내용이 반영되어 모든 빈에 자동으로 등록된다.

여기 모든 프록시 호스트에서 `192.168.0.5`를 제외하는 재정의 예제가 있다:

```

static class ProxyCustomizer implements RestTemplateCustomizer {

    @Override
    public void customize(RestTemplate restTemplate) {
        HttpHost proxy = new HttpHost("proxy.example.com");
        HttpClient httpClient = HttpClientBuilder.create()
            .setRoutePlanner(new DefaultProxyRoutePlanner(proxy) {

                @Override
                public HttpHost determineProxy(HttpHost target,
                    HttpRequest request, HttpContext context)
                    throws HttpException {
                    if (target.getHostName().equals("192.168.0.5")) {
                        return null;
                    }
                    return super.determineProxy(target, request,
context);
                }

            })
            .build();
        restTemplate.setRequestFactory(
            new HttpComponentsClientHttpRequestFactory(httpClient));
    }

}

```

마지막으로 가장 극단적인 선택으로 `RestTemplateBuilder` 빈을 재정의하는 방법이 있다. 자동구성된 `RestTemplateBuilder`가 비활성화되고 `RestTemplateCustomizer` 빈을 차단하게 될 것이다.

3.11. 스프링 세션

스프링 부트는 폭넓은 스프링 세션 자동구성을 제공한다:

- JDBC
- MongoDB
- Redis
- Hazelcast
- HashMap

만약 스프링 세션이 가능하다면, 세션을 저장할 때 사용할 `StoreType`를 선택해야 한다. 백엔드 저장에 JDBC 인스턴스를 사용한다면 애플리케이션 설정은 다음과 같다:

```
spring.session.store-type: jdbc
```

TIP | 스프링 세션을 비활성화할 거라면 `store-type`을 `none`으로 설정한다.

각각 저장에는 추가적인 설정을 정의할 수 있다. jdbc 저장을 위해 테이블명을 재정의하는 것이 가능하다.

```
spring.session.jdbc.table-name: SESSIONS
```

3.12. 테스트

스프링 부트는 애플리케이션 테스트를 지원하는 다양한 유틸리티와 애너테이션을 제공한다. 두 모듈에 의해 테스트 지원이 제공된다: `spring-boot-test`는 핵심 아이템들이 포함되어 있으며, `spring-boot-test-autoconfigure`는 테스트를 위한 자동구성을 지원한다.

대부분의 개발자는 `spring-boot-starter-test` 스타터를 사용하여 스프링 부트 테스트 모듈과 JUnit, AssertJ, Hamcrest 와 다른 유용한 라이브러리를 불러올 수 있다.

3.12.1. 테스트 관련 의존성

`spring-boot-starter-test` 스타터를 사용(`test` 스코프에서)하면 다음 라이브러리를 확인할 수 있다:

- **JUnit**: 사실상 Java 애플리케이션 테스트 표준
- **Spring Test** & Spring Boot test: 스프링 부트 애플리케이션을 위한 유틸리티 및 통합테스트 지원
- **AssertJ**: 플루언트(Fluent) 선언 라이브러리
- **Hamcrest**: 매칭(Matcher) 객체(확정적이거나 예측가능한) 라이브러리
- **Mockito**: Java 목킹 프레임워크
- **JSONassert**: JSON을 위한 선언 라이브러리
- **JsonPath**: JSON 용 XPath

NOTE | 스프링 부트는 기본적으로 모키토 1.x 를 사용한다. 그러나 원한다면 2.x를 사용하는 것도 가능하다.

위에 나열된 라이브러리는 테스트를 작성할 때 유용한 것들이다. 필요한 상황에 적합하지 않다면 자유롭게 테스트 의존성을 추가할 수 있다.

3.12.2. 스프링 애플리케이션 테스트

의존성 주입을 통해 얻게 되는 큰 혜택 중 하나는 코드에 대한 단위테스트를 손쉽게할 수 있다는 것이다. 스프링 없이도 `new` 명령어를 통해 간단하게 객체의 인스턴스를 얻을 수 있다. 또한 실제 의존성을 가진 객체 대신 *mock 객체*를 사용할 수도 있다.

'단위 테스트' 다음에는 '통합 테스트'(진행 과정 중에 실제 스프링 `ApplicationContext`이 필요하기도 하고)으로 넘어가야한다. 다른 인프라에 연결해야하는 상황이나 애플리케이션을 배포하지 않고 통합테스트를 수행하려고 할 때 유용하다.

스프링 프레임워크는 통합 테스트를 위한 테스트 모듈을 포함하고 있다. `org.springframework.spring-`

test이나 spring-boot-starter-test 스타터에 대한 의존성을 선언할 수 있다.

만약 spring-test 모듈을 사용해본 경험이 없다면 스프링 프레임워크 참조 문서에서 [관련된 항목](https://goo.gl/jnXpFV) (<https://goo.gl/jnXpFV>)을 읽기 바란다.

3.12.3. 스프링 부트 애플리케이션 테스트

스프링 부트 애플리케이션은 스프링 `ApplicationContext` 이기에 스프링 컨텍스트에 관해서는 특별히 더 테스트할 것은 없다. 외부 속성, 로깅과 스프링 부트의 다른 기능은 `SpringApplication` 을 사용하여 생성하는 경우 기본적으로 컨텍스트에만 설치된다는 것을 유의해야 한다.

스프링 부트는 스프링 부트 기능이 필요할 때 표준 `spring-test @ContextConfiguration` 애너테이션의 대안으로 사용할 수 있는 `@SpringBootTest` 애너테이션을 제공한다. 이 애너테이션은 `SpringApplication`를 이용한 테스트에서 `ApplicationContext`를 만들어낸다.

`@SpringBootTest`의 `webEnvironment` 속성을 사용하면 테스트가 어떻게 실행될지 깔끔하게 정의할 수 있다:

- **MOCK**: `WebApplicationContext`를 적재하면서 mock 서블릿 환경을 제공한다. 이 애너테이션을 사용하면 내장 서블릿 컨테이너가 시작하지 않는다. 클래스패스상에 서블릿 API가 존재하지 않는다면 이 모드에서는 비웹 `ApplicationContext`이 생성될 것이다. 애플리케이션을 위한 `MockMvc` 기반 테스트를 위해 `@AutoConfigureMockMvc`를 사용할 수도 있다.
- **RANDOM_PORT**: `EmbeddedWebApplicationContext`를 적재하며 실제 서블릿 컨테이너를 제공한다. 내장 서블릿 컨테이너는 무작위 포트로 시작하며 준비한다.
- **DEFINED_PORT**: `EmbeddedWebApplicationContext`를 적재하고 실제 서블릿 환경을 제공한다. 내장 서블릿 컨테이너는 기본 포트(`application.yml`에서 정의하지 않았다면 기본 포트는 8080)로 시작한다.
- **NONE**: `SpringApplication`을 사용하여 `ApplicationContext`이 적재되며 서블릿 환경(mock 혹은 어떤 것도)을 제공하지 않는다.

NOTE

테스트에서 `@Transactional`을 사용하면, 기본적으로 각 테스트 메서드가 끝나는 시점에 트랜잭션에 대한 롤백이 발생한다. `RANDOM_PORT` 혹은 `DEFINED_PORT`를 사용한다면, 테스트가 서버와는 다른 스레드에서 실행될 때 서버에서 시작된 트랜잭션은 롤백되지 않는다.

NOTE

`@SpringBootTest`를 비롯해서 애플리케이션을 테스트하는데 필요한 다양한 애너테이션을 제공한다. 자세한 내용에 대해서는 아래에서 기술한다.

TIP

테스트 코드에 `@RunWith(SpringRunner.class)`를 추가하는 것을 잊지 말자. 그렇지 않으면 다른 애너테이션들은 무시된다.

테스트 구성 탐색

스프링 테스트 프레임워크에 친숙하다면, `@ContextConfiguration(classes=...)`를 사용하여 스프링 `@Configuration`이 적재되는 순서를 지정해봤을 것이다. 그에 대한 대안으로 테스트 코드 내부 클래스에 `@Configuration`를 선언하여 사용할 수도 있다.

스프링 부트 애플리케이션에서는 많이 사용되지 않는 방법이다. 스프링 부트의 `.*Test` 애너테이션은 명확하게 설정하지

않는 경우에는 주구성을 자동으로 탐색한다.

탐색 알고리즘은 `@SpringBootApplication` 혹은 `@SpringBootConfiguration` 애너테이션이 선언되어 있는 클래스를 찾을 때까지 진행된다. 꽤 이전에 '코드 구조'에서 설명했던 것을 잘 이행하고 있다면 주 구성클래스를 찾아낼 것이다.

주 구성을 재정의하고 싶다면 `@TestConfiguration`를 선언한 내부클래스를 사용할 수 있다. `@Configuration`을 사용한 내부 클래스와는 달리 애플리케이션 주구성을 대신할 수 있고 `@TestConfiguration`이 선언된 클래스를 추가할 수도 있다.

NOTE

스프링 테스트 프레임워크는 테스트 사이에 애플리케이션 컨텍스트를 캐시처리한다. 그러므로, 테스트 중에는 동일한 구성을 공유(그것이 어떻게 가능하든지 간에)하며, 테스트가 실행될 때 컨텍스트 적재가 한번만 일어나면서 시간을 절약할 수 있다.

테스트 구성 제외

애플리케이션에서 컴포넌트 스캔시, `@SpringBootApplication` 혹은 `@ComponentScan`을 사용했을 때, 특정 테스트에서만 생성된 최상위 클래스가 모든 곳에 노출될 수 있다.

'테스트 구성 탐색'에서 살펴봤던, `@TestConfiguration`은 테스트할 때 주구성 클래스를 내부 클래스를 사용하여 재정의할 수 있다. 최상위 클래스를 대신할 때, `@TestConfiguration`은 `src/main/test`에 있는 클래스를 탐색해서는 안된다. 이에 필요한 부분에서 클래스를 불러올 수 있는 기능을 제공한다:

```
@RunWith(SpringRunner.class)
@SpringBootTest
@Import(MyTestsConfiguration.class)
public class MyTests {
    @Test
    public void exampleTest() {
        ...
    }
}
```

NOTE

`@ComponentScan(@SpringBootApplication` 없이)을 바로 사용한 경우 `TypeExcluderFilter`를 등록해야 한다. [Javadoc](#)에서 보다 자세한 내용을 살펴보기 바란다.

랜덤포트 이용

테스트를 위해서 애플리케이션을 완벽하게 시작해야 한다면, 랜덤포트를 사용하기를 권한다. 만약 `@SpringBootTest(webEnvironment=WebEnvironment.RANDOM_PORT)`를 사용한다면 테스트를 실행할 때마다 무작위로 포트를 선택할 것이다.

`@LocalServerPort` 애너테이션을 사용하면 테스트에서 실제 포트를 주입받을 수 있다. 편의를 위해 테스트는 REST 호출이 필요한 경우에 서버는 `@Autowired`로 `TestRestTemplate`를 추가하여 실행 서버와 관련된 연결을 처리할 수 있다.

```

import org.junit.Test;
import org.junit.runner.RunWith;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import
org.springframework.boot.test.context.SpringBootTest.WebEnvironment;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.test.context.junit4.SpringRunner;

import static org.assertj.core.api.Assertions.assertThat;

@RunWith(SpringRunner.class)
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
public class RandomPortExampleTests {

    @Autowired
    private TestRestTemplate restTemplate;

    @Test
    public void exampleTest() {
        String body = this.restTemplate.getForObject("/", String.class);
        assertThat(body).isEqualTo("Hello World");
    }
}

```

빈에 대한 모방(Mock) 과 스파이(Spy) 처리

테스트를 실행할 때 애플리케이션 컨텍스트 내에서 특정 구성요소를 모방(Mock)해야하는 경우가 있다. 예를 들어, 개발 동안에 원격 서비스를 흉내내야할 때가 있다. 모킹은 실제 환경에서 구현하기 어려운 실패하는 경우를 원하는 경우 유용하다.

스프링 부트는 **@MockBean** 애너테이션을 통해 **ApplicationContext** 에 있는 빈을 모기로 모방체으로 정의할 수 있다. 애너테이션을 사용하여 새로운 빈을 추가하거나 정의되어 있는 빈을 대체할 수 있다. 테스트 클래스에서 바로 사용하거나 **@configuration** 클래스와 필드에서 사용할 수 있다. 필드에 사용할 경우, 생성된 목 인스턴스가 주입된다. 모방 빈은 각 테스트 메서드 마다 자동으로 초기화된다.

다음 예제는 **RemoteService** 빈을 모방 구현체로 대체하고 있다:

```

import org.junit.*;
import org.junit.runner.*;
import org.springframework.beans.factory.annotation.*;
import org.springframework.boot.test.context.*;
import org.springframework.boot.test.mock.mockito.*;
import org.springframework.test.context.junit4.*;

import static org.assertj.core.api.Assertions.*;
import static org.mockito.BDDMockito.*;

@RunWith(SpringRunner.class)
@SpringBootTest
public class MyTests {
    @MockBean
    private RemoteService remoteService;
    @Autowired
    private Reverser reverser;

    @Test
    public void exampleTest() {
        // RemoteService has been injected into the reverser bean
        given(this.remoteService.someCall()).willReturn("mock");
        String reverse = reverser.reverseSomeCall();
        assertThat(reverse).isEqualTo("kcom");
    }
}

```

거기에 더해 **@SpyBean**을 사용하면 실제하는 빈을 모키토 **spy**로 감싸게 된다. 자세한 내용은 Javadoc 을 살펴보기 바란다.

자동구성된 테스트

스프링 부트의 자동구성된 시스템은 애플리케이션에서 잘 동작한다. 그러나 테스트에 사용하기에는 좀 과한 면이 있다. 이때 애플리케이션 중 테스트에 필요한 부분만 잘라내어 구성 일부만 적재하면 유용할 것이다. 예를 들어, 스프링 MVC 컨트롤러에 URL이 정확히 매핑되었는지만 확인하고 싶거나 데이터베이스를 호출하고 싶지 않다거나 JPA 엔티티만 테스트하고 싶거나 웹 계층에 대해서는 테스트하고 싶지 않은 경우 등 말이다.

spring-boot-test-autoconfigure 모듈은 필요한 부분만 절제한 구성을 사용할 수 있는 다양한 애너테이션을 포함하고 있다. 각각이 동작하는 방식은 유사하며, **@...Test** 애너테이션은 **ApplicationContext**에 적재되고 하나 이상의 **@autoconfigure..** 애너테이션으로 자동구성을 재정의할 수 있다.

NOTE

각 절제부는 제한적인 자동구성 클래스 묶음을 적재한다. 만약 그것들 중 하나를 제외하고 싶다면 **@...Test** 애너테이션에서 **excludeAutoConfiguration** 속성을 제공하고 있다. 대안으로는, **@ImportAutoConfiguration#exclude**를 사용할 수 있다.

TIP

가급적이면 표준 `@SpringBootTest` 애너테이션 과 함께 `@AutoConfigure...`을 사용하기 바란다. 애플리케이션을 절제하는 것에는 관심이 없지만 자동구성된 테스트 빈을 원하는 경우에 이 조합을 사용할 수 있다.

자동구성된 JSON 테스트

객체에 대한 JSON 직렬화와 역직렬화를 테스트하고 싶을 때 `@JsonTest` 애너테이션을 사용할 수 있다. `@JsonTest`는 Jackson `ObjectMapper`, `@JsonComponent` 빈과 Jackson `Module`을 자동구성한다. 물론 Jackson 대신 `Gson` 구성을 사용할 수도 있다. 만약 자동구성된 요소들을 필요로 한다면 `@AutoConfigureJsonTesters` 애너테이션을 사용할 수 있다.

스프링 부트는 AssertJ 기반의 헬퍼 `JSONassert`와 `JsonPath` 라이브러리를 통해서 JSON을 예측할 수 있다.

`JacksonTester`, `GsonTester` 그리고 `BasicJsonTester` 클래스는 Jackson, Gson 과 문자열에 각각 사용할 수 있다. `@JsonTest` 를 사용했을 때 테스트 클래스의 헬퍼 필드에 `@Autowired` 를 사용한 예제다:

```

import org.junit.*;
import org.junit.runner.*;
import org.springframework.beans.factory.annotation.*;
import org.springframework.boot.test.autoconfigure.json.*;
import org.springframework.boot.test.context.*;
import org.springframework.boot.test.json.*;
import org.springframework.test.context.junit4.*;

import static org.assertj.core.api.Assertions.*;

@RunWith(SpringRunner.class)
@JsonTest
public class MyJsonTests {

    @Autowired
    private JacksonTester<VehicleDetails> json;

    @Test
    public void testSerialize() throws Exception {
        VehicleDetails details = new VehicleDetails("Honda", "Civic");
        // Assert against a `.json` file in the same package as the test
        assertThat(this.json.write(details)).isEqualToJson("expected.json");
        // Or use JSON path based assertions
        assertThat(this.json.write(details)).hasJsonPathStringValue(
"@.make");
        assertThat(this.json.write(details).
extractingJsonPathStringValue("@.make")
                .isEqualTo("Honda"));
    }

    @Test
    public void testDeserialize() throws Exception {
        String content = "{\"make\":\"Ford\",\"model\":\"Focus\"}";
        assertThat(this.json.parse(content))
                .isEqualTo(new VehicleDetails("Ford", "Focus"));
        assertThat(this.json.parseObject(content).getMake()).isEqualTo(
"Ford");
    }
}

```

NOTE

JSON 헬퍼 클래스는 표준 단위테스트에 바로 사용할 수 있다. `@JsonTest`를 사용하지 않는다면 `@Before` 메서드에서 헬퍼의 `initFields` 메서드를 호출하면 된다.

`@JsonTest`에 의해 활성화되는 자동구성 목록은 <https://goo.gl/ptPqf3>에서 확인할 수 있다.

자동구성된 스프링 MVC 테스트

스프링 MVC 컨트롤러를 테스트할 때는 `@WebMvcTest` 애너테이션을 사용할 수 있다. `@WebMvcTest`는 스프링 MVC 인프라와 제한적으로 `@Controller`, `@ControllerAdvice`, `@JsonComponent`, `Filter`, `WebMvcConfigurer` 와 `HandlerMethodArgumentResolver` 빈을 남색하여 사용할 수 있다. 이 애너테이션을 사용했을 때는 일반적인 `@Component` 들은 탐색되지 않는다.

`@WebMvcTest`는 하나의 컨트롤러로 제한하거나 `@MockBean` 과 조합하여 필요한 목 구현할 수 있다.

또한 `@WebMvcTest`는 `MockMvc`를 자동구성한다. Mock Mvc 는 HTTP 서버를 완전히 가동할 필요 없이 MVC 컨트롤러 테스트를 빠르게 할 수 있는 강력한 방법을 제공한다.

TIP

`@WebMvcTest` 없이도 `MockMvc`를 자동구성할 수 있다(예: `SpringBootTest`와 `@AutoConfigurationMockMvc` 애너테이션을 함께 사용).

```
import org.junit.*;
import org.junit.runner.*;
import org.springframework.beans.factory.annotation.*;
import org.springframework.boot.test.autoconfigure.web.servlet.*;
import org.springframework.boot.test.mock.mockito.*;

import static org.assertj.core.api.Assertions.*;
import static org.mockito.BDDMockito.*;
import static
    org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static
    org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@RunWith(SpringRunner.class)
@WebMvcTest(UserVehicleController.class)
public class MyControllerTests {
    @Autowired
    private MockMvc mvc;
    @MockBean
    private UserVehicleService userVehicleService;

    @Test
    public void testExample() throws Exception {
        given(this.userVehicleService.getVehicleDetails("sboot"))
            .willReturn(new VehicleDetails("Honda", "Civic"));

        this.mvc.perform(get("/sboot/vehicle").accept(MediaType.TEXT_PLAIN))
            .andExpect(status().isOk()).andExpect(content().string("Honda Civic"));
    }
}
```

TIP

자동구성에서 구성된 요소가 필요하다면(서블릿 필터가 필요하다면) `@AutoConfigureMockMvc` 애너테이션의 속성을 사용하면 된다.

만약에 HtmlUnit이나 Selenium을 사용한다면 자동구성은 `WebClient` 빈과 `WebDriver` 빈을 제공한다. 다음의 예에서는 HtmlUnit을 사용하고 있다:

```
import com.gargoylesoftware.htmlunit.*;
import org.junit.*;
import org.junit.runner.*;
import org.springframework.beans.factory.annotation.*;
import org.springframework.boot.test.autoconfigure.web.servlet.*;
import org.springframework.boot.test.mock.mockito.*;

import static org.assertj.core.api.Assertions.*;
import static org.mockito.BDDMockito.*;

@RunWith(SpringRunner.class)
@WebMvcTest(UserVehicleController.class)
public class MyHtmlUnitTests {
    @Autowired
    private WebClient webClient;
    @MockBean
    private UserVehicleService userVehicleService;

    @Test
    public void testExample() throws Exception {
        given(this.userVehicleService.getVehicleDetails("sboot"))
            .willReturn(new VehicleDetails("Honda", "Civic"));
        HtmlPage page = this.webClient.getPage("/sboot/vehicle.html");
        assertThat(page.getBody().getTextContent()).isEqualTo("Honda
Civic");
    }
}
```

NOTE

기본적으로 스프링 부트는 `WebDriver` 빈에 특별한 "범위(scope)"를 부여하여 각 테스트마다 새로운 인스턴스를 주입한다. 이것을 원하지 않는다면 `WebDriver @Bean` 선언에 `@Scope("singleton")`을 추가하면 된다.

`@WebMvcTest`에 의해 활성화되는 자동구성 목록은 <https://goo.gl/ptPqf3>에서 확인할 수 있다.

자동구성된 Data JPA 테스트

`@DataJpaTest`는 JPA 애플리케이션을 테스트할 때 사용할 수 있다. 기본적으로 인-메모리 내장 데이터베이스가 구성되며, `@Entity` 클래스를 탐색하고 스프링 Data JPA {repo}가 구성된다. 일반적인 `@Component` 빈은 `ApplicationContext`에 적재되지 않는다.

Data JPA 테스트는 트랜잭션이 되며 기본적으로 각 테스트가 끝나면 롤백된다. 스프링 참조문서의 [관련된 세션](#)

(<https://goo.gl/uwxDNt>)에서 보다 자세한 내용을 볼 수 있다. 트랜잭션 처리가 되는 것을 원치 않는다면 다음과 같이 하여 테스트 동안에 트랜잭션 관리를 비활성화할 수 있다:

```
import org.junit.Test;
import org.junit.runner.RunWith;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.test.context.junit4.SpringRunner;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@RunWith(SpringRunner.class)
@DataJpaTest
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public class ExampleNonTransactionalTests {
    // 코드 생략
}
```

Data JPA 테스트에는 `TestEntityManager` 빈을 주입하여 표준 JPA `EntityManager`에 대한 대안으로 테스트에 제공할 수 있다. 만약 `TestEntityManager`를 `@DataJpaTest` 외부에서 사용하고 싶다면 `@AutoConfigureTestEntityManager` 애너테이션을 사용할 수 있다. 필요하다면 `JdbcTemplate`도 사용가능하다:

```
import org.junit.*;
import org.junit.runner.*;
import org.springframework.boot.test.autoconfigure.orm.jpa.*;

import static org.assertj.core.api.Assertions.*;

@RunWith(SpringRunner.class)
@DataJpaTest
public class ExampleRepositoryTests {
    @Autowired
    private TestEntityManager entityManager;
    @Autowired
    private UserRepository repository;

    @Test
    public void testExample() throws Exception {
        this.entityManager.persist(new User("sboot", "1234"));
        User user = this.repository.findByUsername("sboot");
        assertThat(user.getUsername()).isEqualTo("sboot");
        assertThat(user.getVin()).isEqualTo("1234");
    }
}
```

인-메모리 내장 데이터베이스 테스트는 개발자가 추가적으로 설치할 필요없이 빠르게 잘 동작한다. 그러나, 실제

데이터베이스 테스트를 하고 싶다면 `@AutoConfigureTestDatabase` 애너테이션을 사용할 수 있다.

```
@RunWith(SpringRunner.class)
@DataJpaTest
@AutoConfigureTestDatabase(replace=Replace.NONE) ①
public class ExampleRepositoryTests {
    // ...
}
```

① 실제 데이터베이스 테스트를 하고 싶을 때 사용하는 방법

`@DataJpaTest`에 의해 활성화되는 자동구성 목록은 <https://goo.gl/ptPqf3>에서 확인할 수 있다.

이 외에도 스프링 부트는 테스트를 하는 데 필요한 기능과 많은 유틸리티를 제공한다. 보다 자세한 내용은 [테스트 유틸리티 \(https://goo.gl/qudHP8\)](https://goo.gl/qudHP8)를 살펴보기 바란다.

3.13. 웹소켓(WebSocket)

스프링 부트는 내장 톰캣(8과 7), 제티 9와 언더투에 대한 웹소켓 자동구성을 제공한다. 실행중인 컨테이너에 WAR파일로 배포시 스프링 부트는 컨테이너의 종류를 추측하여 웹소켓 지원에 대한 구성을 지원한다.

스프링 프레임워크의 [웹소켓 지원](#)을 `spring-boot-starter-websocket` 모듈을 통해 쉽게 사용할 수 있다.

3.14. 웹 서비스(Web Service)

스프링 부트는 `Endpoints` 정의가 필요한 모든 곳에 웹서비스 자동구성을 제공한다. [스프링 웹서비스 기능](#)은 `spring-boot-starter-web-service` 모듈을 통해 쉽게 접근할 수 있다.

3.15. 자동구성 직접 생성하기

사내에서 개발한 라이브러리를 공유하여 작업하거나 오픈소스 혹은 상업적인 라이브러리로 작업하게 될 경우 자동구성을 개발하고 싶은 상황을 마땅드리게 될 것이다. 자동구성은 외부 jar에서 제공하거나 스프링 부트에 의해 제공될 것이다.

자동구성은 사용하게 되는 일반적인 라이브러리에 대한 자동구성 코드를 "스타터"를 통해서 제공한다. 자동구성을 구성하는데 필요한 것들이 무엇인지 살펴본 후 '[스타터 직접 생성하기](#)' 단계로 이동한다.

TIP | [데모 프로젝트](#)를 통해서 스타터를 만들어 가는 과정을 살펴볼 수 있다.

3.15.1. 자동구성 빈 이해하기

자동구성을 간단하게 정리하자면 표준 `@Configuration` 애너테이션이 선언되어 있는 구현체다. 거기에 더하여 `@Conditional` 애너테이션으로 자동구성이 사용될 상황을 제어한다. 일반적으로 자동구성 클래스는 `@ConditionalOnClass`와 `@ConditionalOnMissingBean` 애너테이션을 사용한다. 이 조건은 관련된 클래스가 있을 때 자동구성이 존재하고 이와 관련된 `@Configuration` 정의가 없는 경우 발동된다.

이와 관련된 소스코드는 `spring-boot-autoconfigure`에서 살펴볼 수 있으며 스프링 부트에서 제공하는 자동구성

목록을 `/META-INF/spring.factories`에서 확인할 수 있다. 파일 내에 `EnableAutoConfiguration` 키에 구성 클래스 목록이 위치한다.

3.15.2. 자동구성 대상 위치

스프링 부트는 게시된 jar 파일 내에 `META-INF/spring.factories` 파일이 존재하는지 검사한다.

```
org.springframework.boot.autoconfigure.EnableAutoConfiguration=\
io.honeymon.boot.springboot.autoconfigure.HoneymonAutoConfiguration,\
io.honeymon.boot.springboot.autoconfigure.HoneymonWebAutoConfiguration
```

지정된 순서에 따라서 구성되어야 한다면 `AutoConfigureAfter` 혹은 `AutoConfigureBefore` 애너테이션을 사용할 수 있다. 예를 들어, 웹과 관련된 구성을 제공하고 싶다면 `WebMvcAutoConfiguration` 구성 후에 실행되도록 할 수 있다.

다른 구성에 영향이 없는 상황에서 자동구성 순서를 확고히 하고 싶다면 `@AutoconfigureOrder`를 사용할 수 있다. 이 애너테이션은 일반적인 `@Order`와 동일한 형태로 동작하지만 자동구성 클래스를 위한 순서를 정의한다.

NOTE

자동구성의 적재방법은 정해져있다. 특정 패키지 공간에 정의되어 있고 컴포넌트 탐색의 대상이 아닌지 확인해야 한다.

3.15.3. 조건 애너테이션

대부분의 자동구성 클래스에는 한 개 이상의 `@Conditional` 애너테이션을 포함하게 된다.

`@ConditionalOnMissingBean`이 가장 일반적인 예로 개발자에게 '기본'적인 구성이 맘에 들지 않을 때 자동구성을 덮어쓰기할 수 있다.

스프링 부트에 포함된 많은 `@Configuration` 애너테이션은 `@Configuration`가 선언된 클래스 혹은 `@Bean` 메서드에서 사용할 수 있다.

클래스 조건

`@ConditionalOnClass` 과 `@ConditionalOnMissingClass` 애너테이션은 지정된 클래스가 존재하거나 부재하는 경우 구성한다. `ASM`을 사용하여 애너테이션 메타데이터를 파싱하여 `value` 속성에 정의되어 있는 클래스가 존재하거나 실행되는 {application} 클래스패스에 부재하는 지를 점검한다. 또한 `name` 속성을 사용하여 클래스 이름을 `String` 값을 사용하여 클래스 이름을 정의할 수 있다.

TIP

`@ConditionalOnClass` 혹은 `@ConditionalOnMissingClass`를 메타 애너테이션으로 사용하여 애너테이션을 작성하는 경우 적용되지 않는 경우 참조하는 클래스에 대해서는 `name`을 사용하여 정의해야 한다.

빈 조건

`@ConditionalOnBean` 과 `@ConditionalOnMissingBean` 애너테이션은 지정한 빈이 존재하거나 부재하는 경우 구성한다. `value` 속성으로 빈 유형을 정의하거나, `name` 속성으로 빈의 이름을 지정할 수 있다. `search` 속성은 `ApplicationContext` 계층에 한정적으로 등록되어 있는 빈을 탐색한다.

@Bean 메서드에 위치하는 경우, 대상 유형은 기본적으로 메서드 반환형이 지정된다. 예를 들어:

```
@Configuration
public class MyAutoConfiguration {
    @Bean
    @ConditionalOnMissingBean ①
    public MyService myService() { ... }
}
```

① myService 빈은 ApplicationContext 에서 MyService 유형의 빈이 없는 경우 생성된다.

TIP

지금까지 처리된 내용에 기반해서 평가되기 때문에 빈을 정의하는 순서에 주의해야 한다. 이런 이유로 @ConditionalOnBean 과 @ConditionalOnMissingBean 애너테이션은 자동구성 클래스에서만 사용하기를 권장한다(사용자가 정의한 빈이 추가된 후에 적재되도록 보장하고 있다).

NOTE

@ConditionalOnBean 과 @ConditionalOnMissingBean은 @Configuration 클래스가 생성될 때는 동작하지 않는다. 클래스 수준에서 이러한 조건을 사용하면 @Bean 메서드에서 사용한 것과 동일하게 취급받는다.

속성 조건

@ConditionalOnProperty 애너테이션은 스프링 Environment 속성 기반으로 구성한다. prefix와 name 으로 점검할 속성을 지정할 수 있다. 속성이 존재하거나 false가 아닌 경우 적용된다. havingValue와 matchIfMissing 속성을 사용하여 조금 더 다양한 점검을 할 수 있다.

자원 조건

@ConditionalOnResource 애너테이션은 지정한 자원이 존재하는 경우에만 구성을 허용한다. 자원은 file:~/test.key처럼 스프링 관례를 사용하여 지정가능하다.

웹애플리케이션 조건

@ConditionalOnWebApplication 과 @ConditionalOnNotWebApplication 애너테이션은 애플리케이션이 '웹 애플리케이션'인 경우에 구성을 허용한다. 웹 애플리케이션은 스프링 WebApplicationContext를 사용하며 session 스코프가 정의되었거나 StandardServletEnvironment를 가지고 있는 경우에 해당한다.

SpEL 표현 조건

@ConditionalOnExpression 애너테이션은 SpEL 표현식의 결과를 기반으로 구성된다.

3.15.4. 스타터 직접 생성하기

라이브러리를 위한 완벽한 스프링 부트 스타터는 다음 조건을 포함해야한다:

- 자동구성 코드를 포함한 autoconfigure 모듈

- **starter** 모듈은 라이브러리에 대한 자동구성된 모듈 의존성을 제공하며 관련된 의존성을 추가한다. 정리하자면 스타터는 라이브러리를 사용하기 시작하는데 필요한 것들을 추가해야 한다.

TIP | 자동구성 코드와 의존성관리를 분리할 필요 없이 하나의 모듈에 묶어주는 것이 좋다.

이름짓기

스타터에 명확한 네임스페이스를 부여하자. **spring-boot**로 모듈을 시작하지 말고, 메이븐 groupId 도 다른 것을 사요오하자. 나중에 개인의 자동구성에 대해 스프링 부트에서 공식지원을 할 수도 있다.

추천할만한 방법이 있다. "acme"를 위한 스타터를 생성한다면, auto-configure 모듈이름은 **acme-spring-boot-autoconfigure**으로 하고 스타터는 **acme-spring-boot-starter**로 한다. 두 모듈을 하나로 엮는다면 **acme-spring-boot-starter**을 사용한다.

반면에 스타터에서 구성키를 제공할 계획이라면 명확한 네임스페이스를 부여하라. 특히 스프링 부트에서 사용하는 네임스페이스를 키로 포함하지 않도록 주의하자(예: **server**, **management**, **spring**, 기타 등등). 공식지원을 하게 될 경우 작성했던 것들을 향상 또는 변경할 수도 있다.

IDE 어시스턴스에서 키를 지원하도록 하려면 **메타데이터 생성 트리거**를 만들어보자. 생성한 메타데이터(**META-INF/spring-configuration-metadata.json**)를 살펴보고 키에 대해서 적절하게 문서화를 제공하자.

auto-configure 모듈

auto-configure 모듈은 라이브러리를 사용하기 위해 필요한 모든 것을 포함해야 한다. 구성키 정의 (**@ConfigurationProperties**)와 컴포넌트 초기화시에 재정의할 수 있도록 콜백 인터페이스를 포함해야 한다.

TIP | 프로젝트에서 auto-configure 모듈을 쉽게 포함시킬 수 있도록 라이브러리에 대한 종속성을 표시해야 한다. 그렇게하면 라이브러리가 제공되지 않고 기본적으로 스프링 부트가 해제된다.

스타터 모듈

스타터는 비어있는 jar 다. 라이브러리가 동작하는데 필요한 의존성을 제공하려는 목적으로 사용된다; 그래서 자세히 살펴보면 시작하는데 필요한 것이 무엇인지 알 수 있다.

프로젝트에 필요할 것이라고 추측한 것들을 스타터에 추가하지 말자. 다른 스타터에서 필요로 하는 일반적인 자동구성 라이브러리라면 그것에 대해서 언급하는 정도면 된다. **기본** 의존성에 대해서 명확하게 제공하면 일반적으로 사용하는 라이브러리를 위한 불필요한 의존성 참조를 피할 수 있다.

3.16. 다음 읽을꺼리

이번 장에서 논의되었던 클래스에 대해서 보다 자세히 살펴보기를 원한다면 [Spring Boot API 문서](#) 혹은 직접 [소스코드](#)를 살펴보는 것이 좋다. 특정 상황에 대해서 어떻게 사용할지 살펴보고자 한다면 "[스프링 부트](#)" [활용 안내](#)를 살펴보기 바란다.

Chapter 4. '스프링 부트' 출시준비: 액추에이터

스프링 부트에는 운영시 사용할 수 있도록 애플리케이션을 관리하고 관측하는데 도움이 되는 여러가지 기능들이 포함되어 있다. 애플리케이션을 관리하고 관측하기 위해 HTTP 종단점(endpoint), JMX 혹은 리모트 셸(SSH 혹은 Telnet) 중에서 선택할 수 있다. 애플리케이션에 대한 감시, 상태확인 그리고 계측을 할 수 있도록 자동구성된다.

액추에이터 HTTP 종단점은 스프링 MVC 기반의 애플리케이션에서만 지원한다. 저지(Jersey)를 이용하면 [스프링 MVC를 사용하지 않아도 동작](#)하게 할 수는 있다.

4.1. 출시준비 기능 활성화

`spring-boot-actuator` 모듈은 스프링 부트 출시준비 기능과 관련된 것들을 제공한다. `spring-boot-starter-actuator` 스타터 의존성만 추가하면 기능들이 활성화된다.

액추에이터 에 대한 정의

액추에이터는 '제조 관련 용어'로서 시스템을 움직이거나 제어하기 위한 기계장치를 말한다. 액추에이터는 작은 변화로부터 많은 양의 산출물을 얻을 수 있다. - Wikipedia: <https://goo.gl/R1LSmU>

그레이들 기반 프로젝트에서 액추에이터를 추가하려면 다음과 같이 스타터 의존성을 추가하면 된다:

```
dependencies {
    //.....
    compile("org.springframework.boot:spring-boot-starter-actuator")
    //.....
}
```

메이븐 기반 프로젝트에서는 다음과 같다:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
</dependencies>
```

4.1.1. 종단점(Endpoint)

액추에이터 종단점은 애플리케이션에 대한 모니터링 및 조작을 가능하도록 한다. 스프링 부트는 빌트-인(built-in, 사전에 제작되어 제공되는) 다양한 종단점을 제공하며 개발자가 필요에 따라서 얼마든지 추가할 수 있다. 예를 들어 `health` 종단점의 경우 기본적인 애플리케이션의 건강 정보를 제공한다.

종단점 노출을 결정하는 것은 개발자가 사용하려는 기능과 그에 대한 사용여부 결정에 따른다. 대부분의 애플리케이션은

종단점에 ID를 부여하고 이를 URL 에 매칭하여 HTTP 모니터링하는 방식을 선택한다. 예를 들어 **health** 종단점은 **/health** 에 매칭된다.

사용할 수 있는 종단점은 다음과 같다:

ID	설명	민감정보여부(Sensitive) 기본설정
actuator	다른 종단점을 살펴볼 수 있는 하이퍼미디어 기반의 페이지를 제공한다. {spring-hateoas}가 클래스패스 에 있어야 한다.	true
auditevents	현재 애플리케이션의 감시 이벤트 정보를 제공한다.	true
autoconfig	모든 자동구성에 대한 자동구성 상태 보고서를 보여준다.	true
beans	애플리케이션에 등록된 모든 빈(bean) 목록을 보여준다.	true
configprops	모든 @ConfigurationProperties 목록을 보여준다.	true
dump	스레드 덤프를 수행한다.	true
env	스프링 ConfigurableEnvironment 속성들을 보여준다.	true
flyway	Flyway를 사용하는 경우 데이터베이스 마이그레이션 정보를 보여준다.	true
health	애플리케이션 건강 정보(애플리케이션에 대한 보안정책이 활성화되어 있는 경우 인증되지 않은 접속인 경우에는 status만 보여주고 인증된 접속인 경우에는 상세정보 제공)를 보여준다.	false
info	애플리케이션 정보를 보여준다.	false
loggers	애플리케이션 로거 구성을 보여주고 변경할 수 있다.	true
liquibase	Liquibase를 사용하는 경우 데이터베이스 마이그레이션 정보를 보여준다.	true
metrics	현재 애플리케이션 '측량' 정보를 보여준다.	true
mappings	모든 @RequestMapping 경로에 대한 목록을 보여준다.	true
shutdown	깔끔하게 애플리케이션을 종료한다(기본적으로는 비활성화 되어 있다).	true
trace	트레이스 정보를 보여준다(기본적으로 최근 HTTP 요청 100개까지).	true

스프링 MVC를 사용하면 다음 종단점을 추가사용할 수 있다:

ID	설명	민감정보여부(Sensitive) 기본설정
docs	액추에이터 종단점에 대한 문서(요청 및 응답 예제를 포함한)를 제공한다. 클래스패스 상에 <code>spring-boot-actuator-docs</code> 를 필요로 한다.	false
heapdump	GZip 으로 압축된 <code>hprof</code> 헤드 덤프파일을 반환한다.	true
jolokia	HTTP으로 JMX 빈을 노출한다(클래스패스 상에 Jolokia가 있는 경우).	true
logfile	로그파일 내용(<code>logging.file</code> 혹은 <code>logging.path</code> 속성을 정의했을 경우)을 반환한다. 로그 파일의 내용에 대해서 HTTP <code>Range</code> 헤더를 사용할 수 있다.	true

NOTE

민감한 정보는 기본적으로 '민감정보여부(Sensitive)'가 `true`로 되어 있어서 종단점이 **비활성**상태다. 필요한 경우 사용하려는 종단점의 `sensitive` 속성을 `true`로 표기하면 바로 **활성**상태가 되어 접근가능하다.

4.1.2. 종단점 보안

기본적으로 모든 HTTP 종단점은 `ACTUATOR` 역할을 가지고 있는 이들만 접근가능하다. 보안은 기본적으로 `HttpServletRequest.isUserInRole` 메서드를 사용하도록 강제한다.

TIP

`ACTUATOR` 와 다른 것을 원하는 경우에는 `management.security.roles` 속성을 사용한다.

방화벽 뒤에서 애플리케이션을 배포한다면, 액추에이터 종단점은 인증할 필요없이 접근하도록 하고 싶을 것이다. 그런 경우에는 `management.security.enabled` 속성을 변경하면 된다.

```
management.security.enabled: false
```

NOTE

기본적으로 액추에이터 종단점은 일반적인 HTTP 트래픽을 제공하는 동일한 포트에 노출된다. 민감한 정보가 의도치 않은 상황에서 노출되지 않도록 신경쓸 필요가 없다면 `management.security.enabled` 속성을 변경하면 된다.

공개적으로 노출되는 애플리케이션을 배포하려 한다면, 스프링 시큐리티를 추가하여 사용자 인증을 제어하도록 하자. 스프링 시큐리티가 추가되면 기본 인증은 username `user` 에 생성된 비밀번호를 애플리케이션 구동시 콘솔에 출력한다.

TIP

생성된 패스워드는 애플리케이션 구동시 로그로 출력된다. `Using default security password`로 찾아보자.

스프링 프로퍼티를 통해서 종단점에 접근하는 접근권한(ROLE)을 변경하고 사용자명(username)과 비밀번호를 변경할 수 있다. 예를 들어, 다음과 같이 `application.yml` 파일을 설정하면 된다:


```
security:
  user.name: admin
  user.password: secret
management.security.roles: SUPERUSER
```

만약 애플리케이션이 사용자정의한 보안구성을 가지고 있고 인증없이 액츄에이터 종단점에 접근가능하길 원한다면, 구성시 `management.security.enabled` 속성을 `false`로 변경해야 한다. 액츄에이터 종단점에 대한 접근권한을 재정의하고 싶다면 `management.security.roles`에 인증된 사용자가 가지고 있어야할 권한을 정의하면 된다.

NOTE

만약 미인증 사용자들에게 기본적인 건강 정보를 노출하고 싶지 않다면, 액츄에이터 종단점에 대한 `management.security.enabled` 보안속성을 `false`로 재정의 하면 된다.

4.1.3. 종단점 재정의

종단점은 스프링 속성을 사용해서 재정의가 가능하다. 종단점의 `enabled`와 `id`를 변경할 수 있다.

예를 들어, `beans` 종단점 에 대한 `id`를 변경하고 `shutdown` 이 가능하도록 한 `application.yml`이 여기있다:

```
endpoints:
  beans.id: springbeans
  shutdown.enabled: true
```

NOTE

접두사 '`endpoints + . + {name}`'은 종단점에 대한 고유한 식별자로 사용된다.

기본적으로 모든 종단점에 대한 `shutdown` 은 활성화되어 있다. 만약 종단점 활성화를 제어하고 싶다면 `endpoint.enabled` 속성을 사용할 수 있다. 예를 들어, 다음 설정에서는 `info`를 제외한 모든 종단점을 비활성화 한다.

```
endpoints:
  enbaled: false
  info.enalbed: true
```

4.1.4. 액츄에이터 MVC 종단점 를 위한 하이퍼미디어

디스커버리 페이지("Discovery Page")는 모든 종단점에 대한 링크를 제공한다. 디스커버리 페이지는 `/application`으로 접근가능하다.

관리 컨텍스트 경로를 재정의한 경우, 디스커버리 페이지는 `/application`에서 자동적으로 관리 컨텍스트의 최상위로 이동한다. 예를 들어, 관리 컨텍스트 경로가 `/management` 인 경우 `/management`에서 디스커버리 페이지를 발견할 수 있다.

4.1.5. CORS 지원

Cross-origin resource sharing(CORS)는 W3C 명세되어 있는 서로 다른 도메인사이 요청에 대한 인증을 유연하게 정의하는 것을 허용한다. 액추에이터는 MVC 종단점에 대해서 여러가지 상황에 따른 구성을 지원한다.

CORS 지원은 기본적으로 비활성화되어 있으며 `endpoints.cors.allowed-origins` 속성을 통해서만 활성화가 가능하다. 다음과 같이 정의하면 `example.com` 도메인으로부터 GET과 POST 요청에 대해서만 허용한다:

```
endpoints.cors:
  allowed-origins: http://example.com
  allowed-method: GET,POST
```

TIP | `CORS 종단점` 속성에서 선택가능한 항목 전체를 살펴볼 수 있다.

4.1.6. 재정의 종단점 추가하기

만약 `EndPoint` 타입의 `@Bean`을 추가한다면 자동으로 JMX 및 HTTP(서버에서 설정되어 있다면)가 가능하다. HTTP 종단점은 `MvcEndpoint` 타입의 빈으로 생성하여 재정의할 수 있다. `MvcEndpoint`는 `@Controller`는 아니지만 `@RequestMapping`(그리고 `@Managed*`)를 사용하여 자원을 노출할 수 있다.

4.1.7. 건강정보

건강 정보는 실행중인 애플리케이션의 상태를 점검하는데 사용할 수 있다. 운영 시스템이 다운되었을 때 누군가에게 경고할 수 있는 모니터링 소프트웨어로 사용된다. 기본 정보는 `health` 종단점을 통해서 노출되며 접근상황에 따라 다른 정보를 보여준다. 비인증 접속의 경우 보안이 활성화된 애플리케이션에서는 간단한 상태 메시지만 반환하며 인증 접속의 경우에는 추가적인 상세정보를 보여줄 수 있다(이와 관련해서는 [HTTP 건강 종단점 접근 제한](#)에서 보다 자세하게 설명하겠다).

건강 정보는 `ApplicationContext` 에 정의되어 있는 모든 `HealthIndicator` 빈으로부터 수집되어 보여진다. 스프링 부트는 자동구성된 다양한 `HealthIndicator` 외에도 개발자가 Indicator 를 작성할 수도 있다. 기본적으로, 시스템 상태는 지정된 목록을 기반으로 각각의 `HealthAggregator`를 통해서 전달한 값을 `HealthIndicator`가 정렬하여 제공한다. 맨먼저 노출되는 상태는 전체적인 건강상태를 보여준다. 만약 `HealthAggregator`가 반환하는 상태가 없다면 `HealthIndicator`는 `UNKNOWN` 상태를 보여준다.

4.1.8. HealthIndicator 보안

건강에 대한 응답은 캐시처리되어 '서비스 거부 공격(Denial of service attack, DDoS)'를 방지한다. 1000 밀리세컨드 단위로 캐시되는 기본값을 변경하고 싶다면 `endpoints.health.time-to-live` 속성을 이용하면 된다.

자동구성 HealthIndicator

스프링 부트에서 사용할 수 있는 자동구성 목록은 다음과 같다:

이름	설명
<code>CassandraHealthIndicator</code>	카산드라 데이터베이스 동작을 확인가능하다.

이름	설명
DiskSpaceHealthIndicator	디스크 공간에 대한 상태를 확인가능하다.
DataSourceHealthIndicator	DataSource에서 얻을 수 있는 연결정보를 확인가능하다.
ElasticsearchHealthIndicator	엘라스틱서치 클러스터의 동작을 확인가능하다.
JmsHealthIndicator	JMS 브로커의 동작을 확인가능하다.
MailHealthIndicator	메일서버의 동작을 확인가능하다.
MongoHealthIndicator	몽고데이터베이스의 동작을 확인가능하다.
RabbitHealthIndicator	래빗서버의 동작을 확인가능하다.
RedisHealthIndicator	레디스서버의 동작을 확인가능하다.
SolrHealthIndicator	솔(Solr)서버의 동작을 확인가능하다.

TIP

`management.health.defaults.enabled` 속성을 사용하면 모든 HealthIndicator에 대한 활성화를 제어할 수 있다.

HealthIndicator 작성하기

사용자정의한 건강정보를 제공하려면 HealthIndicator 인터페이스를 구현한 빈을 등록하면 된다. `health()` 메서드를 구현하면서 Health 응답을 반환하면 된다. Health 응답은 상태와 보여주고 싶은 선택적인 정보들을 추가하면 된다.

```
import org.springframework.boot.actuate.health.Health;
import org.springframework.boot.actuate.health.HealthIndicator;
import org.springframework.stereotype.Component;

@Component
public class MyHealthIndicator implements HealthIndicator {

    @Override
    public Health health() {
        int errorCode = check(); // perform some specific health check
        if (errorCode != 0) {
            return Health.down().withDetail("Error Code", errorCode).
build();
        }
        return Health.up().build();
    }
}
```

NOTE

HealthIndicator의 식별을 위해서 작성한 HealthIndicator의 클래스명에서 HealthIndicator를 제거한 접두사를 사용한다. 위의 경우라면 `my`라는 이름으로 건강정보를 제공가능하다.

추가적으로 스프링 부트에서 사전에 정의한 Status 유형은 새로운 시스템 상태를 표현할 수 있도록 Status를

재정의하여 **Health**로 반환하는 것도 가능하다. 이런 것을 원하는 경우에는 **HealthAggregator** 인터페이스를 재정의한 구현체를 작성하거나 기본 구현체를 이용하려는 경우에는 **management.health.status.order** 속성을 정의하여 구성가능하다.

예를 들어, **FATAL** 이 추가된 새로운 **Status**를 **HealthIndicator** 구현체에서 사용한다고 가정해보자. 다음과 같이 애플리케이션속성을 추가하면 된다:

```
management.health.status.order: FATAL, DOWN, OUT_OF_SERVICE, UNKNOWN, UP
```

HTTP 상태코드는 건강 상태(예: **UP**은 200, **OUT_OF_SERVICE** 혹은 **DOWN**은 503)를 통합하여 보여준다. HTTP를 통해서 건강 종단점에 접근할 수 있다면 **HealthMvcEndpoint**를 통해서 추가적으로 재정의한 상태를 보여주는 것도 가능하다. 예를 들어, **FATAL**을 **HttpStatus.SERVICE_UNAVAILABLE**에 연결하고 싶다면 다음처럼 하면 된다:

```
endpoint.health.mapping.FATAL: 503
```

미리 정의된 상태별로 연결된 값은 다음과 같다:

상태	대응값
DOWN	SERVICE_UNAVAILABLE (503)
OUT_OF_SERVICE	SERVICE_UNAVAILABLE (503)
UP	기본적으로 대응하는 값은 없고, HTTP 상태가 200인 경우
DOWN	기본적으로 대응하는 값은 없고, HTTP 상태가 200인 경우

4.1.9. 애플리케이션 정보

애플리케이션 정보는 **ApplicationContext** 에 등록되어 있는 모든 **InfoContributor** 빈으로부터 수집된 다양한 정보를 제공한다. 스프링 부트는 다양한 자동구성된 **InfoContributor**과 작성한 것들을 포함하여 제공가능하다.

자동구성된 **InfoContributors**

스프링 부트에서 제공하는 자동구성된 **InfoContributor**는 다음과 같다:

이름	설명
EnvironmentInfoContributor	Environment에서 info 키 아래에 값을 노출한다.
GitInfoContributor	git.properties 파일을 읽을 수 있다면 깃정보를 노출한다.
BuildInfoContributor	META-INF/build-info.properties 파일을 읽을 수 있다면 빌드 정보를 노출한다.

TIP

management.info.defaults.enabled 속성을 사용해서 위의 것들을 활성화를 조절할 수 있다.

재정의한 애플리케이션 정보

스프링 속성으로 정의된 `info.*`에 를 노출하는 `info` 종단점을 재정의할 수 있다. 모든 `Environment` 속성에서 `info`키 이하의 것들은 자동으로 노출된다. 예를 들어, `application.yml` 에 다음과 같이 추가할 수 있다:

```
info:
  app:
    encoding: UTF-8
    java.source: 1.8
    java.target: 1.8
```

위에서 처럼 하드코딩을 하는 것보다는 `빌드시 속성이 추가되도록 하는 것도 가능하다`. 메이븐을 이용하는 경우, 다음과 같이 작성하는 것도 가능하다.

`application.properties`

TIP

```
info:
  app:
    encoding: ${file.encoding}
    java.source: ${java.version}
    java.target: ${java.version}
```

깃(Git) 커밋 정보

`info` 종단점에서 유용한 기능이 하나있는데 프로젝트가 빌드될 때 `git` 소스코드 {repo}의 상태를 보여줄 수 있는 기능이 있다. 만약 `GitProperties` 빈이 있다면 `git.branch`, `git.commit.id`과 `git.commit.time` 속성 값을 확인할 수 있다.

TIP

`GitProperties` 빈은 자동구성되며 최상위 클래스패스에 `git.properties` 파일이 있을 때 사용가능하다. [깃 정보를 생성하는 방법](#)을 보면 보다 자세히 알 수 있다.

만약 깃정보 전체를 보여주고 싶다면(예를 들어 `git.properties` 전체 컨텐츠), `management.info.git.mode` 속성을 정의하면 된다:

```
management.info.git.mode: full
```

빌드(build) 정보

`info` 종단점은 `BuildProperties` 빈이 등록되어 있다면 빌드 정보를 제공할 수 있다. `META-INF/build-info.properties` 파일이 클래스패스에 있다면 가능하다.

TIP

메이븐과 그레이들 플러그인을 통해서 파일 생성이 가능한데, [빌드 정보 생성하기](#)를 보면 보다 자세히 알 수 있다.

TIP

메이븐과 그레이들 플러그인을 이용해서 파일을 생성할 수 있는데, [빌드 정보 생성하기](#)에서 보다 자세히 살펴보기 바란다.

InfoContributors 재정의 작성

제공되는 **InfoContributor** 인터페이스를 구현한 스프링 빈을 등록하면 재정의한 애플리케이션 정보를 제공할 수 있다.

4.2. HTTP 로 모니터링하고 관리하기

스프링 MVC 애플리케이션을 개발하고 있다면 스프링 부트 액츄에이터는 자동 구성으로 활성화된 모든 종단점을 HTTP로 노출한다. 기본관례에 의해 URL 경로로 종단점 ID를 사용한다. 예를 들어 **health**의 경우 **/health**로 노출된다.

4.2.1. 관리 종단점 경로 재정의

관리 종단점의 접두사를 재정의하는 것은 유용할 때가 있다. 예를 들어, 애플리케이션에서 이미 **/application**을 다른 목적으로 사용하고 있는 경우가 그렇다. **management.context-path** 속성을 사용하여 관리 종단점 접두사를 변경할 수 있다:

```
management.context-path: /manage
```

위의 **application.yml** 예제에 따르면 **/application/{id}**는 **/manage/{id}**(예로, **/manage/info**)로 변경된다.

NOTE

종단점이 각각 정의되어 있다고 하더라도, **management.context-path** 속성은 상대경로로 적용된다.

4.2.2. 관리 서버 포트 재정의

기본 HTTP 포트를 통해서 노출되는 관리 종단점은 클라우드 기반에 배포될 때 변경해야 할 경우가 생긴다. 그러나 애플리케이션을 데이터 센터에서 애플리케이션을 실행하려면 다른 HTTP 포트를 사용해야 할 수도 있다.

management.port 속성으로 HTTP 포트를 변경할 수 있다.

```
management.port: 8081
```

관리포트는 방화벽에 의해 보호되어 공개되지 않기 때문에 애플리케이션에 보안이 필요하다 하더라도 관리 종단점에 대해서는 보안관리를 하지 않을 수 있다. 스프링 시큐리티가 클래스패스에 있다면, 다음과 같이 하여 관리 보안을 비활성화할 수 있다:

```
management.security.enable: false ①
```

- ① 스프링 시큐리티가 클래스패스에 없다면 이 방법으로는 관리 보안을 비활성화할 필요가 없으며 한다면 애플리케이션이 중단될 것이다.

4.2.3. 관리를 위한 SSL 재정의

재정의한 포트를 사용하도록 구성하는 경우에 관리서버는 `management.ssl.*` 속성을 이용해서 SSL 구성이 가능하다. 예를 들어, 메인 애플리케이션이 HTTPS를 사용하는 동안 관리 서버는 HTTP를 사용하는 것이 가능하다:

```
sever:
  port: 8443
  ssl:
    enabled: true
    key-store: classpath:store.jks
    key-password: secret
management:
  port: 8080
  ssl.enabled: false
```

혹은 메인 애플리케이션과 관리서버가 다른 키를 가지고 SSL을 사용하도록 구성하는 것도 가능하다:

```
sever:
  port: 8443
  ssl:
    enabled: true
    key-store: classpath:store.jks
    key-password: secret
management:
  port: 8080
  ssl:
    enabled: true
    key-store: classpath:management.jks
    key-password: secret
```

4.2.4. 관리 서버 주소 재정의

관리 종단점의 주소를 재정의하고 싶다면 `management.address` 속성으로 설정가능하다. 이 기능은 내부 네트워크 혹은 `localhost`를 통해서만 접속하도록 설정하고 싶을 때 유용하다.

NOTE | 메인 서버 포트와 다른 경우에만 다른 주소를 지정하여 접속가능하다.

여기 `application.yml` 예제에서는 원격지에서 관리접속을 허용하지 않는다:

```
management:
  port: 8081
  address: 127.0.0.1
```

4.2.5. HTTP 종단점 비활성화

HTTP 로 종단점을 노출하고 싶지 않다면 관리포트에 **-1** 을 설정하면 된다:

```
management.port=-1
```

4.2.6. HTTP 건강 종단점 접근 제한

health 종단점은 익명으로 접근했을 때와 인증된 접근인 경우에 노출되는 정보가 다르다. 기본적으로 보안관리 중인 애플리케이션에 익명으로 접근하면 서버의 세부적인 건강 상태는 감춰지며 종단점은 단순히 서버가 작동중인지 작동중지 상태인지 여부만 보여준다.

서버가 **작동중**인 경우에 HTTP 응답(익명으로 요청시):

```
$ curl -i localhost:8080/health
HTTP/1.1 200
X-Application-Context: application:8080
Content-Type: application/vnd.spring-boot.actuator.v1+json;charset=UTF-8
Content-Length: 15

{"status":"UP"}
```

서버가 **작동중지**인 경우에 HTTP 응답으로 "DOWN"과 (503 상태코드 전달):

```
$ curl -i localhost:8080/health
HTTP/1.1 503
X-Application-Context: application:8080
Content-Type: application/vnd.spring-boot.actuator.v1+json;charset=UTF-8
Content-Length: 17

{"status":"DOWN"}
```

HTTP 응답의 상세내용 예제는 다음과 같다(설정에 따라 보다 자세한 내용이 노출된다):


```
$ curl -i localhost:8080/health
HTTP/1.1 200 OK
X-Application-Context: application:8080
Content-Type: application/vnd.spring-boot.actuator.v1+json;charset=UTF-8
Content-Length: 221

{
  "status" : "UP",
  "diskSpace" : {
    "status" : "UP",
    "total" : 63251804160,
    "free" : 31316164608,
    "threshold" : 10485760
  },
  "db" : {
    "status" : "UP",
    "database" : "H2",
    "hello" : 1
  }
}
```

4.3. 측량(Metrics)

스프링 부트 액츄에이터는 [Micrometer\(https://micrometer.io/\)](https://micrometer.io/)를 위한 자동구성과 의존성 관리를 제공하며, 애플리케이션 {metric}를 통해 다양한 모니터링 시스템을 지원하고 있다:

- [Atlas\(https://github.com/Netflix/atlas\)](https://github.com/Netflix/atlas)
- [Datadog\(https://www.datadoghq.com/\)](https://www.datadoghq.com/)
- [Ganglia\(http://ganglia.sourceforge.net/\)](http://ganglia.sourceforge.net/)
- [Graphite\(https://graphiteapp.org/\)](https://graphiteapp.org/)
- [Influx\(https://www.influxdata.com/\)](https://www.influxdata.com/)
- [Prometheus\(https://prometheus.io/\)](https://prometheus.io/)

Micrometer는 지원하는 모니터링 시스템에 맞춰 독립된 모듈을 제공한다. 이 모듈 중 하나(혹은 그 이상)에 따라서 스프링 부트 애플리케이션에서 Micrometer가 실행하기에 충분하다. Micrometer에 대해서 보다 자세히 알아보고자 한다면 [Micrometer 참고문서\(https://micrometer.io/docs\)](https://micrometer.io/docs)를 살펴보기 바란다.

스프링 부트 1.5.X.RELEASE /**metrics** 에서 보여지는 형식은 다음과 같다. 하지만 **2.0.0.RELEASE** 로 업그레이드되면서 측량 형식이 변경됐다.

```

{
  mem: 611405,
  mem.free: 356994,
  processors: 8,
  instance.uptime: 50186,
  uptime: 54526,
  systemload.average: 4.05078125,
  heap.committed: 550400,
  heap.init: 262144,
  heap.used: 193405,
  heap: 3728384,
  nonheap.committed: 62104,
  nonheap.init: 2496,
  nonheap.used: 61007,
  nonheap: 0,
  threads.peak: 41,
  threads.daemon: 33,
  threads.totalStarted: 45,
  threads: 35,
  classes: 9369,
  classes.loaded: 9369,
  classes.unloaded: 0,
  gc.ps_scavenge.count: 8,
  gc.ps_scavenge.time: 84,
  gc.ps_marksweep.count: 2,
  gc.ps_marksweep.time: 112,
  httpsessions.max: -1,
  httpsessions.active: 1,
  datasource.primary.active: 0,
  datasource.primary.usage: 0,
  gauge.response.unmapped: 4,
  gauge.response.login: 2,
  counter.status.302.unmapped: 2,
  counter.status.200.login: 1
}

```

Micrometer에서 사용할 수 있는 형식으로 변경된 듯 하다.

```
{
  names: [
    "jvm.buffer.memory.used",
    "jvm.memory.used",
    "jvm.buffer.count",
    "logback.events",
    "process.uptime",
    "jvm.memory.committed",
    "system.load.average.1m",
    "jvm.buffer.total.capacity",
    "jvm.memory.max",
    "process.start.time",
    "cpu",
    "http.server.requests"
  ]
}
```

각 측량에 대해서는 `/application/metrics/{metricsName}` 으로 상세정보를 확인할 수 있다. 예로 `/application/metrics/http.server.requests` 으로 요청을 하면 다음과 같은 내용을 확인할 수 있다.

```
{
  "name": "http.server.requests",
  "measurements": [
    {
      "statistic": "Count",
      "value": 0
    },
    {
      "statistic": "TotalTime",
      "value": 0
    },
    {
      "statistic": "Max",
      "value": 0
    }
  ],
  "availableTags": [
    {
      "tag": "{tagName}",
      "values": [
        // 예외가 발생할 경우
      ]
    }
  ]
}
```

사용하는 기능에 따라 자동구성된 측량 정보를 확인할 수 있다. /application/metrics/{metricsName}를 살펴보면 살펴볼 수 있는 측량 상세의 이름, 측정치와 사용 가능한 태그 정보를 확인할 수 있다. 지원하는 기능에 대한 보다 자세한 내용은 [스프링 부트 참고문서\(https://goo.gl/NmYMYb\)](https://goo.gl/NmYMYb) 'Metrics' 관련내용을 살펴보기 바란다.

4.4. 감시

스프링 시큐리티가 실행되면 스프링 부트 액츄에이터는 이벤트('인증성공(authentication success)', '실패(failure)'과 '접근거부(access denied)' 예외 발생시)를 게시하는 유연한 감시 프레임워크를 제공한다. 이 기능은 인증실패를 기반으로 한 접근통제 구현 및 보고에 유용하다. 보안 이벤트 게시와 관련해서 재정의의를 하려면 `AbstractAuthenticationAuditListener`와 `AbstractAuthorizationAuditListener`의 구현체를 필요로 한다.

또한 비즈니스 이벤트를 위한 감시 서비스로도 사용할 수 있다. 컴포넌트에 `AuditEventRepository`를 주입하여 직접 사용하거나 스프링 `ApplicationEventPublisher`에서 `AuditApplicationEvent`를 게시(`ApplicationEventPublisherAware` 구현)해야 한다.

4.5. 추적(Tracing)

추적은 모든 HTTP 요청에 대해서 자동으로 활성화된다. `trace` 종단점에서 볼 수 있으며 최근 100개 요청에 관한 기본정보가 유지된다. 기본 구성은 다음과 같다:

```
{
  "traces": [
    {
      "timestamp": 1511157217254,
      "info": {
        "method": "GET",
        "path": "/application/trace",
        "headers": {
          "request": {
            "host": "localhost:20000",
            "connection": "keep-alive",
            "upgrade-insecure-requests": "1",
            "user-agent": "Mozilla/5.0 (Macintosh; Intel Mac
OS X 10_12_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/61.0.3163.100
Safari/537.36",
            "accept":
"text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/ap
ng,*/*;q=0.8",
            "referer": "http://localhost:20000/application",
            "accept-encoding": "gzip, deflate, br",
            "accept-language": "en-US,ko-
KR;q=0.8,ko;q=0.6,en;q=0.4",
            "cookie":
"JSESSIONID=3807753EE776194F5B46F2F6CBAC4ADD"
          },
          "response": {
            "Content-Type": "application/vnd.spring-
boot.actuator.v2+json;charset=UTF-8",
            "Transfer-Encoding": "chunked",
            "Date": "Mon, 20 Nov 2017 05:53:37 GMT",
            "status": "200"
          }
        },
      },
      "timeTaken": "27"
    }
  ]
}
```

추적에 기본적으로 포함되어 있는 정보는 다음과 같다:

이름	설명
요청헤더	요청 헤더정보
응답헤더	응답 헤더정보
쿠키	요청헤더의 Cookie 와 응답헤더의 Set-Cookie
에러	에러가 발생할 경우
소요시간(Time Taken)	요청을 처리하는데 걸린 시간(밀리세컨드)

4.5.1. 추적 재정의

추가적으로 이벤트 추적을 해야한다면, 스프링 빈에 **TraceRepository**를 주입한다. 그리고 **add** 메서드(**void add (Map<String, Object> traceInfo)**)에 **Map** 구조의 객체를 등록하면 JSON과 로그로 변환된다.

기본적으로 **InMemoryTraceRepository**에 최근 100개 이벤트가 보관된다. 만약 용량을 확장해야할 필요성이 있다면 **InMemoryTraceRepository**를 구현하여 정의하면 된다. **TraceRepository**를 구현하여 생성하는 방법도 있다.

4.6. 프로세스 모니터링

spring-boot 모듈에는 프로세스 모니터링에 유용한 두 가지 클래스를 찾아 이용할 수 있다:

- **ApplicationPidFileWriter**는 애플리케이션 PID 파일을 생성할 수 있다(기본적으로 애플리케이션 디렉토리에 **application.pid**라는 이름으로 생성).
- **EmbeddedServerPortFileWriter** 내장 서버의 포트번호를 파일로 생성한다(기본적으로 애플리케이션 디렉토리에 **application.port**라는 이름으로 생성).

이 **Writer**들은 기본적으로 비활성화상태지만 다음에서 설명하는 방법 중에 한가지를 이용하면 활성화된다.

4.6.1. 확장 구성

META-INF/spring.factories 파일에 다음과 같이 **Writer** 들을 리스너로 활성화시킬 수 있다:

```
org.springframework.context.ApplicationListener=\
org.springframework.boot.system.ApplicationPidFileWriter,\
org.springframework.boot.system.EmbeddedServerPortFileWriter
```

4.6.2. 프로그래밍적인 처리

SpringApplication.addListeners(...) 메서드를 통해서 리스너를 활성화하고 **Writer** 객체를 전달한다. 이 방법을 이용하면 **Writer** 생성자를 통해 파일의 경로와 이름을 재정의할 수 있다.

개인적으로는 "프로그래밍적인 처리" 방법을 이용한다.

NOTE

```
@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication app = new SpringApplication
(DemoApplication.class);
        app.addListeners(new ApplicationPidFileWriter(), new
EmbeddedServerPortFileWriter());
        app.run(args);
    }
}
```

4.7. 다음 읽을꺼리

개발한 애플리케이션을 모니터링하고 관리하는 방법을 살펴봤으니, 실제로 다양한 시스템에 배포해보자. 이 과정이 반복되면 웹 애플리케이션을 배포하고 운영하는 노하우가 쌓이게 될 것이고 다양한 응용할 수 있게 될 것이다.

Chapter 5. '스프링 부트' 배포

편리하고 막강한 기능을 제공하는 클라우드 서비스들이 많이 등장했다. 그 클라우드 서비스에 우리가 만든 앱을 배포하는 과정을 살펴보면서 자신에게 맞는 클라우드 서비스를 선택하고 관리해보자.

5.1. 클라우드 배포

스프링 부트 실행가능한 jar의 경우 압축된 패키지 안에 애플리케이션을 실행하는데 필요한 모든 것을 갖추고 대부분의 클라우드 PaaS(Platform-as-a-Service) 프로바이더(제공자, Provider)를 지원할 준비가 되어있다. 클라우드 프로바이더에 크게 상관없이 실행가능한 jar로 빌드된 애플리케이션을 Java 플랫폼에 배포가능하다.

NOTE 구글 앱엔진 클래식은 Java 7 실행환경에서는 서블릿 2.5 API를 지원하므로 "오래된 컨테이너(Servlet 2.5)에 war 배포"를 참고하기 바란다. Java 8 실행환경에서는 서블릿 3.1 과 서블릿 2.5 사양을 지원한다.

클라우드 환경에 배포하는 방법은 클라우드 프로바이더에 따라서 다양하다. 현재 근무하고 있는 회사에서는 젠킨스(jenkins)를 이용하여 실행가능한 jar로 배포본을 빌드하고 Elastic Beanstalk 에서 애플리케이션 정보를 등록하면서 지정한 S3 버킷(Bucket)에 버전을 변경하여 배포하는 방식을 이용하고 있다.

보다 자세한 내용은 [스프링 부트 참고문서 \(Deploying To the cloud, https://goo.gl/dzWtvV\)](https://goo.gl/dzWtvV)를 살펴보기 바란다.

5.2. 스프링 부트 애플리케이션 설치

스프링 부트 애플리케이션은 유닉스 시스템에서 `java -jar`를 사용하여 애플리케이션을 실행하는 것이 가능하다. `init.d` 혹은 `systemd`로 등록하여 실행가능한 바이너리처럼 실행가능한 바이너리 jar를 등록하여 실행하는 것이 가능하다. 일반적인 운영환경에서 스프링 부트 애플리케이션을 설치하고 관리가 손쉬울 것이다.

WARNING 실행가능한 바이너리 jar는 빌드를 하면서 내부에 추가된 스크립트를 통해서 동작한다. 일부 도구는 이 형식을 허용하지 않으므로 항상 이 기술을 사용할 수 있는 것은 아니다. 예를 들어 실행가능한 jar 혹은 war 파일에 대해서 `jar -xf` 명령으로 완벽하게 압축해제하지 못할 수 있다. jar 혹은 war 를 `java -jar`로 실행하거나 서블릿 컨테이너에 배포하는 대신 직접 실행하려는 경우에는 실행가능한 바이너리 jar 또는 war 로 만들 것을 권장한다.

빌드와 함께 실행 스크립트를 생성하려면 2.0.0.RELEASE 그레이들에서는 다음과 같이 선언해야 한다:

`build.gradle`

```
bootJar {
    launchScript()
}
```

2.0.0.RELEASE 이전 버전에서는 다음과 같았다:


```
springBoot {  
    executable = true  
}
```

메이븐에서는 다음과 같이 선언해야 한다:

pom.xml

```
<plugin>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-maven-plugin</artifactId>  
    <configuration>  
        <executable>true</executable>  
    </configuration>  
</plugin>
```

이렇게 생성된 애플리케이션은 **./boot-spring-boot.jar**(프로젝트명이 **boot-spring-boot**)로 실행이 가능하다. jar 파일이 있는 디렉토리가 작업 디렉토리가 된다.

실행가능한 바이너리 jar 설정을 하고 빌드를 하게 되면 그레이들의 경우 **build/libs**에 파일이 생성되고 메이븐의 경우에는 **target** 디렉토리에 생성이 된다. 생성된 실행가능한 바이너리 jar를 살펴보면 **jar -xf**가 아니라 **unzip boot-spring-boot.jar**으로 압축해제 해야 한다.

실행가능한 바이너리 jar 파일을 압축해제하여 살펴보면 **META-INF/MANIFEST.MF** 파일을 열어보면 다음과 같은 내용을 확인할 수 있다:

```
Manifest-Version: 1.0  
Start-Class: io.honeymon.boot.springboot.BootSpringBootApplication  
Spring-Boot-Classes: BOOT-INF/classes/  
Spring-Boot-Lib: BOOT-INF/lib/  
Spring-Boot-Version: 1.5.8.RELEASE  
Main-Class: org.springframework.boot.loader.JarLauncher
```

org.springframework.boot.loader.JarLauncher를 실행하고 매니페스트 파일을 읽어서 **BootSpringBootApplication**를 실행할거라고 유추해볼 수 있다. 파일을 살펴보면 실행가능한 속성(x)도 부여되어 있다.

```
$ ls -la boot-spring-boot-1.0.0.RELEASE.jar  
-rwxr-xr-x 1 honeymon staff 32371950 Nov 21 12:43 boot-spring-boot-  
1.0.0.RELEASE.jar
```

대부분의 리눅스 배포본을 지원하며 CentOS 와 Ubuntu 에서 테스트됐다. OS X와 FreeBSD 같은 플랫폼에서는 **embeddedLaunchScript**를 재정의하여 사용해야한다.

5.2.1. 유닉스/리눅스 서비스

스프링 부트 애플리케이션은 `init.d` 혹은 `systemd`를 이용하여 유닉스/리눅스 서비스로 시작하는 것이 쉽다.

`init.d`로 설치하기

스프링 부트 그레이들 혹은 메이븐 플러그인을 통해서 실행가능한 바이너리 jar를 생성하도록 구성하고 `embeddedLaunchScript`를 재정의하지 않았다면 애플리케이션을 `init.d` 서비스로 사용가능하다. 그렇게 하기 위해서는 jar 파일을 `init.d`에 심볼릭링크(symlink)하여 표준 `start`, `stop`, `restart`와 `status` 명령어를 지원하도록 한다.

스크립트는 다음의 기능들을 지원한다:

- 사용자 서비스로 jar 파일 구동
- `/var/run/<appname>/<appname>.pid`를 통해서 애플리케이션 PID 추적
- 콘솔 로그를 `/var/log/<appname>.log`에 기록

스프링 부트 애플리케이션을 `/var/boot-spring-boot`에 설치했다고 가정하고 설치한 애플리케이션을 `init.d` 서비스로 연결하는 심볼릭링크를 생성한다:

```
$ sudo ln -s /var/boot-spring-boot/boot-spring-boot.jar /etc/init.d/boot-spring-boot
```

설치가 끝나면 서비스를 시작하고 중단할 수 있게 된다. 예를 들어, 데비안 계열 시스템에서는 다음과 같이 시작할 수 있다:

```
$ service boot-spring-boot start
```

TIP 애플리케이션 구동이 실패하면 `/var/log/<appname>.log` 로그파일에 기록된 로그를 살펴보면 된다.

표준 운영체제 도구를 이용해서 자동으로 애플리케이션이 구동되는 것을 조정할 수 있다. 예를 들어, 데비안 계열 시스템에서는 다음과 같이 할 수 있다:

```
$ update-rc.d boot-spring-boot defaults
```

`systemd`로 설치하기

`systemd`는 유닉스 시스템 V init 시스템의 계승자이며 현재는 많은 리눅스 배포판들이 지원하고 있다. 계속 `systemd`와 함께 `init.d` 스크립트를 계속 사용할 수 있지만 `systemd service` 스크립트를 사용하여 스프링 부트 애플리케이션을 시작할 수 있다.

스프링 부트 애플리케이션을 `/var/boot-spring-boot`에 설치했다고 가정하고 설치한 애플리케이션을 `systemd` 서비스로 설치하기 위해서는 `/etc/systemd/system` 디렉토리에서 `boot-spring-boot.service` 라는 이름으로 스크립트를 생성한다. 다음 예제 스크립트를 살펴보자:

```
[Unit]
Description=boot-spring-boot
After=syslog.target

[Service]
User=honeymon
ExecStart=/var/boot-spring-boot/boot-spring-boot.jar
SuccessExitStatus=143

[Install]
WantedBy=multi-user.target
```

IMPORTANT

실제로 사용할 때는 **Description**, **User**와 **ExecStart** 필드를 변경해야한다는 것을 기억해두자.

NOTE

ExecStart 필드에 스크립트 액션 명령어를 선언하지 않으면 **run** 명령이 기본사용된다.

init.d 서비스로 실행할 때와는 달리, 애플리케이션 PID 파일 및 콘솔로그를 실행하는 사용자는 **systemd** 자체에서 관리하므로 **Service** 스크립트의 해당필드를 사용하여 구성해야 한다. 보다 자세한 사항은 **systemd.service** (<https://goo.gl/ycHkj5>)를 살펴보기 바란다.

시스템 구동시 자동으로 시작되도록 조정이 가능한데 다음과 같이 명령하면 된다:

```
$ systemctl enable boot-spring-boot.service
```

보다 자세한 내용에 대해서는 **\$ man systemctl** 을 실행하거나 혹은 <https://goo.gl/CsuFGP> 메뉴얼을 참조하기 바란다.

5.3. MS 윈도우즈 서비스

스프링 부트 애플리케이션 은 **winsw** 를 사용하여 윈도우 서비스로 실행할 수 있다.

예제(**spring-boot-daemon**(<https://goo.gl/jPKzdL>))를 보면서 스프링 부트 애플리케이션을 윈도우즈 서비스로 어떻게 생성하는지 단계별로 살펴볼 수 있다.

5.4. 다음 읽을꺼리

PaaS에서 제공하는 기능 및 종류에 대한 자세한 정보는 각 웹사이트에서 확인하기 바란다. 자바를 실행할 수 있는 스프링 부트는 클라우드 기반 배포에 적합하며 다양한 클라우드 서비스 공급자들을 자유롭게 고려할 수 있다.

메이븐 과 그레이들 에서 제공하는 스프링 부트 플러그인을 살펴본다.

Chapter 6. '스프링 부트' 빌드도구 플러그인

에서는 '실행가능한(Executable)' 배포본(jar or war)를 만들어내는 '빌드 도구' 를 살펴보도록 한다. 스프링 부트는 자바 개발시 주로 사용하는 빌드도구인 메이븐과 그레이들 에서 사용할 수 있는 빌드 플러그인을 제공하고 있다. 이 빌드 플러그인을 이용하여 빌드할 때 고려할 수 있는 다양한 선택사항들을 제공하여 개발자의 다양한 욕구를 수용하고 있다. 여기서 '실행가능한' 배포본을 만들어내는 교묘한 기술 '재포장(리패키징, Repackage)' 에 대해 설명한다.

6.1. 스프링 부트 메이븐 플러그인

스프링 부트는 메이븐과 그레이들에서 사용할 수 있는 빌드도구 플러그인을 제공한다. 플러그인은 다양한 기능을 제공하는데 그중에는 실행가능한 jar의 패키징도 포함되어 있다. 이번 장에서는 플러그인에 대해서 상세히 살펴보고, 빌드 시스템에서 지원하지 않는 기능이 필요할 때 확장하는 방법을 살펴보도록 한다. 이제 막 이 부분부터 시작한다면 "[스프링 부트' 살펴보기](#)"의 "[빌드 시스템](#)"을 읽어보길 바란다.

6.1.1. 플러그인 추가

[스프링 부트 메이븐 플러그인](#)은 메이븐에 대한 스프링 부트를 지원하며, 실행가능한 jar나 war를 패키징하거나 애플리케이션으로 바로 실행하는 기능 등을 제공한다. 이를 위해서는 메이븐 3.2(혹은 그 이상)을 사용해야 한다.

NOTE | 보다 자세한 내용은 [스프링 부트 메이븐 플러그인](#)을 살펴보기 바란다.

6.1.2. 플러그인 추가하기

스프링 부트 메이븐 플러그인을 추가하려면 `pom.xml` 내에 적당한 위치에 `plugins`에 플러그인에 대한 XML을 추가하면 된다.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi=
"http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <!-- ... -->
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
        <version>2.0.0.RELEASE</version>
        <executions>
          <execution>
            <goals>
              <goal>repackage</goal>
            </goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
</project>
```

이 구성은 메이븐 {lifecycle}에서 **package** 단계에서 빌드가 진행되는 동안 jar나 war를 재포장한다. 다음 예제에서는 리패키징된 jar와 원본 jar가 **target** 디렉토리에 있는 것을 보여준다:

```
$ mvn package
// 로그 생략
$ ls target/*.jar
target/boot-spring-boot-1.0.0.RELEASE.jar target/boot-spring-boot-
1.0.0.RELEASE.jar.original
```

만약 위의 플러그인 선언부에서 **<execution/>**구성을 포함시키지 않았다면, 다음과 같이 별도로 플러그인을 실행시켜야 한다(패키징이 잘 되어있다면 잘 동작한다).

```
$ mvn package spring-boot:repackage
// 로그 생략
$ ls target/*.jar
target/boot-spring-boot-1.0.0.RELEASE.jar target/boot-spring-boot-
1.0.0.RELEASE.jar.original
```

만약에 아직 정식 출시가 되지 않은 마일스톤(milestone) 혹은 스냅샷(snapshot) 버전을 사용하고자 한다면 적당한 위치에 **pluginRepository** 요소를 추가하면 된다:

```

<pluginRepositories>
  <pluginRepository>
    <id>spring-snapshots</id>
    <url>http://repo.spring.io/snapshot</url>
  </pluginRepository>
  <pluginRepository>
    <id>spring-milestones</id>
    <url>http://repo.spring.io/milestone</url>
  </pluginRepository>
</pluginRepositories>

```

6.1.3. 실행가능한 jar 와 war 파일 패키징

`pom.xml` 에 `spring-boot-maven-plugin`이 추가되어 있다면 자동적으로 `spring-boot:repackage` 골을 사용하여 실행가능한 패키지로 재작성하는 작업이 진행된다. 프로젝트를 jar로 할 지 war로 할 지 `packaging` 요소에서 정의하는 것이 가능하다:

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi=
"http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <!-- ... -->
  <packaging>jar</packaging> ①
  <!-- ... -->
</project>

```

① 패키징을 jar로 할 지 war로 할 지 정의하는 부분

`package` 페이지가 진행되는 동안 스프링 부트가 압축하는 과정에 개입한다. 실행하고자 하는 메인 클래스를 구성요소로 선택하거나 매니페스트에 `Main-Class` 속성을 추가하는 방법이 있다. 메인 클래스를 정의하지 않았다면 플러그인은 `public static void main(String[] args)` 메서드를 가지고 있는 클래스를 탐색할 것이다.

프로젝트에 대한 빌드와 실행을 하려면 다음과 같이 입력한다:

```

$ mvn package
// 로그 생략
$ java -jar target/boot-spring-boot-1.0.0.RELEASE.jar
// 로그생략

```

실행가능하면서 외부 컨테이너에서 배포가능한 war 파일을 빌드하기 위해서는 내장 컨테이너 의존성에 대해서 "provided"를 선언해야 한다:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi=
"http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <!-- ... -->
  <packaging>war</packaging>
  <!-- ... -->
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
      <scope>provided</scope>
    </dependency>
  <!-- ... -->
</dependencies>
</project>
```

TIP | "배포가능한 war 파일 생성"에서 어떻게 배포 가능한 war 파일을 생성하는지 자세히 살펴볼 수 있다.

보다 자세한 옵션과 예제는 '플러그인 안내 페이지'에서 살펴볼 수 있다.

6.2. 스프링 부트 그레이들 플러그인

스프링 부트 그레이들 플러그인은 그레이들에 대한 {executable-jar}나 war를 패키징하거나 스프링 부트 애플리케이션을 실행하거나 **spring-boot-dependencies**에 의해 제공되는 의존성 관리를 사용하는 등의 스프링 부트에 관련된 기능을 지원한다.

NOTE

스프링 부트 2.0.0.RELEASE부터는 그레이들 4.0 혹은 그 이상의 버전을 필요로 한다. 플러그인과 관련된 내용에 대해서는 다음의 플러그인 문서를 참고하기 바란다.

- 참고문서: [HTML](#) or [PDF](#)
- [API 문서](#)

6.2.1. 플러그인 추가하기

스프링 부트 그레이들 플러그인구성을 사용하려면 **plugin** 영역에 다음과 같이 선언한다:

```
plugins {  
    id 'org.springframework.boot' version '2.0.0.RELEASE'  
}
```

6.2.2. 그레이들 의존성 관리

spring-boot 플러그인은 자동으로 **의존성 관리 플러그인**을 제공하며 **spring-boot-starter-parent** BOM을 불러와 구성한다. 이 기능은 메이븐 사용자에게도 메이븐과 유사한 의존성 관리 경험을 제공한다. 예를 들어, BOM에서 관리하고 있는 의존성을 정의할 때는 버전을 생략하는 것이 가능하다. 이 기능을 사용한다면, 의존성을 선언하면서 버전은 비워두면 된다:

```
dependencies {  
    compile("org.springframework.boot:spring-boot-starter-web")  
    compile("org.thymeleaf:thymeleaf-spring4")  
    compile("nz.net.ultraq.thymeleaf:thymeleaf-layout-dialect")  
}
```

NOTE

spring-boot 그레이들 플러그인의 버전은 **spring-boot-starter-parent** BOM에 버전을 정의하고 불러온다. **spring-boot** 그레이들 플러그인의 버전은 항상 사용하려고 하는 실제 스프링 부트 버전으로 설정해야 한다. 제공하는 의존성 라이브러리에 대한 상세한 내용은 [스프링 부트 부록#Dependency versions\(https://goo.gl/H1VShv\)](https://goo.gl/H1VShv)을 살펴보기 바란다.

의존성 관리 플러그인이 제공하는 다양한 기능을 더 살펴보고 싶다면, **의존성 관리 플러그인**을 살펴보기 바란다.

6.2.3. 실행가능한 jar 와 war 파일 생성

spring-boot 플러그인이 프로젝트에 적용되면 **bootRepackage** 태스크를 이용하여 아카이브를 다시 작성하여 실행가능하게 만든다. jar 또는 war(적절한 경우)로 프로젝트를 빌드하도록 프로젝트를 구성해야 한다.

실행할 메인 클래스는 구성 옵션을 사용하여 정의하거나 **Main-Class** 속성을 매니페스트에 추가하여 지정할 수 있다. 만약 메인 클래스를 정의하지 않았다면 플러그인은 **public static void main(String[] args)** 메서드가 작성되어 있는 클래스를 탐색한다(**ApplicationRunner** 혹은 **CommandLineRunner**를 이용하여 테스트 코드 등을 작성했다면 반드시 메인 클래스를 정의해야 한다).

TIP

구성 옵션에 관해 상세히 살펴보고자 한다면 "[[build-tool-plugins-gradle-repackage-configuration](#)]"을 확인하기 바란다.

빌드한 프로젝트 아티팩트를 실행하려면 다음과 같이 입력하면 된다:

```
$ gradle build  
$ java -jar build/libs/boot-spring-boot-1.0.0.RELEASE.jar
```

war 파일을 빌드하면 실행하거나 외부 컨테이너에 배포하는 것이 가능한데, 내장 컨테이너 의존성에 war 플러그인에

`providedRuntime` 구성할 수 있도록 표시해야 한다:

```
...
apply plugin: 'war'

war {
    baseName = 'boot-spring-boot'
    version = '1.0.0.RELEASE'
}

repositories {
    jcenter()
    maven { url "http://repo.spring.io/milestone" }
}

dependencies {
    compile("org.springframework.boot:spring-boot-starter-web")
    providedRuntime("org.springframework.boot:spring-boot-starter-tomcat")
    ...
}
```

TIP 배포 가능한 파일을 어떻게 생성하는지에 대해서는 "[배포 가능한 war 파일 생성](#)" 섹션을 살펴보기 바란다.

6.2.4. 프로젝트를 빌드하지 않고 실행하기

`bootRun` 태스크를 사용하면 jar 파일을 빌드하지 않아도 프로젝트를 실행하는 것이 가능하다.

```
$ gradle bootRun
```

"[스프링 부트 개발자도구](#)"를 프로젝트에 추가해두면 애플리케이션의 변경을 자동으로 감지한다. 혹은 대안으로, 애플리케이션이 실행했을 때 정적 자원 클래스패스(예: 기본구성은 `src/main/resources`으로 지정)가 동작중인 애플리케이션에서 재적재할 수 있도록 애플리케이션을 실행하는 기능은 개발시 유용할 수 있다.

```
bootRun {
    addResources = true
}
```

정적 자원 클래스패스를 재적재할 수 있게 만들면 `bootRun`이 `processResources` 태스크의 결과물을 사용하지 않음을 의미한다. 즉, `bootRun`을 사용하면 애플리케이션은 처리되지 않은 자원을 사용하게 된다.

6.2.5. 스프링 부트 플러그인 구성

그레이들 플러그인은 전역 구성을 위한 `springBoot` 요소에 대한 빌드 스크립트 DSL을 자동으로 확장한다. 그레이들을 확장하여 속성을 적당히 설정할 수 있다.

```
springBoot {
    backupSource = false
}
```

스프링 부트 플러그인 구성 옵션들

가능한 옵션은 다음과 같다:

이름	설명
<code>mainClass</code>	실행가능한 형태로 압축했을 때 실행할 메인 클래스를 정의한다.
<code>providedConfiguration</code>	제공되는 구성이름 정의(기본적으로 <code>providedRuntime</code>)
<code>backupSource</code>	리패키징하기에 앞서 원본자원 아카이브를 백업하고 싶을 때 정의(기본은 <code>true</code>)
<code>customConfiguration</code>	구성 이름을 재구성하는 이름
<code>layout</code>	의존성을 내부에서 어떻게 레이아웃을 나누는지 결정하는 압축 유형 정의(기본적으로 압축유형을 기반으로 추측가능) 자세한 내용은 가능한 레이아웃(layout) 을 살펴보기 바란다.
<code>layoutFactory</code>	레이아웃팩토리는 레이아웃을 재정의할 때 필요하다. 서드파티에서 제공하는 레이아웃을 구성해야할 때 사용한다. 레이아웃 팩토리를 사용할 때는 <code>layout</code> 을 사용할 수 없다.
<code>requiresUnpack</code>	실행을 위해 육중한 jar에서 압축을 해제할 “groupId:artifactId” 형식으로 정의한 의존성 목록. 대상 파일들은 육중한 jar 안에 압축된 상태로 있지만, 실행을 위해서 자동으로 압축해제된다.

가능한 레이아웃(layout)

`layout` 속성은 압축형태와 부트스트랩 로더를 포함할지 여부를 구성한다. 지원하는 레이아웃은 다음과 같다:

이름	설명	실행가능여부
<code>JAR</code>	일반적인 실행가능한 <code>JAR 레이아웃</code>	예
<code>WAR</code>	실행가능한 <code>WAR 레이아웃</code> . <code>provided</code> 로 정의된 의존성 라이브러리들은 <code>war</code> 가 서블릿 컨테이너에 배포했을 때 충돌을 피하기 위해서 <code>WEB-INF/lib-provided</code> 에 위치한다.	예
<code>ZIP(DIR 동일)</code>	<code>JAR</code> 레이아웃과 유사하며, <code>PropertiesLauncher</code> 를 사용한다.	예
<code>MODULE</code>	기본적인 의존성(<code>provided</code> 스코프를 가진 건 제외)과 프로젝트 자원을 가짐	아니오
<code>NONE</code>	기본적으로 모든 의존성과 프로젝트 자원을 가짐	아니오

개발을 마친 애플리케이션을 배포하기 위해 빌드하는 과정에서 `springBoot`와 `bootRepackage` 태스크에 속성을 정의하여 개발자가 의도하는 메인 클래스를 정의하고 배포하는 환경에 맞춰 레이아웃을 구성하여 배포하는 상황은 빈번하게 발생한다. 별도의 서버에서 톰캣에 배포하던 애플리케이션을 클라우드 플랫폼에 배포하여 운영하려고 할 때 클라우드 플랫폼의 인스턴스에 톰캣을 구성하고 실행하는 것도 가능하지만, 실행가능한 jar 형태로 빌드하여 간단한 스크립트와 함께 실행하는 구성이 가능하다. 스프링 부트를 개발플랫폼으로 사용하면 이런 배포환경의 변화에 대해 유연하게 대처하는 것이 가능하다.

우리회사에도 초창기에는 AWS에서 EC2 인스턴스를 생성하고 그 안에 톰캣을 구성하여 젠킨스로 빌드한 war를 배포하는 형태를 취했었다. AWS Beanstalk 를 이용하여 애플리케이션을 배포 및 실행할 수 있게 되면서 jar 형태로 변경해야 했다.

6.3. 그 외의 빌드 시스템

만약 메이븐, 그레이들 혹은 앤트 이외의 빌드 도구를 사용하길 원한다면 스스로 플러그인을 개발해야할 수도 있다. 실행가능한 jar는 특정한 형식과 압축되지 않은 양식으로 작성된 목록을 포함하고 있어야 한다(보다 자세한 사항은 스프링 부트 참조문서의 '`executable-jar`'를 살펴보기 바란다).

스프링 부트 메이븐과 그레이들 플러그인 모두 `spring-boot-loader-tools`를 이용해서 jar를 생성한다. 필요하다면 자유롭게 이 라이브러리를 직접 사용할 수 있다.

6.3.1. 재압축한 압축파일

이미 생성된 압축파일을 재압축하는 과정에서 컨테이너를 포함한 실행가능한 압축파일을 만들기 위해서 `org.springframework.boot.loader.tools.Repackager`를 사용한다. `Repackager` 클래스는 생성자를 하나 가지고 있으며 생성되어 있는 jar 혹은 war 파일을 참조한다. `repackage()` 메서드 중에서 하나를 사용하여 원본 파일을 대체하거나 새로운 위치에 작성할 수 있다. 변수 설정은 실행하기에 앞서 `Repackager`에거 구성가능하다.

6.3.2. 라이브러리 포함하기

압축파일을 재압축할 때 `org.springframework.boot.loader.tools.Libraries` 인터페이스를 사용하여 의존성 파일들에 대한 참조를 포함시킬 수 있다. `Libraries`에 대한 구현체를 제공하지는 않으며 사용하는 빌드 시스템에 따라 사용하면 된다.

만약 압축파일에 라이브러리가 포함되어 있다면 `Libraries.NONE` 을 사용할 수 있다.

6.3.3. 메인 클래스 찾기

`Repackager.setMainClass()`를 사용하여 메인 클래스를 정의하지 않았다면 repackager는 ASM을 사용하여 클래스 파일을 읽고 `public static void main(String[] args)` 메서드를 가지고 있는 적합한 클래스를 찾아내려고 한다. 한 개 이상을 찾았을 경우에는 예외가 발생한다.

6.3.4. 재압축 구현 예제

기본적인 재압축 예제를 살펴보자:

```

Repackager repackager = new Repackager(sourceJarFile);
repackager.setBackupSource(false);
// repackager.repackage(Libraries.NONE); ①
repackager.repackage(new Libraries() {
    @Override
    public void doWithLibraries(LibraryCallback callback) throws
IOException {
        // Build system specific implementation, callback for each
        dependency
        // callback.library(new Library(nestedFile,
        LibraryScope.COMPILE));
    }
});

```

① 라이브러리가 포함되어 있을 때는 `Libraries.NONE`를 선언하여 라이브러리를 포함시키지 않을 수도 있다.

6.4. 다음 읽을꺼리

빌드도구 플러그인에 흥미가 생겼다면 깃헙에서 [spring-boot-tools](#) 모듈을 살펴보기 바란다. 보다 기술적인 자세한 사항에 대해서는 부록 '[\[appendix-reference-executable-jar-structure\]](#)'에서 살펴볼 수 있다.

빌드와 관련된 부분들에 대해서는 '[스프링 부트](#)' [활용 안내](#)를 살펴보기 바란다.

Chapter 7. '스프링 부트' 활용 안내

이번 장에서는 스프링 부트를 사용하면서 자주 하게되는 '이제 어떻게 해야한담?'와 유사한 형태의 자주 묻게되는 질문에 대한 해답을 제시한다. 완전하다고 할 수는 없지만 상당한 영역에 대해서 다루게 될 것이다(양이 많다. 아주 많다).

만약 여기서 다루지 않은 부분에서 문제를 겪고 있다면, [스택오버플로우\(stackoverflow.com\)](https://stackoverflow.com)에서 누군가 답변한 것을 찾아볼 수 있을 것이다. 이 곳은 새로운 질문을 하기에다 매우 좋은 곳이다([spring-boot](#) 태그를 사용하기 바란다).

스프링 부트는 기본적으로 개발팀에서 권장(Recommend) 하는 방식이 있다. 그렇지만 스프링 부트를 사용하는 개발자에게 다양한 확장 포인트를 제공하고 있다. 이는 스프링 부트의 근간이 되는 스프링 프레임워크가 가지고 있는 특징이기도 하다. '다 써볼 수 있을까?' 싶을만큼 너무나 다양하고 방대한 내용을 다루고 있다.

7.1. 스프링 부트 애플리케이션

7.1.1. 오류분석기 제작

FailureAnalyzer는 구동시 발생하는 예외사항을 수집하여 사람이 읽기 좋은 메시지(**FailureAnalysis**로 포장)로 변환해준다. 스프링 부트는 **JSR-303** 외에도 애플리케이션 컨텍스트와 관련된 예외에 대한 다양한 분석기를 제공한다. 필요한 것을 작성하는 것도 정말 쉽다.

AbstractFailureAnalyzer는 **FailureAnalyzer**를 손쉽게 확장할 수 있도록 하여 특정유형의 예외가 발생했을 때 제어할 수 있도록 한다. 실제로 예외가 발생하면 구현한 부분에서 확장하여 다룰 수 있게 된다. 만약 예외를 제어할 수 없는 상황이 발생한다면, **null** 을 반환하여 다른 구현체에서 예외를 제어할 기회를 줘야한다.

FailureAnalyzer 구현체는 **META-INF/spring.factories**에 등록해야 한다.
ProjectConstraintViolationFailureAnalyzer를 등록한 예가 있다:

```
org.springframework.boot.diagnostics.FailureAnalyzer=\ncom.example.ProjectConstraintViolationFailureAnalyzer
```

7.1.2. 자동구성 문제해결

스프링 부트 자동구성 '가급적 가장 적합하게' 구성하려고 시도하지만, 때때로 실패하는 경우가 발생하고 왜 그런지 설명하기 어려울 때가 있다.

이런 경우에는 스프링 부트 **ApplicationContext** 에서 사용할 수 있는 **ConditionEvaluationReport**가 정말 유용하다. **DEBUG** 레벨의 로그를 출력하도록 했다면 볼 수 있다. 만약 **spring-boot-actuator**를 사용하고 있다면 **autoconfig** 종단점에서 JSON 형태로 리포트가 제공된다.

'소스코드와 Javadoc 을 보면 질문에 대한 답변을 얻을 수 있다'고 고수들이 이야기 합니다. 몇 가지 규칙을 정리해보자면:

- **AutoConfiguration** 이란 이름을 가지고 있는 클래스를 찾아서 소스코드를 살펴보고, **@Conditional** 애너테이션이 선언되어 있는 부분을 살펴보면 그 기능이 언제 활성화될 지 알 수 있다. 커맨드라인에서 실행시 **--debug**이나 **-Ddebug** 시스템 속성을 추가하면 애플리케이션이 구동되면서 구성된 자동구성과 관련된 정보를 콘솔 로그로 볼 수 있다. 실행중인 애플리케이션의 액츄에이터 **autoconf** 종단점

(`/application/autoconfig`)에서도 동일한 정보를 확인할 수 있다.

- `@ConfigurationProperties`를 사용하고 있는 클래스(예: `ServerProperties`)를 찾아보고 그 클래스와 관련된 외부 구성옵션을 찾아본다. `@ConfigurationProperties`는 `prefix` 속성을 가지고 있는데 외부 속성들의 접두사로 동작하며 `ServerProperties`는 `prefix="server"`으로 선언되어 이를 통해 구현된 구성속성들은 `server.port`, `server.address` 와 기타 등등이 있다. 애플리케이션 액츄에이터가 실행중이라면 `configprops` 종단점에서 찾아볼 수 있다.
- `Environment`의 구성값을 `Binder`의 `bind` 메서드를 사용하여 명시적으로 끌어올 수 있다. 접두사를 이용하는 경우가 많다(`server`, `spring` 등).
- `@Value` 애너테이션을 사용하여 `Environment` 에서 직접 연계하여 찾아볼 수 있다.
- `Environment`으로부터 위치변환자에 대하여 SpEL 표현식에 따라서 활성화 혹은 비활성화될 수 있는 `@ConditionalOnExpression` 애너테이션을 찾아본다.

7.1.3. `Environment` 혹은 `ApplicationContext` 가 시작하기 전에 재정의하기

`SpringApplication`은 `ApplicationListeners`와 `ApplicationContextInitializers` 을 가지고 있으며, 이것들은 컨텍스트 혹은 환경을 재정의하는 용도로 사용된다. 스프링 부트는 `META-INF/spring.factories`를 이용하여 다수의 재정의된 것들을 적재한다. 이것들을 등록할 수 있는 방법은 다양하다:

- `SpringApplication`이 실행되기 전에 `addListeners`와 `addInitializers` 메서드를 호출하여 프로그래밍하는 방법
- `context.initializer.classes` 혹은 `context.listener.classes` 속성을 통해서 애플리케이션에 정의하기
- `META-INF/spring.factories`에 작성하여 모든 애플리케이션에 정의하고 애플리케이션에서 라이브러리로 사용하는 jar 로 패키징하는 방법

`SpringApplication`은 몇몇 `ApplicationEvents`를 리스너(컨텍스트가 생성되기 전에 일부)로 보낸 다음 `ApplicationContext`에서 게시(Publish)한 이벤트에 대한 리스너를 등록한다. 이 전체리스트는 [\[boot-features-application-events-and-listeners\]](#)에서 살펴볼 수 있다.

애플리케이션 컨텍스트가 갱신될 때 `EnvironmentPostProcessor`를 사용하기 전에 `Environment`를 재정의하는 것도 가능하다. 각 구현체를 `META-INF/spring.factories` 에 등록하면 된다:

+

```
org.springframework.boot.env.EnvironmentPostProcessor=com.example.YourEnvironmentPostProcessor
```

구현체는 임의의 파일을 적재하고 `Environment`에 추가할 수 있다. 클래스패스에 있는 YAML 구성 파일을 적재하는 예를 들어보자:

+

```

public class EnvironmentPostProcessorExample implements
EnvironmentPostProcessor {

    private final YamlPropertySourceLoader loader = new
YamlPropertySourceLoader();

    @Override
    public void postProcessEnvironment(ConfigurableEnvironment
environment,
        SpringApplication application) {
        Resource path = new ClassPathResource(
"com/example/myapp/config.yml");
        PropertySource<?> propertySource = loadYaml(path);
        environment.getPropertySources().addLast(propertySource);
    }

    private PropertySource<?> loadYaml(Resource path) {
        if (!path.exists()) {
            throw new IllegalArgumentException("Resource " + path + " does
not exist");
        }
        try {
            return this.loader.load("custom-resource", path, null);
        }
        catch (IOException ex) {
            throw new IllegalStateException(
                "Failed to load yaml configuration from " + path, ex);
        }
    }
}

```

TIP

Environment는 기본적으로 스프링 부트에 의해 모든 속성들을 사용할 수 있도록 적재가 되어 있다. 그래서 **Environment**로 부터 파일의 위치를 얻는 것도 가능하다. 이 예서는 목록 마지막에 **custom-resource** 속성을 추가하여 다른 위치에서 정의된 키가 우선 적용되도록 한다. 재정의한 구현을 통해서 순서를 재정의할 수 있다.

NOTE

@SpringBootApplication에서 **@PropertySource**을 사용하는 것은 **Environment**로 부터 재정의한 자원을 쉽게 적재하고 편리하게 사용할 수 있는데, **ApplicationContext**을 새로 고치기 전에 스프링 부트가 **Environment**를 준비하는 것을 권장하지는 않는다. **@PropertySource**를 통해서 정의한 어떤 키는 자동구성에 영향을 끼치기에는 너무 늦게 적재된다.

7.1.4. ApplicationContext 계층 조직(부모 혹은 루트 컨텍스트 추가)

ApplicationBuilder 클래스를 사용하여 부모/자식 **ApplicationContext** 계층을 생성할 수 있다. '스프링 부트 기능' **유연한(Fluent) 빌더 API** 보다 자세한 내용을 살펴볼 수 있다.

7.1.5. 비-웹애플리케이션 생성

모든 스프링 애플리케이션이 웹 애플리케이션(혹은 웹서비스)여야 하는 것은 아니다. `main` 메서드에서 코드를 실행하려면 스프링 애플리케이션 인프라를 구성하고 구동해야 할 것 같지만, 스프링 부트의 `SpringApplication` 기능을 이용하면 너무나 쉽게 처리할 수 있다. `SpringApplication`는 웹 애플리케이션인지 여부에 따라서 `ApplicationContext`를 변경한다. 이를 위한 첫번째 할일은 클래스패스에서 서블릿 API 의존성을 제거하는 것이다. 이렇게 하고 싶지 않은 경우에는 `SpringApplication` 인스턴스에서 `setWebEnvironment(false)`를 호출하거나 `applicationContextClass` 속성을 설정한다(JAVA API를 이용하거나 외부 속성 이용). 실행하고 싶은 비즈니스 로직을 `CommandLineRunner`를 구현하여 애플리케이션 코드로 작성하고 컨텍스트에 `@Bean` 정의를 이용할 수 있다.

7.2. 속성(Properties) 및 구성(Configuration)

7.2.1. 빌드 시 자동으로 속성 확장

프로젝트의 빌드 구성에도 일부 속성을 하드코딩하는 대신 기존 빌드 구성을 사용하여 자동으로 확장하도록 지정할 수 있다. 메이븐과 그레이들에서 가능하다.

메이븐을 사용하여 자동으로 속성 확장하기

메이븐의 리소스 필터링을 사용하여 자동으로 속성을 확장할 수 있다. 만약 `spring-boot-starter-parent`를 사용한다면 메이븐 프로젝트 속성을 `@..@` 위치변환자를 이용하여 참조할 수 있다:

```
app:
  encoding: @project.build.sourceEncoding@
  java.version: @java.version@
  YAML is a superset of JSON and
```

NOTE

운영 구성에서만 적용된다(`src/test/resources`는 필터링되지 않는다.)

만약 starter parent를 사용하지 않는다면, `pom.xml` 파일 `<build/>` 요소 안에 다음 항목을 추가해야 한다:

```
<resources>
  <resource>
    <directory>src/main/resources</directory>
    <filtering>true</filtering>
  </resource>
</resources>
```

그리고 `<plugins/>` 안에 다음내용을 추가한다:


```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-resources-plugin</artifactId>
  <version>2.7</version>
  <configuration>
    <delimiters>
      <delimiter>@</delimiter> ①
    </delimiters>
    <useDefaultDelimiters>false</useDefaultDelimiters>
  </configuration>
</plugin>
```

① 위치변환자 식별자를 @으로 지정

NOTE

`useDefaultDelimiters` 속성은 구성에서 표준 스프링 위치변환자(예: `${foo}`)를 사용하고자 할 때 중요하다. 해당 속성을 `false`로 설정하지 않은 경우 반드시 확장이 가능하다.

그레이들을 사용하여 자동으로 속성 확장하기

그레이들 프로젝트에서 자바 플러그인의 `processResources` 태스크를 사용하여 자동으로 속성을 확장할 수 있다:

```
processResources {
  expand(project.properties) ①
}
```

① 프로젝트 최상위 경로에 있는 `project.properties` 파일을 이용한다.

7.2.2. SpringApplication 외부 구성

`SpringApplication`는 Java API를 사용하여 애플리케이션을 생성하면서 빈속성을 이용해서(세터를 사용) 동작을 조작하는 것이 가능하다. 혹은 `application.yml`에 `spring.main.*`속성을 정의하는 것도 가능하다. 예로 다음과 같다:

```
spring:
  main:
    web-environment: false ①
    banner-mode: off ②
```

① 웹 애플리케이션 아님

② 애플리케이션 구동시 배너를 출력하지 않음

NOTE

위 예제에서 보는 것처럼 속성명을 선언하는데 대시(-)를 사용하는 것과 같이 언더스코어(_)를 사용하는 경우에도 유연하게 대응한다.

속성은 `ApplicationContext`를 생성하기 위해 Java API를 이용하여 값을 정의하며 외부구성을 정의할 수 있다. 다음과 같이 애플리케이션을 정의 가능하다:

```
new SpringApplicationBuilder()  
    .bannerMode(Banner.Mode.OFF)  
    .sources(io.honeymon.boot.BootSpringBootApplication.class)  
    .run(args);
```

위에서 코드로 정의한 부분과 별도로 다음과 같이 구성파일에 정의할 수 있다:

```
spring.main:  
  sources: io.honeymon.boot.springboot.config.AppConfig,  
  io.honeymon.boot.springboot.config.ExtraConfig  
  banner-mode: console
```

위와 같이 정의하면 애플리케이션이 실제로 동작할 때는 배너가 출력되고(구성에 의해 덮어씌어짐)

`ApplicationContext`을 위해서 구성파일 3개를 이용한다:

```
io.honeymon.boot.BootSpringBootApplication,  
io.honeymon.boot.springboot.config.AppConfig,  
io.honeymon.boot.springboot.config.ExtraConfig
```

7.2.3. 애플리케이션 속성 외부구성파일 위치변경

서로 다른 소스에서 기본 속성들이 스프링 `Environment`에 추가될 때에는 정의된 순서에 따른다('스프링 부트' 기능소개'의 '외부 구성(Externalized Configuration)'에서 상세히 설명함). 애플리케이션 소스에 `@PropertySource` 애너테이션을 추가하여 속성을 정의해두면 정의나 변경이 매우 용이하다. `SpringApplication`에 있는 정적 메서드 중 `setSources()`에 추가된 소스에서 `@PropertySource`를 가지고 있는지 검사하여 `ApplicationContext` {lifecycle}의 모든 과정에서 `Environment`에 속성을 추가할 수 있다. 이 방법으로 추가된 속성은 기본 위치 정의된 구성파일(예: `application.yml`), 시스템 속성, 환경변수 혹은 실행인자보다 낮은 순위로 추가된다. 시스템 속성(혹은 환경변수)를 정의하여 동작을 변경할 수 있다:

- `spring.config.name` (`SPRING_CONFIG_NAME`): application 파일명을 정의한다.
- `spring.config.location` (`SPRING_CONFIG_LOCATION`): 파일 위치(클래스패스 혹은 URL)를 정의한다. 분리되어 있는 `Environment` 속성 소스는 시스템 속성, 환경변수 혹은 실행인자 우선순위에 따라 덮어쓰기되며 구성된다.

이 과정에서 어떻게 환경이 구성되는지에 대해서는 걱정할 필요가 없다. 스프링 부트는 `application.yml(or properties)`에서 정의한 속성에 대해서만 반영한다. 구성파일과 관련된 스프링 부트는 `DEBUG` 레벨에서 출력되기 때문에 `TRACE` 레벨에서는 찾을 수가 없다. `ConfigFileApplicationListener`에서 자세한 내용을 살펴보기 바란다.

7.2.4. 커맨드라인 실행인자 축약사용

어떤 이들은 실행인자로 구성속성을 정의하는데 `--server.port=9000` 대신 `--port=9000`을 사용하는 것을 선호한다. `application.yml(or properties)`에서 위치변환자(예: `server.port: ${port:9000}`)를

이용하면 손쉽게 이용할 수 있다.

TIP

만약 `spring-boot-starter-parent` POM을 상속했다면 `maven-resources-plugins`의 기본 필터 토큰에 의해서 `${}`이 `@@`로 변경(예: `${maven.token}` 대신 `@maven.token@`)하여 스프링 스타일의 위치변환자와의 충돌을 방지한다. 만약 `application.yml(or properties)`에도 메이븐 필터가 적용시키거나 기노 필터 토큰을 다른 구분자를 이용할 수 있다.

NOTE

앞에서 살펴본 포트를 정의하는 경우 AWS 혹은 CloudFoundry 와 같은 클라우드 환경에서는 `PORT` 환경변수가 자동으로 설정되어 스프링에서는 환경속성에 적용할 수 있다. 클라우드 환경에서 스프링 애플리케이션을 배포하면서 외부 속성정의를 사용할 때는 시스템변수 혹은 환경변수 형식(예: `SPRING_PROFILES_ACTIVE`, `PORT`)으로 정의를 해야한다. 각각의 플랫폼에서 환경변수를 정의하는 영역이 있다.

7.2.5. 외부속성에 대한 YAML 사용

YAML은 JSON 의 상위체이며 계층적인 형식으로 외부 속성을 관리할 수 있는 매우 편리한 문법을 제공한다.

```
spring:
  application:
    name: boot-spring-boot
  datasource:
    driverClassName: com.mysql.jdbc.Driver
    url: jdbc:mysql://localhost/honeymon
  server:
    port: 9000
```

`application.yml`라는 이름으로 파일을 생성하여 클래스패스의 최상위에 위치시킨 후 snakeyaml 의존성을 추가 (메이븐이나 그레이들에서 `org.yaml:snakeyaml`을 추가하거나, `spring-boot-starter`를 사용한다면 이미 포함되어 있다). YAML 파일은 Java `Map<String, Object>`(JSON 객체처럼) 파싱되고, 스프링 부트에서는 독립된 키를 가지는 1단계 깊이를 가지는 맵으로 구성된다. 몇몇 이들은 `Properties` 파일을 사용하기를 선호하기도 한다.

```
spring.application.name=boot-spring-boot
spring.datasource.driverClassName=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost/springboot
server.port=9000
```

7.2.6. 스프링 프로파일 활성화

스프링 `Environment`는 프로파일 활성화를 위한 API를 가지고 있지만, 일반적으로 시스템 속성 (`spring.profiles.active`) 혹은 운영체제 환경변수(`SPRING_PROFILES_ACTIVE`)로 설정한다. 예를 들어, 커맨드라인을 통해서 실행할 경우에는 `-D`인자를 이용하여 설정가능하다(메인 클래스 혹은 jar 파일을 선언하기 전에 등록해야한다는 것을 기억해두자).

```
$ java -jar -Dspring.profiles.active=prod boot-spring-boot-0.0.1-SNAPSHOT.jar
```

7.2.7. 환경 의존적인 구성 변경

YAML 파일은 `---` 라인으로 문서를 구분하며 각 문서는 평맵으로 파싱된다.

만약 YAML 문서에 `spring.profiles` 키가 있고 프로파일값(콤마(,)로 구분되는 프로파일 목록)은 스프링 `Environment.acceptsProfiles()` 으로 확인가능하며 활성화된 프로파일이 포함된 문서는 최종적으로 하나의 평맵으로 병합된다.

```
server:
  port: 9000
---
spring:
  profiles: dev
server:
  port: 9001
---
spring:
  profiles: prd
server:
  port: 5000
```

위 예에서 기본 포트는 9000 이지만, 스프링 프로파일 `dev`가 활성화되면 포트는 9001이 되고 `prd`가 활성화되면 5000이 된다.

YAML 문서는 발견되는 순서대로 병합된다(나중에 값이 이전 것보다 우선함).

properties 파일에 동일한 내용을 적용하려면 `application-${프로파일}.properties`로 각 프로파일 값을 지정해야 한다.

7.2.8. 외부속성에 대한 빌트인 옵션(built-in option) 발견

스프링 부트는 애플리케이션 실행시 `application.yml` 혹은 (`.properties`)로부터 외부 속성을 연결한다. 애플리케이션 실행에 필요한 속성들을 한 곳에서 제공하는 것은 기술적으로 불가능 하다. 클래스패스에 추가된 jar 파일에서 제공하는 경우도 있기 때문이다.

실행중인 애플리케이션의 액추에이터 기능에서 `configprops` 종단점을 살펴보면 연결되거나 `@ConfigurationProperties`을 통하여 연결가능한 속성들을 모두 보여준다.

스프링 부트 부록에 있는 `application.properties` 예제를 살펴보면, 스프링 부트에서 사용가능한 속성들이 나열되어 있다. 최종목록은 소스 코드에서 `@ConfigurationProperties` 및 `@Value` 주석을 검색하고

`RelaxedPropertyResolver`에서 추출되어 나열된다.

`@Value` 에서 지정된 속성값이 애플리케이션 실행시 정의되어 있지 않다면 실행되지 않는다. 다음과 같이 `app.info` 속성이 정의되어 있지 않다면

```
@Component
public class ComponentService {

    @Value("${app.info}")
    private String info;

    @PostConstruct
    public void setUp() {
        log.debug("App info: {}", this.info);
    }
}
```

다음과 같은 `IllegalArgumentException`이 발생한다.

```
Caused by: java.lang.IllegalArgumentException: Could not resolve
placeholder 'app.info' in value "${app.info}"
    at
org.springframework.util.PropertyPlaceholderHelper.parseStringValue(Proper
tyPlaceholderHelper.java:174)
    .. 생략
```

7.3. 내장서블릿 컨테이너

7.3.1. 애플리케이션에 서블릿, 필터 혹은 리스너 추가하기

서블릿 스펙을 지원하는 `Servlet`, `Filter`, `ServletContextListener`와 다른 리스너를 애플리케이션에 추가하는 두 가지 방법이 있다. 스프링 빈을 제공하거나 서블릿 컴포넌트를 검색하도록 할 수 있다.

스프링 빈을 통해 `Servlet`, `Filter` 혹은 `Listener` 추가하기

`Servlet`, `Filter` 혹은 `Servlet*Listener`을 위한 빈을 정의하여 추가할 수 있다. 이 방법은 사용자가 원할 때 구성이나 의존성을 주입할 수 있어 매우 유용하다. 그러나, 애플리케이션 구동 초기에 컨테이너에 설치하기 때문에 많은 빈을 등록하요 초기화하지 않도록 주의해야 한다(`DataSource` 혹은 JPA 구성을 참조하도록 하는 것은 좋은 생각이 아니다). 초기화될 때 초반에 사용하기보다는 느슨하게 초기화되도록 제한하는 것이 좋다.

`Filter`와 `Servlet`의 경우 기본 컴포넌트 대신 `FilterRegistrationBean` 혹은 `ServletRegistrationBean`을 추가하여 매핑 및 초기화 매개변수를 추가할 수 있다.

필터 등록시 `dispatcherType`을 지정하지 않았다면, `FORWARD`, `INCLUDE` 그리고 `REQUEST`와 일치된다. 만약 비동기가 활성화된다면 `ASYNC`와도 일치한다.
만약 `web.xml`에서 `dispatcher` 속성을 정의하지 않았다면 `dispatcherType`을 지정해주는 줄 필요가 있다:

TIP

```
@Bean
public FilterRegistrationBean myFilterRegistration() {
    FilterRegistrationBean registration = new
    FilterRegistrationBean();
    registration.setDispatcherTypes(DispatcherType.REQUEST);
    ....

    return registration;
}
```

서블릿 혹은 필터 등록 비활성화

위에서 언급했던 `Servlet` 혹은 `Filter` 빈은 서블릿 컨테이너에 자동으로 등록될 것이다. `Filter` 혹은 `Servlet` 빈의 등록을 명시적으로 비활성화할 수 있다. 예로:

```
@Bean
public FilterRegistrationBean registration(MyFilter filter) {
    FilterRegistrationBean registration = new FilterRegistrationBean
    (filter);
    registration.setEnabled(false); ①
    return registration;
}
```

① `Servlet` 혹은 `Filter` 빈의 자동등록 비활성화

클래스패스 탐색을 이용해서 `Servlet`, `Filter` 그리고 `Listener` 추가하기

`@WebServlet`, `@WebFilter` 그리고 `@WebListener` 애너테이션을 선언한 클래스를 내장 서블릿 컨테이너에 등록하려면 `@Configuration`과 함께 `@ServletComponentScan` 애너테이션을 사용하거나 등록을 원하는 컴포넌트가 위치한 패키지를 정의하여 등록할 수 있다. 기본적으로 `@ServletComponentScan`를 선언한 클래스가 위치한 패키지부터 탐색한다.

7.3.2. HTTP 포트 변경

애플리케이션이 실행될 때 기본포트는 `8080`이지만 `server.port`(예: `application.yml` 혹은 시스템 속성)으로 설정할 수 있다. `Environment`의 느슨한 연결방식으로 `SERVER_PORT`(OS 환경변수)를 사용할 수도 있다.

`WebApplicationContext`를 생성하지만 HTTP 종단점을 완벽히 끄고자 한다면 `server.port: -1`을 사용한다 (테스트시 유용하다).

보다 자세한 내용을 살펴보고 싶다면, '스프링 부트' 기능소개의 "내장 서블릿 컨테이너 재정의"를 찾아보거나

`ServerProperties` 소스코드를 살펴보기 바란다.

7.3.3. 할당되지 않은 HTTP 포트 무작위 사용

OS 에서 선점하여 사용하고 있는 포트와 충돌하지 않는 포트를 탐색하여 사용하려면 다음과 같이 설정한다.

```
server.port: 0
```

7.3.4. 실행시 HTTP 포트 할당

서버가 실행되었을 때 로그로 출력되었을 때나 `EmbeddedServletContainer`을 통해 `EmbeddedWebApplicationContext`에서 가져왔을 때 서버 포트에 접근이 가능하다. 가장 확실한 방법은 `ApplicationListener<EmbeddedServletContainerInitializedEvent>` 타입을 `@Bean`으로 추가하고 컨테이너가 구동되는 이벤트가 발생되었을 때 가져오도록 할 수 있다.

테스트를 할 때는 `@SpringBootTest(webEnvironment=WebEnvironment.RANDOM_PORT)`를 사용하거나 `@LocalServerPort` 애너테이션을 필드에 사용하여 실제 포트를 주입받을 수 있다. 예를 들어:

```
@RunWith(SpringJUnit4ClassRunner.class)
@SpringBootTest(webEnvironment=WebEnvironment.RANDOM_PORT)
public class MyWebIntegrationTests {
    @Autowired
    EmbeddedWebApplicationContext server;

    @LocalServerPort
    int port;
    // ...
}
```

NOTE

`@LocalServerPort`는 `@Value("${local.server.port}")`와 대응하는 메타애너테이션이다. 일반적인 애플리케이션에서는 포트를 주입받으려 하지 말라. 방금 보았듯이, 컨테이너가 초기화되면서 값이 설정된다. 테스트와는 달리 애플리케이션 코드에 대한 콜백은 일찍 처리된다(즉, 값을 실제로 사용할 수 있게 되기 전에). 그러면 `IllegalArgumentException`이 발생할 것이다.

7.3.5. SSL 구성

`application.yml` 혹은 `application.properties`에서 다양한 `server.ssl.*` 속성을 설정하여 SSL을 구성할 수 있다. 예로:

```
server.port: 8443
server.ssl:
  key-store: classpath:keystore.jks
  key-store-password: secret
  key-password: another-secret
```

Ssl을 살펴보면 지원하는 속성들을 모두 볼 수 있다.

위에서 살펴본 예제처럼 구성하면 더이상 HTTP 커넥터에서 8080 포트를 지원하지 않는다. 스프링 부트는 `application.yml`을 통해서 HTTP 커넥터와 HTTPS 커넥터를 동시에 구성하는 것은 지원하지 않는다. 만약 두 가지를 모두 지원하길 원한다면 둘 중 하나를 프로그램적으로 구성해야 한다. **HTTP 커넥터를 프로그래밍으로 구성하는 것이 더 쉽기 때문에 `application.yml`으로는 HTTPS를 구성하는 것이 좋다.** `spring-boot-sample-tomcat-multi-connectors` 프로젝트에서 예제를 볼 수 있다.

7.3.6. 접근로그 구성

톰캣과 언더투의 namespace를 통해서 접근로그를 구성할 수 있다.

예를 들어, 톰캣의 접근로그 패턴을 재정의한다.

```
server.tomcat:
  basedir: my-tomcat
  accesslog:
    enabled: true
    pattern: %t %a "%r" %s (%D ms)
```

NOTE

로그의 기본위치는 톰캣 기본 디렉토리와 관련된 `logs` 디렉토리이며, 이 디렉토리는 기본적으로 임시 디렉토리이므로 톰캣의 기본 디렉토리를 수정하거나 로그의 절대경로를 사용할 수 있다. 위의 예에서, 로그는 애플리케이션의 작업 디렉토리에서 `my-tomcat/logs`에 로그가 생성될 것이다.

언더투의 접근로그는 유사한 방식으로 구성할 수 있다:

```
server.undertow.accesslog:
  enabled: true
  patttern: %t %a "%r" %s (%D ms)
```

로그는 애플리케이션의 작업 디렉토리에서 `logs` 디렉토리에 저장된다.
`server.undertow.accesslog.directory` 속성으로 재정의 가능하다.

7.3.7. 톰캣 구성

외부속성에 대한 빌트인 옵션(built-in option) 발견에서 언급했던 `@ConfigurationProperties` (`ServerProperties`가 주요하다) 구성을 따를 수 있지만, `EmbeddedServletContainerCustomizer`를

찾아보고 톰캣을 위한 다양한 *Customizers 중에 하나를 추가하는 방법이 있다. 풍부한 톰캣 API를 통해서 TomcatEmbeddedServletContainerFactory에 접근하면서 여러가지 방법으로 수정할 수 있다. 혹은 TomcatEmbeddedServletContainerFactory를 추가하여 세부적으로 구성하는 방법도 있다.

톰캣을 통한 다중 커넥터 활성화

TomcatEmbeddedServletContainerFactory에 org.apache.catalina.connector.Connector를 추가하여 다중 커넥터(HTTP와 HTTPS 커넥터)를 활성화할 수 있다.

```
@Bean
public EmbeddedServletContainerFactory servletContainer() {
    TomcatEmbeddedServletContainerFactory tomcat = new
    TomcatEmbeddedServletContainerFactory();
    tomcat.addAdditionalTomcatConnectors(createSslConnector());
    return tomcat;
}

private Connector createSslConnector() {
    Connector connector = new Connector(
    "org.apache.coyote.http11.Http11NioProtocol");
    Http11NioProtocol protocol = (Http11NioProtocol) connector
    .getProtocolHandler();
    try {
        File keystore = new ClassPathResource("keystore").getFile();
        File truststore = new ClassPathResource("keystore").getFile();
        connector.setScheme("https");
        connector.setSecure(true);
        connector.setPort(8443);
        protocol.setSSLEnabled(true);
        protocol.setKeystoreFile(keystore.getAbsolutePath());
        protocol.setKeystorePass("changeit");
        protocol.setTruststoreFile(truststore.getAbsolutePath());
        protocol.setTruststorePass("changeit");
        protocol.setKeyAlias("apitester");
        return connector;
    }
    catch (IOException ex) {
        throw new IllegalStateException("can't access keystore: [" +
        "keystore"
            + "] or truststore: [" + "keystore" + "]", ex);
    }
}
```

7.3.8. 톰캣을 언더투우로 대체하기

톰캣 의존성을 제외하고 언더투우 스타터를 추가하면 간단하게 대체가능하다.

그레이들은 다음처럼 선언하면 되고,

```

configurations {
    compile.exclude module: "spring-boot-starter-tomcat"
}

dependencies {
    compile("org.springframework.boot:spring-boot-starter-web")
    compile("org.springframework.boot:spring-boot-starter-undertow")
    // ...
}

```

메이븐은 다음처럼 선언하면 된다.

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-undertow</artifactId>
</dependency>

```

7.3.9. 언더토우 구성

외부속성에 대한 빌트인 옵션(built-in option) 발견에서 언급했던 `@ConfigurationProperties` (`ServerProperties`와 `ServerProperties.Undertow`가 주요하다) 구성을 따르거나, `EmbeddedServletContainerCustomizer`를 찾아볼 수 있다. `UndertowBuilderCustomizer`를 사용하여 `UndertowEmbeddedServletContainerFactory`에 접근하여 언더토우 구성을 변경할 수 있다. 혹은 `UndertowEmbeddedServletContainerFactory`에 세부적인 사항을 추가할 수도 있다.

7.3.10. 언더토우 멀티 리스너 활성화

`UndertowEmbeddedServletContainerFactory`에 `UndertowBuilderCustomizer`를 추가하고 `Builder`로 리스너를 추가할 수 있다.

```

@Bean
public UndertowEmbeddedServletContainerFactory
embeddedServletContainerFactory() {
    UndertowEmbeddedServletContainerFactory factory = new
    UndertowEmbeddedServletContainerFactory();
    factory.addBuilderCustomizers(new UndertowBuilderCustomizer() {

        @Override
        public void customize(Builder builder) {
            builder.addHttpListener(8080, "0.0.0.0");
        }

    });
    return factory;
}

```

7.3.11. @ServerEndpoint 를 사용한 웹소켓 종단점 생성

내장 컨테이너를 사용하는 스프링 부트 app에서 @ServerEndpoint를 사용하려면 ServerEndpointExporter를 빈으로 선언해야 한다.

```

@Bean
public ServerEndpointExporter serverEndpointExporter() {
    return new ServerEndpointExporter();
}

```

이 빈은 @ServerEndpoint 애너테이션이 선언된 모든 빈을 기본 웹소켓 컨테이너에 등록한다. 독립형 서블릿 컨테이너에 배포할 때 서블릿 컨테이너가 초기화되면서 수행하기 때문에 ServerEndpointExporter 빈은 필요하지 않다.

7.3.12. HTTP 응답 압축 활성화

HTTP 응답 압축은 톰캣, 언더투와 제티에서 지원한다. application.yml에서 활성화할 수 있다:

```
server.compression.enabled: true
```

기본적으로 압축을 수행하려면 응답 길이가 적어도 2048 바이트 이상이어야 한다. 이에 대해서는 server.compression.min-response-size 속성으로 사용하여 구성할 수 있다.

기본적으로, 다음 콘텐츠 유형(Content-Type)이 다음 중 하나인 경우에만 응답압축이 가능하다:

- text/html
- text/xml

- `text/plain`
- `text/css`

이에 대한 구성은 `server.compression.mime-types` 속성으로 구성가능하다.

7.4. 스프링 MVC

7.4.1. JSON REST 서비스 작성

스프링 부트 애플리케이션에서 스프링 `@RestController`는 클래스패스에 있는 Jackson2에 의해서 JSON으로 랜더링된다. 예를 들어:

```
@RestController
public class MyController {

    @RequestMapping("/thing")
    public MyThing thing() {
        return new MyThing();
    }
}
```

`MyThing`는 Jackson2에 의해 직렬화(일반적인 POJO 혹은 그루비 객체인 경우)되어 `localhost:8080/thing`은 기본적으로 JSON으로 표현될 것이다. 브라우저에 따라서 XML로 표현될 수도 있는데 이런 경우는 접근헤더가 XML로 정해졌을 때 그렇다.

7.4.2. XML REST 서비스 작성

만약 클래스패스 상에 Jackson XML 확장(`jackson-dataformat-xml`)이 있다면, JSON으로 동작하던 예제를 XML 랜더링을 하도록 만드는 것은 매우 쉽다. 이 기능을 사용하기 위해서는 다음과 같이 의존성을 추가해야 한다.

`build.gradle`

```
dependencies {
    // 생략
    compile("com.fasterxml.jackson.dataformat:jackson-dataformat-xml")
}
```

거기에 추가로 Woodstox 의존성을 추가하고 싶을 것이다. JDK에서 제공하는 기본 StAx 구현체보다 빠르고 출력을 꾸밀 수 있고 보다 나은 namespace 제어가 가능하기 때문이다.

```
dependencies {
    // 생략
    compile("org.codehaus.woodstox:woodstox-core-asl")
}
```

혹시나 Jackson XML 확장을 쓰는 것이 여의치 않아 JAXB(JDK에서 기본제공)를 사용하려면 **MyThing**에 **@XmlRootElement** 애너테이션을 추가해야 한다.

```
@XmlRootElement
public class MyThing {
    private String name;
    // .. getters and setters
}
```

JSON 대신 XML을 제공받으려면 **Accept: text/xml** 헤더를 보내면 된다(혹은 브라우저를 이용).

7.4.3. 잭슨(Jackson) 오브젝트매퍼(ObjectMapper) 재정의

스프링 MVC(클라이언트와 서버)에서는 **HttpMessageConverters**으로 HTTP 교환 중에 콘텐츠 변환을 처리한다. 만약 Jackson이 클래스패스가 있다면 자동구성되어 제공되는 것을 대신하여 **Jackson2ObjectMapperBuilder**으로 기본 컨버터를 얻을 수 있다.

ObjectMapper(혹은 Jackson XML 컨버터를 위한 **XmlMapper**) 인스턴스는 다음 재정의 속성으로 생성된다:

- **MapperFeature.DEFAULT_VIEW_INCLUSION**: 비활성화
JsonView 애너테이션이 없는 경우 속성이 모두 포함시키는데 이를 비활성화
- **DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES**: 비활성화
변환시 속성이 일치하지 않을 경우 실패처리하는 것을 비활성화

스프링 부트는 보다 쉽게 커스터마이징할 수 있게 해주는 몇가지 기능을 가지고 있다.

Environment를 이용해서 **ObjectMapper**와 **XmlMapper**을 구성할 수 있다. Jackson은 구성을 다양한 방법으로 처리할 수 있는 간단한 on/off 기능을 제공한다. 이 기능은 Jackson 의 6가지 enum으로 처리가 가능하다:

Jackson enum	환경속성
com.fasterxml.jackson.databind.DeserializationFeature	spring.jackson.deserialization.<feature_name>: true or false
com.fasterxml.jackson.core.JsonGenerator.Feature	spring.jackson.generator.<feature_name>: true or false
com.fasterxml.jackson.databind.MapperFeature	spring.jackson.mapper.<feature_name>: true or false
com.fasterxml.jackson.core.JsonParser.Feature	spring.jackson.parser.<feature_name>: true or false
com.fasterxml.jackson.databind.SerializationFeature	spring.jackson.serialization.<feature_name>: true or false
com.fasterxml.jackson.annotation.JsonInclude.Include	spring.jackson.default-property-inclusion: always or non_null or non_absent or non_default or non_empty

예를 들어, 들여쓰기하며 출력하고 싶은 경우 `spring.jackson.serialization.indent_output: true`. 속성을 설명하면서 거론했던 '느슨한 연결'을 통해서 `indent_output`로 작성해도 `INDENT_OUTPUT`과 대응되기 때문에 대소문자를 구분할 필요는 없다.

환경값을 기반으로 한 구성은 자동구성된 `Jackson2ObjectMapperBuilder` 빈을 제공하며 빌더를 이용하여 얼마든지, 자동구성된 `ObjectMapper` 빈을 포함한, 매퍼를 생성할 수 있다.

`Jackson2ObjectMapperBuilder`는 하나 이상의 `Jackson2ObjectMapperBuilderCustomizer` 빈을 재정의할 수 있다. 이런 재정의된 빈을 배치가 가능하며 스프링 부트에서 생성한 재정의된 빈의 순서는 0이므로 스프링 부트 재정의의 빈의 전후에 별도로 재정의한 빈을 배치할 수 있다.

`com.fasterxml.jackson.databind.Module` 타입의 빈은 자동구성된 `Jackson2ObjectMapperBuilder`에 자동으로 등록되며 `ObjectMapper` 인스턴스에 적용된다. 애플리케이션에서 새로운 기능을 추가할 때 재정의한 모듈을 제공하는 전역적 기작을 제공한다.

기본 `ObjectMapper`를 완벽하게 대체하고 싶다면, `@Bean`으로 새로이 정의하고 `@Primary`로 지정하거나 빌더 기반의 접근방법으로 `Jackson2ObjectMapperBuilder`을 `@Bean`으로 정의하는 것이다. 이 방법들은 `ObjectMapper`의 모든 자동구성은 비활성화된다.

`MappingJackson2HttpMessageConverter` 타입의 빈을 정의할 경우 MVC 구성의 기본 값들을 대체한다. 또한, `HttpMessageConverters`(기본 MVC 구성을 사용하는 경우 항상 사용가능) 타입의 빈을 제공하며 기본 및 사용자정의한 메시지 컨버터에 접근할 수 있는 유용한 메서드를 가지고 있다.

"`@ResponseBody` 렌더링 재정의" 을 살펴보고 `WebMvcAutoConfiguration`

7.4.4. `@ResponseBody` 렌더링 재정의

스프링은 `@ResponseBody`(혹은 `@RestController`의 응답) 렌더링에 `HttpMessageConverters`를 사용한다. 스프링 부트 컨텍스트에서 `HttpMessageConverters` 타입의 빈을 간단히 추가하여 컨버터를 추가적으로 등록가능하다. 추가하는 빈이 기본적으로 포함되어 있는 타입(JSON 변환을 위한 `MappingJackson2HttpMessageConverter` 같은)이라면 기본값을 대체한다. 컨벤션빈은 `HttpMessageConverters` 타입(기본 MVC 구성을 사용할 경우 항상 사용가능)으로 제공되며 기본 및 재정의한 메시지 컨버터에 접근하는데 유용한 메서드가 있다(예: 재정의한 `RestTemplate`수동으로 주입하려는 경우 유용함).

일반적인 MVC로 이용할 경우, 재정의한 `WebMvcConfigurerAdapter`빈은 `configureMessageConverters` 메서드를 재정의하여 컨버터를 등록할 수 있지만 일반 MVC와는 달리 필요한 컨버터만 추가할 수도 있다(스프링 부트에서도 동일한 메커니즘을 사용한다). 스프링 부트 기본 MVC 구성을 대신하여 `@EnableWebMvc` 구성을 하려한다면 `WebMvcConfigurationSupport`에 있는 `getMessageConverters`를 사용하여 수동으로 모든 것을 제어가능하다.

보다 자세한 내용은 `WebMvcAutoConfiguration`를 살펴보자.

7.4.5. Multipart 파일 업로드 제어

스프링 부트는 파일 업로드를 지원하는 서블릿 3 `javax.servlet.http.Part` API를 품고 있다. 기본 스프링 부트가 구성한 스프링 MVC에서 파일 최대크기는 파일당 1MB이며 단일 요청에서 데이터 최대크기는 10MB다. 이 값을 재정의하고 싶다면, 데이터가 중간에 저장되는 위치(예: `/tmp` 디렉토리) 및 `MultipartProperties` 클래스 속성을 사용하여 데이터가 저장되는 임계값을 지정한다. 파일에 대한 제한을 하고 싶지 않다면

`spring.http.multipart.max-file-size` 속성을 -1로 설정하면 된다.

멀티파트 지원은 스프링 MVC 컨트롤러 핸들러 메서드에서 `@RequestParam`이 선언된 `MultipartFile` 파일 데이터를 받고 싶을 때 유용하다.

보다 자세한 내용은 [link:https://github.com/spring-projects/spring-boot/tree/v스프링부트/spring-boot-autoconfigure/src/main/java/org/springframework/boot/autoconfigure/web/servlet/MultipartAutoConfiguration.java](https://github.com/spring-projects/spring-boot/tree/v스프링부트/spring-boot-autoconfigure/src/main/java/org/springframework/boot/autoconfigure/web/servlet/MultipartAutoConfiguration.java)[`MultipartAutoConfiguration`]를 살펴보기 바란다.

7.4.6. 스프링 MVC `DispatcherServlet` 전환

스프링 부트는 애플리케이션의 `/` 경로 이하 루트에서 모든 콘텐츠를 제공하고자 한다. 별도의 서블릿을 `/`에 매핑하는 하는 것이 가능하지만 다른 스프링 부트의 MVC 기능 중 일부를 손실할 수도 있다. 서블릿을 추가하고 루트 자원에 매핑하려면 `Servlet` 타입의 `@Bean`을 정의하고 `dispatcherServlet` 라는 이름으로 빈을 정의한다(혹은 이름은 유지하며 다른 타입의 빈으로 대체할 수도 있다).

7.4.7. 기본 MVC 구성 전환

MVC 구성을 완벽하게 제어할 수 있는 가장 쉬운 방법은 `@EnableWebMvc` 애너테이션을 `@Configuration`과 함께 사용하는 것이다. 이렇게 하면 모든 MVC 구성을 제어할 수 있게 된다.

7.4.8. `ViewResolver` 재정의

`ViewResolver`는 스프링 MVC에서 핵심 컴포넌트로서, `@Controller`에서 뷰 이름을 번역하여 실제 `View`로 구현한다. 좀더 살펴보자면 `ViewResolvers`는 REST 스타일 서비스(`View`는 `@ResponseBody` 랜더링에 사용되지 않음)보다는 UI 애플리케이션에 사용된다. 선택할 수 있는 `ViewResolver`의 구현체가 많으니, 스프링에서 어느 것만을 사용해야 하는지 강제되는 것은 없다. 반면 스프링 부트에서는 클래스패스와 애플리케이션 컨텍스트에서 필요한 것을 한 개 혹은 두 개 정도 설치한다. `DispatcherServlet`은 애플리케이션 컨텍스트에서 찾아낸 모든 리졸버를 사용하며 결과를 얻을 때까지 차례대로 시도하므로 자신이 추가한 리졸버의 순서와 위치를 알고 있어야 한다.

`WebMvcAutoConfiguration`는 컨텍스트에 다음과 같은 `ViewResolvers`를 추가한다:

- `InternalResourceViewResolver`

빈 ID는 `defaultViewResolver`이다. `DefaultServlet`를 사용하여 랜더링할 수 있는 물리적인 위치를 찾는다(예: 정적 자원과 JSP 페이지를 이용하는 경우). 접두사(prefix)와 접미사(suffix)를 적용한 뷰이름과 서블릿 컨텍스트에서 물리적 리소스를 찾는다(기본적으로는 양쪽다 가능하지만, 외부 구성으로는 `spring.mvc.view.prefix`과 `spring.mvc.view.suffix`으로 설정한 값으로 접근가능하다). 동일한 타입의 빈을 생성하여 덮어쓰는 것이 가능하다.

- `BeanNameViewResolver`

빈 ID는 `beanNameViewResolver`이다. 유용한 뷰리졸버 체인중 하나이며 뷰를 처리하는 과정에서 동일한 이름을 가진 빈을 찾는다. 이 빈을 덮어쓰거나 대체할 필요는 없다.

- `ContentNegotiatingViewResolver`

빈 ID `viewResolver`는 실제로 `View` 타입의 빈이 있는 경우에만 추가된다. 이 빈은 '마스터' 리졸버로서,

클라이언트가 보낸 요청에서 `Accept` HTTP 헤더에 일치하는 것들을 탐색한다.

`ContentNegotiatingViewResolver`에 [관한 블로그 포스팅](#)을 읽어보면서 공부하고 좀 더 자세히 소스코드를 살펴보기 바란다. `viewResolver`라는 이름으로 빈을 정의하여 자동구성된

`ContentNegotiatingViewResolver`를 비활성화할 수 있다.

- `ThymeleafViewResolver`

타임리프를 사용하고 있다면 `thymeleafViewResolver`라는 ID로 `ThymeleafViewResolver`를 사용할 수 있다. 뷰이름에 접두사와 접미사를 적용하여 자원을 탐색한다(`spring.thymeleaf.prefix`와 `spring.thymeleaf.suffix`으로 외부구성이 가능하며, 기본값으로 `classpath:/templates`와 `.html`를 사용한다). 동일한 이름의 빈을 정의하여 덮어쓰기 가능하다.

- `FreeMarkerViewResolver`

프리마커(FreeMarker)를 사용한다면 '`freeMarkerViewResolver`'라는 ID로 `FreeMarkerViewResolver`를 사용할 수 있다. 지정된 위치에(외부구성은 `spring.freemarker.templateLoaderPath`으로 설정가능하며 기본값은 `classpath:/templates/`)서 뷰이름에 접두사와 접미사를 적용하여 자원을 탐색한다(`spring.freemarker.prefix`와 `spring.freemarker.suffix`으로 외부구성이 가능, 각각 빈값과 `.ftl`이 기본값). 동일한 이름의 빈을 정의하여 덮어쓰기 가능하다.

- `GroovyMarkupViewResolver` 그루비 템플릿을 사용한다면(실제로 클래스패스상에 `groovy-templates`가 있다면) `groovyMarkupViewResolver`라는 ID로 `GroovyMarkupViewResolver`를 사용할 수 있다. 뷰이름에 접두사와 접미사를 적용하여 자원을 탐색한다(`spring.groovy.template.prefix`와 `spring.groovy.template.suffix`으로 외부 구성이 가능, 각각 `classpath:/templates/`와 `.tpl`가 기본값)

보다 자세한 내용은 다음 코드들을 살펴보기 바란다:

- `WebMvcAutoConfiguration`
- `ThymeleafAutoConfiguration`
- `FreeMarkerAutoConfiguration`
- `GroovyTemplateAutoConfiguration`

7.5. HTTP 클라이언트

스프링 부트는 HTTP 클라이언트에서 동작하는 다양한 스타터를 제공한다.

7.5.1. `RestTemplate` 가 프록시를 사용하도록 구성

"`RestTemplate` 재정의"에서 설명했듯이, `RestTemplate`를 재정의하는데 `RestTemplateBuilder`에서 사용할 수 있도록 `RestTemplateCustomizer`를 구현하여 빈으로 등록사용할 수 있다. 이 제안은 프록시를 사용할 때 `RestTemplate`를 구성하여 생성할 수 있다.

7.6. 로깅

스프링 부트는 로깅에 대한 의존성을 강제하지 않으며, Common Logging API에 대한 것을 제외하고, 다양한 구현체 중에서 선택가능하다. 로그백(logback, <http://logback.qos.ch/>)을 사용하려고 하는 경우에는 `jcl-over-slf4j` (Common Logging API 구현체)가 클래스패스 상에 있어야 한다. 가장 간단한 추가방법은 로깅스타터(`spring-boot-starter-logging`)를 추가하는 것이다. 웹애플리케이션의 경우, `spring-boot-starter-web`만 추가하면, 로깅스타터가 자동으로 추가된다. 만약 그레이들들을 사용하는 경우에는 다음과 같이 로깅 관련 의존성을 추가할 수 있다:

```
dependencies {
    compile("org.springframework.boot:spring-boot-starter-web")
}
```

스프링 부트는 클래스패스를 기반으로 로깅을 구성하려고 하는 `LoggingSystem` 추상화를 가지고 있다. 로그백을 가능하다면 우선적으로 구성된다.

만약 다양한 로거에서 로깅레벨을 변경하고 싶다면, `application.yml`에서 `logging.level` 접두사를 통해서 다음과 같이 설정가능하다:

```
logging.level:
  org.springframework.web: DEBUG
  org.hibernate: ERROR
```

이 `logging.level` 설정도 스프링 프로파일 설정기능을 통해서 변경이 가능하다. 즉 배포하는 환경에 따라서 로깅레벨 설정이 가능하다는 이야기이다.

NOTE

```
spring.profiles: test
logging.level:
  org.springframework.web: DEBUG
  org.hibernate: ERROR
---
spring.profiles: prd
logging.level:
  org.springframework.web: INFO
  org.hibernate: INFO
```

또한 `logging.file` 속성을 사용하여 콘솔에 출력되는 로그와는 별도로 로그 파일을 생성하는 위치를 지정하는 것도 가능하다(로그파일의 생성위치는 상대경로일 경우 스프링 부트 애플리케이션이 실행되는 위치를 기준으로 한다).

보다 자세한 구성을 하고 싶은 경우에는 `LoggingSystem`에서 지원하는 기존의 구성 포맷을 사용해야 한다. 기본적으로, 스프링 부트는 시스템 기본 위치(예: 로그백의 경우 `classpath:logback.xml`)에서 원시구성파일을 탐색하지만, `logging.config` 속성을 이용해서 구성 파일의 위치를 별도로 설정가능하다.

7.6.1. Logback 구성

`logback.xml` 파일이 클래스패스 루트에 있다면, 이 파일을 기반으로 로깅을 구성한다(혹은 `logback-spring.xml`으로 설정한다면 스프링 부트에서 제공하는 템플릿 기능의 혜택을 볼 수 있다). 스프링 부트에서 기본적으로 제공하는 기본 구성 외에 사용자가 원하는 로그 레벨을 설정하는 것도 가능하다:

`logback-spring.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <include resource="org/springframework/boot/logging/logback/base.xml"
  />
  <logger name="org.springframework.web" level="DEBUG" />
</configuration>
```

`spring-boot.jar`에 있는 `base.xml`를 열어보면 `LoggingSystem`에서 사용하는 몇가지 유용한 시스템 속성을 볼 수 있을 것이다:

- `${PID}`: 현재 프로세스 ID
- `${LOG_FILE}`: 스프링 부트 외부 구성에서 사용하는 `logging.file`와 대응
- `${LOG_PATH}`: 스프링 부트 외부 구성에서 사용하는 `logging.path`와 대응

스프링 부트는 또한 콘솔로 출력되는 로그를 재정의한 로그백의 컨버터를 사용하여 ANSI 색상으로 출력하는 것도 가능하다. 이에 대해서는 `base.xml`를 찾아보면 보더 자세한 내용을 볼 수 있다.

NOTE

<https://goo.gl/d8z8WY>에서 `spring-boot.jar`에 포함되는 `base.xml`과 다른 로깅 구성을 살펴볼 수 있다.

파일만 출력되도록 Logback 구성

만약에 콘솔출력은 비활성화하고 파일만 출력하고 싶다면, 다음의 예제처럼 `console-appender.xml`는 제외하고 `file-appender.xml`만 불러넣은 `logback-spring.xml`를 재정의해야 한다.

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <include resource=
  "org/springframework/boot/logging/logback/defaults.xml" />
  <property name="LOG_FILE" value="${LOG_FILE:-${LOG_PATH:-${LOG_TEMP:-
  ${java.io.tmpdir:-/tmp}}}/spring.log}" />
  <include resource="org/springframework/boot/logging/logback/file-
  appender.xml" />
  <root level="INFO">
    <appender-ref ref="FILE" />
  </root>
</configuration>
```

그런 후에 `application.yml` 에서 `logging.file` 속성을 다음과 같이 정의해야 한다:

```
logging.file: boot-spring-boot.log
```

7.6.2. Log4j 구성

스프링 부트는 [Log4j2](https://logging.apache.org/log4j/2.x)(<https://logging.apache.org/log4j/2.x>)가 클래스패스에서 존재하는 경우 로깅구성을 지원한다. 만약 의존성을 정의하려면, Logback 을 제외시키고 대신에 Log4j2를 포함시켜야 한다. 만약 스타터를 사용하지 않는다면 Log4j2를 추가하고 최소한 `jcl-over-slf4j`를 추가해야 한다.

스타터로 구성하기 위해서는 몇가지를 제외(exclude)하는 작업을 필요로 한다. 메이븐에서는 다음과 같이 구성이 가능하다.

```
dependencies {
    compile("org.springframework.boot:spring-boot-starter-web")
    compile("org.springframework.boot:spring-boot-starter") {
        exclude group: "org.springframework.boot" , module: "spring-boot-starter-logging"
    }
    compile("org.springframework.boot:spring-boot-starter-log4j2")
}
```

NOTE

Log4j 스타터는 공통 로깅 요구사항 의존성을 함께 필요로 한다(톰캣은 `java.util.logging`를 사용하지만 Log4j2를 사용하여 출력하도록 구성).

YAML 혹은 JSON 으로 Log4j2 구성하기

XML 구성을 기본적으로 사용하지만, Log4j2는 YAML과 JSON 구성파일을 지원한다. 다른 구성 파일 형식을 사용하도록 Log4j2를 구성하려면 클래스패스에 적절한 종속성을 추가하고 다음과 같이 선택한 파일과 일치하도록 구성 파일이름을 지정해야 한다.

형식	의존성 라이브러리	파일 이름
YAML	<code>com.fasterxml.jackson.core:jackson-databind</code> <code>com.fasterxml.jackson.dataformat:jackson-dataformat-yaml</code>	<code>log4j2.yaml</code> <code>log4j2.yml</code>
JSON	<code>com.fasterxml.jackson.core:jackson-databind</code>	<code>log4j2.json</code> + <code>log4j2.jsn</code>

7.7. 데이터 접근

스프링 부트는 데이터 자원에 동작하는 다양한 스타터를 포함하고 있다.

7.7.1. DataSource 재정의 구성

DataSource를 구성하기 위해서는 @Bean으로 정의를 해야 한다. 스프링 부트는 데이터베이스 초기화를 포함하여 어느 곳에서나 필요한 DataSource를 재사용한다. 몇가지 설정을 외부설정해야한다면 DataSource에 환경("서드파티 구성")을 연동시킬 수 있다.

다음 예제는 dataSource 빈에서 어떻게 연결하는지 보여준다:

```
@Bean
@ConfigurationProperties(prefix="app.datasource")
public DataSource dataSource() {
    return new FancyDataSource();
}
```

다음 예제는 dataSource에 설정되는 속성을 어떻게 정의하는지 보여준다:

application.yml

```
app.datasource:
  url: jdbc:h2:mem:mydb
  username: sa
  pool-size: 30
```

여기서 추측해볼 수 있는 것은 FancyDataSource는 자바빈 속성으로 URL, username 과 pool 사이즈를 가지고 있으며, 이 설정은 DataSource에 자동으로 연결된다. "데이터베이스 초기화"에서도 동일한 작업이 일어난다 ("spring.datasource.*"의 하위 속성들을 사용하며 이 속성들은 외부 구성이 가능하다).

스프링 부트는 DataSourceBuilder라는 유틸리티성 빌더 클래스를 제공하여 표준 DataSource 중에 하나를 생성할 수 있다(클래스패스에 있다면). 빌더는 클래스패스에서 탐색된 것을 기반으로 하여 구성한다. 또한 JDBC URL을 기반으로 해서 드라이버를 자동탐색한다.

다음과 같이 DataSourceBuilder를 사용하여 DataSource를 어떻게 생성하는지 살펴보자:

```
@Bean
@ConfigurationProperties("app.datasource")
public DataSource dataSource() {
    return DataSourceBuilder.create().build();
}
```

애플리케이션이 실행될 때 DataSource에서는 연결정보를 필요로 한다. Pool 정의 구성도 설정가능하다. 실행에 사용할 수 있는 보다 자세한 내용은 구현체를 살펴보기 바란다.

다음 예제를 통해서 JDBC DataSource를 설정하는 속성을 어떻게 정의하는지 볼 수 있다:

```
app.datasource:
  url: jdbc:mysql://localhost/springboot
  username: springboot
  password: springboot
  pool-size: 30
```

그러나, 이 구성은 오류가 생길 수 있다. 커넥션 풀 타입이 노출되지 않거나, 재정의 `DataSource`를 위한 메타데이터를 위해 생성한 키가 없으면 IDE에서 자동완성을 할 수 없다(`DataSource` 인터페이스는 속성이 노출되지 않는다). 또한, 히카리(Hikari)가 클래스패스 상에 있다면, 이 설정은 동작하지 않는다. 왜냐하면 히카리는 `url` 속성이 없기 때문이다(`jdbcUrl` 속성은 있다). 이런 경우 다음과 같이 변경해야 한다:

```
app.datasource:
  jdbc-url: jdbc:mysql://localhost/springboot
  username: springboot
  password: springboot
  maximum-pool-size: 30
```

`DataSource`보다는 구현체에 따라서 커넥션풀 변경이 강제된다. 실행중에는 구현체를 변경할 수 없다.

다음 예제에서는 `DataSourceBuilder`를 이용해서 `HikariDataSource`를 만드는 과정을 보여준다:

```
@Bean
@ConfigurationProperties("app.datasource")
public HikariDataSource dataSource() {
    return DataSourceBuilder.create().type(HikariDataSource.class).build(
    );
}
```

`DataSourceProperties`를 이용하면 `DataSourceBuilder`를 초기화하는 것이 쉬워진다. 구현체에 따라서 `url/jdbcUrl`을 구분할 필요가 없이 `url`만 이용하면 간단하게 사용가능하다.

다음 예제는 `DataSourceProperties`를 이용해서 초기화된 `DataSourceBuilder`를 생성하면서 `HikariDataSource`를 생성한다.

```

@Bean
@Primary
@ConfigurationProperties("app.datasource")
public DataSourceProperties dataSourceProperties() {
    return new DataSourceProperties();
}

@Bean
@ConfigurationProperties("app.datasource")
public HikariDataSource dataSource(DataSourceProperties properties) {
    return properties.initializeDataSourceBuilder().type(HikariDataSource
.class).build();
}

```

NOTE

위에서는 열심히 `app.datasource`를 설정했지만, `spring.datasource`를 이용하면 외부구성이 가능한 자동구성된 `DataSource`를 사용할 수 있다.

"[chap-03]"에서 "`DataSource` 구성"과 `DataSourceAutoConfiguration`(<https://goo.gl/yknWsm>)를 살펴보면 보다 자세한 내용을 볼 수 있다.

7.7.2. `DataSource` 복수 정의

다중 `DataSource`를 구성해야한다면, 앞에서 설명했던 것과 동일한 방법을 이용할 수 있다. 그러나, `DataSource` 인스턴스 중에 하나에는 `@Primary`를 표시해야한다, 왜냐하면 다양한 자동구성 중에서 `DataSource`를 필요로 하는 경우가 있기 때문이다.

`DataSource`를 생성하면 자동구성은 중단된다. 다음 예제에서는 기본 `DataSource`에서 제공하는 자동구성과 동일한 기능 설정을 제공한다.

```

@Bean
@Primary
@ConfigurationProperties("app.datasource.foo")
public DataSourceProperties fooDataSourceProperties() {
    return new DataSourceProperties();
}

@Bean
@Primary
@ConfigurationProperties("app.datasource.foo")
public DataSource fooDataSource() {
    return fooDataSourceProperties().initializeDataSourceBuilder().build(
);
}

@Bean
@ConfigurationProperties("app.datasource.bar")
public BasicDataSource barDataSource() {
    return DataSourceBuilder.create().type(BasicDataSource.class).build();
}

```

TIP

`fooDataSourceProperties`에 `@Primary`를 선언해두면 데이터베이스 초기화 기능에서도 사용한다(초기화를 사용한다면).

두 `DataSource`는 보다 나은 재정의가 가능하다. 예를 들어, 다음과 같은 구성이 가능하다:

```

app.datasource:
  foo:
    type: com.zaxxer.hikari.HikariDataSource
    maximum-pool-size: 30
  bar:
    url: jdbc:mysql://localhost/test
    username: springboot
    password: springboot
    max-total: 30

```

위에서 사용한 방법을 이용하면 다음과 같이 두번째 `DataSource`를 다음 예제처럼 구성가능하다:

```

@Bean
@Primary
@ConfigurationProperties("app.datasource.foo")
public DataSourceProperties fooDataSourceProperties() {
    return new DataSourceProperties();
}

@Bean
@Primary
@ConfigurationProperties("app.datasource.foo")
public DataSource fooDataSource() {
    return fooDataSourceProperties().initializeDataSourceBuilder().build(
);
}

@Bean
@ConfigurationProperties("app.datasource.bar")
public DataSourceProperties barDataSourceProperties() {
    return new DataSourceProperties();
}

@Bean
@ConfigurationProperties("app.datasource.bar")
public DataSource barDataSource() {
    return barDataSourceProperties().initializeDataSourceBuilder().build(
);
}

```

앞의 예는 스프링 부트가 자동구성에서 사용하는 것과 동일한 방법으로 재정의 네임스페이스에 두 개의 DataSource를 구성한다.

7.7.3. 스프링 Data {repo} 이용

스프링 Data는 다양한 라이브러리에 대한 **@Repository** 인터페이스 구현체를 생성할 수 있다. **@Repository**가 **@EnableAutoConfiguration**클래스와 동일한 패키지(혹은 하위 패키지)에 포함되어 있다면 스프링 부트가 모든 것을 처리한다.

많은 애플리케이션에서 클래스패스 상에 스프링 Data 의존성을 추가(JPA를 위한 **spring-boot-starter-data-jpa**과 MongoDB를 위한 **spring-boot-starter-data-mongodb**)해야 하고 **@Entity** 객체를 다루기 위한 {repo}를 생성해야 한다. JPA 예제(<https://goo.gl/mh6Pc8>)와 MongoDB 예제(<https://goo.gl/6MRnf5>)가 있다.

스프링 부트는 **@EnableAutoConfiguration**를 기반으로 해서 **@Repository** 위치를 추측한다. 좀 더 자세히 제어하고 싶다면 **@EnableJpaRepositories**를 사용한다.

7.7.4. 스프링 구성으로부터 **@Entity** 정의 분리

스프링 부트는 **@EnableAutoConfiguration**를 기반으로 해서 **@Entity**의 위치를 추측한다. 보다 자세히 제어하고

싶다면 `@EntityScan`을 다음과 같이 정의하여 사용한다:

```
@Configuration
@EnableAutoConfiguration
@EntityScan(basePackageClasses=City.class)
public class Application {
    //...
}
```

7.7.5. JPA 속성 구성

스프링 Data JPA는 이미 벤더에 독립적인 구성선택사항(SQL 로깅도 포함)을 제공하였으며 스프링 부트는 하이버네이트를 위한 외부 구성 속성을 제공한다. 일부 속성은 컨텍스트에 따라 자동으로 감지되므로 굳이 설정하지 않아도 된다.

`spring.jpa.hibernate.ddl-auto`는 좀 특별한 경우인데, 실행 조건에 따라서 다른 기본값을 가진다. 내장 데이터베이스를 사용하고 스키마 매니저(Liquibase 혹은 Flyway)를 사용하지 않는다면 `DataSource`는 `create-drop`가 기본설정된다. 그 외의 경우에는 `none`가 기본설정된다.

방언(Dialect)은 현재 `DataSource`를 기반으로 해서 자동으로 감지되지만 `spring.jpa.database` 속성을 정의하여 구동시 점검없이 설정가능하다.

NOTE

데이터베이스를 지정하면 잘 정의된 하이버네이트 방언을 구성할 수 있다. 여러 데이터베이스는 한개 이상의 방언을 가지고 있는데, 개발자가 필요한 것에 맞지 않을 수도 있다. 이 경우, `spring.jpa.database` 속성을 `default`로 정의하고 `spring.jpa.database-platform` 속성을 설정하여 하이버네이트로 하여 상황에 따라 방언을 설정할 수 있도록 할 수 있다.

`spring.jpa.properties.*`에 있는 모든 속성은 로컬 `EntityManagerFactory`를 생성할 때 JPA 속성으로 전달된다.

7.7.6. 재정의 `EntityManagerFactory` 이용

`EntityManagerFactory`의 구성과 관련해서 모든 부분을 제어하고 싶다면, `entityManagerFactory`라는 이름으로 빈을 추가하면 된다. 그러면 스프링 부트는 엔티티매니저에 대한 자동구성을 중지하고 정의한 빈을 사용한다.

7.7.7. 복수 `EntityManager` 이용

기본 `EntityManagerFactory`가 잘 동작하고 있을 때, 새로운 것을 정의해야할 때가 있다. 그러나 동일한 타입으로 두번째 빈이 존재하면 기본 `EntityManagerFactory`가 비활성화된다. 스프링 부트에서 제공하는 `EntityManagerBuilder`를 이용하면 쉽게 만들 수 있다. 다음과 같은 방법으로 스프링 ORM에서 직접 `LocalContainerEntityManagerFactoryBean`를 얻을 수 있다:

```
// add two data sources configured as above

@Bean
public LocalContainerEntityManagerFactoryBean
customerEntityManagerFactory(
    EntityManagerFactoryBuilder builder) {
    return builder
        .dataSource(customerDataSource())
        .packages(Customer.class)
        .persistenceUnit("customers")
        .build();
}

@Bean
public LocalContainerEntityManagerFactoryBean orderEntityManagerFactory(
    EntityManagerFactoryBuilder builder) {
    return builder
        .dataSource(orderDataSource())
        .packages(Order.class)
        .persistenceUnit("orders")
        .build();
}
```

위 구성으로도 거의 잘 동작한다. 기능을 완성하기 위해서는 두 **EntityManager**에 대한 **TransactionManagers**를 구성해야 한다. **@Primary**를 추가한 것이 있다면 스프링 부트에서 기본 **JpaTransactionManager**를 둘 중 하나에 적용할 수 있다. 또는 JTA 트랜잭션 관리자를 사용할 수도 있다.

만약, 스프링 Data를 사용한다면 다음 예제에서 보는 것처럼 **@EnableJpaRepositories**를 구성해야 한다:

```
@Configuration
@EnableJpaRepositories(basePackageClasses = Customer.class,
    entityManagerFactoryRef = "customerEntityManagerFactory")
public class CustomerConfiguration {
    //...
}

@Configuration
@EnableJpaRepositories(basePackageClasses = Order.class,
    entityManagerFactoryRef = "orderEntityManagerFactory")
public class OrderConfiguration {
    //...
}
```

7.7.8. 전통적인 **persistence.xml** 이용

스프링은 더이상 JPA 프로바이더 구성을 위해 XML을 필요로 하지 않지만 스프링 부트는 그 기능을 이용하기 원한다고

가정한다. 만약 `persistence.xml`를 참조하길 원한다면 `LocalEntityManagerFactoryBean` 타입의 `@Bean`을 정의(ID는 `entityManagerFactory`으로 정의)해야 하며 거기에 퍼시스턴스 유닛 이름을 설정해야 한다.

기본적인 구성을 살펴보려면 `JpaBaseConfiguration`(<https://goo.gl/71YUTR>)을 살펴보기 바란다.

7.7.9. 스프링 Data JPA 와 몽고 {repo} 이용

스프링 Data JPA와 스프링 Data Mongo에서는 자동으로 `Repository` 구현체를 생성한다. 만약 클래스패스에 두가지고 존재한다면 스프링 부트에게 어느 {repo}에 대해서(혹은 둘 다) {repo}를 만들 것인지를 알려주는 몇 가지 추가 구성을 해야할 수도 있다. 가장 명확한 방법은 표준 스프링 Data `@EnableJpaRepositories`와 `@EnableMongoRepositories` 애너테이션을 사용하고 `Repository` 인터페이스의 위치를 지정하는 것이다.

또한 `spring.data.*.repositories.enabled` 플래그를 이용하면 외부 구성을 이용해서 자동구성되는 {repo}를 켜고 끌 수 있다. 예를 들어, Mongo {repo}는 끄고 자동구성된 `MongoTemplate`는 사용하고 싶을 때도 유용할 수 있다.

다른 자동구성된 스프링 Data {repo} 타입(Elasticsearch, Solr, 혹은 기타 등등)에도 동일한 장애와 기능이 존재한다. 그것들과 함께 작업하려면 그에 따라 애너테이션과 속성을 변경해야 한다.

7.7.10. REST 종단점 로 스프링 Data {repo} 노출

스프링 Data REST는 애플리케이션에서 스프링 MVC를 사용하고 있을 때 `Repository` 구현체를 REST 종단점으로 노출할 수 있다.

스프링 부트는 `RepositoryRestConfiguration`(<https://goo.gl/iaCfBT>)를 재정의하여 `spring.data.rest` 네임스페이스로부터 쓸만한 속성들을 노출하도록 구성가능하다. 추가적인 재정의의 제공해야한다면 `RepositoryRestConfigurer` 빈을 이용하면 된다.

NOTE

재정의한 `RepositoryRestConfigurer`에 대해서 우선순위를 지정하지 않았다면, 스프링 부트가 내부적으로 사용한 후에 실행된다. 우선순위를 지정할거라면 0 보다는 높게 설정하길 바란다.

7.8. 데이터베이스 초기화

SQL 데이터베이스는 사용하는 기술에 따라 다른 방법으로 초기화된다. 물론 수동으로 데이터베이스 초기화를 독립적으로 수행할 수도 있다.

7.8.1. JPA 를 이용한 데이터베이스 초기화

JPA는 DDL 생성기능을 가지고 있으며, 이 기능은 데이터베이스가 시작할 때 실행하도록 설정가능하다. 이 기능은 두가지 외부 속성으로 제어가 가능하다:

- `spring.jpa.generate-ddl`(boolean): 벤더에 독립적으로 기능을 켜고 끌 수 있다.
- `spring.jpa.hibernate.generate-ddl`(enum): 하이버네이트에서는 기능을 보다 상세히 제어할 수 있는 방법을 제공한다. 이에 대한 기능은 "[Hibernate 를 이용한 데이터베이스 초기화](#)"에서 상세히 다룰 것이다.

7.8.2. Hibernate 를 이용한 데이터베이스 초기화

`spring.jpa.hibernate.ddl-auto` 속성에 부여할 수 있는 표준 하이버네이트 속성값에는 `none`, `validate`, `update`, `create` 그리고 `create-drop`이 있다. 스프링 부트가 기본값을 선택할 때는 데이터베이스는 내장이라고 가정을 한다. 스키마 매니저가 없는 경우에는 `create-drop`을 기본으로 하고 그 외의 경우에는 `none`으로 처리한다. 내장 데이터베이스는 `Connection` 타입을 통해 감지한다. `hsqldb`, `h2` 그리고 `derby`는 내장이고 그 외에는 아니라고 판단한다. 인-메모리에서 실제 데이터베이스로 변환을 할 때는 주의해야 한다. 새 플랫폼에서 테이블과 데이터가 존재할 거라는 가정을 해서는 안된다. `ddl-auto`를 명시적으로 설정하거나 다른 메커니즘 중 하나를 사용하여 데이터베이스를 초기화해야 한다.

WARNING

스프링 프로파일에 따라서 `spring.jpa.hibernate.ddl-auto` 속성을 정의할 때 특히 조심해야 한다. 자칫하다가는 운영 데이터베이스를 초기화시키는 상황이 발생할 수도 있기 때문에 **배포 전 항상 주의**를 기울이기 바란다.

TIP

`org.hibernate.SQL` 로거를 활성화해서 스키마 생성을 출력할 수 있다. 디버그 모드가 활성화 되어 있다면 자동으로 처리된다.

추가적으로 `import.sql`이라는 이름으로 클래스패스 최상위에 파일이 위치해 있으면(`ddl-auto` 속성을 `create` 이거나 `create-drop`으로 설정) 시작시 하이버네이트가 스키마생성을 실행한다. 이 기능은 데모와 테스트 때는 유용하겠지만 운영 단계에서는 클래스패스에서는 곤란할 것이다. 이 기능은 하이버네이트 기능이다(스프링에서 하는 것은 없다).

7.8.3. 데이터베이스 초기화

스프링 부트는 `DataSource`의 스키마(DDL script)를 자동으로 생성하고 초기화(DML script) 가능하다. 클래스패스 최상위 위치에 `schema.sql`와 `data.sql`이 있다면 SQL을 불러온다. 추가로 스프링 부트는 `schema-${platform}.sql`와 `data-${platform}.sql` 파일을 처리한다, `{platform}`은 `spring.datasource.platform`의 값으로 치환된다. 이것을 이용하면 필요한 경우 데이터베이스에 따라 지정하는 것이 가능하다. 예를 들어, 개발자는 데이터베이스의 벤더이름(`hsqldb`, `h2`, `oracle`, `mysql`, `postgresql`와 기타 등등)을 설정할 수 있다.

스프링 부트는 내장 `DataSource`의 스키마를 자동으로 생성한다. `spring.datasource.initialization-mode` 속성(`always` 혹은 `never`)을 사용하여 재저오이 가능하다.

NOTE

JPA 기반의 애플리케이션에서는 스키마를 생성하는데 하이버네이트를 사용하거나 `schema.sql`을 선택할 수 있지만 두 가지 모드 사용할 수는 없다. `schema.sql`을 사용한다면 `spring.jpa.hibernate.ddl-auto`을 비활성화해야 한다.

7.8.4. 스프링 배치 를 이용한 데이터베이스 초기화

스프링 배치를 사용한다면 가장 보편적인 데이터베이스 플랫폼을 위한 SQL 초기화 스크립트를 미리 패키지로 제공한다. 스프링 부트는 데이터베이스 타입을 탐지하여 시작시 적절한 스크립트를 실행한다. 내장 데이터베이스를 사용한다면 기본적으로 발생한다. 다른 데이터베이스 타입을 사용한다면 다음과 같이 활성화할 수 있다.

```
spring.batch.initialize-schema: always
```

초기화를 비활성화하려면 `spring.batch.initialize-schema: never`로 설정하면 된다.

7.8.5. 고차원 데이터베이스 마이그레이션 도구 사용

스프링 부트에서는 두 가지 고차원 데이터베이스 마이그레이션 도구를 제공한다: Flyway(<http://flywaydb.org/>) 혹은 Liquibase(<http://www.liquibase.org/>)

Flyway

시작시 Flyway 데이터베이스 마이그레이션을 자동으로 실행하기 위해서는 클래스패스에 `org.flywaydb:flyway-core`를 추가한다.

마이그레이션은 `V<VERSION>__<NAME>.sql` 형식(<VERSION> 형식에 대해서는 <https://flywaydb.org/documentation/migration/versioned> 참조)을 가지는 스크립트를 이용한다. 기본적으로

`classpath:db/migration` 경로에 있는 폴더를 참조하지만, `spring.flyway.locations` 속성으로 위치를 변경할 수 있다. 거기에 더해서 `{vendor}` 위치변환자를 이용해서 벤더에 따라 스크립트를 지정할 수 있다. 다음과 같은 형식으로 사용가능하다:

```
spring.flyway.locations: db/migration/{vendor}
```

`db/migration`를 사용하는 대신, 데이터베이스 타입(MySQL의 경우 `db/migration/mysql`)을 따라서 폴더를 지정하여 구성할 수 있다. 지원하는 데이터베이스 목록은 `DatabaseDriver`(<https://goo.gl/XzWZmf>)을 살펴보기 바란다.

`flyway-core`에서 `Flyway` 클래스를 살펴보면 설정 가능한 스키마 등을 살펴볼 수 있다. 추가적으로, 스프링 부트는 몇몇 속성(`FlywayProperties`(<https://goo.gl/FvZA7E>))을 제공한다. 이 속성을 통해서 마이그레이션을 비활성화하거나 위치 확인기능을 끌 수 있다. 스프링 부트는 `Flyway.migrate()`를 호출하여 데이터베이스 마이그레이션을 수행한다. 보다 자세히 제어하고 싶다면 `FlywayMigrationStrategy`(<https://goo.gl/z5EZZ3>) 구현체를 `@Bean`으로 정의하면 된다.

Flyway는 SQL과 Java 콜백(<http://flywaydb.org/documentation/callbacks.html>)을 지원한다. SQL 기반 콜백을 사용하면 `classpath:db/migration` 폴더가 콜백스크립트 위치가 된다. Java 기반 콜백의 경우, `FlywayCallback` 구현체 혹은 `BaseFlywayCallback`를 확장한 클래스를 빈으로 생성한다. 생성된 빈은 Flyway에 자동으로 등록된다. `@Order` 혹은 `Ordered`를 구현하여 우선순위를 지정할 수 있다.

기본적으로, Flyway는 마이그레이션을 위해서 컨텍스트에 있는 `(@Primary)DataSource`를 이용한다. 만약 다른 `DataSource`를 사용하려면 하나를 만들고 `@Bean`과 함께 `@FlywayDataSource`를 표시하면 된다. 만약 두 개의 `DataSource`를 사용한다면 둘 중에 하나에는 `@Primary`를 선언해야한다는 것을 잊지말자. 그에 대한 대안으로 Flyway용 `DataSource` 외부 속성(`spring.flyway.[url,user,password]`)을 설정할 수 있다.

Flyway 예제(<https://goo.gl/YGKixA>)를 보면서 어떻게 설정할 수 있는지를 생각해보자.

특정 시나리오에 따라서 데이터를 제공하도록 Flyway를 사용할 수 있다. 예를 들어, 테스트에서만 사용하려는 경우에는 `src/test/resources`에 위치시키면 테스트를 위해서 애플리케이션을 시작할 때만 실행될 것이다. 보다 상세히 하고자 하는 경우, `spring.flyway.locations`를 재정의하여 프로파일에 따라 구성을 달리하여 프로파일이 활성화 되었을 때만 실행되도록 할 수 있다. 예를 들어, `dev` 프로파일일 때, 다음처럼 지정가능하다:

```
spring.flyway.locations=classpath:db/migration,classpath:dev/db/migration
```

`classpath:dev/db/migration`는 `dev` 프로파일이 활성화 되었을 때만 실행된다.

Liquibase

시작 시 Liquibase 데이터베이스 마이그레이션을 자동으로 실행하게 하려면, 클래스패스에서 `org.liquibase:liquibase-core`를 추가해야 한다.

기본적으로 `db/changelog/db.changelog-master.yaml`에서 마스터 변경로그를 읽어오지만 `spring.liquibase.change-log` 속성으로 위치를 변경할 수 있다. YAML 외에도 Liquibase는 JSON, XML 과 SQL 변경 로그를 지원한다.

기본적으로, Liquibase는 마이그레이션을 위해서 컨텍스트에 있는 `@Primary` `DataSource`를 이용한다. 만약 다른 `DataSource`를 사용하려면 하나를 만들고 `@Bean`과 함께 `@LiquibaseDataSource`를 표시하면 된다. 만약 두 개의 `DataSource`를 사용한다면 둘 중에 하나에는 `@Primary`를 선언해야한다는 것을 잊지말자. 그에 대한 대안으로 Liquibase 용 `DataSource` 외부 속성(`spring.liquibase.[url,user,password]`)을 설정할 수 있다.

컨텍스트에서 설정가능한 기본 스키마와 그 외의 기능들에 대해서는 `LiquibaseProperties`(<https://goo.gl/41puQQ>)을 살펴보기 바란다.

Liquibase 예제(<https://goo.gl/Agrala>)를 보면서 어떻게 설정할 수 있는지를 생각해보자.

7.9. 배치 애플리케이션

NOTE

기본적으로 배치 애플리케이션은 상세 Job 정보를 보관하기 위해 `DataSource`를 필요로 한다. 이를 벗어나려면 `BatchConfigurer`를 구현해야 한다. 자세한 내용은 `@EnableBatchProcessing`의 Javadoc(<https://goo.gl/2H7YEy>)를 참조해야 한다.

7.9.1. 구동시 스프링 배치 작업 실행

스프링 배치 자동구성은 컨텍스트 내에서 `@EnableBatchProcessing`를 추가하면 활성화된다.

기본적으로 시작시 애플리케이션 컨텍스트에 있는 모든 `Jobs`을 실행한다(자세한 사항은 `JobLauncherCommandLineRunner`(<https://goo.gl/wH9Lx3>)을 살펴보기 바란다). `spring.batch.job.names`를 지정하여 작업범위를 특정하여 좁힐 수 있다(Job 이름을 쉼표(,)로 구분가능).

애플리케이션 컨텍스트에 `JobRegistry`가 포함되어 있다면 `spring.batch.job.names`의 Job은 컨텍스트에서 자동으로 실행되는 대신 레지스트리에서 조회된다. 이 방법은 여러 작업이 하위 컨텍스트에서 정의하고 중앙에서 등록되는 복잡한 시스템에서 일반적인 패턴이다.

보다 자세한 내용에 대해서는 `BatchAutoConfiguration`(<https://goo.gl/uJ83jh>)과 `@EnableBatchProcessing`(<https://goo.gl/82UqwG>)을 살펴보기 바란다.

7.10. 액추에이터

7.10.1. 액추에이터 종단점 의 포트 혹은 주소 변경

독립형 애플리케이션에서 액추에이터 HTTP 포트는 기본적으로 메인 HTTP 포트와 동일하다. 애플리케이션에서 다른 포트를 사용하도록 하려면 `management.server.port` 속성을 설정하면 된다. 완전히 다른 네트워크 주소를 이용하려는 경우(관리를 위해 내부 네트워크와 애플리케이션을 위한 외부 네트워크가 있는 경우), 서버에서 연결가능한 서버 IP주소를 `management.server.address`으로 설정하면 된다.

보다 자세한 내용은 `ManagementServerProperties`(<https://goo.gl/QcTV4Q>)와 "관리 서버 포트 재정의"를 살펴보기 바란다.

7.10.2. `whitelabel` 오류 페이지 재정의

스프링 부트는 서버에러가 발생하는 경우 브라우저 클라이언트에서 볼 수 있는 'whitelabel' 에러 페이지를 출력한다 (클라이언트가 JSON을 요구했거나 다른 미디어 타입을 사용한다고 판단될 때는 그에 적절한 형태로 에러코드를 보여준다).

NOTE

`server.error.whitelabel.enabled: false`로 설정하면 에러 페이지를 끌 수 있다. 이렇게 설정하면 사용중인 서블릿 컨테이너의 기본값이 복원된다. 추가적으로 스프링 부트는 에러뷰를 처리하려고 시도하기 때문에 완전히 비활성화 하지 않고 오류 페이지를 추가해야 한다.

에러 페이지를 덮어쓰려고 한다면 템플릿 기술에 의존적이 된다. 예를 들어 타임리프를 사용한다면 `error.html` 템플릿을 추가할 수 있다. 만약 프리마커를 사용한다면 `error.ftl` 템플릿을 추가해야 한다. 일반적으로 `error`라는 이름의 `View` 리졸버가 필요하거나 `@Controller`에서 `/error` 경로를 제어하면 된다. 기본 구성에서 일부를 변경하지 않는 경우 `ApplicationContext`에서 `BeanNameViewResolver`를 찾을 수 있으며 `error`라는 이름의 `@Bean`을 정의하는 것도 간단한 처리방법 중 하나다. 보다 자세한 `ErrorMvcAutoConfiguration`(<https://goo.gl/8tM5PS>)을 살펴보기 바란다.

서블릿 컨테이너에서 어떻게 핸들러를 등록하는지에 대해서는 "에러 처리"를 살펴보기 바란다.

7.11. 보안

7.11.1. 스프링 부트 보안 구성 전환

애플리케이션에서 `@Configuration`과 함께 `@EnableWebSecurity`을 정의하면, 스프링 부트 기본 웹앱 시큐리티는 비활성화된다(액추에이터 시큐리티는 활성화 상태 유지). `security.*`의 속성을 설정하여 기본을 변경할 수 있다(설정 가능한 상세내역은 `SecurityProperties`(<https://goo.gl/mVwvVu>)과 스프링 부트 참고문서 "공통 애플리케이션 속성"(<https://goo.gl/pnxMsY>)에서 `SECURITY` 관련 속성을 살펴보기 바란다.

7.11.2. `AuthenticationManager` 변경하고 사용자 계정 추가

만약 `AuthenticationManager` 타입을 별도로 `@Bean`을 선언하여 사용하게 되면 기본값을 가지는 빈이 생성되지 않지만 스프링 시큐리티의 모든 기능을 사용할 수 있다.

스프링 시큐리티는 `AuthenticationManagerBuilder`라는 편의성을 제공하는 빌더를 통해서

`AuthenticationManager`를 구축하는 데 사용할 수 있다. 웹애플리케이션을 사용할 때 권장되는 방식은 다음과 같이 `WebSecurityConfigurerAdapter`를 확장하여 void 메서드에서 주입받아 사용하는 것이다:

```
@Configuration
public class SecurityConfiguration extends WebSecurityConfigurerAdapter {
    @Autowired
    public void configureGlobal(AuthenticationManagerBuilder auth) throws
Exception {
        auth.inMemoryAuthentication()
            .withUser("barry").password("password").roles("USER"); //
... etc.
    }
    // ... other stuff for application security
}
```

중첩(nested)클래스 혹은 독립형클래스에서 이용하면 유용하다(인스턴스 처리되는 순서에 영향을 줄 수 있는 다른 `@Bean`들과 혼합되지 않음). [웹시큐리티 예제\(https://goo.gl/xrDXPT\)](https://goo.gl/xrDXPT)라는 따라해보기 좋은 템플릿이 제공된다.

인스턴스화 문제가 발생하는 경우(JDBC 혹은 JPA를 이용하여 사용자 상세정보를 저장하는 경우)가 있다면, 다음 예제와 같이 `GlobalAuthenticationConfigurerAdapter`(다른 곳에서 `AuthenticationManager`가 필요하기 전에 `init()` 메서드에서 초기화)에서 `AuthenticationManagerBuilder` 콜백을 추출해야 할 수 있다:

```
@Configuration
public class AuthenticationManagerConfiguration extends
GlobalAuthenticationConfigurerAdapter {

    @Override
    public void init(AuthenticationManagerBuilder auth) {
        auth.inMemoryAuthentication() // ... etc.
    }
}
```

7.11.3. 프록시 서버 이용시 HTTPS 활성화

모든 주요한 중단점가 HTTPS를 통해서만 사용가능하도록 보장하는 것은 어느 애플리케이션에서건 중요한 일이다. 만약 톰캣을 서블릿 컨테이너를 사용하고 있다면, 스프링 부트는 톰캣이 가지고 있는 `RemoteIpValve`을 자동으로 추가, `HttpServletRequest`를 사용하여 안전한지 여부를 확인할 수 있다(실제 SSL이 종료 처리하는 프록시 서버의 다운스트림도 가능). 표준 동작은 요청에서 특정헤더의 유무에 따라 결정된다(`x-forwarded-for`와 `x-forwarded-proto`), 관례적인 이름이며, 대부분의 프론트엔드 프록시에서 동작해야 한다. 다음 예시처럼 `application.yml`에서 몇가지 값을 추가하면 활성화할 수 있다:

```
server.tomcat:
  remote-ip-header: x-forwarded-for
  protocol-header: x-forwarded-proto
```


NOTE

이 속성들을 통해서 밸브가 활성화된다. `TomcatServletWebServerFactory` 빈을 추가하여 `RemoteIpValve`를 직접 추가할 수도 있다.

스프링 시큐리티는 모든(혹은 몇몇) 요청에 대해서 필요한 보안채널에 관한 구성이 가능하다. 스프링 부트 애플리케이션에서 활성화하려면 `application.yml`에서 `security.require_ssl`을 `true`로 설정하면 된다.

7.12. 핫스와핑(Hot swapping)

스프링 부트는 핫스와핑을 지원한다.

7.12.1. 정적 콘텐츠 재적재

여러가지 선택사항이 있다. 그 중 가장 권장하는 것은 `spring-boot-devtools`를 사용하는 것인데, 개발시 제공되는 기능으로, 애플리케이션을 빠르게 재시작할 수 있으며 LiveReload 개발시 세밀한 구성(템플릿 캐싱과 같은)을 지원한다. 개발툴은 클래스패스의 변동사항을 살펴보는 방식으로 동작한다. 이 이야기는 정적 자원의 변경사항이 빌드된 후에 변화를 감지하여 적용한다는 뜻이다. 기본적으로 이클립스에서는 변경사항을 저장하면 자동으로 일어난다. 인텔리제이에서는 프로젝트가 필요한 빌드를 발동시키도록 해야한다. "**재시작 대상 제외**"를 이용하면 정적 자원을 변경해도 애플리케이션이 재시작되지 않도록 할 수 있다. 하지만 LiveReload는 발동된다.

반면 IDE에서 실행될 때(특히 디버깅 시)는 기능을 활용하는 것이 좋다(최근의 모든 IDE는 정적 자원의 재적재 및 자바 클래스가 변경되었을 때 핫스와핑이 가능하다).

마지막으로, "'스프링 부트' 빌드도구 플러그인"에서 이야기했던 플러그인에서 구성(`addResources` 속성 확인)하여 커맨드라인에서 실행할때도 소스 파일의 정적 파일을 재적재하도록 할 수 있다. 외부 css/js 컴파일러를 이용할 때도 적용가능해진다.

7.12.2. 컨테이너 재시작없이 템플릿 재적재

스프링 부트에서 지원하는 대부분의 템플릿 기술에 대해서는 캐싱처리를 비활성화할 수 있는 구성 선택사항을 제공한다. 만약 `spring-boot-devtools`를 사용한다면 개발시 이 속성이 자동구성(비활성화)된다.

7.12.3. 빠른 애플리케이션 재시작

`spring-boot-devtools` 모듈은 자동 애플리케이션 재시작 지원기능을 포함하고 있다. `JRebel`처럼 빠른 기술은 아니지만 앱을 종료하고 다시 시작하는 것보다는 빠르다.

NOTE

스프링 부트 애플리케이션은 비교적 작은 크기를 가지다보니 `spring-boot-devtools`의 재시작 기능으로도 충분히 만족스런 재시작 속도를 보여준다. 처음에 인텔리제이와 `JRebel`을 구매해서 사용했지만 `JRebel` 라이선스 종료 후에 `spring-boot-devtools`를 적용하여 쓰고 있지만 큰 불편없이 사용하고 있다. 애플리케이션의 크기가 작고 가벼워서 재시작하는 것이 부담이 없기 때문에 그렇다고 생각한다.

보다 자세한 내용은 "'스프링 부트' 개발자도구"를 살펴보기 바란다.

7.12.4. 컨테이너 재시작없이 자바 클래스 재적재하기

최신 IDE(이클립스, 인텔리제이와 기타 등등)에서는 바이트코드 핫스와핑을 지원하며, 클래스 또는 메서드 구조에 영향을 주지 않는 변경(메서드 내에 비즈니스 로직 변경)을 수행하면 부작용없이 깔끔하게 재적재될 것이다.

7.13. 빌드

스프링 부트는 메이븐과 그레이들을 위한 빌드 플러그인을 지원한다.

7.13.1. 빌드 정보 생성

그레이들과 메이븐 플러그인은 프로젝트의 이름과 버전 등의 빌드 정보를 생성할 수 있다. 또한 플러그인에서 구성에 따라 속성을 추가하는 것이 가능하다. 이 파일에 추가되는 기능은 스프링 부트에서 **BuildProperties** 빈을 자동구성하여 처리한다.

그레이들에서는 다음과 같이 이용하면 빌드정보가 추가된다:

```
jar {
    baseName = 'boot-spring-boot'
    version = '0.0.1-SNAPSHOT'
}

project.version = "${jar.version}"
project.group = "io.honeymon.boot"

// 코드 생략

springBoot {
    buildInfo()
}
```

추가적으로 그레이들 DSL을 사용하여 다음과 같이 속성을 추가할 수 있다:

```
springBoot {
    buildInfo {
        additionalProperties = [
            'written-by': 'honeymon'
        ]
    }
}
```

위처럼 추가하여 빌드를 진행하면 **/META-INF** 디렉토리에 다음과 같은 **build-info.properties** 파일이 생성된다:

```
#Properties
#Thu Nov 16 17:13:19 KST 2017
build.time=2017-11-16T17\:13\:19+0900
build.artifact=boot-spring-boot
build.group=io.honeymon.boot
build.written-by=honeymon
build.name=boot-spring-boot
build.version=0.0.1-SNAPSHOT
```

메이븐에서도 빌드 정보를 생성할 수 있는데 다음과 같이 **build-info** 골을 execution에 추가해야 한다:

pom.xml 에서 **<build></build>** 영역

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <executions>
        <execution>
          <goals>
            <goal>build-info</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

7.13.2. 깃 정보 생성

그레이들과 메이븐에서는 프로젝트 빌드시 **git** 소스코드 리파지토리의 상태에 관한 정보를 가지는 **git.properties**를 생성할 수 있다.

그레이들을 사용하는 경우에는 다음과 같이 **gradle-git-properties**(<https://goo.gl/kPE2Jv>) 플러그인을 추가해야 한다:

build.gradle

```
plugins {  
    id "com.gorylenko.gradle-git-properties" version "1.4.17"  
}  
  
// 생략  
gitProperties {  
    dateFormat = "yyyy-MM-dd'T'HH:mmZ"  
    dateFormatTimeZone = "GMT"  
}
```

① `gradle-git-properties`에서 커밋시간(`Date`를 JSON으로 직렬화)의 형식을 지정한다.

메이븐을 이용하는 경우에는 `spring-boot-starter-parent` POM에 `git.properties` 파일을 생성하도록 미리 구성된 플러그인(`git-commit-id-plugin`, <https://goo.gl/vWHhEZ>)을 포함하고 있기 때문에 다음과 같이 `pom.xml`에 추가하면 된다:

pom.xml

```
<build>  
    <plugins>  
        <plugin>  
            <groupId>pl.project13.maven</groupId>  
            <artifactId>git-commit-id-plugin</artifactId>  
        </plugin>  
    </plugins>  
</build>
```

`gradle-git-properties` 플러그인을 설치하고 나서 빌드를 하고나면 생성된 실행가능한 jar의 `B00T-INF/classes` 디렉토리 내에 `git.properties` 파일이 생성되며 내용은 대략 다음과 같다:

build.gradle

```
git.branch=develop  
git.commit.id=dc3893bd8d56b022c86f0be1a962fac595c07bfa  
git.commit.id.abbrev=dc3893b  
git.commit.user.name=honeymon  
git.commit.user.email=ihoneymon@gmail.com  
git.commit.message.short=Merge branch 'develop' of  
github.com:ihoneymon/boot-spring-boot into develop  
git.commit.message.full=Merge branch 'develop' of  
github.com:ihoneymon/boot-spring-boot into develop  
  
git.commit.time=2017-11-16T08:47+0000 ①
```

① `gitProperties`에서 지정한 `dateFormat` 형식으로 출력된다.

7.13.3. 의존성 버전 재정의

WARNING

각 스프링 부트 출시본은 버전에 따라 서드파티 의존성을 설정하여 설계하고 테스트됐다. 의존성 버전을 변경하면 충돌문제가 발생할 수 있다.

그래도 하겠다면 스프링 부트 참조문서에서 "[Customize dependency versions](#)"을 살펴보고 진행하기 바란다.

7.14. 전통적 배포

스프링 부트는 현대적인 배포뿐 아니라 전통적인 배포(**war** 파일을 생성하여 컨테이너 배포)를 지원한다.

7.14.1. 배포가능한 war 파일 생성

운영을 위한 war파일을 배포하기 위한 첫번째 단계는 `SpringBootServletInitializer`를 확장하고 `configure` 메서드를 재작성하는 것이다. 이렇게 하면 서블릿 컨테이너에서 실행될 애플리케이션이 스프링 프레임워크의 서블릿 3.0 지원 기능을 사용하도록 구성할 수 있다. 일반적으로 `SpringBootServletInitializer`를 확장한 메인 애플리케이션 클래스는 다음과 같은 모습을 하고 있을 것이다:

```
@SpringBootApplication
public class Application extends SpringBootServletInitializer {
    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder application) {
        return application.sources(Application.class);
    }

    public static void main(String[] args) throws Exception {
        SpringApplication.run(Application.class, args);
    }
}
```

다음단계로 jar 파일 대신 war 파일로 배포되도록 빌드 구성을 변경해야 한다. 만약 그레이들들을 사용한다면 다음과 같이 `build.gradle`을 변경해야한다:

```
apply plugin: 'war'
```

메이븐을 사용한다면 다음과 같이 `pom.xml`에서 war로 패키징되도록 변경해야 한다:

```
<packaging>war</packaging>
```

마지막 단계로 war 파일로 배포했을 때 내장 서블릿 컨테이너의 인터페이스를 사용하지 않도록 확실하게 처리해야 한다. 그러기 위해서는 내장 서블릿 컨테이너 의존성을 `provided`로 변경해야 한다.

만약 그레이들을 사용한다면 다음과 같이 서블릿 컨테이너를 `providedRuntime`로 변경한다:

```
dependencies {
    // ...
    providedRuntime 'org.springframework.boot:spring-boot-starter-tomcat'
    // ...
}
```

NOTE

`providedRuntime`는 그레이들 `compileOnly` 구성을 참조한다. `compileOnly` 의존성은 테스트 클래스패스의 영향을 받지 않는다. `providedRuntime`는 컴파일(`compile`)과 실행(`runtime`)은 적용되지만 빌드(`build`) 과정에서는 제외된다고 생각하면 된다.

메이븐을 사용한다면 다음과 같이 서블릿 컨테이너를 `provided`로 변경한다:

```
<dependencies>
  <!-- ... -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
    <scope>provided</scope>
  </dependency>
  <!-- ... -->
</dependencies>
```

"스프링 부트' 빌드도구 플러그인"을 사용하고 있다면, 실행가능한 war를 제작할 때는 내장 서블릿 컨테이너 의존성을 `lib-provided` 디렉토리 패키지에 의존성을 포함하도록 처리한다. 이 말은 서블릿 컨테이너에 배포가능하면서 커맨드라인에서 `java -jar`으로 애플리케이션을 실행할 수도 있다.

TIP

- 메이븐 기반 예제: `pom.xml`
- 그레이들 기반 예제: `build.gradle`

7.14.2. 오래된 서블릿 컨테이너에 배포가능한 war 파일 생성

오래된 서블릿 컨테이너는 서블릿 3.0 에서 사용하는 `ServletContextInitializer` 부트스트랩 프로세스를 지원하지 않는다. 이 컨테이너에서 스프링과 스프링 부트를 계속 사용할 수 있지만 애플리케이션에 `web.xml`을 추가하고 `DispatcherServlet`을 통해서 `ApplicationContext`를 적재할 수 있도록 구성해야 한다.

NOTE

예전에 프로젝트 템플릿으로 쓰려고 올려둔 리파지토리(<https://goo.gl/PD6nXE>)가 있다.

7.14.3. 기존 애플리케이션을 스프링 부트 로 변환

웹애플리케이션이 아닌 경우, 스프링 애플리케이션을 스프링 부트 애플리케이션으로 변환하는 것은 쉽다. 그렇게 하려면 `ApplicationContext`를 생성하는 코드를 버리고 `SpringApplication` 혹은

`SpringApplicationBuilder`를 호출하도록 대체한다. 스프링 MVC 웹애플리케이션은 배포가능한 war 애플리케이션으로 생성할 수 있으며 나중에는 실행가능한 war 혹은 jar로 이관가능하다. 보다 자세한 정보는 [Getting Started Guide on Converting a jar to a war](https://spring.io/guides/gs/convert-jar-to-war/)(<https://spring.io/guides/gs/convert-jar-to-war/>)을 살펴보기 바란다.

배포가능한 war를 생성하기 위해서는 다음 예제 `Application`이라 불리는 클래스처럼 `SpringServletInitializer`를 확장하고 스프링 부트 `@SpringBootApplication` 애너테이션을 추가하면 된다:

```
@SpringBootApplication
public class Application extends SpringServletInitializer {

    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder application) {
        // Customize the application or call application.sources(...) to
        // add sources
        // Since our example is itself a @Configuration class (via
        // @SpringBootApplication)
        // we actually don't need to override this method.
        return application;
    }
}
```

기억해둘 것은 `sources`에 밀언하는 것은 `ApplicationContext`와 유사하다. 일반적이리 이전에 정상적으로 동작하는 것은 변경 후에도 정상적으로 동작해야 한다. 스프링 부트에서 기본 제공하는 몇몇 빈은 나중에 삭제할 수도 있지만, 그러기 보다는 원하는 것에 맞춰 동작하도록 하는 작업이 필요하다.

정적자원은 클래스패스 최상위 `/public`(혹은 `/static` 혹은 `/resources` 혹은 `/META-INF/resources`)으로 이동시킨다. 이와 동일하게 `messages.properties`도 적용한다(스프링 부트에서 자동으로 클래스패스 최상위부터 탐지한다).

순수하게 스프링 `DispatcherServlet`과 스프링 시큐리티를 사용하고 있었다면 변경할 것은 없다. 만약 애플리케이션에서 다른 기능(예를 들어, 다른 서블릿이나 필터)을 사용하고 있다면 `web.xml`에 있는 몇몇 요소들을 `Application` 컨텍스트에서 몇몇 구성을 추가해야할 수 있다:

- `<servlet/>`과 `<servlet-mapping/>`으로 작성된 것은 `Servlet` 혹은 `ServletRegistrationBean` 타입의 `@Bean`으로 정의
- `<filter/>`과 `<filter-mapping/>`으로 작성된 것은 `Filter` 혹은 `FilterRegistrationBean` 타입의 `@Bean`으로 정의
- XML 파일에 있는 `ApplicationContext`은 `Applicatoin`에서 `@ImportResource`를 통해서 추가할 수 있다. 또는 많은 애너테이션을 사용하여 구성한 경우는 몇 줄의 `@Bean` 구성으로 다시 만들 수도 있다.

war 파일이 동작하면 `Application`을 실행할 수 있는 `main` 메서드를 다음 예제처럼 추가할 수 있다:

```
public static void main(String[] args) {
    SpringApplication.run(Application.class, args);
}
```

애플리케이션을 war 혹은 실행가능한 애플리케이션으로 시작하려면 다음과 비슷한 클래스에서 **SpringBootServletInitializer** 콜백과 **main** 메서드에서 사용할 수 있는 빌더를 재정의하여 공유해야 한다:

NOTE

```
@SpringBootApplication
public class Application extends SpringBootServletInitializer {

    @Override
    protected SpringApplicationBuilder configure
(SpringApplicationBuilder builder) {
        return configureApplication(builder);
    }

    public static void main(String[] args) {
        configureApplication(new SpringApplicationBuilder())
        .run(args);
    }

    private static SpringApplicationBuilder
configureApplication(SpringApplicationBuilder builder) {
        return builder.sources(Application.class).bannerMode
        (Banner.Mode.OFF);
    }
}
```

애플리케이션이 다음 범주에서 하나 이상에 속할 수 있다:

- **web.xml** 없는 서블릿 3.0+ 애플리케이션
- **web.xml**을 가진 애플리케이션
- 컨텍스트 계층을 가진 애플리케이션
- 컨텍스트 계층이 없는 애플리케이션

이 범주에 속하는 것들은 전환에 별 다른 무리가 없지만, 조금씩 다른 기술을 필요로 할 수 있다.

서블릿 3.0+ 애플리케이션은 스프링 서블릿 3.0+ 이니셜라이저 지원클래스를 사용하고 있다면 손쉽게 전환이 가능하다. 일반적으로 **WebApplicationInitializer**에 있는 모든 코드를 **SpringBootServletInitializer**으로 이관할 수 있다. 만약 한 개 이상의 **ApplicationContext**이 존재하는 경우(예를 들어, **AbstractDispatcherServletInitializer**를 사용하는 경우)에는 모든 컨텍스트자원을 **SpringApplication**에 넣어야할 수 있다. 접할 수 있는 주요한 문제는 컨텍스트 계층구조를 결합하지 않고 유지해야

하는 경우다. 이와 관련해서는 "[ApplicationContext 계층 조직\(부모 혹은 루트 컨텍스트 추가\)](#)"을 살펴보기 바란다. 웹특정 기능을 포함하고 있는 기존의 상위 컨텍스트는 일반적으로 모든 `ServletContextAware` 컴포넌트가 하위 컨텍스트에 있도록 분할해야 한다.

NOTE

고전적 방법일 수 있겠지만 웹특정 컨텍스트와 비웹특정 컨텍스트를 분리하여 필요에 따라 조합하여 사용하는 방식을 쓸 수 있다.

스프링 애플리케이션이 아닌 경우에도 스프링 부트 애플리케이션으로 변환이 가능하며 앞에서 다루었던 내용들이 도움이 될 것이다. 그러나, 아직 다루지 못한 문제들을 직면하게 될 것이다. 그 때는 [스택오버플로우에서 spring-boot 태그를 통해 질의](#)(<https://goo.gl/b8fRZP>)해보길 바란다.

7.14.4. 오래된 컨테이너(Servlet 2.5)에 war 배포

스프링 부트는 `ServletContext`(`Servlet`과 다른 것들을 등록할 경우)를 초기화하는데 서블릿 3.0 API를 사용하며, 이럴 경우에는 서블릿 2.5 컨테이너에서는 동일한 애플리케이션을 실행할 수 없다. **하지만** 특별한 도구를 사용하면 오래된 컨테이너에서 스프링 부트 애플리케이션을 실행하는 것이 가능하다. `org.springframework.boot:spring-boot-legacy`(<https://goo.gl/F8zcDJ>) 의존성(핵심 스프링 부트와는 별도로 유지관리되며 현재 `1.1.0.RELEASE`를 사용가능)을 추가한 후 `web.xml`을 추가하고 컨텍스트 리스너를 선언하고 애플리케이션 컨텍스트, 필터 및 서블릿을 작성하면 된다. 컨텍스트 리스너는 스프링 부트를 위한 특별한 목적으로 사용되지만, 나머지는 서블릿 2.5에서 스프링 애플리케이션을 위해 사용된다. 다음에서 보는 것처럼 `web.xml`을 작성하면 서블릿 2.5 컨테이너에서 스프링 부트 프로젝트를 실행할 수 있다:

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app version="2.5" xmlns="http://java.sun.com/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">

  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>demo.Application</param-value>
  </context-param>

  <listener>
    <listener-
class>org.springframework.boot.legacy.context.web.SpringBootContextLoaderL
istener</listener-class>
    </listener>

    <filter>
      <filter-name>metricsFilter</filter-name>
      <filter-class>
org.springframework.web.filter.DelegatingFilterProxy</filter-class>
    </filter>

    <filter-mapping>
      <filter-name>metricsFilter</filter-name>
      <url-pattern>/*</url-pattern>
    </filter-mapping>

    <servlet>
      <servlet-name>appServlet</servlet-name>
      <servlet-class>
org.springframework.web.servlet.DispatcherServlet</servlet-class>
      <init-param>
        <param-name>contextAttribute</param-name>
        <param-
value>org.springframework.web.context.WebApplicationContext.ROOT</param-
value>
      </init-param>
      <load-on-startup>1</load-on-startup>
    </servlet>

    <servlet-mapping>
      <servlet-name>appServlet</servlet-name>
      <url-pattern>/</url-pattern>
    </servlet-mapping>

  </web-app>

```

위의 예제에서는 애플리케이션 컨텍스트 하나(컨텍스트 리스너도 하나만 생성)를 사용하고 이것을 `init` 파라미터를 이용해서 `DispatcherServlet`에 등록했다. 이게 일반적인 스프링 부트 애플리케이션이다(일반적으로 애플리케이션 컨텍스트는 하나만 가지고 있다).

Chapter 8. 정리

'스프링 부트'의 특징을 정리하자면 다음과 같다:

- Java, Groovy, Kotlin 지원
- 그레이들과 메이븐 빌드도구 플러그인 제공
- 스프링 프레임워크 기반
- Java 언어에서 유행하고 있는 기술 추가 쉬움
 - 스프링 부트 프로젝트팀이 유행하는 기술을 선정
 - 유행하는 기술을 추가했을 때 적용되는 자동구성
 - 유행하는 기술을 추가하기 용이한 스타터(starter) 제공
- 외부 구성(Externalized Configuration)
 - 프로파일에 따라 애플리케이션 속성정의
 - 단일 배포본을 환경변수, 시스템변수 등을 이용한 애플리케이션 속성 정의
- 운영에 필요한 기능 제공
 - 애플리케이션 모니터링, 로깅, 계측, 보안
 - war 빌드 배포

자세히 살펴보면 위의 기능을 애플리케이션이 실행될 환경에 맞춰 조합하여 배포할 수 있다는 것이 **스프링 부트의 강점**이라고 생각한다.

기본적인 '스프링 부트'의 개발절차는 다음과 같다:

1. 'Spring initializer'를 이용하여 프로젝트 생성
2. 프로토타이핑을 통한 기능확인
3. 비즈니스 로직 작성
4. 빌드
5. 배포
6. 운영과 관리

물론 개발해야하는 애플리케이션이 이렇게 간단한 과정으로 만들어지지는 않는다. 각 과정의 사이사이에 고민해야하는 것들이 있다. 스프링 부트를 이용하면 그런 고민거리들을 덜어낼 수 있다.

스프링 프로젝트 흐름을 살펴보면 각 기술이 가지는 개념들을 적절한 추상화를 통해 코드로 뽑아내고 이 코드를 이용해서 "개발자가 **비즈니스 로직** 구현에만 집중해라"하는 경향이 있다. 기술을 사용하기 위해 알아야 하는 학습에 대한 부담감을 줄여주고 프로젝트 코드를 어떻게 작성하고 실행하는지만 이해하면 되는 것이다. 하지만 이 타성에만 익숙해져 있어서는 한계를 접하게 된다.

"확장(擴張, extension)"

스프링 부트는 스프링이 가지고 있는 **확장성**을 그대로 이어받았다고 생각한다. 개발자가 확장이 필요한 부분에서 제공하는 **인터페이스**를 통해 기본구성되어 있는 컴포넌트를 대신하거나 외부구성으로 기본구성을 대체하는 작업 등을 손쉽게 처리할 수 있다. 물론 확장포인트를 잘 짚어내는 이에게만 해당될 수 있겠지만...

스프링을 이용하는 건 쉽지만 이해하는 것은 어렵다(나는 지금도 제대로 이해하고 쓰는 것 같지 않다). 그래도 포기하지 않고 꾸준히 하다보면 언젠가는 이해할 수 있는 날이 올거라 기대를 하고 있다.

TIP '토비의 스프링 5'가 나오길 기대하고 있다.

{spring-boo}는 1.5 버전에서 2.0 버전으로 변경되면서 많은 것이 바뀌었다. JDK9과 스프링5를 지원하면서 말이다. 또 공부할 게 한가득 던져졌다. 스프링 부트 저장소(<https://github.com/spring-projects/spring-boot>)와 참고문서(<https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>)를 꾸준히 살펴보길 바란다. 물어보다 모르는 문제가 있을 때는 물어보는 것도 좋다.

- 스택오버플로우: <https://stackoverflow.com/tags/spring-boot>
- KSUG 페이스북 그룹: <https://www.facebook.com/groups/springkorea/>
- 그리고... ihoneymon@gmail.com 혹은 <https://twitter.com/ihoneymon>

내 맘대로 결론!

"자바 개발자+스프링 프레임워크 사용자"가 "**스프링 부트**"를 능숙하게 다룰 수 있다면 쉽고 빠르게 애플리케이션을 개발-배포-운영할 수 있는 능력을 가지게 될 것이다.

'스프링 부트'에 어느정도 익숙해졌기를 바라며 책을 마무리 지으려 한다.

어딘가에서 또 봅시다. ^^~

부록

- Java 8:: <https://docs.oracle.com/javase/8/docs/>
- Java 9:: <https://docs.oracle.com/javase/9/index.html>
- 스프링 참고문서
 - 4.X:: <https://goo.gl/oyttYh>
 - 5.X:: <https://goo.gl/gohykG>
- 스프링 부트 참고문서
 - 1.5.8:: <https://goo.gl/KwWjsE>
 - 2.0.0:: <https://goo.gl/rm1njc>
 - 스프링 부트 애플리케이션 공통 속성(properties): <https://goo.gl/CytJjh>

NOTE

이 공통속성은 `spring-boot-autoconfigure` 프로젝트에 있는 `properties` 를 목록으로 정리한 것이며 `application.yml` 혹은 `application.properties` 에서 필요한 속성값을 덮어쓰기(Override)하는 것이 가능하다.

TIP

스프링 부트에서 제공하는 자동구성 시 사용하는 기본값을 확인할 때 관련한 Properties 클래스를 살펴보면 된다.

- 스택오버플로우 `spring-boot`:: <https://stackoverflow.com/questions/tagged/spring-boot>
- 한국스프링유저그룹(KSUG) 페이스북 그룹:: <https://www.facebook.com/groups/springkorea/>
- 그레이들 참고문서:: <https://goo.gl/WLEFEH>
- 메이븐 참고문서:: <http://maven.apache.org/guides/>
- 허니몬 - Boot Spring Boot 관련 영상:: <https://goo.gl/qB8hHa>